Cross-Domain State Preservation Functor

Jinwook Kim (Jay)

June 28, 2026

Abstract

We mechanize, in Isabelle/HOL, the preservation of state transitions synchronized across heterogeneous domains (blockchains and off-chain ledgers). A hierarchy of locales captures cross-domain state preservation as a functor: we prove the three category axioms on preservation morphisms (identity, composition, associativity), lift them to a tower of synchronization-degree functors connected by natural transformations that are closed under composition, and establish degree monotonicity (over-provisioning is safe, under-provisioning is not). A compositional safety/liveness layer yields *guarded bounded convergence*: from an arbitrary finite-domain, lock-free state, with no assumption that cross-chain consistency holds initially, the synchronization oracle reaches a valid state within a bounded number of steps. We couple the functor to an authenticated data structure by lifting the merge and blinding operations of a Merkle interface to the global-state level, building directly on Lochbihler and Marić's `ADS_Functor` entry, and instantiate the construction both on its blindable-position functor and on a recursive model of the Canton transaction tree. The liveness locales abstract priority resolution, timeout-bounded lock release, and fair scheduling under an assume-guarantee fairness bound; the Byzantine threshold ($n \geq 3f + 1$) is load-bearing, yielding a fair schedule that makes the liveness layer inhabitable, witnessed by a non-degenerate four-node instance with one Byzantine node. Every generic locale is instantiated on the regulatory state model of Oraclizer, a state synchronization oracle for tokenized assets, and, to exhibit domain independence, on a TCP connection tracker. The synchronization model is atomic; lifting it to a partially synchronous network is left to subsequent entries.

Contents

1 Overview

Methodological positioning. The title names a *functor*, and the entry earns the term as a theorem rather than a slogan. Regarding state machines as objects and the structure-preserving synchronization maps of `State_Preservation.thy` as morphisms, `Functor_Laws.thy` proves the three category axioms on these morphisms: identity is a preservation morphism (`preservation_id`), preservation morphisms compose (`preservation_compose`), and their composition is associative (`preservation_assoc`). Precisely, the three axioms establish the *category* of state machines and preservation morphisms; the functorial reading is then carried at two places — each transition system is a functor from its one-object action category into partial maps on states, with preservation morphisms as the structure-compatible maps between such functors, and Section 1.11 exhibits a tower of degree-indexed functors with natural transformations between them. This is the cross-domain counterpart of the "closed under composition" structure of Lochbihler and Marić's `ADS_Functor` entry, whose authenticated-data-structure functor is single-domain; the present entry carries the functorial reading across heterogeneous domains and couples it with that authenticated layer through two glue predicates, `state_join` and `state_refines`, that lift the ADS merge and blinding (partial-order) operations to the global-state level.

We claim no new proof technique. The contribution is a mechanization, in three layers, of structure that, to the best of our knowledge, has not previously been formalized across domains: (i) the cross-domain state-preservation *functor* and its category axioms, above; (ii) *guarded bounded convergence* within the Byzantine D-quencer model ($n \geq 3f + 1$, deterministic fair-leader assumption) — from an *arbitrary* finite-domain, lock-free initial global state, with no assumption that the cross-chain consistency invariant holds initially, the synchronization oracle reaches a valid state within a bounded number of steps (`oraclizer_guarded_bounded_convergence`), upgrading the conditional safety of `conditional_safety_preservation` to convergence without an initial-consistency hypothesis; and (iii) a *tower* of synchronization-degree functors connected by natural transformations (`degree_natural_transformation`) that are closed under composition (`nt_compose`, via `nt_vertical_compose`) — a hierarchy of functors with natural transformations between them, which the `ADS_Functor` entry, to our knowledge, does not develop. Within the authenticated layer, the validity of an extracted cross-domain state is supplied by the extraction invariant (`extract_preserves_validity`); the `merkle_interface` locale — through its mergeability assumption `merge_respects_hashes` and its lemmas `join` and `hash`, all invoked explicitly in the soundness proofs — establishes the structural relations between extracted states (join and blinding refinement), not their validity.

This AFP entry consists of ten theory files. Four form the original two "generic locales + regulatory instances" pairs, one for safety and one for liveness; four further theories add the compositional and functorial layer — a generic safety/liveness composition with bounded convergence, the functor laws together with the authenticated-data-structure glue, the synchronization-degree hierarchy, and the authenticated-functor laws over a recursive model of the Canton transaction tree; a generic proof-automation theory contributes reusable discharge methods for the locale obligations; and an instance outside the regulatory domain (a TCP connection tracker) demonstrates that the generic layer carries no hidden regulatory assumptions.

`State_Preservation.thy` defines domain-independent locales for state machines, pairwise state preservation (with naturality), symmetric bidirectional preservation, and multi-domain preservation. All locales are parameterized over abstract types and can be instantiated with any domain that has finite states, deterministic transitions, and optional terminal states.

`Regulatory_Instance.thy` instantiates the generic safety framework with the regulatory state model. It defines the five regulatory states and seven regulatory actions, proves fundamental properties (terminal absorption, universal reachability of confiscation, transition exclusions with legal justification), models a synchronization protocol with preemptive locking, and proves the main safety theorems: *regulatory homomorphism* (cross-chain consistency after sync), *valid state preservation* (sync maintains the global validity invariant), and *multi-domain parametric instantiation*. In addition, it instantiates `state_preservation` (*escalation preservation*, see Section 1.4) and `symmetric_state_preservation` (*onchain DAML bridge*, see Section 1.5) with concrete heterogeneous-action and layer-crossing examples.

`Priority_Resolution.thy` defines domain-independent locales for liveness properties under Byzantine faults: `priority_system` for deterministic selection from a finite message set with injective priorities, and `fair_leader_system` for starvation freedom under periodic honest leader scheduling. The locales are parameterized over abstract types and impose no dependency on the regulatory domain; they are reusable for any distributed system requiring deterministic conflict resolution or fair scheduling.

`DQuencer_Instance.thy` instantiates the generic liveness framework with the regulatory domain. It defines authority levels (Regional, National, International), a four-component priority key with proven injectivity (under well-formedness bounds), a Byzantine fault model with the BFT threshold $n \geq 3f + 1$, and instantiates each of the two liveness locales for the regulatory consensus system. The `priority_system` instantiation is performed on the `priority_key` type as carrier, with the underlying D-quencer message recovered via a unicity corollary inside a well-formedness sublocale (Section 1.6); the `fair_leader_system` instantiation follows directly from the leader schedule. The theory proves the main liveness consequences (*select highest deterministic*, *starvation freedom*, *eventual completion*) and concludes with the *conditional safety preservation* theorem (Section 1.7), a conditional-safety corollary: a valid, unlocked, transition-enabled state synchronizes successfully and stays valid. Its proof invokes only the safety side, so the corollary makes no Byzantine, determinism, deadlock, or starvation claim of its own; those liveness results are the separate theorems listed above.

`Proof_Automation.thy` provides reusable Eisbach discharge methods (`discharge_state_machine`, `discharge_preservation`) for the instance obligations of the generic locales, driven by three named theorem collections (`discharge_simps`, `discharge_intros`, `discharge_dels`) into which an instance theory declares the defining equations of its domain. The theory depends only on `State_Preservation.thy` and Eisbach and carries no application vocabulary; see the *Proof automation* subsection below.

`Composition.thy` provides domain-independent locales for composing a guarded safety invariant with an eventual-progress scheduler (`guarded_invariant`, `eventual_discharger`, `safety_liveness_composition`) and extends them with a natural-number progress measure (`converging_composition`) to derive *guarded bounded convergence*: from an arbitrary carrier state, not assumed to satisfy the invariant, the system reaches an invariant-satisfying state within a bounded number of steps (`bounded_convergence_from_arbitrary`). The development commits to no application vocabulary and depends only on `State_Preservation.thy`.

`Functor_Laws.thy` establishes that cross-domain state preservation is functorial, proving the three category axioms on state-preservation morphisms (`preservation_id`, `preservation_compose`, `preservation_assoc`). It defines two glue predicates (`state_join`, `state_refines`) lifting the ADS merge and blinding order to the global-state level, an `authenticated_state` locale over the `merkle_interface`, and two soundness theorems (`authenticated_preservatio` `blinded_view_preserves_validity`) whose proofs invoke the Merkle interface explicitly. It then instantiates `converging_composition` with the oracle's synchronization protocol and proves `oraclizer_guarded_bounded_convergen` convergence to a valid state from an arbitrary finite-domain state, with no initial consistency assumption.

`Hierarchy.thy` stratifies the construction by *synchronization degree*, building a tower of transition functors related by a forgetful map (`degree_forget`). Each degree is a genuine functor on the free category of regulatory action words — it preserves the identity word and word concatenation (`deg_run_Nil`, `deg_run_append`) and its action on a word collapses to the composite regulatory transition broadcast once (`deg_run_collapse`), not a vacuous fold. The forgetful map is a natural transformation between consecutive degrees, on the whole action category and not merely its generators (`degree_natural_transformation`, `degree_forget_natural_run`), and natural transformations are closed under composition (`nt_vertical_compose`, `nt_compose`), so the whole degree ladder is structurally coherent. The theory also proves reduction containment of the degree classes (`reduction_containment`) and a one-directional degree monotonicity — over-provisioning reconciles all required chains (`over_provisioning_guarantees`) while under-provisioning guarantees nothing (`no_downward_safety`); validity preservation itself is degree-free (`processing_preserves_validity`, recorded in the over-provisioning regime as the auxiliary `hierarchy_monotonicity`). It also shows well-definedness of the causal-consistency boundary (`boundary_well_defined`).

`External_Instance.thy` instantiates the generic locales on a domain disjoint from the regulatory vocabulary: the TCP connection state machine (an RFC 793 subset) observed by a stateful connection tracker. It imports only the generic theory and the generic proof automation —

not `Regulatory_Instance.thy` — and discharges its instances with the generic methods; see the subsection *An instance outside the regulatory domain.*

1.1 Theory Dependency Structure

The ten theories build up in two stages. The four base theories form two generic-instance pairs that meet at `DQuencer_Instance.thy`: `State_Preservation.thy` is imported by `Regulatory_Instance.thy`; `Priority_Resolution.thy` depends only on `Main` and is deliberately kept independent of the regulatory domain to preserve reusability; `DQuencer_Instance.thy` imports both `Priority_Resolution.thy` and `Regulatory_Instance.thy`, serving as the join point where the liveness framework is applied to the regulatory model and combined with the safety results. The compositional theories sit on top: `Composition.thy` imports `State_Preservation.thy`; `Proof_Automation.thy` imports `State_Preservation.thy` and `HOL-Eisbach`; `Functor_Laws.thy` imports `Composition.thy`, `Regulatory_Instance.thy`, `DQuencer_Instance.thy`, `Proof_Automation.thy`, and `ADS_Functor.Merkle_Interface`; `Hierarchy.thy` and `Canton_Bridge.thy` import `Functor_Laws.thy` (`Canton_Bridge.thy` additionally `ADS_Functor.ADS_Construction`); and `External_Instance.thy` imports only `Proof_Automation.thy` (hence, transitively, only the generic `State_Preservation.thy`), deliberately independent of the regulatory instance. The dependency on `ADS_Functor` is the entry's only external session dependency beyond the Isabelle distribution (`HOL-Library`, `HOL-Eisbach`).

1.2 Design Decisions

The regulatory state model incorporates several deliberate design decisions based on legal precedence and operational considerations:

- The state machine models seven regulatory `reg_actions`: the four escalation actions FREEZE, SEIZE, CONFISCATE, RESTRICT and their de-escalation counterparts UNFREEZE, UNRESTRICT, RELEASE. RECOVER and LIQUIDATE, two of the product's six enforcement actions, are excluded as they involve force transfers or external DEX interactions rather than regulatory state transitions.
- The direct transition SEIZED → FROZEN is excluded because seizure is a strictly stronger legal constraint than freezing; relaxation requires explicit release first.
- The direct transition FROZEN → RESTRICTED is excluded to enforce passage through ACTIVE.
- Preemptive locking is modelled as a guard on the synchronization function: `sync` acquires the lock, updates, and releases in a single

atomic step. Under this atomic abstraction the guard expresses the *intended* exclusion of competing regulatory actions on the same asset rather than serialising them dynamically (there is no interleaving in the model to serialise); the dynamic correctness of contention-time locking is deferred to subsequent work. The guard does not address double-spending, which is handled at a different layer.

- Deadlock (a concurrency phenomenon) does not arise within the model's scope: the atomic `sync` model has no concurrent lock contention. Forced lock release under contention is out of scope, deferred to a preemptive-lock-correctness property; the entry states no proved deadlock-freedom theorem.

1.3 Proof Automation

The entry ships a reusable proof-automation layer (`Proof_Automation.thy`): two Eisbach methods, `discharge_state_machine` and `discharge_preservation`, discharge the obligations of `state_machine` and `state_preservation` instances on finite, enumerated domains. An instance theory declares the defining equations of its state/action sets and transition functions into three named theorem collections (`discharge_simps` for equational and conditional-rewrite content, `discharge_intros` for closure rules, `discharge_dels` for default rewrites that must be suppressed to keep structured state maps opaque), after which whole instances are discharged by a single method invocation.

The methods are exercised against the entry's own instances, with the statements kept identical to their manual counterparts. Four instances are discharged by pure search, with no registered interpretation to fall back on: the escalation-scoped DAML state machine (manual proof 22 lines, method one line; 6 obligations), the escalation-scoped layer-crossing bridge (manual 53 lines, method one line; 11 obligations), the composed three-domain morphism of Section 1.4 (re-derived from atomic obligations without invoking `preservation_compose`; 11 obligations), and a forward escalation-scoped bridge with no manual counterpart, which the method constructs outright. Three further one-line restatements of the registered interpretations (escalation preservation and the two layer-crossing directions, manually 57, 42 and 25 lines) are discharged from the registrations. Across the search-backed instances the methods close 39 of 39 atomic obligations; no obligation was weakened or left to a manual fallback. The methods and their collections are part of the entry's reusable surface: an importer instantiating the locales on its own domain populates the collections and reuses the same two entry points, as the instance outside the regulatory domain does below.

The liveness model adopts an assume-guarantee approach for starvation freedom: rather than directly modeling the probabilistic guarantees of

VRF-based leader election (which would require extensive probability theory infrastructure), the `fair_leader_system` locale assumes that within any `fairness_bound` consecutive epochs, at least one epoch has an honest leader. Under $f < n/3$ Byzantine faults, the probability that a Byzantine leader is elected k consecutive times is $(f/n)^k$, which decreases exponentially. The fairness bound captures this as a deterministic upper bound, following standard assume-guarantee reasoning practice. The Byzantine threshold is not decorative. It guarantees that a fair schedule exists (`fair_schedule_exists`): honest-node existence (`honest_nonempty`, in turn from `honest_majority` and the threshold $n \geq 3f+1$) yields a constant witness schedule on an honest node. This lemma is load-bearing, not idle: `liveness_inhabitable` consumes it to show that *every* D-quencer system admits an instantiation of the liveness locale, so the fair-leader assumption is satisfiable for every threshold-meeting system rather than merely postulated. The whole chain — threshold, `honest_majority`, `honest_nonempty`, `fair_schedule_exists` — is therefore load-bearing in the formal sense: deleting any link breaks the proof of `liveness_inhabitable`.

The liveness hosts are moreover inhabited by concrete global interpretations. `singleton_dq` discharges every assumption of the D-quencer liveness locale with a single-honest-node system, and `singleton_dq_priority` does the same for the priority sublocale. Because a zero-fault system meets the threshold only vacuously, a second interpretation `bft_quorum` exercises it at the tight bound with a *real* fault: four nodes, one of them Byzantine ($4 \geq 3 \cdot 1 + 1$, $\texttt{card (byzantine\_nodes)} = 1 \leq 1$), an honest super-majority of three scheduled on one honest node. The assumption sets of the liveness locales are thus consistent absolutely — witnessed both degenerately and with a genuine Byzantine fault — not merely relative to a hypothetical deployment.

1.4 Heterogeneous-Action Instance: Escalation Preservation

The instantiation of `state_preservation` provided in `Regulatory_Instance.thy` models the situation where two chains share a regulatory state space but differ in their on-chain action vocabularies. Some jurisdictions handle de-escalation (Unfreeze, Unrestrict, Release) exclusively through separate judicial procedures rather than as on-chain actions; the corresponding chain therefore exposes only the four escalation actions (Freeze, Seize, Confiscate, Restrict) on chain. We model this asymmetry by using two distinct action types: `reg_action` for the source chain (seven constructors) and a dedicated four-constructor datatype `chain_b_action` for the target chain.

The instantiation exploits two locale features simultaneously. First, the source action set parameter $\texttt{actions}_\texttt{s}$ is instantiated with the strict subset $\{\textsc{Freeze}, \textsc{Seize}, \textsc{Confiscate}, \textsc{Restrict}\}$ of `reg_action`, scoping structural preservation to the escalation actions only. Second, the source and target action types (`'a` vs. `'b`) carry different datatypes, exercising the locale's heterogeneous action signature. The naturality conditions hold by case analysis on the four escalation actions and the five regulatory states, with the target chain's transition function defined to mirror the source on the escalation subset. Sequential preservation follows as a direct corollary of the locale's main theorem: every escalation action sequence on the source chain produces, when mapped, an equivalent sequence on the target chain ending in the same regulatory state.

This instance is more than a convenience: it isolates the locale's $\texttt{actions}_{\texttt{s}}$ parameter as the formal handle for expressing partial action vocabularies, a concern that recurs across multi-jurisdiction protocol coordination, escalation-only consensus systems, and any cross-domain mapping where the two domains expose different on-domain actions.

A further point the instance makes explicit is the deliberate strength of the preservation condition: naturality is required in both the enabled and the disabled direction (the target must reflect undefinedness of the source transition), so a target domain that is strictly more permissive than the source on some action cannot be a morphism target. The $\texttt{actions}_{\texttt{s}}$ parameter is the intended handle for scoping preservation to the action subset on which the two domains agree in permissiveness, exactly as this instance does.

Composition is exercised on a concrete heterogeneous three-domain chain: the DAML permission ledger, restricted a fortiori to the escalation subset (`daml_escalation_state_machine`), is carried by the layer-crossing map into the regulatory enum (`daml_to_reg_escalation_bridge`) and from there into Chain B's four-action vocabulary, and the composite morphism DAML → Chain B is obtained by a single application of `preservation_compose` (`daml_to_chain_b_composed`) — no new naturality argument is needed. In HOL a heterogeneous N-domain chain cannot be packed into one object, since the domains carry genuinely different state and action types; arbitrary finite heterogeneous chains are instead expressed as composites of pairwise morphisms, with associativity (`preservation_assoc`) making longer path composites unambiguous.

1.5 Layer-Crossing Instance: Onchain DAML Bridge

The instantiation of `symmetric_state_preservation` provided in `Regulatory_Instance.thy` models the bidirectional binding between two representations of the same regulatory state: the on-chain `reg_state` enum and an off-chain DAML structured permission record (`daml_perm`). The DAML record carries a status tag together with auxiliary fields (`seized_by`, `restriction_scope`) that hold the party / scope metadata required by the corresponding regulatory status; a type-level invariant `valid_daml_perm` ties the auxiliary fields to the status tag (non-`None` exactly when the status semantically requires them).

The action vocabularies on the two layers coincide: both layers process the seven actions of `reg_action`, so the action map is the identity in both directions. All non-trivial content of the symmetric instance therefore lives in the layer-crossing state mapping: the bijection between the five-element `reg_state` enum and the image of `reg_to_daml` in `daml_perm`. The roundtrip assumptions of `symmetric_state_preservation` reduce to two bijection conditions, both proved by case analysis (on `reg_state` for one direction, on the status tag plus `valid_daml_perm` for the other). As a corollary, `reg_to_daml` is injective on `reg_states`, which is the formal expression of "no information loss in the layer-crossing representation".

This is the abstraction that underlies the semantic interoperability between an on-chain settlement layer (where states are represented as enums on integer-valued storage slots) and an off-chain ledger built on DAML's permission model. The structured-record side captures DAML's *need-to-know* approach where party / scope metadata is bound to the entitlements implied by a regulatory status; the bijection guarantees that synchronizing in either direction is faithful.

The domain guard carving out the valid records is load-bearing, and this is recorded permanently as a machine-checked witness (`guard_is_load_bearing`): a record carrying the seized tag with no seizing party lies outside the bijection domain, the guarded DAML transition rejects Release on it, while the bare enum projection of the same record would accept the action — so dropping the guard would break naturality exactly on such malformed records. The guard is the boundary of the preservation guarantee, not a notational convenience.

1.6 Priority Instance: Well-Formed Messages

The `priority_system` locale assumes priority injectivity unconditionally on its carrier type. In the D-quencer setting, injectivity of `make_priority_key` holds only when the message fields satisfy two well-formedness bounds: `msg_timestamp` $\leq$ `max_time` and `dqm_source_node` $\leq$ `max_node`. These bounds correspond to the system parameters of any concrete deployment (a fixed-precision time clock and a fixed roster of nodes), but they are not properties of the underlying `dq_message` record type as such.

To bridge this gap without imposing the well-formedness bounds at the type level, we instantiate `priority_system` directly on the `priority_key` type (the lexicographic four-tuple) with the identity function as the priority projection; injectivity is then satisfied by reflexivity on the carrier. The connection back to `dq_message` is made inside a sublocale `dquencer_priority_context` extending `dquencer_system` with a fixed message set `msg_set` satisfying the bounds, which defines the priority-key set `msg_priority_keys` = `make_priority_key max_time max_`

A further sublocale dquencer_priority_concrete adds the priority-distinctness assumption on msg_set (distinct messages produce distinct priority keys, a natural BFT-consensus precondition on authority / timestamp / severity / source-node tuples). Under this assumption the priority key uniquely identifies its underlying message, captured by a definition recover_msg :: priority_key ⇒ dq_message together with a unicity lemma derived from priority_key_injectivity and the distinctness hypothesis.

The dquencer_system locale itself is not modified. Concrete deterministic-selection consequences for the D-quencer (dq_select_highest_deterministic, dq_select_highest_in_set, and the message-recovery corollary dq_select_highest_message) are stated inside the sublocale; together they show that running the generic select_highest on msg_priority_keys and then calling recover_msg yields a deterministic choice of the highest-priority well-formed message in msg_set.

1.7 Conditional Safety Preservation

The conditional_safety_preservation theorem in DQuencer_Instance.thy is a conditional-safety corollary. Concretely: given a valid global state, an asset present on the source chain in a regulatory state from which a transition is enabled, and the asset not currently locked, the synchronization function returns Some and the resulting global state is again valid.

Its proof invokes only the safety side — lock_acquire_success and valid_state_preservation, both from Regulatory_Instance.thy — and none of the two liveness interpretations (dq_priority and dq_fair). We are deliberate about its scope: it does *not* discharge the honest-node or fair-leader assumption, and it makes no Byzantine, determinism, deadlock, or starvation claim. The unlocked precondition is the state that the atomic lock discipline re-establishes after each synchronization, but that connection is operational, not a logical consequence of this corollary. The genuine fusion of safety and liveness — where the initial-validity hypothesis is dropped and convergence is driven by a well-founded measure on cross-chain inconsistency, under the fair-leader assumption — is oraclizer_guarded_bounded_convergence (Section 1.8).

1.8 Guarded Bounded Convergence

The safety results of the base entry are *conditional*: conditional_safety_preservation assumes the global state is already valid. Composition.thy removes that assumption at the abstract level. Its converging_composition locale couples a guarded invariant with a bounded-fair discharger and a natural-number progress measure that every discharging step strictly decreases while the invariant fails; bounded fairness drives the measure to zero, and the zero set lies inside the invariant. The result, bounded_convergence_from_arbitrary, reaches an invariant state from an arbitrary carrier state within `progress_measure` $\times$ `window` steps.

`Functor_Laws.thy` discharges this locale for the oracle. The progress measure is `inconsistency_pairs`, the number of connected chain pairs that disagree on a shared asset; the critical step, `sync_reduces_inconsistency`, shows that synchronizing a disagreeing asset strictly decreases this count. Interpreting `converging_composition` inside the D-quencer liveness context (the fair-leader assumption) yields `oraclizer_guarded_bounded_convergence`: from an arbitrary finite-domain, unlocked state — with *no* assumption of initial cross-chain consistency — the oracle reaches a valid state within a bounded number of evolution steps. This strengthens the conditional safety of Section 1.7 by dropping its initial-consistency hypothesis; it still operates under the fair-leader assumption.

The realization map is pinned to *terminal-faithful safe recovery* (`safe_recovery`): a realized synchronization must not only reduce the measure but use CONFISCATE exactly on the assets that already carry the terminal state on some chain (`terminal_present`). This equivalence closes, in both directions, two degrees of freedom that a merely measure-reducing realization would leave open: resolving a plain ACTIVE/FROZEN disagreement by broadcasting CONFISCATED (indiscriminate confiscation), and overwriting a recorded CONFISCATED holding with a non-terminal value (confiscation erasure). A safe recovery exists in every inconsistent carrier state (`inconsistent_has_safe_recovery`), so the headline convergence statement is unchanged. Two safety theorems record the payload: along every recovery run, an asset that starts with no recorded confiscation never acquires one (`recovery_no_fresh_terminal`), and on a terminal-bearing asset a safe recovery can only complete the confiscation (`recovery_terminal_completion`). Both excluded behaviours are additionally pinned by machine-checked regression witnesses on concrete two-chain states (`blind_confiscate_excluded`, `terminal_overwrite_excluded`).

1.9 Authenticated Cross-Domain State

Beyond the functor laws, `Functor_Laws.thy` couples the construction to an authenticated data structure. The two glue predicates read the ADS operations at the global-state level: `state_join sa sb sab` says `sab` is the consistent combination of the partial views `sa` and `sb` (the state-level reading of the ADS merge), and the least such combination on revealed holdings (lock components unconstrained): `sab` carries the value of whichever view knows a chain, the two views agree wherever both are defined, and a containment clause forbids `sab` from revealing any chain beyond the union of the two views; and `state_refines sa sb` says `sa` is a partial view of `sb` (the state-level reading of the ADS blinding order). The `authenticated_state` locale extends the `merkle_interface` with an extraction map from authenticated values to global states. Its two soundness theorems, `authenticated_preservation_soundness` and blinded_view_preserves_validity, show that merging hash-compatible values yields a valid state that both inputs refine, and that a blinded view extracts to a valid refinement. The proofs invoke the Merkle interface explicitly — the mergeability assumption merge_respects_hashes and the lemmas join and hash of the merkle_interface locale — so the dependency is load-bearing rather than a mere import of a signature. Validity itself comes from the extraction invariant (extract_preserves_validity); the glue predicates and the Merkle lemmas supply the structural relations between extracted states, not their validity. The glue layer is abstract within one structural commitment: state_join enforces a single consensus value per object identifier across both views. Any system of that shape is a potential transplant target — for instance Merkle-replicated registries, authenticated cross-shard asset records, or the public/witness layering of zero-knowledge proofs, where state_refines would express that a public view is a blinding refinement of a full witness — but this entry instantiates the layer only on the regulatory model; those domains are not formalized here.

Functor_Laws.thy discharges this locale's model obligation with the interpretation oss_authenticated, so its soundness theorems are not vacuous. The authenticated datum is a consensus regulatory state together with the set of chains that have revealed the shared asset in that state; the hash (auth_hash) commits to the consensus state alone, the merge (auth_merge) unions the revealed-chain sets when the consensus states agree, the blinding order (auth_bo) forgets revealed chains, and the extraction map (auth_extract) returns the global state in which every revealed chain holds the asset in the consensus state. Because the hash fixes the consensus state, mergeable data never disagree, so the extracted join is consistent and the three coherence obligations (extract_respects_merging, extract_under_blinding, extract_preserves_validity) hold non-vacuously, with state_join realised as the union of revealed chains and state_refines as their subset order. A machine-checked regression witness (rogue_join_excluded) pins the containment clause of state_join: a candidate join that extends the union of two views with a disagreeing holding on a chain revealed by neither view is itself inconsistent and is rejected as a join. The instance is at once the oracle's concrete authenticated cross-domain state and a reusable pattern — a consensus-committing hash over a revealed-participant set — that an importer can transplant to the abstract domains just listed.

1.10 Authenticated Functor and the Canton Transaction Tree

Beyond soundness, the authenticated layer of the previous subsection carries a functorial reading, and Canton_Bridge.thy proves its categorical laws. Inside the authenticated_state locale the blinding relation is a preorder on authenticated values and state_refines is a preorder on global states;

the extraction map is the composite functor from the first preorder to the second, sending each blinding morphism to a refinement morphism. The headline law, `cdsp_ads_compose`, closes this functor under composition: two composable blindings $a \to b$ and $b \to c$ compose to $a \to c$ by transitivity of the blinding order, and the functor sends the composite to the composite refinement of the extracted states, with all three states valid. It is the authenticated, cross-domain analogue of `preservation_compose`. The companion law, `cdsp_ads_merge_assoc`, lifts the Merkle interface's merge associativity to the join algebra on extracted states: the three-way combination of authenticated views is independent of its bracketing, so the two groupings land on the same joined state.

That single-step soundness is then raised to whole synchronization sequences. Along a `blinding_path`, a list in which each element is a blinding of its successor, `sequence_authenticity_preservation` shows that every view extracts to a valid state and refines the most-revealed endpoint (validity and need-to-know together); `blinding_path_hash_soundness` and `merge_seq_hash` show that the sequence commits to a single authenticating root in the blinding direction and in the merge direction respectively; and `sequence_inclusion_integrity` establishes inclusion integrity. This sequence-level result is stated at the level of revealed holdings: a holding revealed by any view in the sequence is genuinely present at the endpoint with the same regulatory state, and no view fabricates a chain the endpoint does not authenticate. It is then sharpened to the concrete Merkle *inclusion path* by instantiating the generic rose-tree inclusion-proof machinery of `ADS_Functor.Inclusion_Proof_Construction` on the recursive view tree, with the discrete content embedding and hash. An inclusion proof is a zipper in which the target subview is revealed while the path to the root and the off-path siblings are blinded: `reg_view_inclusion_same_hash` shows that every such proof commits to the same authenticating root hash as the fully revealed tree; `reg_view_inclusion_blinding_of` shows it is a blinding of the canonical inclusion-path tree, which carries that same root hash; and `reg_view_inclusion_chains_sound` shows its attested chains are contained in that tree's. The instantiation adds no assumption, since the discrete content already satisfies the blinding-order locale. Like the inclusion-proof construction it mirrors, the result characterises the proofs generated from a source view tree; it operates on the view tree, so the consensus scope limit recorded next is untouched.

These sequence-level results are not vacuous. `Canton_Bridge.thy` instantiates the `authenticated_state` locale directly on the blindable-position functor of `ADS_Functor.ADS_Construction`, in the interpretation `oss_blindable`: an authenticated datum is either a revealed consensus state paired with its revealed-chain set or a hash-only commitment that reveals nothing, and the Merkle-interface obligation is discharged through `merkle_blindable`, so no fresh interface proof is needed. A machine-checked regression witness, `blinding_path_witness`, exhibits a genuinely increasing need-to-know sequence (a hash-only commitment, then a one-chain view, then a two-chain view, each blinding its successor) whose most-revealed endpoint extracts to a non-degenerate two-chain state, pinning the sequence-level theorems to a non-vacuous instance.

Two further interpretations connect the layer to Canton's own transaction structure, both over concrete regulatory content rather than the opaque `view_data` and `view_metadata` types that the public Canton formalisation leaves as `typedecl`: reading a cross-domain state out of those opaque types would require an inadmissible axiom relating them to the regulatory model, which we do not introduce. The interpretation `canton_authenticated` mirrors the top-level shape of a Canton transaction, a single blindable position wrapping a metadata-and-views payload, transporting the three coherence obligations through the constructor bijection. The interpretation `reg_tx_authenticated` keeps the public *recursive* shape: it instantiates the content-agnostic rose-tree Merkle machinery (the `rose_tree` datatype that the public Canton formalisation itself uses to build views) with concrete regulatory content, so that subviews nest and subview-level blinding stays alive, inheriting the Merkle interface with no new datatype and no new interface proof. The honesty boundary is stated in the sources and held here. The construction is *recursive-faithful* but *content-abstracted*, and the consensus `reg_state` is modelled as a bare, non-independently-blindable field of the payload. That modelling choice structurally forces a single consensus value per transaction — a subview is reachable only by revealing the whole transaction, hence its consensus — and this in turn yields validity with no added assumption; the machine-checked `single_consensus_load_bearing` pins the point, exhibiting an inconsistent two-chain state that a per-node consensus would permit and that the recursive extraction never produces. The accompanying price is a declared scope limit: the case in which a view is revealed while the consensus is blinded lies outside this model, because Canton's `common_metadata` is independently blindable whereas the bare field here is not. Lifting that case is left as future work on the public ground, and the single remaining question for the Canton authors is one of model fidelity — whether the concrete payload faithfully abstracts their datatype — which is a sanity check rather than a proof gap.

1.11 The Synchronization-Degree Hierarchy

`Hierarchy.thy` stratifies the functor by *synchronization degree*, the breadth of cross-chain coupling an asset requires. A degree-k system keeps the chains $0 \dots k$ of an asset in lockstep around a hub chain, giving a tower of transition functors $F\,0, F\,1, F\,2, \dots$ in which the weaker functor is obtained from the stronger one by forgetting the top coupled chain (`degree_forget`). This forgetful map is a genuine natural transformation between consecutive degrees: its naturality square — forget after a transition equals a transition after the forget — commutes (`degree_natural_transformation`). Natural transformations of degree functors are closed under composition (`nt_vertical_compose`), so the two-step demotion is again natural (`nt_compose`) and the whole ladder is structurally coherent rather than a point-to-point collection of maps. Degree demotion is information-discarding: `degree_forget_refines` shows it yields a blinding refinement in the exact sense of `state_refines`, tying the hierarchy back to the authenticated glue of Section 1.9.

The monotonicity payload follows. The degree classes are nested (`reduction_containment`). The degree monotonicity is genuinely one-directional: over-provisioning reconciles all of the asset's required chains (`over_provisioning_guarantees`), while under-provisioning carries no such guarantee (`no_downward_safety`), so the degree hypothesis is load-bearing exactly in this over-/under-provisioning regime pair. Validity preservation itself holds independently of the degrees (`processing_preserves_validity`); the auxiliary lemma `hierarchy_monotonicity` records that validity preservation in the over-provisioning regime and follows directly, its provisioning hypothesis notational rather than load-bearing. A causal-consistency boundary, grounded in a strict timestamp order, separates the lower degrees from the higher and is shown to be well-defined on asset/system pairs (`boundary_well_defined`). This tower of functors with natural transformations between them is a layer that, to our knowledge, the `ADS_Functor` entry does not develop.

The degree-class layer is parametric in the degree assignment: the assignment of required degrees to assets is operational data, so the layer is developed in a context fixing an arbitrary `asset_degree` :: `asset_id` ⇒ `nat`, and every theorem above holds for every assignment. A concrete example assignment witnesses non-vacuity (`example_degree_over_provisioning`), and a static-promotion corollary (`static_promotion_safety`) records that the over-provisioning guarantee transfers verbatim to any re-assignment within the system's capability. In-flight promotion — a re-assignment crossing a live synchronization — is out of scope for this entry; subsequent work on retry-queue semantics covers it.

1.12 An Instance Outside the Regulatory Domain

The generic locales are advertised as domain-independent; `External_Instance.thy` substantiates the claim with an instance from a domain disjoint from the regulatory vocabulary: the TCP connection state machine, in a six-state, five-event subset of RFC 793 (single-endpoint view: passive and active open, handshake completion, FIN-initiated teardown, RST abort), synchronized into a stateful connection tracker (conntrack). The theory imports only the generic theory and the generic proof automation — it does not import `Regulatory_Instance.thy` — so the instantiation itself is the evidence that the locale layer carries no hidden regulatory assumptions.

The instance reuses both structural patterns of the regulatory development outside their original domain. The tracker state is a structured record (a phase tag plus peer-endpoint metadata) whose well-formedness invariant ties the auxiliary field to the tag, mirroring the DAML permission record; the tracker's state space is the image of the representation map and its transition function is the lift of the endpoint transition through that map. The tracker observes a strict subset of the endpoint's events (RST is not tracked: aborted connections are expired by the tracker's timeout machinery rather than by following the reset packet), so the morphism's source action set is instantiated with that subset, mirroring the escalation instance. Two modelling notes fix the terminal semantics: CLOSED is terminal because a re-connection is a new connection identity (a new tracker entry), and a RST in LISTEN is ignored, as in RFC 793.

All three instances — the endpoint machine, the tracker machine, and the preservation morphism `tcp_conntrack_preservation` — are discharged by the generic methods of Section "*Proof Automation*" with the theory's own collection declarations, demonstrating that the automation, too, works outside the domain it was developed in. The locale's sequential payload specializes to the expected statement (`tracked_sequence_mirrored`): every tracked packet sequence accepted by the endpoint is mirrored, packet for packet, by the tracker.

1.13 Beyond the Atomic Sync Model

The atomic sync model is the natural locale-based abstraction for proving structural correctness of cross-domain state preservation inside a single AFP entry. Lifting the model to a partially synchronous network with message delays, redelivery, and out-of-order arrival is the next step. The locales in this entry are designed to support that lift: `state_preservation` carries the structural correctness condition that any concrete protocol must preserve, and `fair_leader_system` provides an operational abstraction through which atomic sync can be realized as a sequence of bounded-duration steps. We leave the construction of the lift to subsequent entries.

theory *State-Preservation*
  imports *Main*
begin

2 Generic State Machine Locale

A state machine with finite state set, deterministic partial transition function, and optional terminal states. Terminal states accept no transitions.

locale *state-machine* =

fixes *states* :: *'s set*
and *actions* :: *'a set*
and *transition* :: *'s* ⇒ *'a* ⇒ *'s option*
and *terminal* :: *'s set*
assumes *finite-states*: *finite states*
and *finite-actions*: *finite actions*
and *terminal-subset*: *terminal* ⊆ *states*
and *terminal-absorbing*: ⟦ $s \in terminal$; $a \in actions$ ⟧ ⟹ *transition s a* = *None*
and *transition-closed*: ⟦ $s \in states$; $a \in actions$; *transition s a* = *Some s'* ⟧ ⟹ $s' \in states$
and *transition-domain*: ⟦ $s \notin states$ ⟧ ⟹ *transition s a* = *None*
begin

Determinism is inherent: transition is a function, not a relation.

lemma *transition-deterministic*:
transition s a = *Some s1* ⟹ *transition s a* = *Some s2* ⟹ *s1* = *s2*
by *simp*

Sequential composition of actions (partial, returns None on first failure).

fun *apply-actions* :: *'s* ⇒ *'a list* ⇒ *'s option* **where**
apply-actions s [] = *Some s*
| *apply-actions s* (*a* # *as*) = (*case transition s a of*
None ⇒ *None*
| *Some s'* ⇒ *apply-actions s' as*)

lemma *apply-actions-closed*:
⟦ $s \in states$; ∀ $a \in$ *set as*. $a \in actions$; *apply-actions s as* = *Some s'* ⟧
⟹ $s' \in states$
by (*induction as arbitrary*: *s*) (*auto split*: *option.splits dest*: *transition-closed*)

Note: *apply-actions ?s* [] = *Some ?s* already provides the simp rule *apply-actions s* [] = *Some s*, so no separate [simp] lemma is needed.

lemma *apply-actions-terminal*:
⟦ $s \in terminal$; $a \in actions$ ⟧ ⟹ *apply-actions s* (*a* # *as*) = *None*
by (*simp add*: *terminal-absorbing*)

end

3 Cross-Domain State Preservation Locale

Two state machines connected by a synchronization map. The key property (naturality / homomorphism) states that synchronizing after a local transition yields the same result as transitioning after synchronization.

More precisely: for state machines (`S_src`, `T_src`) and (`S_tgt`, `T_tgt`) connected by sync (mapping source states to target states), the following diagram commutes:

s_src –T_src(a)–> s_src' | | sync sync | | v v s_tgt –T_tgt(a)–> s_tgt'

That is: sync (T_src s a) = T_tgt (sync s) a

This is the structural preservation guarantee: a transition performed at the source domain, when synchronized, produces the same result as performing the corresponding transition at the target domain.

locale *state-preservation* =
  source: *state-machine* $states_s$ $actions_s$ $transition_s$ $terminal_s$ +
  target: *state-machine* $states_t$ $actions_t$ $transition_t$ $terminal_t$
  for $states_s$ :: *'s set* **and** $actions_s$ :: *'a set*
    and $transition_s$ :: *'s* ⇒ *'a* ⇒ *'s option*
    and $terminal_s$ :: *'s set*
    and $states_t$ :: *'t set* **and** $actions_t$ :: *'b set*
    and $transition_t$ :: *'t* ⇒ *'b* ⇒ *'t option*
    and $terminal_t$ :: *'t set* +
  fixes *state-map* :: *'s* ⇒ *'t*
    and *action-map* :: *'a* ⇒ *'b*
  assumes *state-map-well-defined*: $s \in states_s \Longrightarrow$ *state-map* $s \in states_t$
    and *action-map-well-defined*: $a \in actions_s \Longrightarrow$ *action-map* $a \in actions_t$
    and *terminal-preservation*: $s \in terminal_s \Longrightarrow$ *state-map* $s \in terminal_t$
    — The naturality / homomorphism condition:
    and *naturality*:
      ⟦ $s \in states_s$; $a \in actions_s$; $transition_s$ s a = *Some* s' ⟧
       $\Longrightarrow$ $transition_t$ (*state-map* s) (*action-map* a) = *Some* (*state-map* s')
    and *naturality-none*:
      ⟦ $s \in states_s$; $a \in actions_s$; $transition_s$ s a = *None* ⟧
       $\Longrightarrow$ $transition_t$ (*state-map* s) (*action-map* a) = *None*
begin

The fundamental theorem: state preservation extends to sequences of actions. If a sequence of transitions is valid at the source, the mapped sequence is valid at the target with the mapped final state.

theorem *sequential-preservation*:
  assumes $s \in states_s$
    and ∀ a ∈ *set as*. $a \in actions_s$
    and *source.apply-actions* s *as* = *Some* s'
  shows *target.apply-actions* (*state-map* s) (*map action-map as*) = *Some* (*state-map* s')
  using *assms*
proof (*induction as arbitrary*: s)
  case *Nil*
  then show *?case* **by** *simp*
next
  case (*Cons a as*)
  then obtain *s-mid* **where**
    step: $transition_s$ s a = *Some s-mid* **and**
    rest: *source.apply-actions s-mid as* = *Some* s'
    by (*auto split*: *option.splits*)

```
  from Cons.prems(1) Cons.prems(2) step have s-mid ∈ states_s
    using source.transition-closed by auto
  from Cons.prems(1) Cons.prems(2) step
  have mapped-step: transition_t (state-map s) (action-map a) = Some (state-map
s-mid)
    using naturality by auto
  from ‹s-mid ∈ states_s› Cons.prems(2) rest
  have target.apply-actions (state-map s-mid) (map action-map as) = Some (state-map
s')
    using Cons.IH by auto
  with mapped-step show ?case by simp
qed

theorem sequential-preservation-none:
  assumes s ∈ states_s
    and ∀ a ∈ set as. a ∈ actions_s
    and source.apply-actions s as = None
  shows target.apply-actions (state-map s) (map action-map as) = None
  using assms
proof (induction as arbitrary: s)
  case Nil
  then show ?case by simp
next
  case (Cons a as)
  show ?case
  proof (cases transition_s s a)
    case None
    then have transition_t (state-map s) (action-map a) = None
      using naturality-none Cons.prems(1) Cons.prems(2) by auto
    then show ?thesis by simp
  next
    case (Some s-mid)
    then have mid-in: s-mid ∈ states_s
      using source.transition-closed Cons.prems(1) Cons.prems(2) by auto
    from Some Cons.prems(3) have source.apply-actions s-mid as = None
      by simp
    then have ih: target.apply-actions (state-map s-mid) (map action-map as) =
None
      using Cons.IH mid-in Cons.prems(2) by auto
    from Some have transition_t (state-map s) (action-map a) = Some (state-map
s-mid)
      using naturality Cons.prems(1) Cons.prems(2) by auto
    with ih show ?thesis by simp
  qed
qed
```

Terminal states are preserved: if a source state is terminal, its image is terminal.

```
corollary terminal-image-absorbing:
```

assumes $s \in terminal_s$ **and** $a \in actions_s$
shows $transition_t$ $(state\text{-}map\ s)$ $(action\text{-}map\ a) = None$
proof $-$
from $assms(1)$ **have** $s \in states_s$ **using** *source.terminal-subset* **by** *auto*
from *assms* **have** $transition_s\ s\ a = None$
using *source.terminal-absorbing* **by** *auto*
then show *?thesis*
using *naturality-none* $\langle s \in states_s \rangle$ $assms(2)$ **by** *auto*
qed

end

4 Symmetric State Preservation

For bidirectional synchronization, both directions must preserve structure. This locale composes two `state_preservation` instances with inverse maps, establishing that synchronization in either direction is structure-preserving.

locale *symmetric-state-preservation* =
forward: *state-preservation*
$states_s$ $actions_s$ $transition_s$ $terminal_s$
$states_t$ $actions_t$ $transition_t$ $terminal_t$
state-map action-map +
backward: *state-preservation*
$states_t$ $actions_t$ $transition_t$ $terminal_t$
$states_s$ $actions_s$ $transition_s$ $terminal_s$
state-map-inv action-map-inv
for $states_s$ :: $'s\ set$ **and** $actions_s$ :: $'a\ set$
and $transition_s$ :: $'s \Rightarrow 'a \Rightarrow 's\ option$
and $terminal_s$ :: $'s\ set$
and $states_t$ :: $'t\ set$ **and** $actions_t$ :: $'b\ set$
and $transition_t$ :: $'t \Rightarrow 'b \Rightarrow 't\ option$
and $terminal_t$ :: $'t\ set$
and *state-map* :: $'s \Rightarrow 't$ **and** *action-map* :: $'a \Rightarrow 'b$
and *state-map-inv* :: $'t \Rightarrow 's$ **and** *action-map-inv* :: $'b \Rightarrow 'a$ +
assumes *roundtrip-state-src*: $s \in states_s \Longrightarrow state\text{-}map\text{-}inv\ (state\text{-}map\ s) = s$
and *roundtrip-state-tgt*: $t \in states_t \Longrightarrow state\text{-}map\ (state\text{-}map\text{-}inv\ t) = t$
and *roundtrip-action-src*: $a \in actions_s \Longrightarrow action\text{-}map\text{-}inv\ (action\text{-}map\ a) = a$
and *roundtrip-action-tgt*: $b \in actions_t \Longrightarrow action\text{-}map\ (action\text{-}map\text{-}inv\ b) = b$
begin

Under symmetric preservation with roundtrip maps, `state_map` is injective on the state set distinct source states map to distinct target states. This guarantees no information loss during synchronization.

lemma *state-map-injective*:
$[\![\ s1 \in states_s;\ s2 \in states_s;\ state\text{-}map\ s1 = state\text{-}map\ s2\ ]\!] \Longrightarrow s1 = s2$
using *roundtrip-state-src* **by** *metis*

end

5 Multi-Domain State Preservation

Generalization to N domains sharing the same abstract state machine. This models scenarios where a single asset exists on multiple domains (e.g., blockchain networks), all of which must reflect the same state.

The key property: if one domain transitions, the synchronization brings ALL other domains to the same resulting state.

We model this as: all domains are instances of the same abstract state machine, connected by identity-like state/action maps. The "same abstract state machine" assumption holds when the protocol enforces a uniform state model across all domains.

locale *multi-domain-preservation* =
  fixes *domains* :: $'d$ *set*
    and *states* :: $'s$ *set*
    and *actions* :: $'a$ *set*
    and *transition* :: $'s \Rightarrow 'a \Rightarrow 's$ *option*
    and *terminal* :: $'s$ *set*
    and *domain-state* :: $'d \Rightarrow 'id \Rightarrow 's$ *option*
      — Current state of asset 'id in domain 'd
  assumes *fin-domains*: *finite domains*
    and *sm*: *state-machine states actions transition terminal*
    and *consistent-init*:
      — Precondition: all domains holding an asset agree on its state
      $[\![$ *d1* $\in$ *domains*; *d2* $\in$ *domains*;
        domain-state d1 aid = *Some s1*; *domain-state d2 aid* = *Some s2* $]\!]$
       $\Longrightarrow$ *s1* = *s2*
begin

Which domains hold a given asset.

definition *connected-domains* :: $'id \Rightarrow 'd$ *set* **where**
  connected-domains aid = $\{$*d* $\in$ *domains*. *domain-state d aid* $\neq$ *None*$\}$

The global consensus state of an asset (well-defined by `consistent_init`).

definition *consensus-state* :: $'id \Rightarrow 's$ *option* **where**
  consensus-state aid =
    (*if connected-domains aid* = $\{\}$ *then None*
     else domain-state (*SOME d*. *d* $\in$ *connected-domains aid*) *aid*)

After synchronizing an action on an asset, all connected domains reflect the new state. This is the cross-domain consistency theorem.

definition *sync-all* ::
  $'d \Rightarrow 'a \Rightarrow 'id \Rightarrow ('d \Rightarrow 'id \Rightarrow 's\ option) \Rightarrow ('d \Rightarrow 'id \Rightarrow 's\ option)\ option$
where
  sync-all source action aid ds =
    (*case ds source aid of*
       None $\Rightarrow$ *None*
     | *Some s* $\Rightarrow$
         (*case transition s action of*

```
        None ⇒ None
      | Some s′ ⇒
          Some (λd aid′.
            if aid′ = aid ∧ d ∈ connected-domains aid
            then Some s′
            else ds d aid′)))
```

The cross-domain consistency theorem: after `sync_all`, every connected domain agrees on the new state.

```
theorem cross-domain-consistency:
  assumes source ∈ domains
    and domain-state source aid = Some s
    and s ∈ states
    and action ∈ actions
    and transition s action = Some s′
    and sync-all source action aid domain-state = Some ds′
    and d ∈ connected-domains aid
  shows ds′ d aid = Some s′
proof −
  from assms(2,5,6) have
    ds′ = (λd aid′.
      if aid′ = aid ∧ d ∈ connected-domains aid
      then Some s′
      else domain-state d aid′)
    unfolding sync-all-def by auto
  with assms(7) show ?thesis by simp
qed
```

`sync_all` does not affect other assets.

```
theorem sync-isolation:
  assumes sync-all source action aid domain-state = Some ds′
    and aid′ ≠ aid
  shows ds′ d aid′ = domain-state d aid′
proof −
  from assms(1) obtain s s′ where
    domain-state source aid = Some s
    transition s action = Some s′
    unfolding sync-all-def by (auto split: option.splits)
  then have ds′ = (λd aid′.
    if aid′ = aid ∧ d ∈ connected-domains aid
    then Some s′
    else domain-state d aid′)
    using assms(1) unfolding sync-all-def by auto
  with assms(2) show ?thesis by auto
qed

end

end
```

theory *Regulatory-Instance*
imports *State-Preservation*
begin

6 Regulatory State and Action Definitions

datatype *reg-state* = *ACTIVE* | *FROZEN* | *SEIZED* | *CONFISCATED* | *RESTRICTED*

datatype *reg-action* = *FREEZE* | *SEIZE* | *CONFISCATE* | *RESTRICT*
| *UNFREEZE* | *UNRESTRICT* | *RELEASE*

The regulatory transition function. Partial: returns None for invalid (state, action) combinations. The transition table encodes legal precedence: seizure is stronger than freezing, confiscation is terminal and universally reachable, and intermediate states must return to Active before lateral transitions.

fun *reg-transition* :: *reg-state* ⇒ *reg-action* ⇒ *reg-state option* **where**
— From ACTIVE: all forward actions valid
reg-transition ACTIVE FREEZE = *Some FROZEN*
| *reg-transition ACTIVE SEIZE* = *Some SEIZED*
| *reg-transition ACTIVE CONFISCATE* = *Some CONFISCATED*
| *reg-transition ACTIVE RESTRICT* = *Some RESTRICTED*
| *reg-transition ACTIVE UNFREEZE* = *None*
| *reg-transition ACTIVE UNRESTRICT* = *None*
| *reg-transition ACTIVE RELEASE* = *None*
— From FROZEN: unfreeze, escalate to SEIZED or CONFISCATED
| *reg-transition FROZEN UNFREEZE* = *Some ACTIVE*
| *reg-transition FROZEN SEIZE* = *Some SEIZED*
| *reg-transition FROZEN CONFISCATE* = *Some CONFISCATED*
| *reg-transition FROZEN FREEZE* = *None*
| *reg-transition FROZEN RESTRICT* = *None*
| *reg-transition FROZEN UNRESTRICT* = *None*
| *reg-transition FROZEN RELEASE* = *None*
— From SEIZED: release or final confiscation only
| *reg-transition SEIZED RELEASE* = *Some ACTIVE*
| *reg-transition SEIZED CONFISCATE* = *Some CONFISCATED*
| *reg-transition SEIZED FREEZE* = *None*
| *reg-transition SEIZED SEIZE* = *None*
| *reg-transition SEIZED RESTRICT* = *None*
| *reg-transition SEIZED UNFREEZE* = *None*
| *reg-transition SEIZED UNRESTRICT* = *None*
— From CONFISCATED: terminal no transitions out
| *reg-transition CONFISCATED FREEZE* = *None*
| *reg-transition CONFISCATED SEIZE* = *None*
| *reg-transition CONFISCATED CONFISCATE* = *None*

| *reg-transition CONFISCATED RESTRICT* = *None*
| *reg-transition CONFISCATED UNFREEZE* = *None*
| *reg-transition CONFISCATED UNRESTRICT* = *None*
| *reg-transition CONFISCATED RELEASE* = *None*
— From RESTRICTED: unrestrict, escalate to FROZEN or CONFISCATED
| *reg-transition RESTRICTED UNRESTRICT* = *Some ACTIVE*
| *reg-transition RESTRICTED FREEZE* = *Some FROZEN*
| *reg-transition RESTRICTED CONFISCATE* = *Some CONFISCATED*
| *reg-transition RESTRICTED RESTRICT* = *None*
| *reg-transition RESTRICTED SEIZE* = *None*
| *reg-transition RESTRICTED UNFREEZE* = *None*
| *reg-transition RESTRICTED RELEASE* = *None*

7 Fundamental Properties of the Transition Function

7.1 Invariant I1: CONFISCATED is terminal

lemma *confiscated-terminal*:
reg-transition CONFISCATED a = *None*
by (*cases a*) *auto*

7.2 Invariant I2: CONFISCATE reaches CONFISCATED from any non-terminal state

lemma *confiscate-universal*:
$s \neq$ *CONFISCATED* $\Longrightarrow$ *reg-transition s CONFISCATE* = *Some CONFISCATED*
by (*cases s*) *auto*

7.3 Invariant I3: Determinism (inherent from function definition)

Determinism follows directly from the **fun** definition.

7.4 SEIZED to FROZEN exclusion

SEIZED is a strictly stronger constraint than FROZEN. Direct transition from SEIZED to FROZEN is legally nonsensical (cannot "weaken" a court-ordered seizure to a mere freeze). The path is: RELEASE then ACTIVE then FREEZE.

lemma *seized-no-freeze*:
reg-transition SEIZED FREEZE = *None*
by *simp*

lemma *seized-no-unfreeze*:
reg-transition SEIZED UNFREEZE = *None*

by *simp*

lemma *seized-no-restrict*:
reg-transition SEIZED RESTRICT = None
by *simp*

7.5 FROZEN to RESTRICTED exclusion

Must go through ACTIVE: UNFREEZE then ACTIVE then RESTRICT.

lemma *frozen-no-restrict*:
reg-transition FROZEN RESTRICT = None
by *simp*

7.6 Transition completeness: every non-terminal state has at least one valid action

lemma *non-terminal-has-action*:
assumes $s \neq$ *CONFISCATED*
shows $\exists\, a.$ *reg-transition s a* $\neq$ *None*
proof (*cases s*)
case *ACTIVE*
show *?thesis*
apply (*rule exI*[**where** *x=FREEZE*])
using *ACTIVE* **by** *simp*
next
case *FROZEN*
show *?thesis*
apply (*rule exI*[**where** *x=UNFREEZE*])
using *FROZEN* **by** *simp*
next
case *SEIZED*
show *?thesis*
apply (*rule exI*[**where** *x=RELEASE*])
using *SEIZED* **by** *simp*
next
case *CONFISCATED*
then show *?thesis* **using** *assms* **by** *contradiction*
next
case *RESTRICTED*
show *?thesis*
apply (*rule exI*[**where** *x=UNRESTRICT*])
using *RESTRICTED* **by** *simp*
qed

7.7 Reachability from ACTIVE

Every state is reachable from ACTIVE via some sequence of actions.

lemma *active-reaches-frozen*: *reg-transition ACTIVE FREEZE = Some FROZEN*

by *simp*

lemma *active-reaches-seized*: *reg-transition ACTIVE SEIZE = Some SEIZED*
by *simp*

lemma *active-reaches-confiscated*: *reg-transition ACTIVE CONFISCATE = Some CONFISCATED*
by *simp*

lemma *active-reaches-restricted*: *reg-transition ACTIVE RESTRICT = Some RESTRICTED*
by *simp*

7.8 Return paths to ACTIVE

lemma *frozen-returns*: *reg-transition FROZEN UNFREEZE = Some ACTIVE*
by *simp*

lemma *seized-returns*: *reg-transition SEIZED RELEASE = Some ACTIVE*
by *simp*

lemma *restricted-returns*: *reg-transition RESTRICTED UNRESTRICT = Some ACTIVE*
by *simp*

8 State Machine Instance

We show that `reg_state` / `reg_action` / `reg_transition` satisfy the `state_machine` locale assumptions.

definition *reg-states* :: *reg-state set* **where**
reg-states = {*ACTIVE*, *FROZEN*, *SEIZED*, *CONFISCATED*, *RESTRICTED*}

definition *reg-actions* :: *reg-action set* **where**
reg-actions = {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*, *UNFREEZE*, *UNRESTRICT*, *RELEASE*}

definition *reg-terminal* :: *reg-state set* **where**
reg-terminal = {*CONFISCATED*}

lemma *reg-states-UNIV*: *reg-states = UNIV*
unfolding *reg-states-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *reg-actions-UNIV*: *reg-actions = UNIV*
unfolding *reg-actions-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *reg-transition-closed*:
$\llbracket$ *reg-transition s a = Some s'* $\rrbracket \Longrightarrow s' \in$ *reg-states*
unfolding *reg-states-def* **by** (*cases s*; *cases a*; *auto*)

```
interpretation reg-sm: state-machine reg-states reg-actions reg-transition reg-terminal
proof unfold-locales
  show finite reg-states unfolding reg-states-def by auto
next
  show finite reg-actions unfolding reg-actions-def by auto
next
 show reg-terminal ⊆ reg-states unfolding reg-terminal-def reg-states-def by auto
next
  fix s a
  assume s ∈ reg-terminal a ∈ reg-actions
  then show reg-transition s a = None
    unfolding reg-terminal-def by (auto simp: confiscated-terminal)
next
  fix s a s'
  assume s ∈ reg-states a ∈ reg-actions reg-transition s a = Some s'
  then show s' ∈ reg-states
    using reg-transition-closed by auto
next
  fix s :: reg-state and a :: reg-action
  assume s ∉ reg-states
  then show reg-transition s a = None
    using reg-states-UNIV by auto
qed
```

9 Asset and Global State Modeling

```
type-synonym asset-id = nat
type-synonym chain-id = nat
type-synonym authority-id = nat
type-synonym timestamp = nat

record asset-state =
  as-asset-id  :: asset-id
  as-reg-state :: reg-state

type-synonym chain-state = asset-id ⇒ asset-state option

record global-state =
  gs-chains    :: chain-id ⇒ chain-state
  gs-locks     :: asset-id ⇒ bool

record oss-message =
  msg-action    :: reg-action
  msg-asset-id  :: asset-id
  msg-authority :: authority-id
  msg-timestamp :: timestamp
  msg-source    :: chain-id
  msg-targets   :: chain-id set
```

10 State Accessors and Predicates

definition *get-asset-state* :: *global-state* ⇒ *chain-id* ⇒ *asset-id* ⇒ *asset-state option* **where**
get-asset-state gs cid aid = *gs-chains gs cid aid*

definition *get-reg-state* :: *global-state* ⇒ *chain-id* ⇒ *asset-id* ⇒ *reg-state option* **where**
get-reg-state gs cid aid =
(*case get-asset-state gs cid aid of*
None ⇒ *None*
| *Some ast* ⇒ *Some* (*as-reg-state ast*))

definition *asset-exists* :: *global-state* ⇒ *chain-id* ⇒ *asset-id* ⇒ *bool* **where**
asset-exists gs cid aid = (*get-asset-state gs cid aid* ≠ *None*)

definition *is-locked* :: *global-state* ⇒ *asset-id* ⇒ *bool* **where**
is-locked gs aid = *gs-locks gs aid*

Connected chains: all chains holding a given asset.

definition *connected-chains* :: *global-state* ⇒ *asset-id* ⇒ *chain-id set* **where**
connected-chains gs aid = {*cid. asset-exists gs cid aid*}

11 Global State Validity

A global state is valid if all chains holding the same asset agree on its regulatory state. This is the cross-chain consistency invariant that synchronization must preserve.

definition *consistent-state* :: *global-state* ⇒ *bool* **where**
consistent-state gs ≡
∀ *c1 c2 aid s1 s2*.
get-reg-state gs c1 aid = *Some s1* ⟶
get-reg-state gs c2 aid = *Some s2* ⟶
s1 = *s2*

definition *no-locked-without-reason* :: *global-state* ⇒ *bool* **where**
no-locked-without-reason gs ≡
∀ *aid*. ¬ *is-locked gs aid*
— In a quiescent valid state, no assets are locked. Locks are transient, held only during sync operations.

definition *valid-state* :: *global-state* ⇒ *bool* **where**
valid-state gs ≡ *consistent-state gs* ∧ *no-locked-without-reason gs*

12 Synchronization Protocol

12.1 Lock acquisition

definition *acquire-lock* :: *global-state* ⇒ *asset-id* ⇒ *global-state option* **where**
  acquire-lock gs aid =
    (*if is-locked gs aid then None*
     else Some (*gs*⦇ *gs-locks* := (*gs-locks gs*)(*aid* := *True*) ⦈))

definition *release-lock* :: *global-state* ⇒ *asset-id* ⇒ *global-state* **where**
  release-lock gs aid = *gs*⦇ *gs-locks* := (*gs-locks gs*)(*aid* := *False*) ⦈

12.2 State update across chains

Update the regulatory state of an asset on a specific chain. Preserves the other asset fields (the asset identifier).

definition *update-chain-reg-state* ::
  global-state ⇒ *chain-id* ⇒ *asset-id* ⇒ *reg-state* ⇒ *global-state*
where
  update-chain-reg-state gs cid aid new-st =
    (*let old-chain* = *gs-chains gs cid in*
     let new-chain = (λ*aid*′.
       if aid′ = *aid then*
         (*case old-chain aid of*
            None ⇒ *None*
          | *Some ast* ⇒ *Some* (*ast*⦇ *as-reg-state* := *new-st* ⦈))
       else old-chain aid′)
     in gs⦇ *gs-chains* := (*gs-chains gs*)(*cid* := *new-chain*) ⦈)

Update all connected chains to a new regulatory state.

We define this directly rather than using `Finite_Set`.fold, because the update for a given asset on each chain is independent we simply construct a new global state where every chain's view of the asset is updated. This avoids the need for `comp_fun_commute` proofs.

definition *update-all-chains* ::
  global-state ⇒ *asset-id* ⇒ *reg-state* ⇒ *chain-id set* ⇒ *global-state*
where
  update-all-chains gs aid new-st targets =
    gs⦇ *gs-chains* := (λ*cid*.
      if cid ∈ *targets then*
        (λ*aid*′. *if aid*′ = *aid then*
           (*case gs-chains gs cid aid of*
              None ⇒ *None*
            | *Some ast* ⇒ *Some* (*ast*⦇ *as-reg-state* := *new-st* ⦈))
         else gs-chains gs cid aid′)
      else gs-chains gs cid) ⦈

12.3 The synchronization function

sync: the core synchronization operation.

Given a source chain, action, and `asset_id`: 1. Verify the asset exists on the source chain 2. Check the transition is valid 3. Acquire global lock (fails if already locked) 4. Update all connected chains to the new state 5. Release lock

Returns None if any step fails (invalid transition, lock contention, etc.)

definition *sync* ::
chain-id ⇒ *reg-action* ⇒ *asset-id* ⇒ *global-state* ⇒ *global-state option*
where
sync source action aid gs =
(*case get-reg-state gs source aid of*
None ⇒ *None*
| *Some current-st* ⇒
(*case reg-transition current-st action of*
None ⇒ *None*
| *Some new-st* ⇒
(*case acquire-lock gs aid of*
None ⇒ *None*
| *Some gs-locked* ⇒
let targets = *connected-chains gs aid in*
let gs-updated = *update-all-chains gs-locked aid new-st targets in*
Some (*release-lock gs-updated aid*))))

13 Synchronization Properties

13.1 Idempotency

After synchronization, applying the same action again yields None (the state has already transitioned, so the same transition from the new state is typically invalid) or is a no-op.

More precisely: if sync succeeds and produces `new_st`, then `reg_transition new_st` action = None for most action/state pairs (the transition table has no self-loops).

lemma *no-self-loops*:
reg-transition s a = *Some s* $\Longrightarrow$ *False*
by (*cases s*; *cases a*; *auto*)

lemma *transition-changes-state*:
reg-transition s a = *Some s'* $\Longrightarrow$ $s \neq s'$
using *no-self-loops* **by** *auto*

13.2 Sync isolation: other assets are not affected

Synchronization on asset aid does not change the state of any other asset (where the asset identifier differs) on any chain.

lemma *update-chain-other-asset*:
  assumes $aid' \neq aid$
  shows *gs-chains* (*update-chain-reg-state gs cid aid new-st*) cid' aid' =
    gs-chains gs cid' aid'
proof (*cases* $cid' = cid$)
  case *True*
  then show *?thesis*
    unfolding *update-chain-reg-state-def Let-def*
    using *assms* **by** *auto*
next
  case *False*
  then show *?thesis*
    unfolding *update-chain-reg-state-def Let-def* **by** *auto*
qed

13.3 Sync correctness: source chain updated

After `update_chain_reg_state`, the target chain reflects the new state.

lemma *update-chain-same-asset*:
  assumes *gs-chains gs cid aid* = *Some ast*
  shows *gs-chains* (*update-chain-reg-state gs cid aid new-st*) *cid aid* =
    Some (*ast*(| *as-reg-state* := *new-st* |))
  unfolding *update-chain-reg-state-def Let-def*
  using *assms* **by** *auto*

13.4 Properties of update_all_chains

`update_all_chains` correctly updates any chain in the target set.

lemma *update-all-chains-in-targets*:
  assumes $cid \in targets$
    and *gs-chains gs cid aid* = *Some ast*
  shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid aid* =
    Some (*ast*(| *as-reg-state* := *new-st* |))
  unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-in-targets-none*:
  assumes $cid \in targets$
    and *gs-chains gs cid aid* = *None*
  shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid aid* = *None*
  unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-outside-targets*:
  assumes $cid \notin targets$
  shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid* = *gs-chains gs cid*
  unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-other-asset*:
  assumes $aid' \neq aid$
  shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid* aid' =

gs-chains gs cid aid′
unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-locks*:
gs-locks (*update-all-chains gs aid new-st targets*) = *gs-locks gs*
unfolding *update-all-chains-def* **by** *auto*

Key lemma: `get_reg_state` after `update_all_chains` for a chain in targets.

lemma *update-all-chains-reg-state*:
assumes *cid* ∈ *targets*
and *get-asset-state gs cid aid* = *Some ast*
shows *get-reg-state* (*update-all-chains gs aid new-st targets*) *cid aid* = *Some new-st*
proof −
from *assms*(*2*) **have** *chain*: *gs-chains gs cid aid* = *Some ast*
unfolding *get-asset-state-def* **by** *simp*
then have *gs-chains* (*update-all-chains gs aid new-st targets*) *cid aid* =
Some (*ast*(| *as-reg-state* := *new-st* |))
using *update-all-chains-in-targets*[*OF assms*(*1*)] **by** *simp*
then show *?thesis*
unfolding *get-reg-state-def get-asset-state-def* **by** *simp*
qed

13.5 Lock mutual exclusion

If a lock is held, a further lock acquisition on the same asset fails (`acquire_lock` returns `None`). In the atomic model this guard expresses the intended mutual exclusion of regulatory actions on an asset; there is no concurrent execution in the model to serialise.

lemma *lock-acquire-success*:
assumes ¬ *is-locked gs aid*
shows ∃ *gs′*. *acquire-lock gs aid* = *Some gs′* ∧ *is-locked gs′ aid*
proof −
from *assms* **have** ¬ *gs-locks gs aid*
unfolding *is-locked-def* **by** *simp*
then have *acquire-lock gs aid* = *Some* (*gs*(| *gs-locks* := (*gs-locks gs*)(*aid* := *True*) |))
unfolding *acquire-lock-def is-locked-def* **by** *simp*
moreover have *is-locked* (*gs*(| *gs-locks* := (*gs-locks gs*)(*aid* := *True*) |)) *aid*
unfolding *is-locked-def* **by** *simp*
ultimately show *?thesis* **by** *auto*
qed

14 Main Theorem: Cross-Chain Regulatory Consistency

The central theorem of this theory. Under a valid global state (all chains consistent, no outstanding locks), if a regulatory action is applied via sync, the resulting state is also valid (all chains consistent) and every connected chain reflects the new regulatory state.

This is the regulatory instantiation of the cross-domain consistency theorem from `State_Preservation`.thy.

The proof proceeds by: 1. Unfolding sync into lock, then `update_all_chains`, then unlock 2. Showing `acquire_lock` and `release_lock` preserve chain state 3. Showing `update_all_chains` updates every chain in the target set 4. Combining these facts to establish the conclusion

Auxiliary: update on a single chain preserves consistency for other assets.

lemma *update-chain-preserves-other-consistency*:
assumes *consistent-state gs*
and $aid' \neq aid$
and *get-reg-state gs c1 aid'* = *Some s1*
and *get-reg-state gs c2 aid'* = *Some s2*
shows *get-reg-state* (*update-chain-reg-state gs cid aid new-st*) *c1 aid'* = *Some s1*
$\wedge$ *get-reg-state* (*update-chain-reg-state gs cid aid new-st*) *c2 aid'* = *Some s2*
using *assms* **unfolding** *get-reg-state-def get-asset-state-def*
update-chain-reg-state-def Let-def consistent-state-def
by *auto*

Auxiliary: `release_lock` does not change chains.

lemma *release-lock-chains*:
gs-chains (*release-lock gs aid*) = *gs-chains gs*
unfolding *release-lock-def* **by** *simp*

lemma *release-lock-reg-state*:
get-reg-state (*release-lock gs aid*) *cid aid'* = *get-reg-state gs cid aid'*
unfolding *get-reg-state-def get-asset-state-def release-lock-chains* **by** *simp*

Auxiliary: `acquire_lock` does not change chains.

lemma *acquire-lock-chains*:
assumes *acquire-lock gs aid* = *Some gs'*
shows *gs-chains gs'* = *gs-chains gs*
using *assms* **unfolding** *acquire-lock-def is-locked-def*
by (*auto split*: *if-splits*)

Auxiliary: `connected_chains` uses original gs, not locked gs.

The cross-chain regulatory homomorphism theorem.

theorem *regulatory-homomorphism*:
assumes *valid*: *valid-state gs*
and *exists*: *asset-exists gs source aid*

and *current*: *get-reg-state gs source aid* = *Some s*
and *trans*: *reg-transition s action* = *Some s′*
and *synced*: *sync source action aid gs* = *Some gs′*
and *connected*: *c* ∈ *connected-chains gs aid*
and *fin*: *finite* (*connected-chains gs aid*)
shows *get-reg-state gs′ c aid* = *Some s′*
proof −
— Step 1: Unfold sync to extract intermediate states
from *synced current trans*
obtain *gs-locked* **where**
lock: *acquire-lock gs aid* = *Some gs-locked*
and *gs′-def*: *gs′* = *release-lock*
(*update-all-chains gs-locked aid s′* (*connected-chains gs aid*)) *aid*
unfolding *sync-def*
by (*auto split*: *option.splits simp*: *Let-def*)

— Step 2: `acquire_lock` preserves chains
have *locked-chains*: *gs-chains gs-locked* = *gs-chains gs*
using *acquire-lock-chains*[*OF lock*] **by** *simp*

— Step 3: c is in `connected_chains`, so the asset exists on c
from *connected* **have** *asset-exists gs c aid*
unfolding *connected-chains-def* **by** *simp*
then obtain *ast-c* **where** *ast-c*: *gs-chains gs c aid* = *Some ast-c*
unfolding *asset-exists-def get-asset-state-def* **by** *auto*

— Step 4: Therefore asset also exists in `gs_locked` on chain c
have *ast-locked*: *gs-chains gs-locked c aid* = *Some ast-c*
using *ast-c locked-chains* **by** *simp*

— Step 5: `update_all_chains` updates chain c
have *updated*: *gs-chains* (*update-all-chains gs-locked aid s′* (*connected-chains gs aid*)) *c aid* =
Some (*ast-c*⦇ *as-reg-state* := *s′* ⦈)
using *update-all-chains-in-targets*[*OF connected ast-locked*] **by** *simp*

— Step 6: `release_lock` does not change chains
have *gs-chains gs′ c aid* = *Some* (*ast-c*⦇ *as-reg-state* := *s′* ⦈)
using *updated* **unfolding** *gs′-def release-lock-chains* **by** *simp*

— Step 7: Extract `reg_state`
then show *?thesis*
unfolding *get-reg-state-def get-asset-state-def* **by** *simp*
qed

15 Valid State Preservation

The central safety property: synchronization preserves global state validity.

If the global state is valid (consistent and unlocked) before sync, and sync succeeds, then the resulting global state is also valid.

This closes the inductive invariant: valid initial state + `valid_state` preservation under sync = `valid_state` holds at all reachable states.

The proof decomposes into two parts: (1) `consistent_state` preservation: sync updates all connected chains to the same new state and leaves other assets untouched. (2) `no_locked_without_reason` preservation: sync acquires and then releases the lock within the same operation, so the output has no outstanding locks (assuming no other locks were held).

15.1 Part 1: consistent_state preservation

After sync, any two chains holding asset aid agree on its new regulatory state (it is s'), and any two chains holding a different asset aid' still agree (their states are unchanged from the consistent input).

lemma *sync-preserves-consistent-state*:
  assumes *valid*: *valid-state gs*
    and *exists*: *asset-exists gs source aid*
    and *current*: *get-reg-state gs source aid* = *Some s*
    and *trans*: *reg-transition s action* = *Some s′*
    and *synced*: *sync source action aid gs* = *Some gs′*
    and *fin*: *finite* (*connected-chains gs aid*)
  shows *consistent-state gs′*
proof −
  — Unfold sync to extract the structure
  from *synced current trans*
  obtain *gs-locked* **where**
    lock: *acquire-lock gs aid* = *Some gs-locked*
    and *gs′-def*: *gs′* = *release-lock*
      (*update-all-chains gs-locked aid s′* (*connected-chains gs aid*)) *aid*
    unfolding *sync-def*
    by (*auto split*: *option.splits simp*: *Let-def*)

  have *locked-chains*: *gs-chains gs-locked* = *gs-chains gs*
    using *acquire-lock-chains*[*OF lock*] **by** *simp*

  show *?thesis*
  unfolding *consistent-state-def*
  proof (*intro allI impI*)
    fix *c1 c2 aid′ s1-final s2-final*
    assume *s1-eq*: *get-reg-state gs′ c1 aid′* = *Some s1-final*
      and *s2-eq*: *get-reg-state gs′ c2 aid′* = *Some s2-final*

    show *s1-final* = *s2-final*
    proof (*cases aid′* = *aid*)
      case *True*
      — Case: the synchronized asset. Two sub-cases per chain.

```
    show ?thesis
    proof (cases c1 ∈ connected-chains gs aid)
      case c1-in: True
      then have ae1: asset-exists gs c1 aid
        unfolding connected-chains-def by simp
      then obtain ast1 where ast1: gs-chains gs c1 aid = Some ast1
        unfolding asset-exists-def get-asset-state-def by auto
      have ast1-locked: gs-chains gs-locked c1 aid = Some ast1
        using ast1 locked-chains by simp
       have upd1: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c1 aid =
                Some (ast1(| as-reg-state := s' |))
        using update-all-chains-in-targets[OF c1-in ast1-locked] by simp
      have reg1: get-reg-state gs' c1 aid = Some s'
        unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
        using upd1 by simp
      show ?thesis
      proof (cases c2 ∈ connected-chains gs aid)
        case c2-in: True
        then have ae2: asset-exists gs c2 aid
          unfolding connected-chains-def by simp
        then obtain ast2 where ast2: gs-chains gs c2 aid = Some ast2
          unfolding asset-exists-def get-asset-state-def by auto
        have ast2-locked: gs-chains gs-locked c2 aid = Some ast2
          using ast2 locked-chains by simp
        have reg2: get-reg-state gs' c2 aid = Some s'
          unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
          using update-all-chains-in-targets[OF c2-in ast2-locked] by simp
        from reg1 reg2 s1-eq s2-eq True show ?thesis by simp
      next
        case c2-out: False
        — c2 is not connected, so asset doesn't exist on c2 for aid
          have gs-chains (update-all-chains gs-locked aid s' (connected-chains gs
aid)) c2 =
              gs-chains gs-locked c2
          using update-all-chains-outside-targets[OF c2-out] by simp
        then have gs-chains gs' c2 aid = gs-chains gs c2 aid
          unfolding gs'-def release-lock-chains using locked-chains by simp
        moreover have gs-chains gs c2 aid = None
       using c2-out unfolding connected-chains-def asset-exists-def get-asset-state-def
          by auto
        ultimately have get-reg-state gs' c2 aid = None
          unfolding get-reg-state-def get-asset-state-def by simp
        with s2-eq True show ?thesis by simp
      qed
    next
      case c1-out: False
      — c1 not connected: asset doesn't exist there
     have gs-chains (update-all-chains gs-locked aid s' (connected-chains gs aid))
```

c1 =

gs-chains gs-locked c1

using *update-all-chains-outside-targets*[*OF c1-out*] **by** *simp*

then have *gs-chains gs′ c1 aid* = *gs-chains gs c1 aid*

unfolding *gs′-def release-lock-chains* **using** *locked-chains* **by** *simp*

moreover have *gs-chains gs c1 aid* = *None*

using *c1-out* **unfolding** *connected-chains-def asset-exists-def get-asset-state-def*

by *auto*

ultimately have *get-reg-state gs′ c1 aid* = *None*

unfolding *get-reg-state-def get-asset-state-def* **by** *simp*

with *s1-eq True* **show** *?thesis* **by** *simp*

qed

next

case *diff*: *False*

— Case: a different asset. Sync does not touch it.

have *other1*: *gs-chains* (*update-all-chains gs-locked aid s′* (*connected-chains gs aid*)) *c1 aid′* =

gs-chains gs-locked c1 aid′

using *update-all-chains-other-asset*[*OF diff*] **by** *simp*

have *other2*: *gs-chains* (*update-all-chains gs-locked aid s′* (*connected-chains gs aid*)) *c2 aid′* =

gs-chains gs-locked c2 aid′

using *update-all-chains-other-asset*[*OF diff*] **by** *simp*

have *gs-chains gs′ c1 aid′* = *gs-chains gs c1 aid′*

unfolding *gs′-def release-lock-chains* **using** *other1 locked-chains* **by** *simp*

moreover have *gs-chains gs′ c2 aid′* = *gs-chains gs c2 aid′*

unfolding *gs′-def release-lock-chains* **using** *other2 locked-chains* **by** *simp*

ultimately have *get-reg-state gs′ c1 aid′* = *get-reg-state gs c1 aid′*

and *get-reg-state gs′ c2 aid′* = *get-reg-state gs c2 aid′*

unfolding *get-reg-state-def get-asset-state-def* **by** *simp-all*

with *s1-eq s2-eq* **have**

get-reg-state gs c1 aid′ = *Some s1-final*

get-reg-state gs c2 aid′ = *Some s2-final*

by *simp-all*

with *valid* **show** *?thesis*

unfolding *valid-state-def consistent-state-def* **by** *blast*

qed

qed

qed

15.2 Part 2: no_locked_without_reason preservation

After sync, no assets are locked. The sync operation acquires a lock on the target asset and releases it before returning. For all other assets, the lock state is unchanged (and was False by `valid_state`).

lemma *sync-preserves-no-locks*:

assumes *valid*: *valid-state gs*

and *exists*: *asset-exists gs source aid*

and *current*: *get-reg-state gs source aid* = *Some s*

```
    and trans: reg-transition s action = Some s'
    and synced: sync source action aid gs = Some gs'
  shows no-locked-without-reason gs'
proof -
  from synced current trans
  obtain gs-locked where
    lock: acquire-lock gs aid = Some gs-locked
    and gs'-def: gs' = release-lock
      (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
    unfolding sync-def
    by (auto split: option.splits simp: Let-def)

  — Locks after acquire: only aid is locked
  from lock have locked-locks: gs-locks gs-locked = (gs-locks gs)(aid := True)
    unfolding acquire-lock-def is-locked-def
    by (auto split: if-splits)

  — update_all_chains does not change locks
  have upd-locks: gs-locks (update-all-chains gs-locked aid s' (connected-chains gs
aid)) =
                  gs-locks gs-locked
    using update-all-chains-locks by simp

  — release_lock clears the lock on aid
  have final-locks: gs-locks gs' = (gs-locks gs-locked)(aid := False)
    unfolding gs'-def release-lock-def using upd-locks by simp

  show ?thesis
  unfolding no-locked-without-reason-def is-locked-def
  proof
    fix aid'
    show ¬ gs-locks gs' aid'
    proof (cases aid' = aid)
      case True
      then show ?thesis using final-locks by simp
    next
      case False
      then have gs-locks gs' aid' = gs-locks gs-locked aid'
        using final-locks by simp
      also have ... = gs-locks gs aid'
        using locked-locks False by simp
      also have ... = False
      using valid unfolding valid-state-def no-locked-without-reason-def is-locked-def
        by simp
      finally show ?thesis by simp
    qed
  qed
qed
```

15.3 The valid_state preservation theorem

Combining the two parts: sync preserves `valid_state`. This is the inductive invariant that guarantees `valid_state` holds at every reachable global state.

theorem *valid-state-preservation*:
  assumes *valid*: *valid-state gs*
    and *exists*: *asset-exists gs source aid*
    and *current*: *get-reg-state gs source aid = Some s*
    and *trans*: *reg-transition s action = Some s'*
    and *synced*: *sync source action aid gs = Some gs'*
    and *fin*: *finite* (*connected-chains gs aid*)
  shows *valid-state gs'*
  unfolding *valid-state-def*
  using *sync-preserves-consistent-state*[*OF assms*]
      sync-preserves-no-locks[*OF valid exists current trans synced*]
  by *auto*

16 Heterogeneous-Action State Preservation Instance

This section instantiates the `state_preservation` locale with a concrete heterogeneous-action scenario between two chains.

The scenario models the following operational situation. Two chains A and B share the same regulatory state space (ACTIVE, FROZEN, SEIZED, CONFISCATED, RESTRICTED), but their on-chain action vocabularies differ. Chain A supports the full seven-action set used elsewhere in this theory. Chain B is a chain whose jurisdiction handles de-escalation (UNFREEZE, UNRESTRICT, RELEASE) exclusively through separate judicial procedures rather than as on-chain actions; correspondingly, Chain B's on-chain action vocabulary is the four-action escalation subset only. The synchronisation map between the two chains is therefore non-trivial in two ways:

- The source action type is `reg_action` and the target action type is a separate datatype `chain_b_action` with four constructors. This exercises the locale's heterogeneous source / target action types (`'a` vs. `'b`).
- The locale parameter `actions\<^sub>s` is instantiated with a strict subset of the full `reg_action` set, namely the four escalation actions. De-escalation actions exist in `reg_action` as a datatype but are out of scope for this synchronisation instance.

  The naturality assumption then has to hold only on the escalation subset, which is exactly the design intent of the locale's `actions\<^sub>s` parameter: the user can scope structural preservation to a subset of source actions rather than to the full source action type.

datatype *chain-b-action* = *B-FREEZE* | *B-SEIZE* | *B-CONFISCATE* | *B-RESTRICT*

The escalation subset of `reg_action` that this instance synchronises across to Chain B.

definition *escalation-actions* :: *reg-action set* **where**
escalation-actions = {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*}

definition *chain-b-actions* :: *chain-b-action set* **where**
chain-b-actions = {*B-FREEZE*, *B-SEIZE*, *B-CONFISCATE*, *B-RESTRICT*}

Chain B's transition function. Its values mirror Chain A's transition function on the four escalation actions: when Chain A maps `(s, escalation action)` to `Some s'`, Chain B does the same; when Chain A rejects, Chain B rejects. Both chains share the regulatory state space, so no state translation is required (the state map is the identity).

fun *chain-b-transition* :: *reg-state* ⇒ *chain-b-action* ⇒ *reg-state option* **where**
chain-b-transition s B-FREEZE = *reg-transition s FREEZE*
| *chain-b-transition s B-SEIZE* = *reg-transition s SEIZE*
| *chain-b-transition s B-CONFISCATE* = *reg-transition s CONFISCATE*
| *chain-b-transition s B-RESTRICT* = *reg-transition s RESTRICT*

The escalation action map: a 1-to-1 correspondence between Chain A's four escalation actions and Chain B's four constructors.

fun *escalation-action-map* :: *reg-action* ⇒ *chain-b-action* **where**
escalation-action-map FREEZE = *B-FREEZE*
| *escalation-action-map SEIZE* = *B-SEIZE*
| *escalation-action-map CONFISCATE* = *B-CONFISCATE*
| *escalation-action-map RESTRICT* = *B-RESTRICT*
| *escalation-action-map UNFREEZE* = *B-FREEZE*
| *escalation-action-map UNRESTRICT* = *B-FREEZE*
| *escalation-action-map RELEASE* = *B-FREEZE*
— The last three clauses are unused by the locale instance, since `actions\<^sub>s = escalation_actions` excludes the de-escalation actions. They are present only to make the function total on `reg_action`.

Chain B is also a state machine over the same regulatory state space.

lemma *chain-b-transition-closed*:
chain-b-transition s a = *Some s'* ⟹ *s'* ∈ *reg-states*
proof −
assume *chain-b-transition s a* = *Some s'*
then have ∃ *a'*. *reg-transition s a'* = *Some s'*
by (*cases a*) *auto*
then show *s'* ∈ *reg-states* **using** *reg-transition-closed* **by** *auto*
qed

lemma *chain-b-terminal*:
s ∈ *reg-terminal* ⟹ *chain-b-transition s a* = *None*
unfolding *reg-terminal-def*

by (*cases a*) (*auto simp*: *confiscated-terminal*)

lemma *chain-b-state-machine*:
state-machine reg-states chain-b-actions chain-b-transition reg-terminal
proof −
have *f1*: *finite reg-states* **unfolding** *reg-states-def* **by** *auto*
have *f2*: *finite chain-b-actions* **unfolding** *chain-b-actions-def* **by** *auto*
have *f3*: *reg-terminal* $\subseteq$ *reg-states*
unfolding *reg-terminal-def reg-states-def* **by** *auto*
have *f4*: $\bigwedge s\ a.\ s \in$ *reg-terminal* $\Longrightarrow a \in$ *chain-b-actions* $\Longrightarrow$ *chain-b-transition* $s\ a =$ *None*
using *chain-b-terminal* **by** *auto*
have *f5*: $\bigwedge s\ a\ s'.\ s \in$ *reg-states* $\Longrightarrow a \in$ *chain-b-actions* $\Longrightarrow$
chain-b-transition $s\ a =$ *Some* $s' \Longrightarrow s' \in$ *reg-states*
using *chain-b-transition-closed* **by** *auto*
have *f6*: $\bigwedge s\ a.$ (s :: *reg-state*) $\notin$ *reg-states* $\Longrightarrow$ *chain-b-transition* $s\ a =$ *None*
using *reg-states-UNIV* **by** *auto*
show *?thesis*
by (*intro state-machine.intro*) (*use f1 f2 f3 f4 f5 f6* **in** *auto*)
qed

The naturality conditions of `state_preservation` hold for the escalation subset. Both directions (`Some` and `None` branches) follow by case analysis on the escalation action: by construction, Chain B's transition function on each `B_X` constructor mirrors Chain A's transition function on the corresponding `X` action.

lemma *escalation-naturality-some*:
assumes $a \in$ *escalation-actions*
and *reg-transition* $s\ a =$ *Some* s'
shows *chain-b-transition* s (*escalation-action-map* a) = *Some* s'
proof −
from *assms*(*1*) **have** $a \in$ {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*}
unfolding *escalation-actions-def* **by** *simp*
then consider a = *FREEZE* | a = *SEIZE* | a = *CONFISCATE* | a = *RESTRICT*
by *auto*
then show *?thesis* **using** *assms*(*2*) **by** (*cases*) *auto*
qed

lemma *escalation-naturality-none*:
assumes $a \in$ *escalation-actions*
and *reg-transition* $s\ a =$ *None*
shows *chain-b-transition* s (*escalation-action-map* a) = *None*
proof −
from *assms*(*1*) **have** $a \in$ {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*}
unfolding *escalation-actions-def* **by** *simp*
then consider a = *FREEZE* | a = *SEIZE* | a = *CONFISCATE* | a = *RESTRICT*
by *auto*

  then show *?thesis* **using** *assms(2)* **by** (*cases*) *auto*
qed

The escalation instance: `state_preservation` with `actions\<^sub>s` the escalation subset of `reg_action`, target action type `chain_b_action`, and identity state map (both chains share the regulatory state space).

Source and target state-machine bundles for the heterogeneous-action instance. The target side reuses the existing `chain_b_state_machine` lemma; the source side is on the escalation subset of `reg_actions` rather than the full action set, so we discharge it inline.

lemma *escalation-source-state-machine*:
  state-machine reg-states escalation-actions reg-transition reg-terminal
proof *unfold-locales*
  show *finite reg-states* **unfolding** *reg-states-def* **by** *auto*
next
  show *finite escalation-actions* **unfolding** *escalation-actions-def* **by** *auto*
next
  show *reg-terminal* ⊆ *reg-states* **unfolding** *reg-terminal-def reg-states-def* **by** *auto*
next
  fix *s a*
  assume *s* ∈ *reg-terminal a* ∈ *escalation-actions*
  then show *reg-transition s a* = *None*
    unfolding *reg-terminal-def* **by** (*auto simp*: *confiscated-terminal*)
next
  fix *s a s′*
  assume *s* ∈ *reg-states a* ∈ *escalation-actions reg-transition s a* = *Some s′*
  then show *s′* ∈ *reg-states* **using** *reg-transition-closed* **by** *auto*
next
  fix *s* :: *reg-state* **and** *a* :: *reg-action*
  assume *s* ∉ *reg-states*
  then show *reg-transition s a* = *None* **using** *reg-states-UNIV* **by** *auto*
qed

Heterogeneous-action instance: the source action set is the `escalation_actions` subset of `reg_action`, the target action set is `chain_b_actions`, and the state map is the identity. Both source and target state machines share the regulatory state space, so two of the state-machine obligations (`finite reg_states` and `reg_terminal \<subseteq> reg_states`) collapse between source and target; the proof structure below reflects this by issuing each `show` exactly once for the merged obligation.

interpretation *escalation-preservation*:
  state-preservation
    reg-states escalation-actions reg-transition reg-terminal
    reg-states chain-b-actions chain-b-transition reg-terminal
    id :: *reg-state* ⇒ *reg-state escalation-action-map*
proof *unfold-locales*
  show *finite reg-states* **unfolding** *reg-states-def* **by** *auto*

next
  show *finite escalation-actions* **unfolding** *escalation-actions-def* **by** *auto*
next
  show *reg-terminal* ⊆ *reg-states* **unfolding** *reg-terminal-def reg-states-def* **by** *auto*
next
  fix *s a*
  assume *s* ∈ *reg-terminal a* ∈ *escalation-actions*
  then show *reg-transition s a* = *None*
    unfolding *reg-terminal-def* **by** (*auto simp*: *confiscated-terminal*)
next
  fix $s\ a\ s'$
  assume *s* ∈ *reg-states a* ∈ *escalation-actions reg-transition s a* = *Some* s'
  then show s' ∈ *reg-states* **using** *reg-transition-closed* **by** *auto*
next
  fix *s* :: *reg-state* **and** *a* :: *reg-action*
  assume *s* ∉ *reg-states*
  then show *reg-transition s a* = *None* **using** *reg-states-UNIV* **by** *auto*
next
  show *finite chain-b-actions* **unfolding** *chain-b-actions-def* **by** *auto*
next
  fix *s a*
  assume *s* ∈ *reg-terminal a* ∈ *chain-b-actions*
  then show *chain-b-transition s a* = *None* **using** *chain-b-terminal* **by** *auto*
next
  fix $s\ a\ s'$
  assume *s* ∈ *reg-states a* ∈ *chain-b-actions chain-b-transition s a* = *Some* s'
  then show s' ∈ *reg-states* **using** *chain-b-transition-closed* **by** *auto*
next
  fix *s* :: *reg-state* **and** *a* :: *chain-b-action*
  assume *s* ∉ *reg-states*
  then show *chain-b-transition s a* = *None* **using** *reg-states-UNIV* **by** *auto*
next
  fix *s*
  assume *s* ∈ *reg-states*
  then show *id s* ∈ *reg-states* **by** *simp*
next
  fix *a*
  assume *a* ∈ *escalation-actions*
  then show *escalation-action-map a* ∈ *chain-b-actions*
    unfolding *escalation-actions-def chain-b-actions-def* **by** *auto*
next
  fix *s*
  assume *s* ∈ *reg-terminal*
  then show *id s* ∈ *reg-terminal* **by** *simp*
next
  fix $s\ a\ s'$
  assume *s* ∈ *reg-states a* ∈ *escalation-actions reg-transition s a* = *Some* s'
  then show *chain-b-transition* (*id s*) (*escalation-action-map a*) = *Some* (*id* s')
    using *escalation-naturality-some* **by** *simp*

next
  fix *s a*
  assume $s \in$ *reg-states* $a \in$ *escalation-actions reg-transition s a* = *None*
  then show *chain-b-transition* (*id s*) (*escalation-action-map a*) = *None*
    using *escalation-naturality-none* **by** *simp*
qed

As a corollary of the locale interpretation, sequential preservation (the locale's main theorem) applies to escalation action sequences: any valid sequence of escalation actions on Chain A produces, when mapped through `escalation_action_map`, a valid sequence on Chain B ending in the same regulatory state.

corollary *escalation-sequential-preservation*:
  assumes $s \in$ *reg-states*
    and $\forall$ $a \in$ *set as*. $a \in$ *escalation-actions*
    and *escalation-preservation.source.apply-actions s as* = *Some s′*
  shows *escalation-preservation.target.apply-actions s* (*map escalation-action-map as*)
      = *Some s′*
  using *assms escalation-preservation.sequential-preservation* [**where** *as* = *as*]
  by *simp*

17 Layer-Crossing Symmetric State Preservation Instance

This section instantiates the `symmetric_state_preservation` locale with a concrete bidirectional binding between two representations of the same regulatory state.

The on-chain representation is the five-element `reg_state` enum used throughout this theory. The off-chain representation is a structured permission record (`daml_perm`) modelling a DAML party-permission ledger entry: a status tag plus auxiliary fields that carry the party / scope metadata associated with a non-trivial regulatory status (the seizing party for SEIZED, the restriction scope for RESTRICTED). A type-level invariant (`valid_daml_perm`) carves out the bijection domain by tying the auxiliary fields to the status tag.

The action vocabularies on the two layers coincide: both layers process the seven actions of `reg_action`, and the action map is the identity. All of the non-trivial content of the symmetric instance therefore lives in the layer-crossing state mapping. The roundtrip assumptions of `symmetric_state_preservation` become a pair of bijection conditions between `reg_state` and the valid `daml_perm` records.

datatype *daml-status-tag* =
  D-Active | *D-Frozen* | *D-Seized* | *D-Confiscated* | *D-Restricted*

record *daml-perm* =
  status-tag :: *daml-status-tag*
  seized-by :: *nat option*
  restriction-scope :: *nat option*

definition *default-party-id* :: *nat* **where**
  default-party-id = *0*

definition *default-scope-id* :: *nat* **where**
  default-scope-id = *0*

The well-formedness invariant: the auxiliary fields are non-`None` exactly on the status tags that semantically require them.

definition *valid-daml-perm* :: *daml-perm* ⇒ *bool* **where**
  valid-daml-perm p ⟷
    (*status-tag p* = *D-Seized* ⟷ *seized-by p* ≠ *None*) ∧
    (*status-tag p* = *D-Restricted* ⟷ *restriction-scope p* ≠ *None*)

The two state maps between the two representations.

fun *reg-to-daml* :: *reg-state* ⇒ *daml-perm* **where**
  reg-to-daml ACTIVE = ⦇ *status-tag* = *D-Active*,
    seized-by = *None*,
    restriction-scope = *None* ⦈
| *reg-to-daml FROZEN* = ⦇ *status-tag* = *D-Frozen*,
    seized-by = *None*,
    restriction-scope = *None* ⦈
| *reg-to-daml SEIZED* = ⦇ *status-tag* = *D-Seized*,
    seized-by = *Some default-party-id*,
    restriction-scope = *None* ⦈
| *reg-to-daml CONFISCATED* = ⦇ *status-tag* = *D-Confiscated*,
    seized-by = *None*,
    restriction-scope = *None* ⦈
| *reg-to-daml RESTRICTED* = ⦇ *status-tag* = *D-Restricted*,
    seized-by = *None*,
    restriction-scope = *Some default-scope-id* ⦈

fun *daml-to-reg* :: *daml-perm* ⇒ *reg-state* **where**
  daml-to-reg p = (*case status-tag p of*
      D-Active ⇒ *ACTIVE*
    | *D-Frozen* ⇒ *FROZEN*
    | *D-Seized* ⇒ *SEIZED*
    | *D-Confiscated* ⇒ *CONFISCATED*
    | *D-Restricted* ⇒ *RESTRICTED*)

The DAML target state space: the image of `reg_to_daml`.

definition *daml-states* :: *daml-perm set* **where**
  daml-states = *range reg-to-daml*

definition *daml-terminal* :: *daml-perm set* **where**

daml-terminal = {*reg-to-daml CONFISCATED*}

The DAML transition function: lifts `reg_transition` through the representation map. By construction, `daml_transition` respects the `reg_to_daml` map.

definition *daml-transition* :: *daml-perm* ⇒ *reg-action* ⇒ *daml-perm option* **where**
daml-transition p a =
(*if p* ∈ *daml-states then*
(*case reg-transition* (*daml-to-reg p*) *a of*
None ⇒ *None*
| *Some s′* ⇒ *Some* (*reg-to-daml s′*))
else None)

Roundtrip lemmas establishing the layer-crossing bijection.

lemma *daml-to-reg-to-daml-id*:
daml-to-reg (*reg-to-daml s*) = *s*
by (*cases s*) *auto*

lemma *reg-to-daml-to-reg-on-image*:
assumes *p* ∈ *daml-states*
shows *reg-to-daml* (*daml-to-reg p*) = *p*
proof −
from *assms* **obtain** *s* **where** *p* = *reg-to-daml s*
unfolding *daml-states-def* **by** *auto*
then show *?thesis* **using** *daml-to-reg-to-daml-id* **by** *simp*
qed

lemma *reg-to-daml-valid*:
valid-daml-perm (*reg-to-daml s*)
unfolding *valid-daml-perm-def* **by** (*cases s*) *auto*

lemma *reg-to-daml-injective*:
reg-to-daml s1 = *reg-to-daml s2* ⟹ *s1* = *s2*
using *daml-to-reg-to-daml-id* **by** *metis*

The DAML side is a state machine on the image of `reg_to_daml`.

lemma *daml-states-finite*: *finite daml-states*
proof −
have *fin-reg*: *finite reg-states* **unfolding** *reg-states-def* **by** *auto*
hence *fin-img*: *finite* (*reg-to-daml* ‘ *reg-states*) **by** (*rule finite-imageI*)
have *eq*: *daml-states* = *reg-to-daml* ‘ *reg-states*
unfolding *daml-states-def* **using** *reg-states-UNIV* **by** *simp*
from *fin-img eq* **show** *?thesis* **by** *simp*
qed

lemma *daml-terminal-subset*: *daml-terminal* ⊆ *daml-states*
unfolding *daml-terminal-def daml-states-def* **by** *blast*

lemma *daml-terminal-absorbing*:

assumes $p \in$ *daml-terminal* **and** $a \in$ *reg-actions*
shows *daml-transition* $p\ a$ = *None*
proof −
from *assms*(*1*) **have** p = *reg-to-daml CONFISCATED*
unfolding *daml-terminal-def* **by** *simp*
then have *daml-to-reg* p = *CONFISCATED* **using** *daml-to-reg-to-daml-id* **by** *simp*
moreover have $p \in$ *daml-states*
using *assms*(*1*) *daml-terminal-subset* **by** *auto*
ultimately show *?thesis*
unfolding *daml-transition-def* **using** *confiscated-terminal* **by** *simp*
qed

lemma *daml-transition-closed*:
assumes $p \in$ *daml-states* **and** $a \in$ *reg-actions*
and *daml-transition* $p\ a$ = *Some* p'
shows $p' \in$ *daml-states*
proof −
from *assms*(*1*,*3*) **obtain** s' **where**
s'-*eq*: *reg-transition* (*daml-to-reg* p) a = *Some* s'
and p'-*def*: p' = *reg-to-daml* s'
unfolding *daml-transition-def*
by (*auto split*: *option.splits if-splits*)
then show *?thesis*
unfolding *daml-states-def* **by** *auto*
qed

lemma *daml-transition-outside-states*:
$p \notin$ *daml-states* $\Longrightarrow$ *daml-transition* $p\ a$ = *None*
unfolding *daml-transition-def* **by** *simp*

lemma *daml-state-machine*:
state-machine daml-states reg-actions daml-transition daml-terminal
proof
show *finite daml-states* **by** (*rule daml-states-finite*)
next
show *finite reg-actions* **unfolding** *reg-actions-def* **by** *auto*
next
show *daml-terminal* $\subseteq$ *daml-states* **by** (*rule daml-terminal-subset*)
next
fix $s\ a$
assume $s \in$ *daml-terminal* $a \in$ *reg-actions*
then show *daml-transition* $s\ a$ = *None* **by** (*rule daml-terminal-absorbing*)
next
fix $s\ a\ s'$
assume $s \in$ *daml-states* $a \in$ *reg-actions* *daml-transition* $s\ a$ = *Some* s'
then show $s' \in$ *daml-states* **by** (*rule daml-transition-closed*)
next
fix s :: *daml-perm* **and** a :: *reg-action*

assume $s \notin$ *daml-states*
then show *daml-transition s a = None* **by** (*rule daml-transition-outside-states*)
qed

Forward naturality: `reg_to_daml` commutes with transitions.

lemma *reg-to-daml-naturality-some*:
assumes $s \in$ *reg-states* **and** $a \in$ *reg-actions*
and *reg-transition s a = Some s'*
shows *daml-transition* (*reg-to-daml s*) *a = Some* (*reg-to-daml s'*)
proof −
have *in-states*: *reg-to-daml s* $\in$ *daml-states*
unfolding *daml-states-def* **by** *auto*
have *daml-to-reg* (*reg-to-daml s*) = *s* **using** *daml-to-reg-to-daml-id* **by** *simp*
with *assms*(*3*) **have**
reg-transition (*daml-to-reg* (*reg-to-daml s*)) *a = Some s'* **by** *simp*
with *in-states* **show** *?thesis*
unfolding *daml-transition-def* **by** *simp*
qed

lemma *reg-to-daml-naturality-none*:
assumes $s \in$ *reg-states* **and** $a \in$ *reg-actions*
and *reg-transition s a = None*
shows *daml-transition* (*reg-to-daml s*) *a = None*
proof −
have *in-states*: *reg-to-daml s* $\in$ *daml-states*
unfolding *daml-states-def* **by** *auto*
have *daml-to-reg* (*reg-to-daml s*) = *s* **using** *daml-to-reg-to-daml-id* **by** *simp*
with *assms*(*3*) **have**
reg-transition (*daml-to-reg* (*reg-to-daml s*)) *a = None* **by** *simp*
with *in-states* **show** *?thesis*
unfolding *daml-transition-def* **by** *simp*
qed

Backward naturality: `daml_to_reg` on the image of `reg_to_daml` commutes with transitions in the opposite direction.

lemma *daml-to-reg-naturality-some*:
assumes $p \in$ *daml-states* **and** $a \in$ *reg-actions*
and *daml-transition p a = Some p'*
shows *reg-transition* (*daml-to-reg p*) *a = Some* (*daml-to-reg p'*)
proof −
from *assms*(*1*,*3*) **obtain** *s'* **where**
s'-eq: *reg-transition* (*daml-to-reg p*) *a = Some s'*
and *p'-def*: *p' = reg-to-daml s'*
unfolding *daml-transition-def*
by (*auto split*: *option.splits if-splits*)
from *p'-def* **have** *daml-to-reg p' = s'* **using** *daml-to-reg-to-daml-id* **by** *simp*
with *s'-eq* **show** *?thesis* **by** *simp*
qed

lemma *daml-to-reg-naturality-none*:
assumes $p \in$ *daml-states* **and** $a \in$ *reg-actions*
and *daml-transition p a* = *None*
shows *reg-transition* (*daml-to-reg p*) *a* = *None*
proof (*rule ccontr*)
assume *reg-transition* (*daml-to-reg p*) *a* $\neq$ *None*
then obtain s' **where** *reg-transition* (*daml-to-reg p*) *a* = *Some* s' **by** *auto*
with *assms*(*1*) **have** *daml-transition p a* = *Some* (*reg-to-daml* s')
unfolding *daml-transition-def* **by** *simp*
with *assms*(*3*) **show** *False* **by** *simp*
qed

Forward state preservation: `reg_to_daml` as a structure-preserving map.

Forward state preservation. The source state machine is on `(reg_states, reg_actions, reg_transition, reg_terminal)`, which exactly matches the `reg_sm` interpretation; consequently `unfold_locales` auto-discharges all six source state-machine obligations. The target side is the DAML structured-permission record, for which only individual lemmas (not a registered interpretation) are available, so all six target state-machine obligations remain pending alongside the five preservation-specific axioms eleven obligations in total. The asymmetry with `backward_layer_preservation` below (which has only the five preservation axioms remaining) reflects the order in which the locale extension presents the two state-machine sub-locales.

interpretation *forward-layer-preservation*:
state-preservation
reg-states reg-actions reg-transition reg-terminal
daml-states reg-actions daml-transition daml-terminal
reg-to-daml id
proof *unfold-locales*
show *finite daml-states* **by** (*rule daml-states-finite*)
next
show *finite reg-actions* **unfolding** *reg-actions-def* **by** *auto*
next
show *daml-terminal* $\subseteq$ *daml-states* **by** (*rule daml-terminal-subset*)
next
fix *s a*
assume $s \in$ *daml-terminal* $a \in$ *reg-actions*
then show *daml-transition s a* = *None* **by** (*rule daml-terminal-absorbing*)
next
fix *s a* s'
assume $s \in$ *daml-states* $a \in$ *reg-actions* *daml-transition s a* = *Some* s'
then show $s' \in$ *daml-states* **by** (*rule daml-transition-closed*)
next
fix *s* :: *daml-perm* **and** *a* :: *reg-action*
assume $s \notin$ *daml-states*
then show *daml-transition s a* = *None* **by** (*rule daml-transition-outside-states*)
next
fix *s*

assume *s* ∈ *reg-states*
then show *reg-to-daml s* ∈ *daml-states* **unfolding** *daml-states-def* **by** *auto*
next
fix *a*
assume *a* ∈ *reg-actions*
then show *id a* ∈ *reg-actions* **by** *simp*
next
fix *s*
assume *s* ∈ *reg-terminal*
then show *reg-to-daml s* ∈ *daml-terminal*
unfolding *reg-terminal-def daml-terminal-def* **by** *simp*
next
fix *s a s′*
assume *s* ∈ *reg-states a* ∈ *reg-actions reg-transition s a* = *Some s′*
then show *daml-transition* (*reg-to-daml s*) (*id a*) = *Some* (*reg-to-daml s′*)
using *reg-to-daml-naturality-some* **by** *simp*
next
fix *s a*
assume *s* ∈ *reg-states a* ∈ *reg-actions reg-transition s a* = *None*
then show *daml-transition* (*reg-to-daml s*) (*id a*) = *None*
using *reg-to-daml-naturality-none* **by** *simp*
qed

Backward state preservation: `daml_to_reg` as a structure-preserving map on the image of `reg_to_daml`.

Backward state preservation. The source side is DAML and the target side matches `reg_sm`. `unfold_locales` auto-discharges all twelve state-machine obligations (six target via `reg_sm`, six source via the bundled `daml_state_machine` lemma combined with the individual `daml_*` lemmas), so only the five preservation axioms remain pending.

interpretation *backward-layer-preservation*:
state-preservation
daml-states reg-actions daml-transition daml-terminal
reg-states reg-actions reg-transition reg-terminal
daml-to-reg id
proof *unfold-locales*
fix *p*
assume *p* ∈ *daml-states*
then show *daml-to-reg p* ∈ *reg-states* **using** *reg-states-UNIV* **by** *auto*
next
fix *a*
assume *a* ∈ *reg-actions*
then show *id a* ∈ *reg-actions* **by** *simp*
next
fix *p*
assume *p* ∈ *daml-terminal*
then have *p* = *reg-to-daml CONFISCATED* **unfolding** *daml-terminal-def* **by** *simp*

then have *daml-to-reg p = CONFISCATED* **using** *daml-to-reg-to-daml-id* **by** *simp*
then show *daml-to-reg p ∈ reg-terminal* **unfolding** *reg-terminal-def* **by** *simp*
next
fix *p a p′*
assume *p ∈ daml-states a ∈ reg-actions daml-transition p a = Some p′*
then show *reg-transition (daml-to-reg p) (id a) = Some (daml-to-reg p′)*
using *daml-to-reg-naturality-some* **by** *simp*
next
fix *p a*
assume *p ∈ daml-states a ∈ reg-actions daml-transition p a = None*
then show *reg-transition (daml-to-reg p) (id a) = None*
using *daml-to-reg-naturality-none* **by** *simp*
qed

The symmetric instance: forward and backward state preservation composed with roundtrip guarantees. The non-trivial content is the bijection between `reg_state` and the image of `reg_to_daml` in `daml_perm`; the action map is the identity because both layers share the same regulatory action vocabulary.

interpretation *onchain-daml-bridge*:
symmetric-state-preservation
reg-states reg-actions reg-transition reg-terminal
daml-states reg-actions daml-transition daml-terminal
reg-to-daml id
daml-to-reg id
proof *unfold-locales*
fix *s*
assume *s ∈ reg-states*
then show *daml-to-reg (reg-to-daml s) = s* **using** *daml-to-reg-to-daml-id* **by** *simp*
next
fix *p*
assume *p ∈ daml-states*
then show *reg-to-daml (daml-to-reg p) = p* **using** *reg-to-daml-to-reg-on-image* **by** *simp*
next
fix *a*
assume *a ∈ reg-actions*
then show *id (id a) = a* **by** *simp*
qed

As a corollary of the symmetric interpretation, `reg_to_daml` is injective on `reg_states`: distinct on-chain states map to distinct off-chain DAML records, so the layer-crossing representation incurs no information loss.

corollary *reg-to-daml-injective-on-states*:
⟦ *s1 ∈ reg-states*; *s2 ∈ reg-states*; *reg-to-daml s1 = reg-to-daml s2* ⟧
⟹ *s1 = s2*
using *onchain-daml-bridge.state-map-injective* **.**

18 Multi-Domain Locale Instantiation

We now instantiate the `multi_domain_preservation` locale from `State_Preservation`.thy with the regulatory state machine.

The instantiation is parametric: given ANY finite set of domains and ANY initial `domain_state` function satisfying the consistency precondition, the locale is satisfied. This proves that the abstract framework applies to the concrete regulatory model without fixing a specific topology.

The key bridge: we construct a `domain_state` function from our `global_state`, and show that `valid_state` implies the consistency precondition required by `multi_domain_preservation`.

Bridge from our `global_state` model to the `multi_domain_preservation` locale's `domain_state` signature.

`multi_domain_preservation` expects: `domain_state :: 'd => 'id => 's option`

We instantiate with: 'd = `chain_id`, 'id = `asset_id`, 's = `reg_state`

The bridge function extracts `reg_state` from our richer `asset_state`.

definition *reg-domain-state* :: *global-state* ⇒ *chain-id* ⇒ *asset-id* ⇒ *reg-state option* **where**
reg-domain-state gs cid aid = *get-reg-state gs cid aid*

The parametric instantiation theorem: for any finite set of `chain_ids` and any valid `global_state`, we can interpret the `multi_domain_preservation` locale with the regulatory state machine.

The `state_machine` predicate as a standalone fact. This is needed for instantiating `multi_domain_preservation`, which takes `state_machine` as an opaque predicate assumption.

lemma *reg-state-machine-pred*:
state-machine reg-states reg-actions reg-transition reg-terminal
proof −
have *f1*: *finite reg-states* **unfolding** *reg-states-def* **by** *auto*
have *f2*: *finite reg-actions* **unfolding** *reg-actions-def* **by** *auto*
have *f3*: *reg-terminal* ⊆ *reg-states*
unfolding *reg-terminal-def reg-states-def* **by** *auto*
have *f4*: $\bigwedge s\ a.\ s \in reg\text{-}terminal \Longrightarrow a \in reg\text{-}actions \Longrightarrow reg\text{-}transition\ s\ a = None$
unfolding *reg-terminal-def* **by** (*auto simp*: *confiscated-terminal*)
have *f5*: $\bigwedge s\ a\ s'.\ s \in reg\text{-}states \Longrightarrow a \in reg\text{-}actions \Longrightarrow$
$reg\text{-}transition\ s\ a = Some\ s' \Longrightarrow s' \in reg\text{-}states$
using *reg-transition-closed* **by** *auto*
have *f6*: $\bigwedge s\ a.\ (s :: reg\text{-}state) \notin reg\text{-}states \Longrightarrow reg\text{-}transition\ s\ a = None$
using *reg-states-UNIV* **by** *auto*
show *?thesis*
by (*intro state-machine.intro*) (*use f1 f2 f3 f4 f5 f6* **in** *auto*)
qed

theorem *reg-multi-domain-instantiation*:
  assumes *fin-doms*: *finite* (*doms* :: *chain-id set*)
    and *valid*: *valid-state gs*
  shows *multi-domain-preservation doms reg-states reg-actions reg-transition*
        reg-terminal (*reg-domain-state gs*)
proof (*intro multi-domain-preservation.intro*)
  show *finite doms* **using** *fin-doms* **by** *simp*
next
  show *state-machine reg-states reg-actions reg-transition reg-terminal*
    by (*rule reg-state-machine-pred*)
next
  fix *d1 d2* :: *chain-id* **and** *aid* :: *asset-id* **and** *s1 s2* :: *reg-state*
  assume *d1* ∈ *doms d2* ∈ *doms*
    and *reg-domain-state gs d1 aid* = *Some s1*
    and *reg-domain-state gs d2 aid* = *Some s2*
  then have *get-reg-state gs d1 aid* = *Some s1*
    and *get-reg-state gs d2 aid* = *Some s2*
    unfolding *reg-domain-state-def* **by** *simp-all*
  with *valid* **show** *s1* = *s2*
    unfolding *valid-state-def consistent-state-def* **by** *blast*
qed

Corollary: the generic `cross_domain_consistency` theorem from `State_Preservation`.thy applies to regulatory synchronization.

This connects the abstract theory to the concrete model: the generic theorem guarantees that after `sync_all` (the abstract synchronization), all connected domains agree. Combined with our concrete `regulatory_homomorphism` theorem, we have both the abstract and concrete guarantees.

The instantiation above (`reg_multi_domain_instantiation`) enables using all theorems proven in `multi_domain_preservation` in particular `cross_domain_consistency` and `sync_isolation` for the regulatory model without reproving them. Any consumer of this theory can invoke:

`multi_domain_preservation.cross_domain_consistency` [OF `reg_multi_domain_instantiat`
`fin_doms` `valid_gs`]]

to obtain the concrete guarantee for their domain set and global state.

Summary of what this theory establishes:

1. The regulatory state machine (`reg_state`, `reg_action`, `reg_transition`) satisfies the `state_machine` locale from `State_Preservation`.thy.

2. CONFISCATED is terminal (I1), CONFISCATE is universally reachable (I2), and the transition function is deterministic (I3).

3. SEIZED to FROZEN and FROZEN to RESTRICTED direct transitions are excluded by design, with legal and complexity-reduction justifications.

4. The synchronization protocol (lock, then update, then unlock) preserves cross-chain consistency for the same asset while isolating other assets.

5. Preemptive locking guards the sync function against a second action

while the lock is held; under the atomic model this expresses the intended exclusion of competing regulatory actions on the same asset (NOT double-spend prevention that is handled at a different layer).

6. The regulatory_homomorphism theorem establishes that after sync, all connected chains agree on the new regulatory state.

7. The valid_state_preservation theorem establishes that sync preserves the global validity invariant: consistent_state AND no outstanding locks. This closes the inductive invariant.

8. The heterogeneous-action instance (escalation_preservation) instantiates state_preservation with the four-action escalation subset of reg_action on the source side, the dedicated four-constructor datatype chain_b_action on the target side, and the identity state map. This exercises both the locale's actions\<^sub>s subset parameter and the heterogeneous source / target action types.

9. The layer-crossing instance (onchain_daml_bridge) instantiates symmetric_state_preservat with the on-chain enum representation and a structured DAML permission record (daml_perm) carrying auxiliary fields. The action vocabularies coincide so the action map is the identity; the non-trivial content is the bijection between the enum and the valid records, with a type-level invariant (valid_daml_perm) carving out the bijection domain.

10. The multi_domain_preservation locale is instantiated parametrically: for any finite domain set and valid global state, the abstract framework from State_Preservation.thy applies to the regulatory model.

Open work for subsequent entries: - Eventual consistency under the partially synchronous network model (with message delays, redelivery, and out-of-order arrival).

end

theory *Priority-Resolution*
  imports *Main*
begin

19 Priority-Based Deterministic Selection

A system where messages carry priorities from a linear order. Given a finite non-empty set of messages with injective priorities, there exists a unique highest-priority message, and selection is deterministic.

The injectivity assumption models tiebreaking: in practice, a composite key (e.g., authority level, timestamp, severity, node ID) ensures no two distinct messages share the same priority.

locale *priority-system* =
  fixes *priority* :: $'m \Rightarrow 'k$::*linorder*
  assumes *priority-injective*:

$\llbracket$ *priority m1* = *priority m2* $\rrbracket \Longrightarrow$ *m1* = *m2*
begin

In a finite non-empty set with injective priorities, there exists a unique element with the maximum priority.

lemma *highest-priority-exists*:
  assumes *finite S* **and** $S \neq \{\}$
  shows $\exists!m.\ m \in S \wedge (\forall m' \in S.\ priority\ m' \leq priority\ m)$
proof −
  have *fin-img*: *finite* (*priority* ‘ *S*) **using** *assms*(*1*) **by** *simp*
  have *ne-img*: *priority* ‘ $S \neq \{\}$ **using** *assms*(*2*) **by** *simp*
  obtain *m* **where** *m-in*: $m \in S$ **and** *m-pri*: *priority m* = *Max* (*priority* ‘ *S*)
    using *Max-in*[*OF fin-img ne-img*] **by** *auto*
  have *m-max*: $\forall m' \in S.\ priority\ m' \leq priority\ m$
  proof
    fix m' **assume** $m' \in S$
    hence *priority* $m' \in$ *priority* ‘ *S* **by** *simp*
    hence *priority* $m' \leq$ *Max* (*priority* ‘ *S*) **using** *Max-ge*[*OF fin-img*] **by** *simp*
    thus *priority* $m' \leq$ *priority m* **using** *m-pri* **by** *simp*
  qed
  show *?thesis*
  proof (*rule ex1I*[*of - m*])
    show $m \in S \wedge (\forall m' \in S.\ priority\ m' \leq priority\ m)$
      using *m-in m-max* **by** *auto*
  next
    fix *m2* **assume** $m2 \in S \wedge (\forall m' \in S.\ priority\ m' \leq priority\ m2)$
    then have *m2-in*: $m2 \in S$ **and** *m2-max*: $\forall m' \in S.\ priority\ m' \leq priority\ m2$
by *auto*
    from *m-max m2-in* **have** *priority m2* $\leq$ *priority m* **by** *auto*
    from *m2-max m-in* **have** *priority m* $\leq$ *priority m2* **by** *auto*
    then have *priority m* = *priority m2* **using** ‹*priority m2* $\leq$ *priority m*› **by** *auto*
    then show *m2* = *m* **using** *priority-injective* **by** *auto*
  qed
qed

definition *select-highest* :: *'m set* $\Rightarrow$ *'m option* **where**
  select-highest S =
    (*if S* = {} *then None*
     else Some (*THE m.* $m \in S \wedge (\forall m' \in S.\ priority\ m' \leq priority\ m)$))

theorem *select-highest-deterministic*:
  assumes *fin*: *finite S* **and** *ne*: $S \neq \{\}$
  shows $\exists!m.$ *select-highest S* = *Some m*
proof −
  from *ne* **obtain** *v* **where** *hv*: *select-highest S* = *Some v*
    unfolding *select-highest-def* **by** *simp*
  show *?thesis*
  proof (*rule ex1I*[*of - v*])
    show *select-highest S* = *Some v* **by** (*rule hv*)

```
  next
    fix w assume select-highest S = Some w
    with hv show w = v by simp
  qed
qed

theorem select-highest-in-set:
  assumes fin: finite S and sel: select-highest S = Some m
  shows m ∈ S
proof −
  from sel have ne: S ≠ {}
    unfolding select-highest-def by (simp split: if-splits)
  from highest-priority-exists[OF fin ne] have uniq:
    ∃!x. x ∈ S ∧ (∀ m' ∈ S. priority m' ≤ priority x) .
  from theI'[OF uniq] have the-in:
    (THE x. x ∈ S ∧ (∀ m' ∈ S. priority m' ≤ priority x)) ∈ S by auto
  from sel ne have m = (THE x. x ∈ S ∧ (∀ m' ∈ S. priority m' ≤ priority x))
    unfolding select-highest-def by simp
  with the-in show ?thesis by simp
qed

theorem select-highest-is-max:
  assumes fin: finite S and ne: S ≠ {} and sel: select-highest S = Some m
  shows ∀ m' ∈ S. priority m' ≤ priority m
proof −
  from highest-priority-exists[OF fin ne] have uniq:
    ∃!x. x ∈ S ∧ (∀ m' ∈ S. priority m' ≤ priority x) .
  from theI'[OF uniq] have the-max:
    ∀ m' ∈ S. priority m' ≤
      priority (THE x. x ∈ S ∧ (∀ m' ∈ S. priority m' ≤ priority x))
    by auto
  from sel ne have m = (THE x. x ∈ S ∧ (∀ m' ∈ S. priority m' ≤ priority x))
    unfolding select-highest-def by simp
  with the-max show ?thesis by simp
qed

end
```

20 Fair Leader Starvation Freedom

A leader-based system where an honest leader always processes at least one pending request, and a fairness assumption guarantees that honest leaders appear periodically.

The fairness assumption is a sufficient condition that abstracts over the probabilistic guarantees of VRF-based leader election. Under $f < n\ /\ (3{::}'a)$ Byzantine faults, the probability that a Byzantine leader is elected k consecutive times is $(f\ /\ n)^k$, which decreases exponentially. The fairness bound k captures this as a deterministic assumption.

The key theorem: under the fairness assumption, any pending request is processed within a bounded number of epochs.

locale *fair-leader-system* =
fixes *leader-at* :: *nat* ⇒ *'n*
and *is-honest* :: *'n* ⇒ *bool*
and *pending* :: *nat* ⇒ *nat*
and *fairness-bound* :: *nat*
assumes *fairness-bound-positive*: *fairness-bound* > *0*
and *fair-leader*:
∀ *epoch*. ∃ *e*. *epoch* ≤ *e* ∧ *e* < *epoch* + *fairness-bound* ∧ *is-honest* (*leader-at* *e*)
and *honest-progress*:
⟦ *is-honest* (*leader-at* *e*); *pending* *e* > *0* ⟧ ⟹ *pending* (*Suc* *e*) < *pending* *e*
and *non-honest-bounded*:
pending (*Suc* *e*) ≤ *pending* *e*
begin

Pending count is monotonically non-increasing.

lemma *pending-monotone*:
assumes *e1* ≤ *e2*
shows *pending* *e2* ≤ *pending* *e1*
using *assms*
proof (*induction rule*: *inc-induct*)
case *base*
then show *?case* **by** *simp*
next
case (*step* *e*)
have *pending* *e2* ≤ *pending* (*Suc* *e*) **by** (*rule* *step.IH*)
also have *pending* (*Suc* *e*) ≤ *pending* *e* **using** *non-honest-bounded* **by** *auto*
finally show *?case* **.**
qed

If there are pending requests, within *fairness-bound* epochs at least one will be processed (the pending count strictly decreases).

theorem *starvation-bound*:
assumes *pos*: *pending* *epoch* > *0*
shows ∃ *e*. *epoch* ≤ *e* ∧ *e* < *epoch* + *fairness-bound* ∧ *pending* (*Suc* *e*) < *pending* *e*
proof −
from *fair-leader* **obtain** *h* **where**
h-range: *epoch* ≤ *h* *h* < *epoch* + *fairness-bound* **and**
h-honest: *is-honest* (*leader-at* *h*)
by *auto*
show *?thesis*
proof (*cases* *pending* *h* > *0*)
case *True*
with *h-honest* **have** *pending* (*Suc* *h*) < *pending* *h* **using** *honest-progress* **by** *auto*
with *h-range* **show** *?thesis* **by** *auto*

```
next
  case False
  then have ph0: pending h = 0 by simp
  — pending dropped from >0 (at epoch) to 0 (at h): find a strict decrease step
  have key: ∀ n. 0 < n ⟶ pending (epoch + n) = 0 ⟶
      (∃ e. epoch ≤ e ∧ e < epoch + n ∧ pending (Suc e) < pending e)
  proof (rule allI)
    fix n
    show 0 < n ⟶ pending (epoch + n) = 0 ⟶
        (∃ e. epoch ≤ e ∧ e < epoch + n ∧ pending (Suc e) < pending e)
    proof (induction n)
      case 0 show ?case by simp
    next
      case (Suc k)
      show ?case
      proof (intro impI)
        assume hpk: pending (epoch + Suc k) = 0
        show ∃ e. epoch ≤ e ∧ e < epoch + Suc k ∧ pending (Suc e) < pending e
        proof (cases k = 0)
          case True
         from hpk True have pending (Suc epoch) < pending epoch using pos by
simp
          thus ?thesis by (intro exI[of - epoch]) auto
        next
          case False
          then have kpos: 0 < k by simp
          show ?thesis
          proof (cases pending (epoch + k) = 0)
            case True
            then have pk0: pending (epoch + k) = 0 .
            from Suc.IH[THEN mp, OF kpos, THEN mp, OF pk0]
           obtain e′ where epoch ≤ e′ e′ < epoch + k pending (Suc e′) < pending
e′
              by auto
            thus ?thesis by auto
          next
            case False
            from False hpk have pending (epoch + Suc k) < pending (epoch + k)
by simp
            thus ?thesis by (intro exI[of - epoch + k]) auto
          qed
        qed
      qed
    qed
  qed
  have dpos: 0 < h − epoch
  proof −
    have h ≠ epoch using ph0 pos by auto
    with h-range(1) show ?thesis by arith
```

qed
have *pd*: *pending* (*epoch* + (*h* − *epoch*)) = *0*
proof −
have *epoch* + (*h* − *epoch*) = *h* **using** *h-range*(*1*) **by** *arith*
with *ph0* **show** *?thesis* **by** *simp*
qed
from *key*[*rule-format*, *OF dpos*, *OF pd*]
obtain *e′* **where** *he′*: *epoch* ≤ *e′* *e′* < *epoch* + (*h* − *epoch*) *pending* (*Suc e′*) < *pending e′*
by *auto*
have *epoch* + (*h* − *epoch*) ≤ *h* **using** *h-range*(*1*) **by** *arith*
with *he′* *h-range*(*2*) **show** *?thesis* **by** *auto*
qed
qed

Corollary: all pending requests are eventually processed. By induction on the pending count (which is a natural number and thus a well-founded decreasing measure), repeated application of `starvation_bound` drives the count to zero.

theorem *eventual-completion*:
shows ∃ *e-final*. *pending e-final* = *0*
proof −
define *P* **where**
P n ≡ ∀ *epoch*. *pending epoch* = *n* ⟶ (∃ *e-final*. *pending e-final* = *0*)
for *n* :: *nat*
have *all-P*: ⋀*n*. *P n*
proof −
fix *n* **show** *P n*
unfolding *P-def*
proof (*induction n rule*: *less-induct*)
case (*less n*)
show ∀ *epoch*. *pending epoch* = *n* ⟶ (∃ *e-final*. *pending e-final* = *0*)
proof (*intro allI impI*)
fix *epoch* **assume** *eq*: *pending epoch* = *n*
show ∃ *e-final*. *pending e-final* = *0*
proof (*cases n* = *0*)
case *True* **thus** *?thesis* **using** *eq* **by** *auto*
next
case *False*
have *pos*: *pending epoch* > *0* **using** *eq False* **by** *auto*
from *starvation-bound*[*OF pos*] **obtain** *e* **where**
e-ge: *epoch* ≤ *e* **and** *e-dec*: *pending* (*Suc e*) < *pending e*
by *auto*
have *lt*: *pending* (*Suc e*) < *n*
using *pending-monotone*[*OF e-ge*] *eq e-dec* **by** *simp*
from *less.IH*[*OF lt*] **show** *?thesis*
unfolding *P-def* **by** *blast*
qed
qed

```
    qed
  qed
  from all-P[of pending 0] show ?thesis
    unfolding P-def by blast
qed

end

end
```

```
theory DQuencer-Instance
  imports Priority-Resolution Regulatory-Instance HOL−Library.Product-Lexorder
begin
```

21 Authority Level and Action Severity

Regulatory authority hierarchy. International authorities (e.g., FATF) take precedence over national (e.g., SEC), which take precedence over regional authorities. This reflects the RCP framework's jurisdictional priority model.

```
datatype authority-level = Regional | National | International

fun authority-rank :: authority-level ⇒ nat where
  authority-rank Regional = 1
| authority-rank National = 2
| authority-rank International = 3

lemma authority-rank-injective:
  authority-rank a1 = authority-rank a2 ⟹ a1 = a2
  by (cases a1; cases a2; auto)
```

Action severity determines priority among actions from the same authority level. Stronger enforcement actions (CONFISCATE, SEIZE) take precedence over weaker ones (RESTRICT, UNFREEZE). This reflects the legal principle that more conservative (asset-protective) actions prevail when conflicting orders must be ranked against one another.

```
fun action-severity :: reg-action ⇒ nat where
  action-severity UNRESTRICT = 1
| action-severity UNFREEZE = 2
| action-severity RELEASE = 3
| action-severity RESTRICT = 4
| action-severity FREEZE = 5
| action-severity SEIZE = 6
| action-severity CONFISCATE = 7

lemma action-severity-injective:
  action-severity a1 = action-severity a2 ⟹ a1 = a2
  by (cases a1; cases a2; auto)
```

22 Priority Key

The priority key is a 4-tuple of natural numbers, using Isabelle's built-in lexicographic order on products. No manual linorder instance registration is needed.

Components (highest to lowest significance):

1. `authority_rank`: higher authority = higher priority
2. `inverted_timestamp`: earlier timestamp = higher priority (inverted)
3. `action_severity`: stronger action = higher priority
4. `node_id`: deterministic tiebreaker (lower node ID = higher priority, so we invert to make larger = higher in the nat order)

   For timestamp and `node_id` inversion, we use *max-val* − *actual-val* to convert "smaller is better" to "larger is better" for compatibility with the nat product order where larger = higher.

type-synonym *priority-key* = *nat* × *nat* × *nat* × *nat*

23 Extended Message Type

Extends `oss_message` from `Regulatory_Instance` with authority level and source node ID. Uses Isabelle record inheritance so that existing functions on `oss_message` remain applicable.

record *dq-message* = *oss-message* +
  dqm-authority-level :: *authority-level*
  dqm-source-node :: *nat*

24 Priority Construction

Construct a priority key from message fields. Uses two upper bounds for inversion: `max_time` for timestamps and `max_node` for node IDs.

definition *make-priority-key* ::
  nat ⇒ *nat* ⇒ *dq-message* ⇒ *priority-key*
where
  make-priority-key max-time max-node msg =
    (*authority-rank* (*dqm-authority-level msg*),
     max-time − *msg-timestamp msg*,
     action-severity (*msg-action msg*),
     max-node − *dqm-source-node msg*)

25 Node and System Model

```
datatype node-behavior = Honest | Byzantine

record node-info =
  ni-id       :: nat
  ni-behavior :: node-behavior

definition honest-nodes :: node-info set ⇒ node-info set where
  honest-nodes ns = {n ∈ ns. ni-behavior n = Honest}

definition byzantine-nodes :: node-info set ⇒ node-info set where
  byzantine-nodes ns = {n ∈ ns. ni-behavior n = Byzantine}

lemma honest-byzantine-partition:
  honest-nodes ns ∪ byzantine-nodes ns = ns
proof (rule set-eqI)
  fix x
  show x ∈ honest-nodes ns ∪ byzantine-nodes ns ⟷ x ∈ ns
    unfolding honest-nodes-def byzantine-nodes-def
    by (cases ni-behavior x; auto)
qed

lemma honest-byzantine-disjoint:
  honest-nodes ns ∩ byzantine-nodes ns = {}
  unfolding honest-nodes-def byzantine-nodes-def by auto
```

26 D-quencer System Locale

The D-quencer system locale encapsulates the BFT assumption $((3::'a) * f + 1 \leq n)$ and system parameters.

```
locale dquencer-system =
  fixes nodes :: node-info set
    and f-max :: nat
    and fairness-bound :: nat
    and max-time :: nat
    and max-node :: nat
  assumes finite-nodes: finite nodes
    and bft-threshold: card nodes ≥ 3 * f-max + 1
    and byzantine-bound: card (byzantine-nodes nodes) ≤ f-max
    and nonempty-nodes: nodes ≠ {}
    and fairness-positive: fairness-bound > 0
begin

lemma honest-majority:
  card (honest-nodes nodes) > 2 * f-max
proof −
  have card nodes = card (honest-nodes nodes) + card (byzantine-nodes nodes)
```

using *honest-byzantine-partition honest-byzantine-disjoint finite-nodes*
by (*metis card-Un-disjoint finite-Un*)
with *bft-threshold byzantine-bound* **show** *?thesis* **by** *linarith*
qed

lemma *honest-nonempty*:
honest-nodes nodes $\neq$ {}
proof −
have *fin*: *finite* (*honest-nodes nodes*)
unfolding *honest-nodes-def* **using** *finite-nodes* **by** *simp*
have *pos*: *0* < *card* (*honest-nodes nodes*)
using *honest-majority* **by** *linarith*
with *fin pos* **show** *?thesis* **by** (*auto simp*: *card-eq-0-iff*)
qed

end

27 Priority System Instantiation

We instantiate the `priority_system` locale from `Priority_Resolution.thy` with D-quencer message priorities.

The priority function maps messages to `priority_key` (which is `nat \<times> nat \<times> nat \<times> nat` with the built-in lexicographic order).

Injectivity follows from: distinct messages have either different authority levels, different timestamps, different actions, or different source nodes. The source node ID serves as the final tiebreaker ensuring no two distinct messages share the same key.

For a concrete message set with the injectivity precondition, the `priority_system` locale provides deterministic selection.

lemma *priority-key-injectivity*:
assumes *make-priority-key mt mn m1* = *make-priority-key mt mn m2*
and *msg-timestamp m1* $\leq$ *mt* **and** *msg-timestamp m2* $\leq$ *mt*
and *dqm-source-node m1* $\leq$ *mn* **and** *dqm-source-node m2* $\leq$ *mn*
shows *dqm-authority-level m1* = *dqm-authority-level m2*
$\wedge$ *msg-timestamp m1* = *msg-timestamp m2*
$\wedge$ *msg-action m1* = *msg-action m2*
$\wedge$ *dqm-source-node m1* = *dqm-source-node m2*
proof −
from *assms*(*1*) **have**
eq1: *authority-rank* (*dqm-authority-level m1*) = *authority-rank* (*dqm-authority-level m2*) **and**
eq2: *mt* − *msg-timestamp m1* = *mt* − *msg-timestamp m2* **and**
eq3: *action-severity* (*msg-action m1*) = *action-severity* (*msg-action m2*) **and**
eq4: *mn* − *dqm-source-node m1* = *mn* − *dqm-source-node m2*
unfolding *make-priority-key-def* **by** *auto*
from *eq1* **have** *al*: *dqm-authority-level m1* = *dqm-authority-level m2*

using *authority-rank-injective* **by** *auto*
from *eq2 assms(2,3)* **have** *ts*: *msg-timestamp m1 = msg-timestamp m2*
by *linarith*
from *eq3* **have** *act*: *msg-action m1 = msg-action m2*
using *action-severity-injective* **by** *auto*
from *eq4 assms(4,5)* **have** *nd*: *dqm-source-node m1 = dqm-source-node m2*
by *linarith*
from *al ts act nd* **show** *?thesis* **by** *auto*
qed

Auxiliary corollary: under the well-formedness bounds, equal priority keys imply equal messages on every priority-relevant field.

The generic `priority_system` locale assumes priority injectivity unconditionally on its carrier type. We instantiate it with the identity function on `priority_key` as the carrier; injectivity holds by reflexivity on the carrier. The recovery of the unique D-quencer message corresponding to a chosen priority key is provided as a corollary inside the `dquencer_priority_concrete` locale, using the well-formedness bounds and the additional priority-key distinctness assumption.

interpretation *dq-priority*:
priority-system id :: *priority-key* ⇒ *priority-key*
by *unfold-locales auto*

The `dquencer_priority_context` locale extends `dquencer_system` with a fixed message set whose elements satisfy the well-formedness bounds. Together with the bound-preservation, this set induces the priority key set `msg_priority_keys` on which priority-based selection operates.

locale *dquencer-priority-context* = *dquencer-system* +
fixes *msg-set* :: *dq-message set*
assumes *msg-set-nonempty*: *msg-set* $\neq$ {}
and *msg-set-well-formed*:
$\bigwedge m.\ m \in$ *msg-set* $\Longrightarrow$ *msg-timestamp m* $\leq$ *max-time*
$\wedge$ *dqm-source-node m* $\leq$ *max-node*
begin

definition *msg-priority-keys* :: *priority-key set* **where**
msg-priority-keys = *make-priority-key max-time max-node ‘ msg-set*

lemma *msg-priority-keys-nonempty*: *msg-priority-keys* $\neq$ {}
using *msg-set-nonempty* **unfolding** *msg-priority-keys-def* **by** *simp*

lemma *msg-priority-keys-finite*:
finite msg-set $\Longrightarrow$ *finite msg-priority-keys*
unfolding *msg-priority-keys-def* **by** *simp*

end

The `dquencer_priority_concrete` locale strengthens the context with the priority-key distinctness assumption on `msg_set`. This is the natural

BFT-consensus precondition: distinct authority / timestamp / severity / source-node tuples ensure determinism. Under this assumption the priority key uniquely identifies a message in `msg_set`, so selecting the highest priority key in `msg_priority_keys` is equivalent to selecting the highest-priority message in `msg_set`.

locale *dquencer-priority-concrete* = *dquencer-priority-context* +
  assumes *msg-set-priority-distinct*:
    $\bigwedge m1\ m2.\ m1 \in msg\text{-}set \Longrightarrow m2 \in msg\text{-}set$
      $\Longrightarrow make\text{-}priority\text{-}key\ max\text{-}time\ max\text{-}node\ m1$
        $= make\text{-}priority\text{-}key\ max\text{-}time\ max\text{-}node\ m2$
      $\Longrightarrow m1 = m2$
begin

Recovery: given a priority key from `msg_priority_keys`, return the unique `msg_set` message with that key.

definition *recover-msg* :: *priority-key* ⇒ *dq-message* **where**
  recover-msg k = (*THE m. m* ∈ *msg-set* ∧ *make-priority-key max-time max-node m* = *k*)

lemma *recover-msg-unique-existence*:
  assumes *k* ∈ *msg-priority-keys*
  shows ∃!*m. m* ∈ *msg-set* ∧ *make-priority-key max-time max-node m* = *k*
proof −
  from *assms* **obtain** *m*
    where *m-in*: *m* ∈ *msg-set*
      and *m-key*: *make-priority-key max-time max-node m* = *k*
    unfolding *msg-priority-keys-def* **by** *auto*
  show *?thesis*
  proof (*rule ex1I*[*of - m*])
    show *m* ∈ *msg-set* ∧ *make-priority-key max-time max-node m* = *k*
      using *m-in m-key* **by** *auto*
  next
    fix *m′*
    assume *m′* ∈ *msg-set* ∧ *make-priority-key max-time max-node m′* = *k*
    then have *m′-in*: *m′* ∈ *msg-set*
      and *m′-key*: *make-priority-key max-time max-node m′* = *k*
      by *auto*
    from *m-key m′-key*
    have *make-priority-key max-time max-node m′*
        = *make-priority-key max-time max-node m*
      by *simp*
    then show *m′* = *m*
      using *msg-set-priority-distinct*[*OF m′-in m-in*] **by** *simp*
  qed
qed

lemma *recover-msg-correct*:
  assumes *k* ∈ *msg-priority-keys*

shows *recover-msg k* $\in$ *msg-set*
$\wedge$ *make-priority-key max-time max-node* (*recover-msg k*) = *k*
proof –
from *recover-msg-unique-existence*[*OF assms*]
have *ex1*: $\exists!m.$ *m* $\in$ *msg-set* $\wedge$ *make-priority-key max-time max-node m* = *k* **.**
show *?thesis*
using *theI'*[*OF ex1*] **unfolding** *recover-msg-def* **.**
qed

Concrete consequence: deterministic selection of the highest-priority key from a non-empty finite `msg_priority_keys` set, and recovery of the unique well-formed D-quencer message that produced it.

corollary *dq-select-highest-deterministic*:
assumes *finite msg-set*
shows $\exists!k.$ *dq-priority.select-highest msg-priority-keys* = *Some k*
using *dq-priority.select-highest-deterministic*
[*OF msg-priority-keys-finite*[*OF assms*] *msg-priority-keys-nonempty*] **.**

corollary *dq-select-highest-in-set*:
assumes *finite msg-set*
and *dq-priority.select-highest msg-priority-keys* = *Some k*
shows *k* $\in$ *msg-priority-keys*
using *dq-priority.select-highest-in-set*
[*OF msg-priority-keys-finite*[*OF assms*(*1*)] *assms*(*2*)] **.**

corollary *dq-select-highest-message*:
assumes *finite msg-set*
and *dq-priority.select-highest msg-priority-keys* = *Some k*
shows *recover-msg k* $\in$ *msg-set*
$\wedge$ *make-priority-key max-time max-node* (*recover-msg k*) = *k*
using *recover-msg-correct*[*OF dq-select-highest-in-set*[*OF assms*]] **.**

end

28 Fair Leader Starvation Freedom Instantiation

We instantiate the `fair_leader_system` locale with the D-quencer's leader election and pending request processing.

The fairness assumption abstracts VRF randomness: within any `fairness_bound` consecutive epochs, at least one epoch has an honest leader. Under $f < n / (3::'a)$, the probability of *fairness-bound* consecutive Byzantine leaders is $(f / n)^{fairness\text{-}bound} < (1 / (3::'a))^{fairness\text{-}bound}$.

For concrete instantiation, we need a leader schedule and pending count function that satisfy the locale assumptions. These are provided as parameters in the instantiation context.

locale *dquencer-liveness* = *dquencer-system* +

fixes *leader-schedule* :: *nat* ⇒ *node-info*
and *pending-count* :: *nat* ⇒ *nat*
assumes *fair-leader*:
$\forall$ *epoch*. $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* $<$ *epoch* + *fairness-bound* $\wedge$
ni-behavior (*leader-schedule e*) = *Honest*
and *honest-processes*:
⟦ *ni-behavior* (*leader-schedule e*) = *Honest*; *pending-count e* $>$ *0* ⟧
$\Longrightarrow$ *pending-count* (*Suc e*) $<$ *pending-count e*
and *pending-non-increasing*:
pending-count (*Suc e*) $\leq$ *pending-count e*
begin

interpretation *dq-fair*: *fair-leader-system*
leader-schedule λ*n*. *ni-behavior n* = *Honest pending-count fairness-bound*
proof *unfold-locales*
show *0* $<$ *fairness-bound* **using** *fairness-positive* **.**
next
show $\forall$ *epoch*. $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* $<$ *epoch* + *fairness-bound* $\wedge$
ni-behavior (*leader-schedule e*) = *Honest*
using *fair-leader* **.**
next
fix *e*
assume *ni-behavior* (*leader-schedule e*) = *Honest 0* $<$ *pending-count e*
then show *pending-count* (*Suc e*) $<$ *pending-count e*
using *honest-processes* **by** *auto*
next
fix *e*
show *pending-count* (*Suc e*) $\leq$ *pending-count e*
using *pending-non-increasing* **.**
qed

Concrete starvation freedom for the D-quencer: if there are pending regulatory requests, at least one will be processed within `fairness_bound` epochs.

corollary *dq-starvation-bound*:
assumes *pending-count epoch* $>$ *0*
shows $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* $<$ *epoch* + *fairness-bound* $\wedge$
pending-count (*Suc e*) $<$ *pending-count e*
using *dq-fair.starvation-bound*[*OF assms*] **.**

end

29 Combined Safety and Liveness

This section records how the safety result of Property 1 (cross-domain state preservation) sits alongside the liveness results of Property 2 (deterministic selection, timeout-bounded locking, fair scheduling).

The theorem below, `conditional_safety_preservation`, is a *condi-*

tional safety statement: from a valid global state in which the target asset is unlocked and a transition is enabled, the synchronization function succeeds and preserves the global validity invariant. Its proof uses only the safety side (`lock_acquire_success` and `valid_state_preservation`); it does not invoke the liveness interpretations (`dq_priority`, `dq_fair`) and makes no Byzantine, determinism, deadlock, or starvation claim of its own. Those liveness properties are the separate theorems of this theory (`dq_select_highest_deterministic` and `dq_starvation_bound`), each stated in its own right.

The genuine fusion of the two sides — safety made *unconditional* by a well-founded progress measure on cross-chain inconsistency, under the fair-leader assumption — is `oraclizer_guarded_bounded_convergence` in `Functor_Laws.thy`, which drops the initial-validity hypothesis assumed here.

The statement below is a leaf corollary: it restates `valid_state_preservation` under the explicit precondition that the asset is unlocked — the precondition the lock discipline is designed to establish. It is deliberately not the place where liveness and safety are fused (see `Functor_Laws.thy`).

theorem *conditional-safety-preservation*:
  assumes *valid*: *valid-state gs*
    and *exists*: *asset-exists gs source aid*
    and *current*: *get-reg-state gs source aid* = *Some s*
    and *trans*: *reg-transition s action* = *Some s′*
    and *not-locked*: ¬ *is-locked gs aid*
    and *fin*: *finite* (*connected-chains gs aid*)
  shows ∃ *gs′*. *sync source action aid gs* = *Some gs′* ∧ *valid-state gs′*
proof −
  from *not-locked* **obtain** *gs-locked* **where**
    lock: *acquire-lock gs aid* = *Some gs-locked*
    using *lock-acquire-success* **by** *auto*
  from *valid current trans lock* **have**
    ∃ *gs′*. *sync source action aid gs* = *Some gs′*
    unfolding *sync-def*
    using *exists current trans lock*
    by (*auto simp*: *get-reg-state-def get-asset-state-def*
        split: *option.splits*
        intro!: *exI*)
  then obtain *gs′* **where** *synced*: *sync source action aid gs* = *Some gs′*
    by *auto*
  have *valid-state gs′*
    using *valid-state-preservation*[*OF valid exists current trans synced fin*] **.**
  with *synced* **show** *?thesis* **by** *auto*
qed

Summary of what Properties 1 + 2 together establish:

1. **Safety** (Property 1): After synchronization, all connected chains agree on the regulatory state. The global validity invariant is preserved.

2. **Determinism** (Property 2): Conflicting regulatory actions are resolved by a total order on priority keys. The BFT consensus output is unique. The `priority_system` locale is instantiated on the `priority_key` type as carrier (with the corresponding messages recovered inside `dquencer_priority_concrete` via `recover_msg`), yielding deterministic selection from any finite non-empty set of valid candidate messages.

3. **Deadlock** (Property 2, scope note): deadlock is a concurrency phenomenon. The atomic sync model has no concurrent lock contention (lock holding is a boolean without contended holders), so deadlock does not arise within the model's scope; forced lock release under contention is out of scope, deferred to the preemptive-lock property. The theory states no proved deadlock-freedom theorem.

4. **Starvation freedom** (Property 2): Under the fair leader assumption, every pending regulatory request is processed within a bounded number of epochs.

   Open work for subsequent entries: - Compositional assurance across heterogeneous verification regimes (Canton + OSS + EVM). - Refinement: formal correspondence between these models and the Rust / Solidity implementation.

end

theory *Composition*
  imports *State-Preservation*
begin

30 Guarded Invariants

A guarded transition system: a carrier set closed under stepping, together with an invariant that is preserved by every *guarded* step. The guard isolates exactly the operations that may fire while maintaining the invariant.

locale *guarded-invariant* =
  fixes *carrier* :: $'s$ *set*
    and *step* :: $'s \Rightarrow 'op \Rightarrow 's$ *option*
    and *ops* :: $'op$ *set*
    and *inv* :: $'s \Rightarrow bool$
    and *guard* :: $'s \Rightarrow 'op \Rightarrow bool$
  assumes *step-closed*:
    $[\![\, s \in carrier;\ opn \in ops;\ step\ s\ opn = Some\ s' \,]\!] \Longrightarrow s' \in carrier$
    and *guarded-preservation*:
    $[\![\, s \in carrier;\ opn \in ops;\ inv\ s;\ guard\ s\ opn;\ step\ s\ opn = Some\ s' \,]\!]$
    $\Longrightarrow inv\ s'$

31 Eventual Dischargers

A scheduler that, within every window of length *window*, produces at least one discharging event. This is a bounded-fairness abstraction: progress-bearing events are never starved beyond *window* time units.

locale *eventual-discharger* =
  fixes *schedule* :: *nat* ⇒ *'event*
    and *discharges* :: *'event* ⇒ *bool*
    and *window* :: *nat*
  assumes *window-positive*: *window* > *0*
    and *bounded-occurrence*:
      $\forall\, t.\ \exists\, t'.\ t \leq t' \wedge t' < t + \mathit{window} \wedge \mathit{discharges}\ (\mathit{schedule}\ t')$

32 Compositional Safety and Liveness

The composition couples a guarded invariant with an eventual discharger through a realization map *realize* that turns a discharging event, in the current state, into an operation. The two coupling assumptions state that (i) any realized operation is admissible and guarded, and (ii) from an invariant-satisfying state a realized operation makes invariant-preserving progress.

locale *safety-liveness-composition* =
  safe: *guarded-invariant carrier step ops inv guard* +
  live: *eventual-discharger schedule discharges window*
  for *carrier* :: *'s set*
    and *step* :: *'s* ⇒ *'op* ⇒ *'s option*
    and *ops* :: *'op set*
    and *inv* :: *'s* ⇒ *bool*
    and *guard* :: *'s* ⇒ *'op* ⇒ *bool*
    and *schedule* :: *nat* ⇒ *'event*
    and *discharges* :: *'event* ⇒ *bool*
    and *window* :: *nat* +
  fixes *realize* :: *'event* ⇒ *'s* ⇒ *'op option*
  assumes *realize-discharges*:
      $[\![\ s \in \mathit{carrier};\ \mathit{discharges}\ \mathit{ev};\ \mathit{realize}\ \mathit{ev}\ s = \mathit{Some}\ \mathit{opn}\ ]\!]$
       $\Longrightarrow \mathit{opn} \in \mathit{ops} \wedge \mathit{guard}\ s\ \mathit{opn}$
    and *realize-progresses*:
      $[\![\ s \in \mathit{carrier};\ \mathit{inv}\ s;\ \mathit{discharges}\ \mathit{ev};\ \mathit{realize}\ \mathit{ev}\ s = \mathit{Some}\ \mathit{opn}\ ]\!]$
       $\Longrightarrow \exists\, s'.\ \mathit{step}\ s\ \mathit{opn} = \mathit{Some}\ s' \wedge \mathit{inv}\ s'$
begin

One evolution step at absolute time index *t*: a scheduled discharging event is realized and applied; any non-discharging event (or a realization that does not enable a step) stutters, leaving the state unchanged.

definition *evolve-step* :: *nat* ⇒ *'s* ⇒ *'s* **where**
  evolve-step t s =
    (*if discharges* (*schedule t*) *then*
      (*case realize* (*schedule t*) *s of*

None ⇒ *s*
| *Some opn* ⇒ (*case step s opn of None* ⇒ *s* | *Some s′* ⇒ *s′*))
else s)

The trajectory of length n started at time index k.

fun *run-from* :: *nat* ⇒ *′s* ⇒ *nat* ⇒ *′s* **where**
run-from k s 0 = *s*
| *run-from k s* (*Suc n*) = *evolve-step* (*k* + *n*) (*run-from k s n*)

The trajectory started at time *0*, and the induced evolution relation.

definition *run* :: *′s* ⇒ *nat* ⇒ *′s* **where**
run s n = *run-from 0 s n*

definition *evolves-to* :: *′s* ⇒ *nat* ⇒ *′s* ⇒ *bool* **where**
evolves-to s t s′ ⟷ *run s t* = *s′*

Trajectories compose: continuing for b more steps re-roots the schedule index by a.

lemma *run-from-add*:
run-from k s (*a* + *b*) = *run-from* (*k* + *a*) (*run-from k s a*) *b*
proof (*induction b*)
case *0*
show *?case* **by** *simp*
next
case (*Suc b*)
have *run-from k s* (*a* + *Suc b*) = *evolve-step* (*k* + (*a* + *b*)) (*run-from k s* (*a* + *b*))
by *simp*
also have ... = *evolve-step* (*k* + (*a* + *b*)) (*run-from* (*k* + *a*) (*run-from k s a*) *b*)
using *Suc.IH* **by** *simp*
also have ... = *run-from* (*k* + *a*) (*run-from k s a*) (*Suc b*)
by (*simp add*: *add.assoc*)
finally show *?case* **.**
qed

lemma *evolve-step-stutter*:
¬ *discharges* (*schedule t*) ⟹ *evolve-step t s* = *s*
by (*simp add*: *evolve-step-def*)

A single evolution step keeps the carrier.

lemma *evolve-step-carrier*:
assumes *s* ∈ *carrier*
shows *evolve-step t s* ∈ *carrier*
proof (*cases discharges* (*schedule t*))
case *False*
then show *?thesis* **using** *assms* **by** (*simp add*: *evolve-step-def*)
next
case *True*

```
  show ?thesis
  proof (cases realize (schedule t) s)
    case None
    with assms True show ?thesis by (simp add: evolve-step-def)
  next
    case (Some opn)
    note r = this
    show ?thesis
    proof (cases step s opn)
      case None
      with assms True r show ?thesis by (simp add: evolve-step-def)
    next
      case (Some s')
      note st = this
      have op-in: opn ∈ ops using realize-discharges[OF assms True r] by simp
      have s' ∈ carrier using safe.step-closed[OF assms op-in st] .
      with True r st show ?thesis by (simp add: evolve-step-def)
    qed
  qed
qed
```

A single evolution step preserves the invariant on the carrier.

```
lemma evolve-step-inv:
  assumes s ∈ carrier and inv s
  shows inv (evolve-step t s)
proof (cases discharges (schedule t))
  case False
  then show ?thesis using assms by (simp add: evolve-step-def)
next
  case True
  show ?thesis
  proof (cases realize (schedule t) s)
    case None
    with assms True show ?thesis by (simp add: evolve-step-def)
  next
    case (Some opn)
    note r = this
    show ?thesis
    proof (cases step s opn)
      case None
      with assms True r show ?thesis by (simp add: evolve-step-def)
    next
      case (Some s')
      note st = this
      have c: opn ∈ ops ∧ guard s opn using realize-discharges[OF assms(1) True
r] .
      have inv s'
        using safe.guarded-preservation[OF assms(1) conjunct1[OF c] assms(2)
conjunct2[OF c] st] .
```

```
    with True r st show ?thesis by (simp add: evolve-step-def)
   qed
  qed
qed

lemma run-from-carrier:
  assumes s ∈ carrier
  shows run-from k s n ∈ carrier
proof (induction n)
  case 0
  show ?case using assms by simp
next
  case (Suc n)
  show ?case using evolve-step-carrier[OF Suc.IH] by simp
qed

lemma run-from-inv:
  assumes s ∈ carrier and inv s
  shows inv (run-from k s n)
proof (induction n)
  case 0
  show ?case using assms(2) by simp
next
  case (Suc n)
  have car: run-from k s n ∈ carrier using run-from-carrier[OF assms(1)] .
  have inv (evolve-step (k + n) (run-from k s n)) using evolve-step-inv[OF car
Suc.IH] .
  then show ?case by simp
qed
```

If no discharging event occurs in the first n slots from k, the state is unchanged.

```
lemma run-from-stutter:
  assumes ∀ j<n. ¬ discharges (schedule (k + j))
  shows run-from k s n = s
  using assms
proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have ∀ j<n. ¬ discharges (schedule (k + j)) using Suc.prems by simp
  then have ih: run-from k s n = s using Suc.IH by simp
  have ¬ discharges (schedule (k + n)) using Suc.prems by simp
  then have evolve-step (k + n) (run-from k s n) = run-from k s n
    by (simp add: evolve-step-stutter)
  then show ?case using ih by simp
qed
```

Invariant preservation along the whole trajectory: from an invariant carrier state, every reachable state still satisfies the invariant and stays in the carrier.

lemma *invariant-preserved*:
  assumes $s \in carrier$ **and** *inv s*
  shows *inv* (*run s t*) $\wedge$ *run s t* $\in$ *carrier*
proof −
  have *inv* (*run-from 0 s t*) **using** *run-from-inv*[*OF assms*] **.**
  moreover have *run-from 0 s t* $\in$ *carrier* **using** *run-from-carrier*[*OF assms*(*1*)]
.
  ultimately show *?thesis* **by** (*simp add*: *run-def*)
qed

The eventuality form retained under the convergence fallback: from an invariant carrier state the system stays, at every horizon, in an invariant carrier state.

theorem *unconditional-invariant-eventuality*:
  assumes $s \in carrier$ **and** *inv s*
  shows $\exists\, t\ s'.$ *evolves-to s t s'* $\wedge$ *inv s'* $\wedge$ $s' \in carrier$
proof −
  have *inv* (*run s 0*) $\wedge$ *run s 0* $\in$ *carrier* **using** *invariant-preserved*[*OF assms*] **by** *blast*
  then have *evolves-to s 0* (*run s 0*) $\wedge$ *inv* (*run s 0*) $\wedge$ *run s 0* $\in$ *carrier*
    by (*simp add*: *evolves-to-def*)
  then show *?thesis* **by** *blast*
qed

Liveness within the safe region: from an invariant carrier state, a realized discharging operation makes progress to another invariant carrier state. This couples *realize-progresses* with closure and admissibility.

lemma *safe-region-progress*:
  assumes $s \in carrier$ **and** *inv s* **and** *discharges ev* **and** *realize ev s* = *Some opn*
  shows $\exists\, s'.$ *step s opn* = *Some s'* $\wedge$ *inv s'* $\wedge$ $s' \in carrier$
proof −
  obtain *s'* **where** *st*: *step s opn* = *Some s'* **and** *inv'*: *inv s'*
    using *realize-progresses*[*OF assms*(*1*,*2*,*3*,*4*)] **by** *blast*
  have *op-in*: $opn \in ops$ **using** *realize-discharges*[*OF assms*(*1*,*3*,*4*)] **by** *simp*
  have $s' \in carrier$ **using** *safe.step-closed*[*OF assms*(*1*) *op-in st*] **.**
  with *st inv'* **show** *?thesis* **by** *blast*
qed

end

33 Guarded Bounded Convergence

safety-liveness-composition is extended with a natural-number progress measure whose zero set is contained in the invariant (*measure-zero-inv*) and which every discharging step strictly decreases while the invariant fails (*discharge-progresses*). This is exactly the "well-founded measure connecting non-invariant states to invariant states via discharging events" that drives convergence; well-foundedness is supplied by the natural-number order (*measure progress-measure*).

Together with bounded fairness this yields convergence to the invariant within *progress-measure s ∗ window* steps from an arbitrary carrier state, with *no* assumption that the invariant holds initially.

locale *converging-composition* =
  safety-liveness-composition carrier step ops inv guard schedule discharges window realize
  for *carrier* :: *′s set*
    and *step* :: *′s* ⇒ *′op* ⇒ *′s option*
    and *ops* :: *′op set*
    and *inv* :: *′s* ⇒ *bool*
    and *guard* :: *′s* ⇒ *′op* ⇒ *bool*
    and *schedule* :: *nat* ⇒ *′event*
    and *discharges* :: *′event* ⇒ *bool*
    and *window* :: *nat*
    and *realize* :: *′event* ⇒ *′s* ⇒ *′op option* +
  fixes *progress-measure* :: *′s* ⇒ *nat*
  assumes *measure-zero-inv*:
    ⟦ *s* ∈ *carrier*; *progress-measure s* = *0* ⟧ ⟹ *inv s*
  and *discharge-progresses*:
    ⟦ *s* ∈ *carrier*; ¬ *inv s*; *discharges ev* ⟧
     ⟹ ∃ *opn s′*. *realize ev s* = *Some opn* ∧ *step s opn* = *Some s′*
         ∧ *progress-measure s′* < *progress-measure s*
begin

definition *convergence-bound* :: *′s* ⇒ *nat* **where**
  convergence-bound s = *progress-measure s* ∗ *window*

lemma *progress-measure-wf*: *wf* (*measure progress-measure*)
  by (*rule wf-measure*)

The core convergence lemma, by strong induction on the measure. From any carrier state with measure *m*, an invariant state is reached within *m ∗ window* steps, regardless of the starting time index *k*.

lemma *convergence-bound-reachable*:
  progress-measure s = *m* ⟹ *s* ∈ *carrier*
    ⟹ ∃ *t* ≤ *m* ∗ *window*. *inv* (*run-from k s t*)
proof (*induction m arbitrary*: *s k rule*: *less-induct*)
  case (*less m*)
  show *?case*
  proof (*cases inv s*)
    case *True*
    have *inv* (*run-from k s 0*) **using** *True* **by** *simp*
    moreover have (*0*::*nat*) ≤ *m* ∗ *window* **by** *simp*
    ultimately show *?thesis* **by** *blast*
  next

case *False*
have *m-eq*: *progress-measure s* = *m* **using** *less.prems*(*1*) **.**
have *car*: *s* ∈ *carrier* **using** *less.prems*(*2*) **.**
— Locate the least discharging offset within the fairness window.
obtain *t′* **where** *t′1*: *k* ≤ *t′* **and** *t′2*: *t′* < *k* + *window*
and *t′3*: *discharges* (*schedule t′*)
using *live.bounded-occurrence* **by** *blast*
define *Q* **where** *Q* = (*λj. j* < *window* ∧ *discharges* (*schedule* (*k* + *j*)))
have *w*: *t′* − *k* < *window* **using** *t′1 t′2* **by** *linarith*
have *kk*: *k* + (*t′* − *k*) = *t′* **using** *t′1* **by** *simp*
have *Qwit*: *Q* (*t′* − *k*) **unfolding** *Q-def* **using** *w t′3 kk* **by** *simp*
define *j0* **where** *j0* = (*LEAST j. Q j*)
have *Qj0*: *Q j0* **unfolding** *j0-def* **using** *Qwit* **by** (*rule LeastI*[**where** *P* = *Q*])
have *j0w*: *j0* < *window* **using** *Qj0* **unfolding** *Q-def* **by** *simp*
have *disc0*: *discharges* (*schedule* (*k* + *j0*)) **using** *Qj0* **unfolding** *Q-def* **by** *simp*
have *before*: ∀ *j*<*j0*. ¬ *discharges* (*schedule* (*k* + *j*))
proof (*intro allI impI*)
fix *j* **assume** *jl*: *j* < *j0*
have *lt*: *j* < *Least Q* **using** *jl* **by** (*simp add*: *j0-def*)
have *nQ*: ¬ *Q j* **using** *not-less-Least*[*OF lt*] **.**
have *jw*: *j* < *window* **using** *jl j0w* **by** *simp*
show ¬ *discharges* (*schedule* (*k* + *j*)) **using** *nQ jw* **by** (*simp add*: *Q-def*)
qed
have *stut*: *run-from k s j0* = *s* **using** *run-from-stutter*[*OF before*] **.**
— The discharging step at offset *j0* strictly decreases the measure.
obtain *opn s′* **where** *r*: *realize* (*schedule* (*k* + *j0*)) *s* = *Some opn*
and *st*: *step s opn* = *Some s′*
and *dec*: *progress-measure s′* < *progress-measure s*
using *discharge-progresses*[*OF car False disc0*] **by** *blast*
have *op-in*: *opn* ∈ *ops* **using** *realize-discharges*[*OF car disc0 r*] **by** *simp*
have *s′-car*: *s′* ∈ *carrier* **using** *safe.step-closed*[*OF car op-in st*] **.**
have *ev*: *run-from k s* (*Suc j0*) = *s′*
proof −
have *run-from k s* (*Suc j0*) = *evolve-step* (*k* + *j0*) (*run-from k s j0*) **by** *simp*
also have . . . = *evolve-step* (*k* + *j0*) *s* **using** *stut* **by** *simp*
also have . . . = *s′* **using** *disc0 r st* **by** (*simp add*: *evolve-step-def*)
finally show *?thesis* **.**
qed
have *decm*: *progress-measure s′* < *m* **using** *dec m-eq* **by** *simp*
— Apply the induction hypothesis to the smaller-measure state *s′*.
have ∃ *t* ≤ *progress-measure s′* ∗ *window*. *inv* (*run-from* (*k* + *Suc j0*) *s′ t*)
using *less.IH*[*OF decm refl s′-car*] **by** *blast*
then obtain *t2* **where** *t2b*: *t2* ≤ *progress-measure s′* ∗ *window*
and *inv2*: *inv* (*run-from* (*k* + *Suc j0*) *s′ t2*) **by** *blast*
have *comp*: *run-from k s* (*Suc j0* + *t2*) = *run-from* (*k* + *Suc j0*) *s′ t2*
proof −
have *run-from k s* (*Suc j0* + *t2*)
= *run-from* (*k* + *Suc j0*) (*run-from k s* (*Suc j0*)) *t2*

```
      by (rule run-from-add)
    also have ... = run-from (k + Suc j0) s′ t2 by (simp only: ev)
    finally show ?thesis .
  qed
  have inv-fin: inv (run-from k s (Suc j0 + t2)) using comp inv2 by simp
  have Suc j0 + t2 ≤ window + progress-measure s′ ∗ window
    using j0w t2b by linarith
  also have ... = (1 + progress-measure s′) ∗ window
    by (simp add: add-mult-distrib)
  also have ... ≤ m ∗ window
  proof −
    have 1 + progress-measure s′ ≤ m using decm by linarith
    then show ?thesis by (rule mult-le-mono1)
  qed
  finally have Suc j0 + t2 ≤ m ∗ window .
  then show ?thesis using inv-fin by blast
 qed
qed
```

Guarded bounded convergence (Plan A). From an arbitrary carrier state with no assumption that the invariant holds the system reaches, within *convergence-bound s* evolution steps, a state satisfying the invariant.

```
theorem bounded-convergence-from-arbitrary:
  assumes s ∈ carrier
  shows ∃ t ≤ convergence-bound s. ∃ s′. evolves-to s t s′ ∧ inv s′
proof −
  obtain t where tb: t ≤ progress-measure s ∗ window
    and inv-t: inv (run-from 0 s t)
    using convergence-bound-reachable[OF refl assms] by blast
  have evolves-to s t (run s t) by (simp add: evolves-to-def)
  with tb inv-t show ?thesis
    by (auto simp: convergence-bound-def run-def evolves-to-def)
qed

end

end


theory Proof-Automation
  imports State-Preservation HOL−Eisbach.Eisbach
begin
```

34 Discharge Methods for Instance Obligations

Two named theorem collections feed the methods. *discharge-simps* holds the equational content of a concrete instance: defining equations of the state/action/terminal sets, finiteness facts, terminal-absorption equations, and naturality equations of structured state maps (conditional rewrites whose premises the simplifier discharges from the goal's assumptions). *discharge-intros* holds introduction rules whose extra premises must be instantiated by unification against the goal's assumptions — typically closure lemmas, whose condition variables do not all occur in the conclusion and which are therefore out of reach of conditional rewriting.

named-theorems *discharge-simps*
‹*defining equations and conditional rewrites consumed by the discharge methods*›

named-theorems *discharge-intros*
‹*introduction rules consumed by the discharge methods*›

named-theorems *discharge-dels*
‹*default simplification rules suppressed by the discharge methods ———
typically the defining equations of structured state maps introduced with
‹fun›, whose eager unfolding would destroy the redexes that the
conditional naturality rewrites of ‹discharge-simps› match on*›

The common skeleton: unfold the locale predicate into its atomic obligations (*unfold-locales* also discharges sub-locale obligations already available from registered interpretations), then close each remaining obligation by simplifier-driven exhaustive case analysis — membership of an action in an enumerated set splits into finitely many cases, after which the defining equations of the transition functions decide the goal, with *option*/*if* splits for partially defined transitions.

discharge-state-machine is the entry point for *state-machine* obligations (finiteness, terminal absorption, closure, domain); *discharge-preservation* is the entry point for *state-preservation* obligations, whose unfolding contains the two sub-machines' obligations together with the morphism axioms (well-definedness, terminal preservation, and the two naturality directions).

method *discharge-state-machine* **declares** *discharge-simps discharge-intros discharge-dels* =
(*unfold-locales*;
 (*auto simp add*: *discharge-simps simp del*: *discharge-dels*
   intro: *discharge-intros*
   split: *option.splits if-splits*))

method *discharge-preservation* **declares** *discharge-simps discharge-intros discharge-dels* =
(*unfold-locales*;
 (*auto simp add*: *discharge-simps simp del*: *discharge-dels*
   intro: *discharge-intros*
   split: *option.splits if-splits*))

The two entry points share their skeleton deliberately: a *state-preservation* goal unfolds into the union of the two obligation families, so the preservation method must subsume the machine method. Keeping both names separates the reusable surface by intent — an importer discharging a bare machine instance is not invited to reach for the morphism-level method — and lets the two evolve independently if the obligation families diverge in a future revision.

end

theory *Functor-Laws*
  imports
    Composition
    Regulatory-Instance
    DQuencer-Instance
    Proof-Automation
    ADS-Functor.Merkle-Interface
begin

35 Functor Laws on State-Preservation Morphisms

We regard the locale *state-preservation* as a morphism between two objects of a category. An object is a state machine (*states*, *actions*, *transition*, *terminal*); a morphism from one object to another is a pair (*state-map*, *action-map*) satisfying the well-definedness, terminal-preservation and naturality conditions of *state-preservation*. The following three theorems are the category axioms for this notion of morphism.

35.1 Identity morphism

The identity pair (*id*, *id*) is a state-preservation morphism from any object to itself: F sends identities to identities.

theorem *preservation-id*:
  assumes *state-machine states actions transition terminal*
  shows *state-preservation states actions transition terminal*
            states actions transition terminal id id
  using *assms*
  unfolding *state-preservation-def state-preservation-axioms-def*
  by *simp*

35.2 Composition of morphisms

The composite of two state-preservation morphisms is a state-preservation morphism: the naturality square of the second morphism is stacked on that of the first. This is the cross-domain analogue of "closed under composition".

theorem *preservation-compose*:
  assumes *f*: *state-preservation* S_a A_a T_a F_a S_b A_b T_b F_b $f\,f'$
    and *g*: *state-preservation* S_b A_b T_b F_b S_c A_c T_c F_c $g\;g'$

shows *state-preservation* S_a A_a T_a F_a S_c A_c T_c F_c $(g \circ f)$ $(g' \circ f')$
proof −
interpret *f*: *state-preservation* S_a A_a T_a F_a S_b A_b T_b F_b f f' **by** (*rule f*)
interpret *g*: *state-preservation* S_b A_b T_b F_b S_c A_c T_c F_c g g' **by** (*rule g*)
show *?thesis*
proof (*unfold-locales*)
fix *s* **assume** $s \in S_a$
then show $(g \circ f)$ $s \in S_c$
by (*simp add*: *f.state-map-well-defined g.state-map-well-defined*)
next
fix *a* **assume** $a \in A_a$
then show $(g' \circ f')$ $a \in A_c$
by (*simp add*: *f.action-map-well-defined g.action-map-well-defined*)
next
fix *s* **assume** $s \in F_a$
then show $(g \circ f)$ $s \in F_c$
by (*simp add*: *f.terminal-preservation g.terminal-preservation*)
next
fix *s a s'*
assume *A*: $s \in S_a$ **and** *B*: $a \in A_a$ **and** *C*: T_a s a = *Some* s'
have *fs*: f $s \in S_b$ **using** *A f.state-map-well-defined* **by** *simp*
have *fa*: f' $a \in A_b$ **using** *B f.action-map-well-defined* **by** *simp*
have T_b $(f\ s)$ $(f'\ a)$ = *Some* $(f\ s')$ **using** *f.naturality A B C* **by** *simp*
then have T_c $(g\ (f\ s))$ $(g'\ (f'\ a))$ = *Some* $(g\ (f\ s'))$
using *g.naturality fs fa* **by** *simp*
then show T_c $((g \circ f)\ s)$ $((g' \circ f')\ a)$ = *Some* $((g \circ f)\ s')$
by (*simp add*: *comp-def*)
next
fix *s a*
assume *A*: $s \in S_a$ **and** *B*: $a \in A_a$ **and** *C*: T_a s a = *None*
have *fs*: f $s \in S_b$ **using** *A f.state-map-well-defined* **by** *simp*
have *fa*: f' $a \in A_b$ **using** *B f.action-map-well-defined* **by** *simp*
have T_b $(f\ s)$ $(f'\ a)$ = *None* **using** *f.naturality-none A B C* **by** *simp*
then have T_c $(g\ (f\ s))$ $(g'\ (f'\ a))$ = *None*
using *g.naturality-none fs fa* **by** *simp*
then show T_c $((g \circ f)\ s)$ $((g' \circ f')\ a)$ = *None*
by (*simp add*: *comp-def*)
qed
qed

35.3 Associativity of morphism composition

Composition of state-preservation morphisms is associative on both the state and the action components. Since the components are functions, this is exactly associativity of function composition; the three morphism hypotheses fix the objects across which the equation is read, completing the category axioms (identity, composition, associativity).

theorem *preservation-assoc*:

assumes *state-preservation* S_a A_a T_a F_a S_b A_b T_b F_b f f'
  and *state-preservation* S_b A_b T_b F_b S_c A_c T_c F_c g g'
  and *state-preservation* S_c A_c T_c F_c S_d A_d T_d F_d k k'
shows $((k \circ g) \circ f = k \circ (g \circ f)) \wedge ((k' \circ g') \circ f' = k' \circ (g' \circ f'))$
by (*simp add*: *comp-assoc*)

36 Composition Exercised: A Heterogeneous Three-Domain Chain

preservation-compose is exercised on a concrete chain of three distinct domains: the off-chain DAML permission ledger (structured records), the on-chain regulatory enum, and Chain B with its dedicated four-action vocabulary. In HOL the morphisms of a heterogeneous chain cannot be packed into one object — the domains carry genuinely different state and action types — so the correct formal expression of an N-domain chain is exactly the composite of pairwise morphisms, with *preservation-assoc* making longer path composites unambiguous. Both legs are scoped to the escalation subset of the action vocabulary: the DAML machine restricts a fortiori (a machine restricted to a subset of its action set is again a machine), and the layer-crossing naturality of *daml-to-reg* holds on the subset because it holds on the full vocabulary.

lemma *daml-escalation-state-machine*:
  state-machine daml-states escalation-actions daml-transition daml-terminal
proof *unfold-locales*
  show *finite daml-states* **by** (*rule daml-states-finite*)
next
  show *finite escalation-actions* **unfolding** *escalation-actions-def* **by** *simp*
next
  show *daml-terminal* $\subseteq$ *daml-states* **by** (*rule daml-terminal-subset*)
next
  fix p a
  assume $p \in$ *daml-terminal* **and** $a \in$ *escalation-actions*
  then show *daml-transition* p a = *None*
    using *daml-terminal-absorbing reg-actions-UNIV* **by** *auto*
next
  fix p a p'
  assume $p \in$ *daml-states* **and** $a \in$ *escalation-actions*
    and *daml-transition* p a = *Some* p'
  then show $p' \in$ *daml-states*
    using *daml-transition-closed reg-actions-UNIV* **by** *auto*
next
  fix p :: *daml-perm* **and** a :: *reg-action*
  assume $p \notin$ *daml-states*
  then show *daml-transition* p a = *None* **by** (*rule daml-transition-outside-states*)
qed

The escalation-scoped backward bridge: *daml-to-reg* as a morphism from the DAML machine to the regulatory machine, both restricted to the escalation actions.

lemma *daml-to-reg-escalation-bridge*:
state-preservation daml-states escalation-actions daml-transition daml-terminal reg-states escalation-actions reg-transition reg-terminal daml-to-reg id
proof *unfold-locales*
— Source state-machine obligations (the DAML machine on the escalation subset is not registered, so its six obligations remain pending); the target machine coincides with the source machine of the registered *escalation-preservation* interpretation and is discharged automatically.
show *finite daml-states* **by** (*rule daml-states-finite*)
next
show *finite escalation-actions* **unfolding** *escalation-actions-def* **by** *simp*
next
show *daml-terminal* $\subseteq$ *daml-states* **by** (*rule daml-terminal-subset*)
next
fix p a
assume $p \in$ *daml-terminal* **and** $a \in$ *escalation-actions*
then show *daml-transition* p a = *None*
using *daml-terminal-absorbing reg-actions-UNIV* **by** *auto*
next
fix p a p'
assume $p \in$ *daml-states* **and** $a \in$ *escalation-actions*
and *daml-transition* p a = *Some* p'
then show $p' \in$ *daml-states*
using *daml-transition-closed reg-actions-UNIV* **by** *auto*
next
fix p :: *daml-perm* **and** a :: *reg-action*
assume $p \notin$ *daml-states*
then show *daml-transition* p a = *None* **by** (*rule daml-transition-outside-states*)
next
fix p
assume $p \in$ *daml-states*
then show *daml-to-reg* $p \in$ *reg-states* **using** *reg-states-UNIV* **by** *auto*
next
fix a
assume $a \in$ *escalation-actions*
then show *id* $a \in$ *escalation-actions* **by** *simp*
next
fix p
assume $p \in$ *daml-terminal*
then have p = *reg-to-daml CONFISCATED* **unfolding** *daml-terminal-def* **by** *simp*
then have *daml-to-reg* p = *CONFISCATED* **using** *daml-to-reg-to-daml-id* **by** *simp*
then show *daml-to-reg* $p \in$ *reg-terminal* **unfolding** *reg-terminal-def* **by** *simp*
next
fix p a p'

assume $p \in$ *daml-states* **and** $a \in$ *escalation-actions*
  and *daml-transition p a* = *Some p′*
then show *reg-transition* (*daml-to-reg p*) (*id a*) = *Some* (*daml-to-reg p′*)
  using *daml-to-reg-naturality-some reg-actions-UNIV* **by** *auto*
next
fix *p a*
assume $p \in$ *daml-states* **and** $a \in$ *escalation-actions*
  and *daml-transition p a* = *None*
then show *reg-transition* (*daml-to-reg p*) (*id a*) = *None*
  using *daml-to-reg-naturality-none reg-actions-UNIV* **by** *auto*
qed

The escalation morphism of *Cross-Domain-State-Preservation.Regulatory-Instance* in predicate (bundle) form, so that it can be fed to *preservation-compose*.

lemma *escalation-preservation-bundle*:
state-preservation reg-states escalation-actions reg-transition reg-terminal
  reg-states chain-b-actions chain-b-transition reg-terminal
  id escalation-action-map
— This very instance is registered as the *escalation-preservation* interpretation of *Cross-Domain-State-Preservation.Regulatory-Instance*, so every obligation is discharged from the registration.
by *unfold-locales*

The composed three-domain morphism: DAML permission records, synchronized through the regulatory enum, arrive on Chain B's vocabulary. The composite is literally *preservation-compose* applied to the two legs; no new naturality argument is needed, which is the point of the functor laws. Arbitrary finite heterogeneous chains are expressed the same way, as composites of pairwise morphisms.

theorem *daml-to-chain-b-composed*:
state-preservation daml-states escalation-actions daml-transition daml-terminal
  reg-states chain-b-actions chain-b-transition reg-terminal
  (*id* ∘ *daml-to-reg*) (*escalation-action-map* ∘ *id*)
using *preservation-compose*[*OF daml-to-reg-escalation-bridge*
  escalation-preservation-bundle] **.**

The domain guard of the layer-crossing bridge is load-bearing, not a notational convenience. The record below carries the *D-Seized* tag with no seizing party: it violates *valid-daml-perm* and lies outside *daml-states*. The guarded DAML transition rejects *RELEASE* on it, while the bare enum projection of the same record would accept it — so dropping the guard would break naturality on exactly such records. Preservation holds on the carved-out domain and only there; the guard is the boundary of the preservation guarantee, recorded here permanently as a machine-checked witness.

definition *invalid-daml-perm* :: *daml-perm* **where**
invalid-daml-perm =
  (| *status-tag* = *D-Seized*, *seized-by* = *None*, *restriction-scope* = *None* |)

lemma *guard-is-load-bearing*:
  ¬ *valid-daml-perm invalid-daml-perm*
  invalid-daml-perm ∉ *daml-states*
  daml-transition invalid-daml-perm RELEASE = *None*
  reg-transition (*daml-to-reg invalid-daml-perm*) *RELEASE* = *Some ACTIVE*
proof −
  show ¬ *valid-daml-perm invalid-daml-perm*
    by (*simp add*: *valid-daml-perm-def invalid-daml-perm-def*)
  have $\bigwedge$*s*. *invalid-daml-perm* ≠ *reg-to-daml s*
  proof −
    fix *s* **show** *invalid-daml-perm* ≠ *reg-to-daml s*
      by (*cases s*) (*auto simp*: *invalid-daml-perm-def*)
  qed
  then show *notin*: *invalid-daml-perm* ∉ *daml-states*
    by (*auto simp*: *daml-states-def*)
  from *notin* **show** *daml-transition invalid-daml-perm RELEASE* = *None*
    by (*rule daml-transition-outside-states*)
  show *reg-transition* (*daml-to-reg invalid-daml-perm*) *RELEASE* = *Some AC-TIVE*
    by (*simp add*: *invalid-daml-perm-def*)
qed

37 Proof Automation Exercised on the Regulatory Instances

The discharge methods of *Cross-Domain-State-Preservation.Proof-Automation* are fed here with the equational content of the regulatory, Chain B and DAML domains, and then exercised: each *-via-method* lemma below restates a preservation or machine instance — proved manually above or registered in *Cross-Domain-State-Preservation.Regulatory-Instance* — and discharges it with a single method invocation, giving a one-to-one comparison between the manual proofs and the automated ones. The instance obligations are not weakened in any way: the statements are identical to their manual counterparts.

Finiteness of the enumerated regulatory types, in the normal form the simplifier reaches after rewriting the set definitions to *UNIV*.

lemma *finite-reg-state-UNIV* [*discharge-simps*]: *finite* (*UNIV* :: *reg-state set*)
  using *reg-sm.finite-states* **by** (*simp add*: *reg-states-UNIV*)

lemma *finite-reg-action-UNIV* [*discharge-simps*]: *finite* (*UNIV* :: *reg-action set*)
  using *reg-sm.finite-actions* **by** (*simp add*: *reg-actions-UNIV*)

Terminal and well-definedness helpers for the layer-crossing maps, in conditional-rewrite form.

lemma *daml-to-reg-terminal-val* [*discharge-simps*]:
  p ∈ *daml-terminal* ⟹ *daml-to-reg p* = *CONFISCATED*

by (*auto simp*: *daml-terminal-def daml-to-reg-to-daml-id*)

lemma *reg-to-daml-terminal* [*discharge-simps*]:
s ∈ *reg-terminal* ⟹ *reg-to-daml s* ∈ *daml-terminal*
by (*auto simp*: *reg-terminal-def daml-terminal-def*)

lemma *reg-to-daml-in-states* [*discharge-simps*]:
reg-to-daml s ∈ *daml-states*
by (*simp add*: *daml-states-def*)

The collections. Equational and conditional-rewrite content goes to *discharge-simps*; closure rules, whose condition variables do not all occur in their conclusions, go to *discharge-intros*.

lemmas [*discharge-simps*] =
reg-states-UNIV reg-actions-UNIV
escalation-actions-def chain-b-actions-def reg-terminal-def
confiscated-terminal chain-b-terminal
daml-states-finite daml-terminal-subset
daml-terminal-absorbing daml-transition-outside-states
reg-to-daml-naturality-some reg-to-daml-naturality-none
daml-to-reg-naturality-some daml-to-reg-naturality-none
escalation-naturality-some escalation-naturality-none

lemmas [*discharge-intros*] =
reg-transition-closed chain-b-transition-closed daml-transition-closed

The layer-crossing maps are introduced with *fun*, so their defining equations are default simplification rules; the methods must keep the maps opaque so that the conditional naturality and terminal rewrites above can match their redexes.

lemmas [*discharge-dels*] =
daml-to-reg.simps reg-to-daml.simps

The comparison lemmas. The first three restate instances proved manually in this theory; the method rediscovers each proof from the collections alone (none of these instances is registered, so *unfold-locales* receives every obligation atomically).

lemma *daml-escalation-state-machine-via-method*:
state-machine daml-states escalation-actions daml-transition daml-terminal
by *discharge-state-machine*

lemma *daml-to-reg-escalation-bridge-via-method*:
state-preservation daml-states escalation-actions daml-transition daml-terminal
reg-states escalation-actions reg-transition reg-terminal
daml-to-reg id
by *discharge-preservation*

lemma *daml-to-chain-b-composed-via-method*:

state-preservation daml-states escalation-actions daml-transition daml-terminal
reg-states chain-b-actions chain-b-transition reg-terminal
(*id* ∘ *daml-to-reg*) (*escalation-action-map* ∘ *id*)
by *discharge-preservation*

The forward escalation-scoped bridge is a new instance with no manual counterpart: the method constructs it outright, which is the reuse claim in its sharpest form.

lemma *reg-to-daml-escalation-bridge-via-method*:
state-preservation reg-states escalation-actions reg-transition reg-terminal
daml-states escalation-actions daml-transition daml-terminal
reg-to-daml id
by *discharge-preservation*

The remaining three restate instances registered as interpretations in *Cross-Domain-State-Preservation.Regulatory-Instance*; for these, *unfold-locales* discharges the obligations from the registrations, so the one-line proofs are registration-backed rather than search-backed. The search-backed counterparts above show that the method does not depend on the registrations.

lemma *escalation-preservation-via-method*:
state-preservation reg-states escalation-actions reg-transition reg-terminal
reg-states chain-b-actions chain-b-transition reg-terminal
id escalation-action-map
by *discharge-preservation*

lemma *forward-layer-preservation-via-method*:
state-preservation reg-states reg-actions reg-transition reg-terminal
daml-states reg-actions daml-transition daml-terminal
reg-to-daml id
by *discharge-preservation*

lemma *backward-layer-preservation-via-method*:
state-preservation daml-states reg-actions daml-transition daml-terminal
reg-states reg-actions reg-transition reg-terminal
daml-to-reg id
by *discharge-preservation*

38 State Glue: Join and Refinement

The two glue predicates lift the authenticated-data-structure operations to the global-state level. *state-join sa sb sab* says that *sab* is the consistent combination of the partial views *sa* and *sb*: on every chain connected in either view, *sab* carries the value of whichever view knows the chain (preferring *sa*), the two views agree wherever both are defined, and *sab* reveals no chain beyond the union of the two views (the containment clause). This is the state-level reading of the ADS *join*: on revealed holdings *sab* is the least consistent combination of *sa* and *sb* — the lock components are unconstrained — and the agreement clause is the state-level reading of equal hashes (mergeability). Without the containment clause a candidate join could carry an arbitrary holding on a chain outside both views and still be accepted; the regression witness *rogue-join-excluded* below records that such a candidate is now rejected.

state-refines sa sb says that *sa* is a partial view of *sb*: every chain known to *sa* is known to *sb* with the same regulatory state. This is the state-level reading of the ADS blinding order *bo*: a more-blinded value agrees with a less-blinded one on everything it can observe.

definition *state-join* :: *global-state* ⇒ *global-state* ⇒ *global-state* ⇒ *bool* **where**
state-join sa sb sab ≡
(∀ *aid c.*
(*c* ∈ *connected-chains sa aid* ∨ *c* ∈ *connected-chains sb aid*)
⟶ *get-reg-state sab c aid* =
(*if c* ∈ *connected-chains sa aid then get-reg-state sa c aid*
else get-reg-state sb c aid))
∧ (∀ *aid c1 c2*.
c1 ∈ *connected-chains sa aid* ∧ *c2* ∈ *connected-chains sb aid*
⟶ *get-reg-state sa c1 aid* = *get-reg-state sb c2 aid*)
∧ (∀ *aid c.*
c ∈ *connected-chains sab aid*
⟶ *c* ∈ *connected-chains sa aid* ∨ *c* ∈ *connected-chains sb aid*)

definition *state-refines* :: *global-state* ⇒ *global-state* ⇒ *bool* **where**
state-refines sa sb ≡
∀ *aid c. c* ∈ *connected-chains sa aid*
⟶ *c* ∈ *connected-chains sb aid* ∧ *get-reg-state sa c aid* = *get-reg-state sb c aid*

Refinement is reflexive and transitive: a partial-view preorder on states.

lemma *state-refines-refl*: *state-refines sa sa*
unfolding *state-refines-def* **by** *simp*

lemma *state-refines-trans*:
state-refines sa sb ⟹ *state-refines sb sc* ⟹ *state-refines sa sc*
unfolding *state-refines-def* **by** *metis*

A consistent state refined by another transports its consistency upward: if *sa* refines a consistent *sb*, then *sa* is consistent.

lemma *state-refines-preserves-consistency*:
assumes *state-refines sa sb* **and** *consistent-state sb*
shows *consistent-state sa*
unfolding *consistent-state-def*
proof (*intro allI impI*)
fix *c1 c2 aid s1 s2*
assume *get-reg-state sa c1 aid* = *Some s1* **and** *get-reg-state sa c2 aid* = *Some s2*
moreover have *c1* ∈ *connected-chains sa aid* **and** *c2* ∈ *connected-chains sa aid*

using *calculation* **unfolding** *connected-chains-def asset-exists-def get-reg-state-def get-asset-state-def*
by (*auto split*: *option.splits*)
ultimately have *get-reg-state sb c1 aid* = *Some s1* **and** *get-reg-state sb c2 aid* = *Some s2*
using *assms*(*1*) **unfolding** *state-refines-def* **by** *metis*+
then show *s1* = *s2* **using** *assms*(*2*) **unfolding** *consistent-state-def* **by** *blast*
qed

39 Authenticated Cross-Domain State

An authenticated state structure equips a Merkle interface (*h*, *bo*, *m*) with an extraction map *extract-map* that reads a global state out of an authenticated value. The three coherence assumptions tie the authenticated layer to the state-glue layer:

* *extract-respects-merging*: merging hash-compatible authenticated values yields a state that is the join of the extracted views; * *extract-under-blinding*: a blinding of an authenticated value extracts to a refinement of the extracted state; * *extract-preserves-validity*: every extracted state is valid.

The merging and blinding assumptions are guarded by the hash-equality condition *h a* = *h b*, which is the ADS notion of mergeability. The two soundness theorems below discharge this condition from the ADS lemmas *merge-respects-hashes* and *hash*, and use *join* to exhibit the join as a refinement of each input so the Merkle interface is used in the proofs, not merely imported.

The extraction map is named *extract-map* rather than *extract*, because *extract* is a reserved Isabelle command keyword and cannot be used as a locale parameter name.

locale *authenticated-state* =
mk: *merkle-interface h bo m*
for *h* :: $'d \Rightarrow 'e$
and *bo* :: $'d \Rightarrow 'd \Rightarrow$ *bool*
and *m* :: $'d \Rightarrow 'd \Rightarrow 'd$ *option*
and *extract-map* :: $'d \Rightarrow$ *global-state option* +
assumes *extract-respects-merging*:
⟦ *h a* = *h b*; *m a b* = *Some ab*; *extract-map a* = *Some sa*; *extract-map b* = *Some sb* ⟧
⟹ ∃ *sab*. *extract-map ab* = *Some sab* ∧ *state-join sa sb sab*
and *extract-under-blinding*:
⟦ *h a* = *h b*; *bo a b*; *extract-map b* = *Some sb* ⟧
⟹ ∃ *sa*. *extract-map a* = *Some sa* ∧ *state-refines sa sb*
and *extract-preserves-validity*:
extract-map a = *Some s* ⟹ *valid-state s*
begin

Blinding preserves hashes: the explicit state-level use of the *mk.hash* lemma of the Merkle interface.

```
lemma bo-hash-eq:
  assumes bo x y
  shows h x = h y
proof −
  have vimage2p h h (=) x y using mk.hash assms by (rule predicate2D)
  then show ?thesis by (simp add: vimage2p-def)
qed
```

Soundness of authenticated preservation under merging. If two authenticated values merge, then their extracted states have a valid join, and that join refines each input view. The proof calls *merge-respects-hashes* (to certify mergeability as hash-equality), *join* (to obtain the blinding relation of each input to the merge), and *hash* (via *bo-hash-eq*, to transport that blinding relation to the extracted states).

```
theorem authenticated-preservation-soundness:
  assumes mab: m a b = Some ab
    and ea: extract-map a = Some sa and va: valid-state sa
    and eb: extract-map b = Some sb and vb: valid-state sb
  shows ∃ sab. extract-map ab = Some sab ∧ valid-state sab
            ∧ state-refines sa sab ∧ state-refines sb sab
proof −
  — ADS lemma 1: a merge exists, so the hashes agree (mergeability).
  have ex-merge: ∃ x. m a b = Some x using mab by blast
  have hab: h a = h b using mk.merge-respects-hashes ex-merge by blast
  — ADS lemma 2: the merge ab is the least upper bound of a and b (join).
  have lub: bo a ab ∧ bo b ab ∧ (∀ u. bo a u ⟶ bo b u ⟶ bo ab u)
    using mk.join mab by blast
  then have boa: bo a ab and bob: bo b ab by simp-all
  — State-level join from the merging assumption, using the hash equality hab.
  obtain sab where esab: extract-map ab = Some sab and sj: state-join sa sb sab
    using extract-respects-merging[OF hab mab ea eb] by blast
  have vsab: valid-state sab using extract-preserves-validity[OF esab] .
  — ADS lemma 3 (hash): each input blinds to ab, hence shares its hash, so the
extracted join refines each input view.
  have h a = h ab using bo-hash-eq[OF boa] .
  then have ra: state-refines sa sab
    using extract-under-blinding[OF ‹h a = h ab› boa esab] ea by auto
  have h b = h ab using bo-hash-eq[OF bob] .
  then have rb: state-refines sb sab
    using extract-under-blinding[OF ‹h b = h ab› bob esab] eb by auto
  from esab vsab ra rb show ?thesis by blast
qed
```

Blinded views preserve validity. A blinding a of an authenticated value b extracts to a valid state that refines the extraction of b. This is the need-to-know guarantee: a partial (blinded) view is still a valid, consistency-respecting view. The proof calls *hash* (via *bo-hash-eq*) to certify the hash equality that licenses the blinding coherence assumption.

theorem *blinded-view-preserves-validity*:
  assumes *boab*: *bo a b* **and** *eb*: *extract-map b = Some sb* **and** *vb*: *valid-state sb*
  shows $\exists$ *sa. extract-map a = Some sa* $\wedge$ *valid-state sa* $\wedge$ *state-refines sa sb*
proof –
  have *hab*: *h a = h b* **using** *bo-hash-eq*[*OF boab*] **.**
  obtain *sa* **where** *ea*: *extract-map a = Some sa* **and** *sr*: *state-refines sa sb*
    using *extract-under-blinding*[*OF hab boab eb*] **by** *blast*
  have *valid-state sa* **using** *extract-preserves-validity*[*OF ea*] **.**
  with *ea sr* **show** *?thesis* **by** *blast*
qed

end

40 Authenticated Cross-Domain State: the Oraclizer Instance

We discharge the model obligation of *authenticated-state* by exhibiting a concrete, non-degenerate instance. Without it the two soundness theorems above would range over a possibly empty class of structures; the instance below pins them to an extraction map that reads a genuine cross-domain state out of authenticated data.

An authenticated datum is a pair (r, P): a consensus regulatory state r together with the set P of chains that have revealed — are known to hold — the shared asset in that state. The hash *auth-hash* commits to the consensus state r alone, so two data are mergeable exactly when they agree on r; the merge *auth-merge* then unions the revealed-chain sets, and the blinding order *auth-bo* forgets some of them. This is the state-level reading the glue layer asks for: *auth-merge* realises the ADS *join*, *auth-bo* the ADS blinding order, and *auth-extract* sends (r, P) to the global state in which every chain of P holds the asset in state r. Because the hash fixes r, merging never forces two chains to disagree, so the extracted join is consistent and the three obligations hold with no weakening of the hypotheses.

definition *auth-hash* :: *reg-state* $\times$ *chain-id set* $\Rightarrow$ *reg-state* **where**
  auth-hash a = fst a

definition *auth-bo* :: *reg-state* $\times$ *chain-id set* $\Rightarrow$ *reg-state* $\times$ *chain-id set* $\Rightarrow$ *bool*
where
  auth-bo a b $\longleftrightarrow$ *fst a = fst b* $\wedge$ *snd a* $\subseteq$ *snd b*

definition *auth-merge* ::
  reg-state $\times$ *chain-id set* $\Rightarrow$ *reg-state* $\times$ *chain-id set* $\Rightarrow$ (*reg-state* $\times$ *chain-id set*)
option **where**
  auth-merge a b = (*if fst a = fst b then Some* (*fst a*, *snd a* $\cup$ *snd b*) *else None*)

The merge is the union of revealed-chain sets, guarded by agreement on the consensus state. Idempotence, commutativity and associativity descend from the corresponding laws of set union, and the blinding order is exactly the subset order on revealed chains; hence the triple is a Merkle interface.

lemma *merkle-interface-auth* [*locale-witness*]:
  merkle-interface auth-hash auth-bo auth-merge
proof (*rule merkle-interface.intro*)
  show $\bigwedge a\ b.$ (*auth-hash a* = *auth-hash b*) = ($\exists$ *ab. auth-merge a b* = *Some ab*)
    by (*auto simp*: *auth-hash-def auth-merge-def*)
  show $\bigwedge a.$ *auth-merge a a* = *Some a*
    by (*simp add*: *auth-merge-def*)
  show $\bigwedge a\ b.$ *auth-merge a b* = *auth-merge b a*
    by (*auto simp*: *auth-merge-def Un-commute*)
  show $\bigwedge a\ b\ c.$ *auth-merge a b* $\gg=$ *auth-merge c* = *auth-merge b c* $\gg=$ *auth-merge a*
    by (*auto simp*: *auth-merge-def ac-simps split*: *if-splits*)
  show $\bigwedge a\ b.$ *auth-bo a b* = (*auth-merge a b* = *Some b*)
    by (*auto simp*: *auth-bo-def auth-merge-def prod-eq-iff subset-Un-eq*)
qed

The extraction map. *auth-state r P* is the global state in which each chain of *P* holds asset *0* in regulatory state *r*, with nothing locked.

definition *auth-state* :: *reg-state* $\Rightarrow$ *chain-id set* $\Rightarrow$ *global-state* **where**
  auth-state r P =
    ⦇ *gs-chains* = (λ*c a. if c* $\in$ *P* $\wedge$ *a* = *0*
                      then Some ⦇ *as-asset-id* = *0*, *as-reg-state* = *r* ⦈
                      else None),
      gs-locks = (λ*a. False*) ⦈

definition *auth-extract* :: *reg-state* $\times$ *chain-id set* $\Rightarrow$ *global-state option* **where**
  auth-extract a = *Some* (*auth-state* (*fst a*) (*snd a*))

Accessors of the extracted state, read off the definition.

lemma *auth-state-get-reg*:
  get-reg-state (*auth-state r P*) *c a* = (*if c* $\in$ *P* $\wedge$ *a* = *0 then Some r else None*)
  by (*simp add*: *auth-state-def get-reg-state-def get-asset-state-def split*: *if-splits*)

lemma *auth-state-connected*:
  connected-chains (*auth-state r P*) *a* = (*if a* = *0 then P else* {})
  by (*auto simp*: *connected-chains-def asset-exists-def get-asset-state-def auth-state-def*
        split: *if-splits*)

Every extracted state is valid: all chains carry the same consensus state *r*, so the state is consistent, and no asset is locked.

lemma *auth-state-valid*: *valid-state* (*auth-state r P*)
proof –
  have *consistent-state* (*auth-state r P*)
    unfolding *consistent-state-def* **by** (*auto simp*: *auth-state-get-reg split*: *if-splits*)
  moreover have *no-locked-without-reason* (*auth-state r P*)

    by (*simp add*: *no-locked-without-reason-def is-locked-def auth-state-def*)
  ultimately show *?thesis* **by** (*simp add*: *valid-state-def*)
qed

The merge of two views with the same consensus state extracts to the join of the two extracted states: the union of revealed chains, agreeing on *r*.

lemma *state-join-auth*:
  state-join (*auth-state r Pa*) (*auth-state r Pb*) (*auth-state r* (*Pa* $\cup$ *Pb*))
  unfolding *state-join-def*
  by (*auto simp*: *auth-state-connected auth-state-get-reg split*: *if-splits*)

A view over fewer chains refines a view over more chains with the same consensus state.

lemma *state-refines-auth*:
  Pa $\subseteq$ *Pb* $\Longrightarrow$ *state-refines* (*auth-state r Pa*) (*auth-state r Pb*)
  unfolding *state-refines-def*
  by (*auto simp*: *auth-state-connected auth-state-get-reg subset-iff split*: *if-splits*)

The instance. The three coherence obligations are discharged from the lemmas above: merging extracts to the join (*state-join-auth*), blinding extracts to a refinement (*state-refines-auth*), and every extracted state is valid (*auth-state-valid*). The Merkle interface obligation is *merkle-interface-auth*.

interpretation *oss-authenticated*:
  authenticated-state auth-hash auth-bo auth-merge auth-extract
proof *unfold-locales*
  fix *a b ab sa sb*
  assume *hab*: *auth-hash a* = *auth-hash b* **and** *mab*: *auth-merge a b* = *Some ab*
    and *ea*: *auth-extract a* = *Some sa* **and** *eb*: *auth-extract b* = *Some sb*
  from *hab* **have** *rfst*: *fst a* = *fst b* **by** (*simp add*: *auth-hash-def*)
  with *mab* **have** *ab-eq*: *ab* = (*fst a*, *snd a* $\cup$ *snd b*) **by** (*simp add*: *auth-merge-def*)
  have *sa-eq*: *sa* = *auth-state* (*fst a*) (*snd a*) **using** *ea* **by** (*simp add*: *auth-extract-def*)
  have *sb-eq*: *sb* = *auth-state* (*fst a*) (*snd b*) **using** *eb rfst* **by** (*simp add*: *auth-extract-def*)
  have *auth-extract ab* = *Some* (*auth-state* (*fst a*) (*snd a* $\cup$ *snd b*))
    by (*simp add*: *auth-extract-def ab-eq*)
  moreover have *state-join sa sb* (*auth-state* (*fst a*) (*snd a* $\cup$ *snd b*))
    unfolding *sa-eq sb-eq* **by** (*rule state-join-auth*)
  ultimately show $\exists$ *sab*. *auth-extract ab* = *Some sab* $\wedge$ *state-join sa sb sab*
    by *blast*
next
  fix *a b sb*
  assume *bo*: *auth-bo a b* **and** *eb*: *auth-extract b* = *Some sb*
  from *bo* **have** *rfst*: *fst a* = *fst b* **and** *sub*: *snd a* $\subseteq$ *snd b*
    by (*simp-all add*: *auth-bo-def*)
  have *sb-eq*: *sb* = *auth-state* (*fst a*) (*snd b*) **using** *eb rfst* **by** (*simp add*: *auth-extract-def*)
  have *auth-extract a* = *Some* (*auth-state* (*fst a*) (*snd a*)) **by** (*simp add*: *auth-extract-def*)
  moreover have *state-refines* (*auth-state* (*fst a*) (*snd a*)) *sb*
    unfolding *sb-eq* **using** *sub* **by** (*rule state-refines-auth*)
  ultimately show $\exists$ *sa*. *auth-extract a* = *Some sa* $\wedge$ *state-refines sa sb*
    by *blast*

next
  fix *a s*
  assume *auth-extract a* = *Some s*
  then have *s* = *auth-state* (*fst a*) (*snd a*) **by** (*simp add*: *auth-extract-def*)
  then show *valid-state s* **by** (*simp add*: *auth-state-valid*)
qed

The model is non-degenerate: the extraction map returns rich states, and the merging law is exercised by genuinely distinct partial views. The witnesses below record that a two-chain view extracts to a state with two connected chains carrying *ACTIVE*, and that merging the single-chain views over chains *0* and *Suc 0* joins them into the two-chain view.

lemma *oss-authenticated-extract-nontrivial*:
  auth-extract (*ACTIVE*, {*0*, *Suc 0*}) = *Some* (*auth-state ACTIVE* {*0*, *Suc 0*})
  connected-chains (*auth-state ACTIVE* {*0*, *Suc 0*}) *0* = {*0*, *Suc 0*}
  get-reg-state (*auth-state ACTIVE* {*0*, *Suc 0*}) *0 0* = *Some ACTIVE*
  by (*simp-all add*: *auth-extract-def auth-state-connected auth-state-get-reg*)

lemma *oss-authenticated-merge-nontrivial*:
  auth-merge (*ACTIVE*, {*0*}) (*ACTIVE*, {*Suc 0*}) = *Some* (*ACTIVE*, {*0*, *Suc 0*})
  auth-extract (*ACTIVE*, {*0*}) = *Some* (*auth-state ACTIVE* {*0*})
  auth-extract (*ACTIVE*, {*Suc 0*}) = *Some* (*auth-state ACTIVE* {*Suc 0*})
  state-join (*auth-state ACTIVE* {*0*}) (*auth-state ACTIVE* {*Suc 0*}) (*auth-state ACTIVE* {*0*, *Suc 0*})
  using *state-join-auth*[*of ACTIVE* {*0*} {*Suc 0*}]
  by (*simp-all add*: *auth-merge-def auth-extract-def insert-commute*)

Regression witness for the containment clause of *state-join*. The rogue candidate extends the union of the two single-chain *ACTIVE* views with a *FROZEN* holding on chain *7* — a chain revealed by neither view. The candidate is itself inconsistent, and the containment clause rejects it as a join of the two views.

definition *rogue-join-state* :: *global-state* **where**
  rogue-join-state =
    (| *gs-chains* = (λ*c a*. *if a* = *0* ∧ (*c* = *0* ∨ *c* = *Suc 0*)
                then Some (| *as-asset-id* = *0*, *as-reg-state* = *ACTIVE* |)
                else if a = *0* ∧ *c* = *7*
                then Some (| *as-asset-id* = *0*, *as-reg-state* = *FROZEN* |)
                else None),
      gs-locks = (λ*a*. *False*) |)

lemma *rogue-join-state-get-reg*:
  get-reg-state rogue-join-state c a =
    (*if a* = *0* ∧ (*c* = *0* ∨ *c* = *Suc 0*) *then Some ACTIVE*
     else if a = *0* ∧ *c* = *7 then Some FROZEN else None*)
  by (*simp add*: *rogue-join-state-def get-reg-state-def get-asset-state-def*)

lemma *rogue-join-excluded*:
¬ *consistent-state rogue-join-state*
¬ *state-join* (*auth-state ACTIVE* {*0*}) (*auth-state ACTIVE* {*Suc 0*}) *rogue-join-state*
proof −
have *a*: *get-reg-state rogue-join-state 0 0* = *Some ACTIVE*
and *f*: *get-reg-state rogue-join-state 7 0* = *Some FROZEN*
by (*simp-all add*: *rogue-join-state-get-reg*)
show ¬ *consistent-state rogue-join-state*
proof
assume *consistent-state rogue-join-state*
then have *ACTIVE* = *FROZEN* **using** *a f* **unfolding** *consistent-state-def* **by** *blast*
then show *False* **by** *simp*
qed
have *c7*: (*7*::*nat*) ∈ *connected-chains rogue-join-state 0*
by (*simp add*: *connected-chains-def asset-exists-def get-asset-state-def rogue-join-state-def*)
have (*7*::*nat*) ∉ *connected-chains* (*auth-state ACTIVE* {*0*}) *0*
and (*7*::*nat*) ∉ *connected-chains* (*auth-state ACTIVE* {*Suc 0*}) *0*
by (*simp-all add*: *auth-state-connected*)
then show ¬ *state-join* (*auth-state ACTIVE* {*0*}) (*auth-state ACTIVE* {*Suc 0*}) *rogue-join-state*
using *c7* **unfolding** *state-join-def* **by** *blast*
qed

41 Inconsistency Measure on Global States

The convergence argument needs a well-founded progress measure that a synchronization strictly decreases while the global state is inconsistent. We use the number of unordered pairs of connected chains that disagree on the regulatory state of a shared asset. On a state with finitely many asset holdings this count is finite, it is zero exactly when the state is consistent, and a synchronization of an inconsistent asset strictly decreases it.

definition *finite-domain* :: *global-state* ⇒ *bool* **where**
finite-domain gs ⟷ *finite* {(*c*, *aid*). *asset-exists gs c aid*}

definition *inconsistency-set* ::
global-state ⇒ (*chain-id* × *chain-id* × *asset-id*) *set* **where**
inconsistency-set gs =
{(*c1*, *c2*, *aid*). *c1* ∈ *connected-chains gs aid* ∧ *c2* ∈ *connected-chains gs aid* ∧ *c1* < *c2* ∧ *get-reg-state gs c1 aid* ≠ *get-reg-state gs c2 aid*}

definition *inconsistency-pairs* :: *global-state* ⇒ *nat* **where**
inconsistency-pairs gs = *card* (*inconsistency-set gs*)

A connected chain always carries a defined regulatory state for the asset.

lemma *connected-chains-get-reg-state*:
assumes *c* ∈ *connected-chains gs aid*

shows $\exists$ *s. get-reg-state gs c aid* = *Some s*
using *assms*
unfolding *connected-chains-def asset-exists-def get-reg-state-def get-asset-state-def*
by (*auto split*: *option.splits*)

On a finite-domain state, the connected-chain set of any asset is finite.

lemma *connected-chains-finite*:
assumes *finite-domain gs*
shows *finite* (*connected-chains gs aid*)
proof −
have *connected-chains gs aid* $\subseteq$ *fst* ‘ {(*c*, *a*). *asset-exists gs c a*}
by (*auto simp*: *connected-chains-def image-iff*)
moreover have *finite* (*fst* ‘ {(*c*, *a*). *asset-exists gs c a*})
using *assms* **unfolding** *finite-domain-def* **by** *simp*
ultimately show *?thesis* **by** (*rule finite-subset*)
qed

On a finite-domain state, the inconsistency set is finite.

lemma *inconsistency-set-finite*:
assumes *finite-domain gs*
shows *finite* (*inconsistency-set gs*)
proof −
let *?E* = {(*c*, *a*). *asset-exists gs c a*}
have *inconsistency-set gs* $\subseteq$ (λ(*p*, *q*). (*fst p*, *fst q*, *snd p*)) ‘ (*?E* $\times$ *?E*)
proof
fix *t* **assume** *t* $\in$ *inconsistency-set gs*
then obtain *c1 c2 aid* **where** *t*: *t* = (*c1*, *c2*, *aid*)
and *m1*: *c1* $\in$ *connected-chains gs aid* **and** *m2*: *c2* $\in$ *connected-chains gs aid*
unfolding *inconsistency-set-def* **by** *auto*
from *m1* **have** *e1*: (*c1*, *aid*) $\in$ *?E* **by** (*simp add*: *connected-chains-def*)
from *m2* **have** *e2*: (*c2*, *aid*) $\in$ *?E* **by** (*simp add*: *connected-chains-def*)
have *t* = (λ(*p*, *q*). (*fst p*, *fst q*, *snd p*)) ((*c1*, *aid*), (*c2*, *aid*)) **using** *t* **by** *simp*
then show *t* $\in$ (λ(*p*, *q*). (*fst p*, *fst q*, *snd p*)) ‘ (*?E* $\times$ *?E*)
using *e1 e2* **by** *blast*
qed
moreover have *finite* ((λ(*p*, *q*). (*fst p*, *fst q*, *snd p*)) ‘ (*?E* $\times$ *?E*))
using *assms* **unfolding** *finite-domain-def* **by** *simp*
ultimately show *?thesis* **by** (*rule finite-subset*)
qed

The measure is zero exactly on consistent states.

lemma *inconsistency-pairs-zero-iff-consistent*:
assumes *finite-domain gs*
shows *inconsistency-pairs gs* = *0* $\longleftrightarrow$ *consistent-state gs*
proof
assume *inconsistency-pairs gs* = *0*
then have *empty*: *inconsistency-set gs* = {}
using *inconsistency-set-finite*[*OF assms*] **unfolding** *inconsistency-pairs-def* **by** *simp*

show *consistent-state gs*
unfolding *consistent-state-def*
proof (*intro allI impI*)
fix *c1 c2 aid s1 s2*
assume *a1*: *get-reg-state gs c1 aid* = *Some s1* **and** *a2*: *get-reg-state gs c2 aid* = *Some s2*
have *in1*: *c1* ∈ *connected-chains gs aid* **and** *in2*: *c2* ∈ *connected-chains gs aid*
using *a1 a2* **unfolding** *connected-chains-def asset-exists-def*
get-reg-state-def get-asset-state-def **by** (*auto split*: *option.splits*)
show *s1* = *s2*
proof (*rule ccontr*)
assume *ne*: *s1* ≠ *s2*
then have *rne*: *get-reg-state gs c1 aid* ≠ *get-reg-state gs c2 aid* **using** *a1 a2* **by** *simp*
have *c1* ≠ *c2* **using** *a1 a2 ne* **by** *auto*
then consider *c1* < *c2* | *c2* < *c1* **by** *linarith*
then show *False*
proof *cases*
case *1*
then have (*c1*, *c2*, *aid*) ∈ *inconsistency-set gs*
using *in1 in2 rne* **unfolding** *inconsistency-set-def* **by** *simp*
with *empty* **show** *False* **by** *simp*
next
case *2*
then have (*c2*, *c1*, *aid*) ∈ *inconsistency-set gs*
using *in1 in2 rne* **unfolding** *inconsistency-set-def* **by** *auto*
with *empty* **show** *False* **by** *simp*
qed
qed
qed
next
assume *cons*: *consistent-state gs*
have *inconsistency-set gs* = {}
proof (*rule ccontr*)
assume *inconsistency-set gs* ≠ {}
then obtain *c1 c2 aid* **where**
m1: *c1* ∈ *connected-chains gs aid* **and** *m2*: *c2* ∈ *connected-chains gs aid*
and *rne*: *get-reg-state gs c1 aid* ≠ *get-reg-state gs c2 aid*
unfolding *inconsistency-set-def* **by** *auto*
obtain *s1* **where** *s1*: *get-reg-state gs c1 aid* = *Some s1*
using *connected-chains-get-reg-state*[*OF m1*] **by** *blast*
obtain *s2* **where** *s2*: *get-reg-state gs c2 aid* = *Some s2*
using *connected-chains-get-reg-state*[*OF m2*] **by** *blast*
have *s1* = *s2* **using** *cons s1 s2* **unfolding** *consistent-state-def* **by** *blast*
with *s1 s2 rne* **show** *False* **by** *simp*
qed
then show *inconsistency-pairs gs* = *0* **unfolding** *inconsistency-pairs-def* **by** *simp*
qed

42 Structural Effect of Synchronization

The following lemmas isolate the effect of a successful synchronization on the regulatory states and on asset existence. They are proved without assuming the input state is valid (consistent and unlocked), because the convergence argument starts from an arbitrary state. The existing theorem *regulatory-homomorphism* establishes the analogous fact under the *valid-state* hypothesis; here we re-establish the parts needed for convergence with no such hypothesis.

lemma *sync-components*:
  assumes *sync src act aid0 gs = Some gs′*
  shows ∃ *current-st new-st gs-locked.*
      get-reg-state gs src aid0 = Some current-st
    ∧ *reg-transition current-st act = Some new-st*
    ∧ *acquire-lock gs aid0 = Some gs-locked*
    ∧ *gs′ = release-lock*
        (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
aid0
  using *assms* **unfolding** *sync-def* **by** (*auto split*: *option.splits simp*: *Let-def*)

Synchronizing asset *aid0* leaves the chain map of every other asset unchanged.

lemma *sync-chains-other-asset*:
  assumes *synced*: *sync src act aid0 gs = Some gs′* **and** *ne*: *aid ≠ aid0*
  shows *gs-chains gs′ c aid = gs-chains gs c aid*
proof −
  from *sync-components*[*OF synced*] **obtain** *current-st new-st gs-locked* **where**
    lock: *acquire-lock gs aid0 = Some gs-locked*
    and *gs′*: *gs′ = release-lock*
        (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
aid0
    by *blast*
  have *lc*: *gs-chains gs-locked = gs-chains gs* **using** *acquire-lock-chains*[*OF lock*] **.**
  have *gs-chains* (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*)) *c aid*
      = gs-chains gs-locked c aid
    using *update-all-chains-other-asset*[*OF ne*] **by** *simp*
  then show *?thesis* **unfolding** *gs′ release-lock-chains* **using** *lc* **by** *simp*
qed

lemma *sync-reg-other-asset*:
  assumes *synced*: *sync src act aid0 gs = Some gs′* **and** *ne*: *aid ≠ aid0*
  shows *get-reg-state gs′ c aid = get-reg-state gs c aid*
  unfolding *get-reg-state-def get-asset-state-def*
  using *sync-chains-other-asset*[*OF synced ne*] **by** *simp*

After synchronizing asset *aid0*, all chains connected to *aid0* agree on its new regulatory state.

lemma *sync-synced-asset-uniform*:
  assumes *synced*: *sync src act aid0 gs* = *Some gs'*
  shows $\exists$ *ns*. $\forall$ *c'* $\in$ *connected-chains gs aid0*. *get-reg-state gs' c' aid0* = *Some ns*
proof −
  from *sync-components*[*OF synced*] **obtain** *current-st new-st gs-locked* **where**
    lock: *acquire-lock gs aid0* = *Some gs-locked*
    and *gs'*: *gs'* = *release-lock*
              (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
aid0
    by *blast*
  have *lc*: *gs-chains gs-locked* = *gs-chains gs* **using** *acquire-lock-chains*[*OF lock*] **.**
  have $\forall$ *c'* $\in$ *connected-chains gs aid0*. *get-reg-state gs' c' aid0* = *Some new-st*
  proof
    fix *c'* **assume** *c'conn*: *c'* $\in$ *connected-chains gs aid0*
    then have *asset-exists gs c' aid0* **by** (*simp add*: *connected-chains-def*)
    then obtain *ast* **where** *ast*: *get-asset-state gs c' aid0* = *Some ast*
      unfolding *asset-exists-def* **by** *auto*
    then have *astl*: *get-asset-state gs-locked c' aid0* = *Some ast*
      unfolding *get-asset-state-def* **using** *lc* **by** *simp*
    have *get-reg-state*
          (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*)) *c'*
aid0 = *Some new-st*
      using *update-all-chains-reg-state*[*OF c'conn astl*] **.**
    then show *get-reg-state gs' c' aid0* = *Some new-st*
      unfolding *gs'* **by** (*simp add*: *release-lock-reg-state*)
  qed
  then show *?thesis* **by** *blast*
qed

Synchronization preserves asset existence, hence the connected-chain sets.

lemma *sync-asset-exists-preserved*:
  assumes *synced*: *sync src act aid0 gs* = *Some gs'*
  shows *asset-exists gs' c aid* = *asset-exists gs c aid*
proof (*cases aid* = *aid0*)
  case *False*
  then have *gs-chains gs' c aid* = *gs-chains gs c aid*
    using *sync-chains-other-asset*[*OF synced*] **by** *simp*
  then show *?thesis* **unfolding** *asset-exists-def get-asset-state-def* **by** *simp*
next
  case *True*
  show *?thesis*
  proof (*cases c* $\in$ *connected-chains gs aid0*)
    case *in-conn*: *True*
    then have *e-gs*: *asset-exists gs c aid0* **by** (*simp add*: *connected-chains-def*)
     obtain *ns* **where** $\forall$ *c'*$\in$*connected-chains gs aid0*. *get-reg-state gs' c' aid0* =
Some ns
      using *sync-synced-asset-uniform*[*OF synced*] **by** *blast*
    then have *get-reg-state gs' c aid0* = *Some ns* **using** *in-conn* **by** *blast*

  then have *asset-exists gs′ c aid0*
   unfolding *asset-exists-def get-reg-state-def get-asset-state-def*
   by (*auto split*: *option.splits*)
  with *e-gs True* **show** *?thesis* **by** *simp*
 next
  case *out-conn*: *False*
  then have *ngs*: ¬ *asset-exists gs c aid0* **by** (*simp add*: *connected-chains-def*)
  from *sync-components*[*OF synced*] **obtain** *current-st new-st gs-locked* **where**
   lock: *acquire-lock gs aid0* = *Some gs-locked*
   and *gs′*: *gs′* = *release-lock*
     (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
aid0
   by *blast*
  have *lc*: *gs-chains gs-locked* = *gs-chains gs* **using** *acquire-lock-chains*[*OF lock*]
.
  have *gs-chains gs′ c* = *gs-chains gs c*
   unfolding *gs′ release-lock-chains*
   using *update-all-chains-outside-targets*[*OF out-conn*] *lc* **by** *simp*
  then have *asset-exists gs′ c aid0* = *asset-exists gs c aid0*
   unfolding *asset-exists-def get-asset-state-def* **by** *simp*
  with *True* **show** *?thesis* **by** *simp*
 qed
qed

lemma *sync-connected-chains-preserved*:
 assumes *sync src act aid0 gs* = *Some gs′*
 shows *connected-chains gs′ aid* = *connected-chains gs aid*
 unfolding *connected-chains-def* **using** *sync-asset-exists-preserved*[*OF assms*] **by**
simp

lemma *sync-preserves-finite-domain*:
 assumes *finite-domain gs* **and** *sync src act aid0 gs* = *Some gs′*
 shows *finite-domain gs′*
proof −
 have {(*c*, *aid*). *asset-exists gs′ c aid*} = {(*c*, *aid*). *asset-exists gs c aid*}
  using *sync-asset-exists-preserved*[*OF assms*(*2*)] **by** *simp*
 then show *?thesis* **using** *assms*(*1*) **unfolding** *finite-domain-def* **by** *simp*
qed

Synchronization preserves the unlocked invariant. This is the lock half of *valid-state-preservation*, re-established without the *consistent-state* hypothesis.

lemma *sync-preserves-unlocked*:
 assumes *nl*: *no-locked-without-reason gs* **and** *synced*: *sync src act aid0 gs* = *Some*
gs′
 shows *no-locked-without-reason gs′*
proof −
 from *sync-components*[*OF synced*] **obtain** *current-st new-st gs-locked* **where**
  lock: *acquire-lock gs aid0* = *Some gs-locked*

and *gs'*: *gs'* = *release-lock*
(*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
aid0
by *blast*
from *lock* **have** *locked-locks*: *gs-locks gs-locked* = (*gs-locks gs*)(*aid0* := *True*)
unfolding *acquire-lock-def is-locked-def* **by** (*auto split*: *if-splits*)
have *upd-locks*: *gs-locks* (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
= *gs-locks gs-locked*
using *update-all-chains-locks* **by** *simp*
have *final-locks*: *gs-locks gs'* = (*gs-locks gs-locked*)(*aid0* := *False*)
unfolding *gs' release-lock-def* **using** *upd-locks* **by** *simp*
show *?thesis*
unfolding *no-locked-without-reason-def is-locked-def*
proof
fix *aid'*
show ¬ *gs-locks gs' aid'*
proof (*cases aid'* = *aid0*)
case *True* **then show** *?thesis* **using** *final-locks* **by** *simp*
next
case *False*
then have *gs-locks gs' aid'* = *gs-locks gs aid'*
using *final-locks locked-locks* **by** *simp*
also have . . . = *False*
using *nl* **unfolding** *no-locked-without-reason-def is-locked-def* **by** *simp*
finally show *?thesis* **by** *simp*
qed
qed
qed

43 The Critical Path: Synchronization Reduces Inconsistency

A connected chain witnesses asset existence, so a defined regulatory state places the chain in the connected set.

lemma *get-reg-state-connected*:
assumes *get-reg-state gs c aid* = *Some s*
shows *c* ∈ *connected-chains gs aid*
using *assms* **unfolding** *connected-chains-def asset-exists-def*
get-reg-state-def get-asset-state-def **by** (*auto split*: *option.splits*)

The critical convergence step. Synchronizing an asset *aid0* on which two connected chains disagree strictly decreases the inconsistency measure: every inconsistent pair of *aid0* is removed (all its connected chains are driven to one regulatory state), pairs of other assets are untouched, and the witness pair certifies that at least one pair really disappears.

lemma *sync-reduces-inconsistency*:

assumes *fin*: *finite-domain gs*
and *synced*: *sync src act aid0 gs* = *Some gs'*
and *ca*: *ca* ∈ *connected-chains gs aid0*
and *cb*: *cb* ∈ *connected-chains gs aid0*
and *dis*: *get-reg-state gs ca aid0* ≠ *get-reg-state gs cb aid0*
shows *inconsistency-pairs gs'* < *inconsistency-pairs gs*
proof −
obtain *ns* **where** *uniform*: ∀ *c'*∈*connected-chains gs aid0*. *get-reg-state gs' c' aid0* = *Some ns*
using *sync-synced-asset-uniform*[*OF synced*] **by** *blast*
have *conn-eq*: ⋀*aid*. *connected-chains gs' aid* = *connected-chains gs aid*
using *sync-connected-chains-preserved*[*OF synced*] **by** *simp*
— Step 1: the inconsistency set of *gs'* is contained in that of *gs*.
have *subset*: *inconsistency-set gs'* ⊆ *inconsistency-set gs*
proof
fix *t* **assume** *t* ∈ *inconsistency-set gs'*
then obtain *c1 c2 aid* **where** *t*: *t* = (*c1*, *c2*, *aid*)
and *i1*: *c1* ∈ *connected-chains gs' aid* **and** *i2*: *c2* ∈ *connected-chains gs' aid*
and *lt*: *c1* < *c2* **and** *rne*: *get-reg-state gs' c1 aid* ≠ *get-reg-state gs' c2 aid*
unfolding *inconsistency-set-def* **by** *auto*
have *ane*: *aid* ≠ *aid0*
proof
assume *eq*: *aid* = *aid0*
have *c1* ∈ *connected-chains gs aid0 c2* ∈ *connected-chains gs aid0*
using *i1 i2 conn-eq eq* **by** *auto*
then have *get-reg-state gs' c1 aid0* = *Some ns get-reg-state gs' c2 aid0* = *Some ns*
using *uniform* **by** *auto*
with *rne eq* **show** *False* **by** *simp*
qed
have *c1* ∈ *connected-chains gs aid c2* ∈ *connected-chains gs aid*
using *i1 i2 conn-eq* **by** *auto*
moreover have *get-reg-state gs c1 aid* ≠ *get-reg-state gs c2 aid*
using *rne sync-reg-other-asset*[*OF synced ane*] **by** *simp*
ultimately have (*c1*, *c2*, *aid*) ∈ *inconsistency-set gs*
using *lt* **unfolding** *inconsistency-set-def* **by** *simp*
then show *t* ∈ *inconsistency-set gs* **using** *t* **by** *simp*
qed
— Step 2: the witness pair on *aid0* is in *gs* but not in *gs'*.
have *caneb*: *ca* ≠ *cb* **using** *dis* **by** *auto*
define *c1* **where** *c1* = *min ca cb*
define *c2* **where** *c2* = *max ca cb*
have *lt*: *c1* < *c2* **using** *caneb* **unfolding** *c1-def c2-def* **by** *auto*
have *c1conn*: *c1* ∈ *connected-chains gs aid0* **and** *c2conn*: *c2* ∈ *connected-chains gs aid0*
using *ca cb* **unfolding** *c1-def c2-def* **by** (*auto simp*: *min-def max-def*)
have *wdis*: *get-reg-state gs c1 aid0* ≠ *get-reg-state gs c2 aid0*
using *dis* **unfolding** *c1-def c2-def* **by** (*auto simp*: *min-def max-def*)
have *wit-in*: (*c1*, *c2*, *aid0*) ∈ *inconsistency-set gs*

using *c1conn c2conn lt wdis* **unfolding** *inconsistency-set-def* **by** *simp*
have *wit-out*: $(c1, c2, aid0) \notin$ *inconsistency-set gs'*
proof
assume $(c1, c2, aid0) \in$ *inconsistency-set gs'*
then have *rne*: *get-reg-state gs' c1 aid0* $\neq$ *get-reg-state gs' c2 aid0*
and *c1* $\in$ *connected-chains gs' aid0* **and** *c2* $\in$ *connected-chains gs' aid0*
unfolding *inconsistency-set-def* **by** *auto*
then have *c1* $\in$ *connected-chains gs aid0 c2* $\in$ *connected-chains gs aid0*
using *conn-eq* **by** *auto*
then have *get-reg-state gs' c1 aid0* = *Some ns get-reg-state gs' c2 aid0* = *Some ns*
using *uniform* **by** *auto*
with *rne* **show** *False* **by** *simp*
qed
— Step 3: a strict subset of a finite set has strictly smaller cardinality.
have *psub*: *inconsistency-set gs'* $\subset$ *inconsistency-set gs*
using *subset wit-in wit-out* **by** *blast*
have *card* (*inconsistency-set gs'*) < *card* (*inconsistency-set gs*)
using *psubset-card-mono*[*OF inconsistency-set-finite*[*OF fin*] *psub*] **.**
then show *?thesis* **unfolding** *inconsistency-pairs-def* **.**
qed

44 Existence of a Reducing Synchronization

From any inconsistent, finite-domain, unlocked global state there is a synchronization that strictly reduces the inconsistency measure. An inconsistency exhibits two chains disagreeing on a shared asset; at least one carries a non-terminal (non-*CONFISCATED*) regulatory state, from which the *CONFISCATE* action is always enabled (*confiscate-universal*), so a synchronization succeeds and, by *sync-reduces-inconsistency*, reduces the measure.

definition *reducing-sync* ::
global-state $\Rightarrow$ (*chain-id* $\times$ *reg-action* $\times$ *asset-id*) $\Rightarrow$ *bool* **where**
reducing-sync gs p $\longleftrightarrow$ (*case p of* (*src*, *act*, *aid*) $\Rightarrow$
($\exists$ *gs'*. *sync src act aid gs* = *Some gs'* $\wedge$ *inconsistency-pairs gs'* < *inconsistency-pairs gs*))

lemma *inconsistent-has-reducing-sync*:
assumes *fin*: *finite-domain gs* **and** *nl*: *no-locked-without-reason gs*
and *inc*: $\neg$ *consistent-state gs*
shows $\exists$ *p*. *reducing-sync gs p*
proof −
from *inc* **obtain** *c1 c2 aid0 s1 s2* **where**
g1: *get-reg-state gs c1 aid0* = *Some s1* **and** *g2*: *get-reg-state gs c2 aid0* = *Some s2*
and *ne*: *s1* $\neq$ *s2*
unfolding *consistent-state-def* **by** *blast*
have *conn1*: *c1* $\in$ *connected-chains gs aid0* **using** *get-reg-state-connected*[*OF g1*] **.**

have *conn2*: *c2* ∈ *connected-chains gs aid0* **using** *get-reg-state-connected*[*OF g2*]
.
— One of the two disagreeing chains is non-terminal.
obtain *src ssrc* **where** *src-conn*: *src* ∈ *connected-chains gs aid0*
and *src-reg*: *get-reg-state gs src aid0* = *Some ssrc*
and *src-nonterm*: *ssrc* ≠ *CONFISCATED*
proof (*cases s1* = *CONFISCATED*)
case *True*
then have *s2* ≠ *CONFISCATED* **using** *ne* **by** *auto*
then show *thesis* **using** *that conn2 g2* **by** *blast*
next
case *False*
then show *thesis* **using** *that conn1 g1* **by** *blast*
qed
— *CONFISCATE* is enabled from any non-terminal state, and the asset is unlocked.
have *ctrans*: *reg-transition ssrc CONFISCATE* = *Some CONFISCATED*
using *confiscate-universal*[*OF src-nonterm*] **.**
have *notlocked*: ¬ *is-locked gs aid0*
using *nl* **unfolding** *no-locked-without-reason-def* **by** *simp*
obtain *gsl* **where** *acq*: *acquire-lock gs aid0* = *Some gsl*
using *lock-acquire-success*[*OF notlocked*] **by** *blast*
have *sync src CONFISCATE aid0 gs*
= *Some* (*release-lock*
(*update-all-chains gsl aid0 CONFISCATED* (*connected-chains gs aid0*)) *aid0*)
unfolding *sync-def* **using** *src-reg ctrans acq* **by** (*simp add*: *Let-def*)
then obtain *gs′* **where** *synced*: *sync src CONFISCATE aid0 gs* = *Some gs′* **by** *blast*
have *dis*: *get-reg-state gs c1 aid0* ≠ *get-reg-state gs c2 aid0* **using** *g1 g2 ne* **by** *simp*
have *inconsistency-pairs gs′* < *inconsistency-pairs gs*
using *sync-reduces-inconsistency*[*OF fin synced conn1 conn2 dis*] **.**
then have *reducing-sync gs* (*src*, *CONFISCATE*, *aid0*)
unfolding *reducing-sync-def* **using** *synced* **by** *auto*
then show *?thesis* **by** *blast*
qed

45 Terminal-Faithful Safe Recovery

A measure-reducing synchronization alone leaves the realization map two degrees of freedom that no recovery procedure should have. First, it may resolve a mere *ACTIVE*/*FROZEN* disagreement by broadcasting *CONFISCATED*: an indiscriminate-confiscation implementation would refine the model. Second, in the reverse direction: the broadcast overwrites every connected chain without consulting the target chain's own transition relation, so it may overwrite a recorded *CONFISCATED* holding with a non-terminal value, erasing a confiscation. The predicate *safe-recovery* closes both: a recovery is safe when it reduces the measure *and* uses *CONFISCATE* exactly on the assets that already carry the terminal state on some chain. The equivalence is needed in both directions: left-to-right forbids fresh confiscations, and right-to-left forbids completing an inconsistency on a terminal-bearing asset with anything weaker than the terminal value.

definition *terminal-present* :: *global-state* ⇒ *asset-id* ⇒ *bool* **where**
terminal-present gs aid ⟷ (∃ *c*. *get-reg-state gs c aid* = *Some CONFISCATED*)

definition *safe-recovery* ::
global-state ⇒ (*chain-id* × *reg-action* × *asset-id*) ⇒ *bool* **where**
safe-recovery gs p ⟷ *reducing-sync gs p* ∧
(*case p of* (*src*, *act*, *aid*) ⇒ (*act* = *CONFISCATE*) = *terminal-present gs aid*)

lemma *safe-recovery-reducing*:
safe-recovery gs p ⟹ *reducing-sync gs p*
by (*simp add*: *safe-recovery-def*)

The only regulatory action whose result is the terminal state is *CONFISCATE* itself: the transition table admits no other entry to *CONFISCATED*.

lemma *to-confiscated-only-confiscate*:
reg-transition s a = *Some CONFISCATED* ⟹ *a* = *CONFISCATE*
by (*cases s*; *cases a*) *simp-all*

From every non-terminal regulatory state some non-*CONFISCATE* action is enabled; the witnesses are the de-escalation returns and, from *ACTIVE*, an escalation that is not a confiscation.

lemma *non-terminal-non-confiscate-step*:
assumes $s \neq$ *CONFISCATED*
shows ∃ *a s'*. $a \neq$ *CONFISCATE* ∧ *reg-transition s a* = *Some s'*
proof (*cases s*)
case *ACTIVE*
then have *FREEZE* ≠ *CONFISCATE* ∧ *reg-transition s FREEZE* = *Some FROZEN* **by** *simp*
then show *?thesis* **by** *blast*
next
case *FROZEN*
then have *UNFREEZE* ≠ *CONFISCATE* ∧ *reg-transition s UNFREEZE* = *Some ACTIVE* **by** *simp*
then show *?thesis* **by** *blast*
next
case *SEIZED*
then have *RELEASE* ≠ *CONFISCATE* ∧ *reg-transition s RELEASE* = *Some ACTIVE* **by** *simp*
then show *?thesis* **by** *blast*
next
case *CONFISCATED*
then show *?thesis* **using** *assms* **by** *contradiction*
next

case *RESTRICTED*
then have *UNRESTRICT* $\neq$ *CONFISCATE* $\wedge$ *reg-transition s UNRESTRICT* = *Some ACTIVE* **by** *simp*
then show *?thesis* **by** *blast*
qed

Existence of a safe recovery from any inconsistent carrier state. The proof refines *inconsistent-has-reducing-sync* by selecting the synchronization action according to the terminal census of the inconsistent asset: if the terminal state is already present on some chain, the recovery completes it by confiscating from a non-terminal disagreeing chain; if it is absent, the recovery synchronizes with a non-*CONFISCATE* action enabled on a disagreeing chain. Either way the synchronization succeeds and strictly reduces the inconsistency measure, and the chosen action satisfies the terminal-faithfulness equivalence.

lemma *inconsistent-has-safe-recovery*:
assumes *fin*: *finite-domain gs* **and** *nl*: *no-locked-without-reason gs*
and *inc*: $\neg$ *consistent-state gs*
shows $\exists$ *p. safe-recovery gs p*
proof −
from *inc* **obtain** *c1 c2 aid0 s1 s2* **where**
g1: *get-reg-state gs c1 aid0* = *Some s1* **and** *g2*: *get-reg-state gs c2 aid0* = *Some s2*
and *ne*: *s1* $\neq$ *s2*
unfolding *consistent-state-def* **by** *blast*
have *conn1*: *c1* $\in$ *connected-chains gs aid0* **using** *get-reg-state-connected*[*OF g1*]
.
have *conn2*: *c2* $\in$ *connected-chains gs aid0* **using** *get-reg-state-connected*[*OF g2*]
.
have *dis*: *get-reg-state gs c1 aid0* $\neq$ *get-reg-state gs c2 aid0* **using** *g1 g2 ne* **by** *simp*
have *notlocked*: $\neg$ *is-locked gs aid0*
using *nl* **unfolding** *no-locked-without-reason-def* **by** *simp*
obtain *gsl* **where** *acq*: *acquire-lock gs aid0* = *Some gsl*
using *lock-acquire-success*[*OF notlocked*] **by** *blast*
show *?thesis*
proof (*cases terminal-present gs aid0*)
case *True*
— Terminal completion: one of the disagreeing chains is non-terminal, and confiscating from it completes the recorded confiscation.
obtain *src ssrc* **where** *src-reg*: *get-reg-state gs src aid0* = *Some ssrc*
and *src-nonterm*: *ssrc* $\neq$ *CONFISCATED*
proof (*cases s1* = *CONFISCATED*)
case *True*
then have *s2* $\neq$ *CONFISCATED* **using** *ne* **by** *simp*
then show *thesis* **using** *that g2* **by** *blast*
next
case *False*

then show *thesis* **using** *that g1* **by** *blast*
qed
have *ctrans*: *reg-transition ssrc CONFISCATE* = *Some CONFISCATED*
using *confiscate-universal*[*OF src-nonterm*] **.**
have *sync src CONFISCATE aid0 gs*
= *Some* (*release-lock*
(*update-all-chains gsl aid0 CONFISCATED* (*connected-chains gs aid0*)) *aid0*)
unfolding *sync-def* **using** *src-reg ctrans acq* **by** (*simp add*: *Let-def*)
then obtain *gs'* **where** *synced*: *sync src CONFISCATE aid0 gs* = *Some gs'* **by** *blast*
have *inconsistency-pairs gs'* < *inconsistency-pairs gs*
using *sync-reduces-inconsistency*[*OF fin synced conn1 conn2 dis*] **.**
then have *reducing-sync gs* (*src*, *CONFISCATE*, *aid0*)
unfolding *reducing-sync-def* **using** *synced* **by** *auto*
then have *safe-recovery gs* (*src*, *CONFISCATE*, *aid0*)
using *True* **by** (*simp add*: *safe-recovery-def*)
then show *?thesis* **by** *blast*
next
case *False*
— No terminal holding: the first disagreeing chain is non-terminal, and a non-*CONFISCATE* action is enabled there.
have *s1-nonterm*: *s1* ≠ *CONFISCATED*
using *False g1* **unfolding** *terminal-present-def* **by** *blast*
obtain *act ns* **where** *act-nc*: *act* ≠ *CONFISCATE*
and *atrans*: *reg-transition s1 act* = *Some ns*
using *non-terminal-non-confiscate-step*[*OF s1-nonterm*] **by** *blast*
have *sync c1 act aid0 gs*
= *Some* (*release-lock*
(*update-all-chains gsl aid0 ns* (*connected-chains gs aid0*)) *aid0*)
unfolding *sync-def* **using** *g1 atrans acq* **by** (*simp add*: *Let-def*)
then obtain *gs'* **where** *synced*: *sync c1 act aid0 gs* = *Some gs'* **by** *blast*
have *inconsistency-pairs gs'* < *inconsistency-pairs gs*
using *sync-reduces-inconsistency*[*OF fin synced conn1 conn2 dis*] **.**
then have *reducing-sync gs* (*c1*, *act*, *aid0*)
unfolding *reducing-sync-def* **using** *synced* **by** *auto*
then have *safe-recovery gs* (*c1*, *act*, *aid0*)
using *False act-nc* **by** (*simp add*: *safe-recovery-def*)
then show *?thesis* **by** *blast*
qed
qed

Terminal completion is definitional: on a terminal-bearing asset a safe recovery can only be the completing confiscation.

corollary *recovery-terminal-completion*:
assumes *safe-recovery gs* (*src*, *act*, *aid*) **and** *terminal-present gs aid*
shows *act* = *CONFISCATE*
using *assms* **by** (*simp add*: *safe-recovery-def*)

The value a successful synchronization writes onto a connected chain is the output of the regulatory transition fired at the source. This exposes, for the safety argument below, the link between the broadcast value and the transition table that produced it.

lemma *sync-connected-value*:
  assumes *synced*: *sync src act aid0 gs = Some gs′*
    and *conn*: *c* ∈ *connected-chains gs aid0*
  shows ∃ *cur new-st*. *get-reg-state gs src aid0 = Some cur*
      ∧ *reg-transition cur act = Some new-st*
      ∧ *get-reg-state gs′ c aid0 = Some new-st*
proof −
  from *sync-components*[*OF synced*] **obtain** *cur new-st gs-locked* **where**
    cs: *get-reg-state gs src aid0 = Some cur*
    and *tr*: *reg-transition cur act = Some new-st*
    and *lock*: *acquire-lock gs aid0 = Some gs-locked*
    and *gs′*: *gs′ = release-lock*
        (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*))
aid0
    by *blast*
  have *lc*: *gs-chains gs-locked = gs-chains gs* **using** *acquire-lock-chains*[*OF lock*] **.**
  from *conn* **have** *asset-exists gs c aid0* **by** (*simp add*: *connected-chains-def*)
  then obtain *ast* **where** *get-asset-state gs c aid0 = Some ast*
    unfolding *asset-exists-def* **by** *auto*
  then have *astl*: *get-asset-state gs-locked c aid0 = Some ast*
    unfolding *get-asset-state-def* **using** *lc* **by** *simp*
  have *get-reg-state*
      (*update-all-chains gs-locked aid0 new-st* (*connected-chains gs aid0*)) *c aid0*
      = Some new-st
    using *update-all-chains-reg-state*[*OF conn astl*] **.**
  then have *get-reg-state gs′ c aid0 = Some new-st*
    unfolding *gs′* **by** (*simp add*: *release-lock-reg-state*)
  with *cs tr* **show** *?thesis* **by** *blast*
qed

The single-step safety payload of the terminal-faithfulness equivalence: a safe-recovery synchronization never makes the terminal state appear on an asset that did not already carry it. If the synchronized value were *CONFISCATED*, then by *to-confiscated-only-confiscate* the action was *CONFISCATE*, so by the equivalence the asset already carried the terminal state — a contradiction.

lemma *safe-recovery-sync-no-fresh-terminal*:
  assumes *sr*: *safe-recovery gs* (*src*, *act*, *aid0*)
    and *synced*: *sync src act aid0 gs = Some gs′*
    and *nt*: ¬ *terminal-present gs aid*
  shows ¬ *terminal-present gs′ aid*
proof
  assume *terminal-present gs′ aid*
  then obtain *c* **where** *c*: *get-reg-state gs′ c aid = Some CONFISCATED*
    unfolding *terminal-present-def* **by** *blast*

show *False*
proof (*cases aid* = *aid0*)
case *False*
then have *get-reg-state gs c aid* = *Some CONFISCATED*
using *c sync-reg-other-asset*[*OF synced False*] **by** *simp*
then show *False* **using** *nt* **unfolding** *terminal-present-def* **by** *blast*
next
case *True*
have *c* ∈ *connected-chains gs′ aid0*
using *c True get-reg-state-connected* **by** *blast*
then have *conn*: *c* ∈ *connected-chains gs aid0*
using *sync-connected-chains-preserved*[*OF synced*] **by** *simp*
obtain *cur new-st* **where**
tr: *reg-transition cur act* = *Some new-st*
and *val*: *get-reg-state gs′ c aid0* = *Some new-st*
using *sync-connected-value*[*OF synced conn*] **by** *blast*
have *new-st* = *CONFISCATED* **using** *c val True* **by** *simp*
then have *act* = *CONFISCATE* **using** *to-confiscated-only-confiscate tr* **by** *blast*
then have *terminal-present gs aid0*
using *sr* **by** (*simp add*: *safe-recovery-def*)
then show *False* **using** *nt True* **by** *simp*
qed
qed

Machine-checked regression witnesses for the two excluded behaviours. The two-chain builder places the given regulatory values on chains *0* and *Suc 0* of asset *0*; both witnesses evaluate by unfolding.

definition *mixed-pair-state* :: *reg-state* ⇒ *reg-state* ⇒ *global-state* **where**
mixed-pair-state v0 v1 =
(| *gs-chains* = (λ*c a*. *if a* = *0* ∧ *c* = *0*
then Some (| *as-asset-id* = *0*, *as-reg-state* = *v0* |)
else if a = *0* ∧ *c* = *Suc 0*
then Some (| *as-asset-id* = *0*, *as-reg-state* = *v1* |)
else None),
gs-locks = (λ*a*. *False*) |)

lemma *mixed-pair-get-reg*:
get-reg-state (*mixed-pair-state v0 v1*) *c a* =
(*if a* = *0* ∧ *c* = *0 then Some v0 else if a* = *0* ∧ *c* = *Suc 0 then Some v1 else None*)
by (*simp add*: *mixed-pair-state-def get-reg-state-def get-asset-state-def*)

Indiscriminate confiscation is excluded: on a plain *ACTIVE*/*FROZEN* disagreement with no terminal holding, no *CONFISCATE* propagation is a safe recovery, from any source chain.

lemma *blind-confiscate-excluded*:
¬ *terminal-present* (*mixed-pair-state ACTIVE FROZEN*) *0*
¬ *safe-recovery* (*mixed-pair-state ACTIVE FROZEN*) (*c*, *CONFISCATE*, *0*)

by (*auto simp*: *terminal-present-def safe-recovery-def mixed-pair-get-reg split*: *if-splits*)

Confiscation erasure is excluded: on a *CONFISCATED*/*ACTIVE* disagreement the terminal state is present, so no non-*CONFISCATE* broadcast is a safe recovery — a recorded confiscation cannot be revived by overwriting it with a non-terminal value.

lemma *terminal-overwrite-excluded*:
terminal-present (*mixed-pair-state CONFISCATED ACTIVE*) *0*
act $\neq$ *CONFISCATE* $\Longrightarrow$ $\neg$ *safe-recovery* (*mixed-pair-state CONFISCATED ACTIVE*) (*c*, *act*, *0*)
proof –
have *get-reg-state* (*mixed-pair-state CONFISCATED ACTIVE*) *0 0* = *Some CONFISCATED*
by (*simp add*: *mixed-pair-get-reg*)
then show *terminal-present* (*mixed-pair-state CONFISCATED ACTIVE*) *0*
unfolding *terminal-present-def* **by** *blast*
then show *act* $\neq$ *CONFISCATE* $\Longrightarrow$ $\neg$ *safe-recovery* (*mixed-pair-state CONFISCATED ACTIVE*) (*c*, *act*, *0*)
by (*simp add*: *safe-recovery-def*)
qed

46 The Oraclizer as a Converging Composition

We package the oraclizer synchronization protocol as the data of a *converging-composition*: the carrier is the set of finite-domain, unlocked global states; the operations are synchronization triples; the invariant is cross-chain consistency; the progress measure is the inconsistency count; and the realization map picks, in any inconsistent state, a terminal-faithful safe recovery — a synchronization that reduces the measure and uses *CONFISCATE* exactly on terminal-bearing assets (existence guaranteed by *inconsistent-has-safe-recovery*).

definition *oss-carrier* :: *global-state set* **where**
oss-carrier = {*gs*. *finite-domain gs* $\wedge$ *no-locked-without-reason gs*}

definition *oss-step* ::
global-state $\Rightarrow$ (*chain-id* $\times$ *reg-action* $\times$ *asset-id*) $\Rightarrow$ *global-state option* **where**
oss-step gs p = (*case p of* (*src*, *act*, *aid*) $\Rightarrow$ *sync src act aid gs*)

definition *oss-ops* :: (*chain-id* $\times$ *reg-action* $\times$ *asset-id*) *set* **where**
oss-ops = *UNIV*

definition *oss-guard* ::
global-state $\Rightarrow$ (*chain-id* $\times$ *reg-action* $\times$ *asset-id*) $\Rightarrow$ *bool* **where**
oss-guard gs p = (*case p of* (*src*, *act*, *aid*) $\Rightarrow$ ($\exists$ *gs'*. *sync src act aid gs* = *Some gs'*))

definition *oss-realize* ::
node-info ⇒ *global-state* ⇒ (*chain-id* × *reg-action* × *asset-id*) *option* **where**
oss-realize ev gs =
(*if consistent-state gs then None else Some* (*SOME p. safe-recovery gs p*))

lemma *oss-step-closed*:
assumes *gs* ∈ *oss-carrier* **and** *oss-step gs p* = *Some gs'*
shows *gs'* ∈ *oss-carrier*
proof −
obtain *src act aid* **where** *p*: *p* = (*src*, *act*, *aid*) **by** (*cases p*)
with *assms*(*2*) **have** *synced*: *sync src act aid gs* = *Some gs'* **by** (*simp add*: *oss-step-def*)
from *assms*(*1*) **have** *finite-domain gs* **and** *no-locked-without-reason gs*
by (*simp-all add*: *oss-carrier-def*)
then have *finite-domain gs'* **and** *no-locked-without-reason gs'*
using *sync-preserves-finite-domain*[*OF* - *synced*] *sync-preserves-unlocked*[*OF* - *synced*] **by** *auto*
then show *?thesis* **by** (*simp add*: *oss-carrier-def*)
qed

lemma *oss-guarded-preservation*:
assumes *carr*: *gs* ∈ *oss-carrier* **and** *cons*: *consistent-state gs*
and *synced*: *oss-step gs p* = *Some gs'*
shows *consistent-state gs'*
proof −
obtain *src act aid* **where** *p*: *p* = (*src*, *act*, *aid*) **by** (*cases p*)
with *synced* **have** *s*: *sync src act aid gs* = *Some gs'* **by** (*simp add*: *oss-step-def*)
from *carr* **have** *nl*: *no-locked-without-reason gs* **and** *fd*: *finite-domain gs*
by (*simp-all add*: *oss-carrier-def*)
have *valid*: *valid-state gs* **using** *cons nl* **by** (*simp add*: *valid-state-def*)
from *sync-components*[*OF s*] **obtain** *current-st new-st gs-locked* **where**
cur: *get-reg-state gs src aid* = *Some current-st*
and *tr*: *reg-transition current-st act* = *Some new-st* **by** *blast*
have *ex*: *asset-exists gs src aid*
using *cur* **unfolding** *asset-exists-def get-reg-state-def get-asset-state-def*
by (*auto split*: *option.splits*)
have *finc*: *finite* (*connected-chains gs aid*) **using** *connected-chains-finite*[*OF fd*] .
show *?thesis*
using *sync-preserves-consistent-state*[*OF valid ex cur tr s finc*] **.**
qed

lemma *oss-realize-discharges*:
assumes *carr*: *gs* ∈ *oss-carrier* **and** *r*: *oss-realize ev gs* = *Some opn*
shows *opn* ∈ *oss-ops* ∧ *oss-guard gs opn*
proof −
from *r* **have** *ninc*: ¬ *consistent-state gs* **and** *opn*: *opn* = (*SOME p. safe-recovery gs p*)
by (*auto simp*: *oss-realize-def split*: *if-splits*)

from *carr* **have** *fd*: *finite-domain gs* **and** *nl*: *no-locked-without-reason gs*
by (*simp-all add*: *oss-carrier-def*)
have $\exists$ *p. safe-recovery gs p* **using** *inconsistent-has-safe-recovery*[*OF fd nl ninc*]
.
then have *safe-recovery gs opn* **using** *opn* **by** (*metis someI-ex*)
then have *reducing-sync gs opn* **by** (*rule safe-recovery-reducing*)
then obtain *src act aid gs'* **where** *opn* = (*src*, *act*, *aid*) **and** *sync src act aid gs* = *Some gs'*
unfolding *reducing-sync-def* **by** (*auto split*: *prod.splits*)
then have *oss-guard gs opn* **by** (*auto simp*: *oss-guard-def*)
then show *?thesis* **by** (*simp add*: *oss-ops-def*)
qed

lemma *oss-realize-progresses*:
assumes *consistent-state gs* **and** *oss-realize ev gs* = *Some opn*
shows $\exists$ *gs'*. *oss-step gs opn* = *Some gs'* $\wedge$ *consistent-state gs'*
using *assms* **by** (*simp add*: *oss-realize-def*)

lemma *oss-measure-zero-inv*:
assumes *gs* $\in$ *oss-carrier* **and** *inconsistency-pairs gs* = *0*
shows *consistent-state gs*
proof –
from *assms*(*1*) **have** *finite-domain gs* **by** (*simp add*: *oss-carrier-def*)
then show *?thesis* **using** *assms*(*2*) *inconsistency-pairs-zero-iff-consistent* **by** *blast*
qed

lemma *oss-discharge-progresses*:
assumes *carr*: *gs* $\in$ *oss-carrier* **and** *ninc*: $\neg$ *consistent-state gs*
shows $\exists$ *opn gs'*. *oss-realize ev gs* = *Some opn* $\wedge$ *oss-step gs opn* = *Some gs'*
$\wedge$ *inconsistency-pairs gs'* < *inconsistency-pairs gs*
proof –
from *carr* **have** *fd*: *finite-domain gs* **and** *nl*: *no-locked-without-reason gs*
by (*simp-all add*: *oss-carrier-def*)
have *ex*: $\exists$ *p. safe-recovery gs p* **using** *inconsistent-has-safe-recovery*[*OF fd nl ninc*] **.**
define *opn* **where** *opn* = (*SOME p. safe-recovery gs p*)
have *sr*: *safe-recovery gs opn* **unfolding** *opn-def* **using** *ex* **by** (*rule someI-ex*)
have *rs*: *reducing-sync gs opn* **using** *sr* **by** (*rule safe-recovery-reducing*)
have *realize*: *oss-realize ev gs* = *Some opn*
using *ninc* **unfolding** *oss-realize-def opn-def* **by** *simp*
from *rs* **obtain** *src act aid gs'* **where**
p: *opn* = (*src*, *act*, *aid*) **and** *s*: *sync src act aid gs* = *Some gs'*
and *dec*: *inconsistency-pairs gs'* < *inconsistency-pairs gs*
unfolding *reducing-sync-def* **by** (*auto split*: *prod.splits*)
have *oss-step gs opn* = *Some gs'* **using** *p s* **by** (*simp add*: *oss-step-def*)
with *realize dec* **show** *?thesis* **by** *blast*
qed

47 Guarded Bounded Convergence for the Oraclizer

Inside the D-quencer liveness context (a fair-leader assumption over a Byzantine-threshold cardinality bound), the oraclizer data above forms a *converging-composition*: the honest leader within every fairness window discharges a measure-reducing synchronization. The fairness assumption is exactly the *fair-leader* assumption of *dquencer-liveness* and the window positivity is its *fairness-positive* assumption (inherited from *dquencer-system*).

The fairness assumption of *dquencer-liveness* is satisfiable in every D-quencer system: the BFT threshold yields an honest node (*dquencer-system.honest-nonempty*), and the constant schedule on that node meets every fairness window. This grounds the assume-guarantee abstraction: the fair-leader hypothesis is the deterministic residue of VRF-based election, and the system's own threshold already supplies a witness schedule, so the hypothesis is satisfiable rather than merely plausible.

```
context dquencer-system
begin

lemma fair-schedule-exists:
  ∃ sched :: nat ⇒ node-info.
    ∀ epoch. ∃ e. epoch ≤ e ∧ e < epoch + fairness-bound ∧
             ni-behavior (sched e) = Honest
proof −
  obtain h where h ∈ honest-nodes nodes
    using honest-nonempty by blast
  then have hb: ni-behavior h = Honest by (simp add: honest-nodes-def)
  define sched :: nat ⇒ node-info where sched = (λ-. h)
  have body: ∀ epoch. ∃ e. epoch ≤ e ∧ e < epoch + fairness-bound
            ∧ ni-behavior (sched e) = Honest
  proof
    fix epoch :: nat
    have epoch ≤ epoch ∧ epoch < epoch + fairness-bound
        ∧ ni-behavior (sched epoch) = Honest
      using fairness-positive hb by (simp add: sched-def)
    then show ∃ e. epoch ≤ e ∧ e < epoch + fairness-bound
              ∧ ni-behavior (sched e) = Honest
      by blast
  qed
  show ?thesis
    by (rule exI[where P = λs :: nat ⇒ node-info.
                        ∀ epoch. ∃ e. epoch ≤ e ∧ e < epoch + fairness-bound
                          ∧ ni-behavior (s e) = Honest
              and x = sched, OF body])
qed
```

The Byzantine threshold is load-bearing. The fair schedule witnessed above is not idle: it instantiates the assume-guarantee liveness locale. For *every* D-quencer system the threshold therefore guarantees that *dquencer-liveness* is inhabitable — the fair-leader assumption is satisfiable rather than merely postulated. This is the downstream consumer that makes the entire Byzantine chain load-bearing: deleting any of *bft-threshold*, *honest-majority*, *honest-nonempty*, or *fair-schedule-exists* breaks the proof below. This load-bearing role is for the inhabitability result (`liveness_inhabitable`, a satisfiability witness for the locale, terminal); the headline bounded-convergence theorem does not route through it, being driven instead by the cross-chain inconsistency measure.

theorem *liveness-inhabitable*:
  $\exists$ (*ls* :: *nat* $\Rightarrow$ *node-info*) (*pc* :: *nat* $\Rightarrow$ *nat*).
    dquencer-liveness nodes f-max fairness-bound ls pc
proof −
  obtain *sched* :: *nat* $\Rightarrow$ *node-info* **where** *fair*:
    $\forall$ *epoch*. $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* $<$ *epoch* + *fairness-bound* $\wedge$
        ni-behavior (*sched e*) = *Honest*
   using *fair-schedule-exists* **by** *metis*
  have *dquencer-liveness nodes f-max fairness-bound sched* (λ-. *0*)
  proof *unfold-locales*
   show $\forall$ *epoch*. $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* $<$ *epoch* + *fairness-bound* $\wedge$
        ni-behavior (*sched e*) = *Honest*
    using *fair* **.**
  next
   fix *e* :: *nat*
   assume *0* $<$ (λ-::*nat*. *0*::*nat*) *e*
   then show (λ-::*nat*. *0*::*nat*) (*Suc e*) $<$ (λ-::*nat*. *0*::*nat*) *e* **by** *simp*
  next
   fix *e* :: *nat*
   show (λ-::*nat*. *0*::*nat*) (*Suc e*) $\leq$ (λ-::*nat*. *0*::*nat*) *e* **by** *simp*
  qed
  thus *?thesis* **by** *blast*
qed

end

context *dquencer-liveness*
begin

interpretation *oss*: *converging-composition*
  oss-carrier oss-step oss-ops consistent-state oss-guard
  leader-schedule λ*n*. *ni-behavior n* = *Honest fairness-bound oss-realize inconsistency-pairs*
proof *unfold-locales*
  show $\bigwedge$*s opn s'*. *s* $\in$ *oss-carrier* $\Longrightarrow$ *opn* $\in$ *oss-ops* $\Longrightarrow$ *oss-step s opn* = *Some s'*
     $\Longrightarrow$ *s'* $\in$ *oss-carrier*
   using *oss-step-closed* **by** *blast*

next
  show $\bigwedge s\ opn\ s'.\ s \in oss\text{-}carrier \Longrightarrow opn \in oss\text{-}ops \Longrightarrow consistent\text{-}state\ s$
  $\Longrightarrow oss\text{-}guard\ s\ opn \Longrightarrow oss\text{-}step\ s\ opn = Some\ s' \Longrightarrow consistent\text{-}state\ s'$
    using *oss-guarded-preservation* **by** *blast*
next
  show $0 < fairness\text{-}bound$ **using** *fairness-positive* **.**
next
  show $\forall t.\ \exists t'.\ t \leq t' \wedge t' < t + fairness\text{-}bound \wedge ni\text{-}behavior\ (leader\text{-}schedule\ t') = Honest$
    using *fair-leader* **.**
next
  show $\bigwedge s\ ev\ opn.\ s \in oss\text{-}carrier \Longrightarrow ni\text{-}behavior\ ev = Honest \Longrightarrow oss\text{-}realize\ ev\ s = Some\ opn$
  $\Longrightarrow opn \in oss\text{-}ops \wedge oss\text{-}guard\ s\ opn$
    using *oss-realize-discharges* **by** *blast*
next
  show $\bigwedge s\ ev\ opn.\ s \in oss\text{-}carrier \Longrightarrow consistent\text{-}state\ s \Longrightarrow ni\text{-}behavior\ ev = Honest$
  $\Longrightarrow oss\text{-}realize\ ev\ s = Some\ opn \Longrightarrow \exists s'.\ oss\text{-}step\ s\ opn = Some\ s' \wedge consistent\text{-}state\ s'$
    using *oss-realize-progresses* **by** *blast*
next
  show $\bigwedge s.\ s \in oss\text{-}carrier \Longrightarrow inconsistency\text{-}pairs\ s = 0 \Longrightarrow consistent\text{-}state\ s$
    using *oss-measure-zero-inv* **by** *blast*
next
  show $\bigwedge s\ ev.\ s \in oss\text{-}carrier \Longrightarrow \neg\ consistent\text{-}state\ s \Longrightarrow ni\text{-}behavior\ ev = Honest$
  $\Longrightarrow \exists\ opn\ s'.\ oss\text{-}realize\ ev\ s = Some\ opn \wedge oss\text{-}step\ s\ opn = Some\ s'$
  $\wedge\ inconsistency\text{-}pairs\ s' < inconsistency\text{-}pairs\ s$
    using *oss-discharge-progresses* **by** *blast*
qed

The headline of this entry's convergence layer. From an *arbitrary* finite-domain, unlocked global state — in particular with *no* assumption that the cross-chain consistency invariant holds initially — the oraclizer reaches, within $inconsistency\text{-}pairs\ gs_0 * fairness\text{-}bound$ evolution steps, a state that is valid (consistent and unlocked). Contrast *conditional-safety-preservation*, which assumes *valid-state gs*: the convergence layer upgrades conditional safety to unconditional bounded convergence.

theorem *oraclizer-guarded-bounded-convergence*:
  assumes *fin*: $finite\text{-}domain\ gs_0$
    and *unlocked*: $no\text{-}locked\text{-}without\text{-}reason\ gs_0$
  shows $\exists t \leq oss.convergence\text{-}bound\ gs_0.\ \exists gs_t.$
  $oss.evolves\text{-}to\ gs_0\ t\ gs_t \wedge valid\text{-}state\ gs_t$
proof −
  have *carr*: $gs_0 \in oss\text{-}carrier$ **using** *fin unlocked* **by** (*simp add*: *oss-carrier-def*)
  obtain $t\ gs_t$ **where** *tb*: $t \leq oss.convergence\text{-}bound\ gs_0$
    and *ev*: $oss.evolves\text{-}to\ gs_0\ t\ gs_t$ **and** *cons*: $consistent\text{-}state\ gs_t$
    using *oss.bounded-convergence-from-arbitrary*[*OF carr*] **by** *blast*
  — The reached state remains in the carrier, hence is unlocked, hence valid.

have gs_t = *oss.run-from 0* gs_0 *t*
using *ev* **by** (*simp add*: *oss.evolves-to-def oss.run-def*)
then have $gs_t \in$ *oss-carrier* **using** *oss.run-from-carrier*[*OF carr*] **by** *simp*
then have *no-locked-without-reason* gs_t **by** (*simp add*: *oss-carrier-def*)
then have *valid-state* gs_t **using** *cons* **by** (*simp add*: *valid-state-def*)
with *tb ev* **show** *?thesis* **by** *blast*
qed

Terminal faithfulness along whole recovery runs. Every evolution step either stutters or applies a realized safe recovery, and a safe recovery never confiscates an asset that carried no terminal holding (*safe-recovery-sync-no-fresh-terminal*); by induction the guarantee extends to the entire trajectory: an asset that starts with no recorded confiscation never acquires one, no matter how the run is scheduled. Together with *recovery-terminal-completion* — on a terminal-bearing asset a safe recovery can only complete the confiscation — this pins the recovery layer to the two behaviours the regulatory semantics admits, excluding both indiscriminate confiscation and confiscation erasure.

lemma *oss-evolve-step-no-fresh-terminal*:
assumes *carr*: *gs* ∈ *oss-carrier*
and *nt*: ¬ *terminal-present gs aid*
shows ¬ *terminal-present* (*oss.evolve-step t gs*) *aid*
proof (*cases ni-behavior* (*leader-schedule t*) = *Honest*)
case *False*
then show *?thesis* **using** *nt* **by** (*simp add*: *oss.evolve-step-def*)
next
case *True*
show *?thesis*
proof (*cases oss-realize* (*leader-schedule t*) *gs*)
case *None*
with *True* **show** *?thesis* **using** *nt* **by** (*simp add*: *oss.evolve-step-def*)
next
case (*Some opn*)
note *r* = *this*
show *?thesis*
proof (*cases oss-step gs opn*)
case *None*
with *True r* **show** *?thesis* **using** *nt* **by** (*simp add*: *oss.evolve-step-def*)
next
case (*Some gs′*)
note *st* = *this*
from *r* **have** *ninc*: ¬ *consistent-state gs*
and *opn-eq*: *opn* = (*SOME p. safe-recovery gs p*)
by (*auto simp*: *oss-realize-def split*: *if-splits*)
from *carr* **have** *fd*: *finite-domain gs* **and** *nl*: *no-locked-without-reason gs*
by (*simp-all add*: *oss-carrier-def*)
have ∃ *p. safe-recovery gs p* **using** *inconsistent-has-safe-recovery*[*OF fd nl ninc*] **.**
then have *sr*: *safe-recovery gs opn* **using** *opn-eq* **by** (*metis someI-ex*)

obtain *src act aid0* **where** *p*: *opn* = (*src*, *act*, *aid0*) **by** (*cases opn*)
have *synced*: *sync src act aid0 gs* = *Some gs′* **using** *st p* **by** (*simp add*: *oss-step-def*)
have ¬ *terminal-present gs′ aid*
using *safe-recovery-sync-no-fresh-terminal*[*OF sr*[*unfolded p*] *synced nt*] **.**
with *True r st* **show** *?thesis* **by** (*simp add*: *oss.evolve-step-def*)
qed
qed
qed

lemma *oss-run-from-no-fresh-terminal*:
assumes *carr*: gs_0 ∈ *oss-carrier*
and *nt*: ¬ *terminal-present* gs_0 *aid*
shows ¬ *terminal-present* (*oss.run-from k* gs_0 *n*) *aid*
proof (*induction n*)
case *0*
show *?case* **using** *nt* **by** *simp*
next
case (*Suc n*)
have *c*: *oss.run-from k* gs_0 *n* ∈ *oss-carrier* **using** *oss.run-from-carrier*[*OF carr*] **.**
show *?case* **using** *oss-evolve-step-no-fresh-terminal*[*OF c Suc.IH*] **by** *simp*
qed

theorem *recovery-no-fresh-terminal*:
assumes *carr*: gs_0 ∈ *oss-carrier*
and *nt*: ¬ *terminal-present* gs_0 *aid*
and *ev*: *oss.evolves-to* gs_0 *t* gs_t
shows ¬ *terminal-present* gs_t *aid*
proof −
have gs_t = *oss.run-from 0* gs_0 *t*
using *ev* **by** (*simp add*: *oss.evolves-to-def oss.run-def*)
then show *?thesis* **using** *oss-run-from-no-fresh-terminal*[*OF carr nt*] **by** *simp*
qed

end

48 Global Model Witnesses for the Liveness Hosts

The liveness interpretations above live inside *dquencer-liveness*, so they establish the consistency of their conclusions only relative to the host locale's assumptions. The interpretations below discharge those assumptions globally, with no ambient hypotheses: a single-honest-node system with zero Byzantine budget, unit lock timeout and unit fairness window, scheduled constantly on its one node, with no pending requests. Every assumption of *dquencer-system* and *dquencer-liveness* holds outright (the BFT threshold reads $1 \geq 3 \cdot 0 + 1$, the Byzantine census is empty, and the constant schedule meets every unit fairness window), so the assumption sets of the liveness hosts are consistent absolutely, not merely relative to a hypothetical deployment.

definition *singleton-node* :: *node-info* **where**
singleton-node = (| *ni-id* = *0*, *ni-behavior* = *Honest* |)

lemma *singleton-fair-leader*:
$\forall$ *epoch*. $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* < *epoch* + (*1*::*nat*)
$\wedge$ *ni-behavior* ((λ-. *singleton-node*) *e*) = *Honest*
proof
fix *epoch* :: *nat*
have *epoch* $\leq$ *epoch* $\wedge$ *epoch* < *epoch* + *1*
$\wedge$ *ni-behavior* ((λ-. *singleton-node*) *epoch*) = *Honest*
by (*simp add*: *singleton-node-def*)
then show $\exists$ *e*. *epoch* $\leq$ *e* $\wedge$ *e* < *epoch* + *1*
$\wedge$ *ni-behavior* ((λ-. *singleton-node*) *e*) = *Honest*
by *blast*
qed

interpretation *singleton-dq*: *dquencer-liveness*
{*singleton-node*} *0 1 0 0* λ-. *singleton-node* λ-. *0*
by *unfold-locales*
(*auto simp*: *byzantine-nodes-def singleton-node-def intro*: *singleton-fair-leader*)

The priority sublocale receives the analogous witness: the one-message set over the same singleton system. The well-formedness bounds hold with both maxima at *0*, and priority-key distinctness on a one-element set is immediate.

definition *singleton-msg* :: *dq-message* **where**
singleton-msg =
(| *msg-action* = *FREEZE*, *msg-asset-id* = *0*, *msg-authority* = *0*, *msg-timestamp* = *0*,
msg-source = *0*, *msg-targets* = {}, *dqm-authority-level* = *National*,
dqm-source-node = *0* |)

interpretation *singleton-dq-priority*: *dquencer-priority-concrete*
{*singleton-node*} *0 1 0 0* {*singleton-msg*}
by *unfold-locales*
(*auto simp*: *byzantine-nodes-def singleton-node-def singleton-msg-def*
intro: *singleton-fair-leader*)

A non-degenerate Byzantine witness. The singleton system above carries a zero Byzantine budget, so it meets the threshold vacuously ($1 \geq 3 \cdot 0 + 1$). The four-node system below exercises the threshold at its tight bound with a *real* fault: one Byzantine node among four ($4 \geq 3 \cdot 1 + 1$ and *card* (*byzantine-nodes*) = $1 \leq 1$), an honest super-majority of three, scheduled constantly on one honest node. It discharges every assumption of *dquencer-liveness* with *f-max* = *1*, so the Byzantine threshold is exercised at its tight bound by a real Byzantine-tagged node (a cardinality witness), not only the degenerate zero-fault case.

definition *quorum-nodes* :: *node-info set* **where**
quorum-nodes =
{⦇ *ni-id* = *0*, *ni-behavior* = *Byzantine* ⦈,
⦇ *ni-id* = *1*, *ni-behavior* = *Honest* ⦈,
⦇ *ni-id* = *2*, *ni-behavior* = *Honest* ⦈,
⦇ *ni-id* = *3*, *ni-behavior* = *Honest* ⦈}

definition *quorum-leader* :: *nat* ⇒ *node-info* **where**
quorum-leader = (λ-. ⦇ *ni-id* = *1*, *ni-behavior* = *Honest* ⦈)

lemma *quorum-finite*: *finite quorum-nodes*
by (*simp add*: *quorum-nodes-def*)

lemma *quorum-nonempty*: *quorum-nodes* ≠ {}
by (*simp add*: *quorum-nodes-def*)

lemma *quorum-card*: *card quorum-nodes* = *4*
by (*simp add*: *quorum-nodes-def*)

lemma *quorum-byzantine-eq*:
byzantine-nodes quorum-nodes = {⦇ *ni-id* = *0*, *ni-behavior* = *Byzantine* ⦈}
by (*auto simp*: *quorum-nodes-def byzantine-nodes-def*)

lemma *quorum-byz-card*: *card* (*byzantine-nodes quorum-nodes*) = *1*
by (*simp add*: *quorum-byzantine-eq*)

interpretation *bft-quorum*: *dquencer-liveness*
quorum-nodes 1 1 0 0 quorum-leader λ*e*. *1* − *e*
by *unfold-locales*
(*auto simp*: *quorum-card quorum-byz-card quorum-finite quorum-nonempty*
quorum-leader-def)

end

theory *Hierarchy*
imports *Functor-Laws*
begin

49 Sync Degree Stratification

A degree-k broadcast drives every chain $c \leq k$ that currently holds the asset *aid* to the regulatory value v, leaving every other chain and every other asset untouched. It is the state-level effect of synchronizing the asset across the chains coupled at degree k.

definition *broadcast-le* ::

```
  nat ⇒ asset-id ⇒ reg-state ⇒ global-state ⇒ global-state where
  broadcast-le k aid v gs =
    gs⦇ gs-chains :=
        (λc. if c ≤ k
             then (λaid′. if aid′ = aid
                          then map-option (λast. ast⦇ as-reg-state := v ⦈)
                                          (gs-chains gs c aid)
                          else gs-chains gs c aid′)
             else gs-chains gs c) ⦈
```

```
lemma broadcast-le-locks [simp]:
  gs-locks (broadcast-le k aid v gs) = gs-locks gs
  by (simp add: broadcast-le-def)
```

```
lemma broadcast-le-exists [simp]:
  asset-exists (broadcast-le k aid v gs) c aid′ = asset-exists gs c aid′
  by (auto simp: broadcast-le-def asset-exists-def get-asset-state-def
        split: option.splits)
```

```
lemma broadcast-le-reg-other-asset:
  aid′ ≠ aid ⟹
    get-reg-state (broadcast-le k aid v gs) c aid′ = get-reg-state gs c aid′
  by (simp add: broadcast-le-def get-reg-state-def get-asset-state-def)
```

```
lemma broadcast-le-reg-out:
  ¬ c ≤ k ⟹
    get-reg-state (broadcast-le k aid v gs) c aid′ = get-reg-state gs c aid′
  by (simp add: broadcast-le-def get-reg-state-def get-asset-state-def)
```

```
lemma broadcast-le-reg-in:
  c ≤ k ⟹
    get-reg-state (broadcast-le k aid v gs) c aid =
      (case get-reg-state gs c aid of None ⇒ None | Some - ⇒ Some v)
  by (simp add: broadcast-le-def get-reg-state-def get-asset-state-def
        map-option-case split: option.splits)
```

Pointwise reading of the chain map after a broadcast: a clean rewrite that avoids unfolding the raw record update inside nested terms.

```
lemma broadcast-le-chains:
  gs-chains (broadcast-le k aid v gs) c =
    (if c ≤ k
     then (λaid′. if aid′ = aid
                  then map-option (λast. ast⦇ as-reg-state := v ⦈) (gs-chains gs c aid)
                  else gs-chains gs c aid′)
     else gs-chains gs c)
  by (simp add: broadcast-le-def)
```

Degree demotion: *degree-forget j* drops the holdings of chain *j*, the top chain coupled one degree up. It keeps locks and every other chain.

definition *degree-forget* :: *nat* ⇒ *global-state* ⇒ *global-state* **where**
degree-forget j gs = *gs*⦇ *gs-chains* := (*gs-chains gs*)(*j* := (λ*aid*. *None*)) ⦈

lemma *degree-forget-locks* [*simp*]:
gs-locks (*degree-forget j gs*) = *gs-locks gs*
by (*simp add*: *degree-forget-def*)

lemma *degree-forget-reg-eq*:
c ≠ *j* ⟹ *get-reg-state* (*degree-forget j gs*) *c aid* = *get-reg-state gs c aid*
by (*simp add*: *degree-forget-def get-reg-state-def get-asset-state-def*)

lemma *degree-forget-exists*:
asset-exists (*degree-forget j gs*) *c aid* =
(*c* ≠ *j* ∧ *asset-exists gs c aid*)
by (*simp add*: *degree-forget-def asset-exists-def get-asset-state-def*)

lemma *degree-forget-chains*:
gs-chains (*degree-forget j gs*) *c* =
(*if c* = *j then* (λ*aid*. *None*) *else gs-chains gs c*)
by (*simp add*: *degree-forget-def*)

Demotion forgets, never invents: *degree-forget j gs* is a blinding refinement of *gs* in the sense of the glue layer of *Cross-Domain-State-Preservation.Functor-Laws*. Every chain still known after demotion carries exactly the regulatory state it had before. This is the state-level reading of the ADS blinding order *bo* (*state-refines*): degree demotion lands inside the partial-view preorder on which the authenticated layer is built, so the hierarchy sits on top of the functor laws rather than beside them.

lemma *degree-forget-refines*: *state-refines* (*degree-forget j gs*) *gs*
unfolding *state-refines-def*
proof (*intro allI impI*)
fix *aid c*
assume *c* ∈ *connected-chains* (*degree-forget j gs*) *aid*
then have *asset-exists* (*degree-forget j gs*) *c aid*
by (*simp add*: *connected-chains-def*)
then have *cne*: *c* ≠ *j* **and** *ce*: *asset-exists gs c aid*
by (*simp-all add*: *degree-forget-exists*)
from *ce* **have** *c* ∈ *connected-chains gs aid* **by** (*simp add*: *connected-chains-def*)
moreover have *get-reg-state* (*degree-forget j gs*) *c aid* = *get-reg-state gs c aid*
using *cne* **by** (*rule degree-forget-reg-eq*)
ultimately show
c ∈ *connected-chains gs aid* ∧
get-reg-state (*degree-forget j gs*) *c aid* = *get-reg-state gs c aid*
by *blast*
qed

The degree-k transition step. An operation (*act*, *aid*) reads the hub chain's regulatory state for *aid*, fires the regulatory transition, and broadcasts the result across the chains coupled at degree k. It is undefined exactly when the hub does not hold the asset or the regulatory action is not enabled there.

definition *deg-step* ::
$nat \Rightarrow (reg\text{-}action \times asset\text{-}id) \Rightarrow global\text{-}state \Rightarrow global\text{-}state\ option$ **where**
$deg\text{-}step\ k\ opn\ gs =$
$\quad (case\ opn\ of\ (act,\ aid) \Rightarrow$
$\quad\quad (case\ get\text{-}reg\text{-}state\ gs\ 0\ aid\ of$
$\quad\quad\quad None \Rightarrow None$
$\quad\quad | \ Some\ s \Rightarrow (case\ reg\text{-}transition\ s\ act\ of$
$\quad\quad\quad\quad\quad None \Rightarrow None$
$\quad\quad\quad\quad | \ Some\ s' \Rightarrow Some\ (broadcast\text{-}le\ k\ aid\ s'\ gs))))$

The carrier of the degree-k functor: states supported on chains $0..k$, i.e. only chains within the degree's coupling reach hold assets.

definition *deg-carrier* :: $nat \Rightarrow global\text{-}state\ set$ **where**
$deg\text{-}carrier\ k = \{gs.\ \forall\, c\ aid.\ asset\text{-}exists\ gs\ c\ aid \longrightarrow c \le k\}$

50 Degree Functors and Their Natural Transformations

We package each degree as the data of a transition functor: a set of states and an action-indexed family of partial transitions. Reading the one-object action category into *Set*, a natural transformation between two such functors is a single state-component map whose naturality squares commute. The conditions below mirror, on *global-state*, the well-definedness and naturality conditions of *state-preservation* (with the action map the identity, since the regulatory action vocabulary is shared across degrees); this is the same commuting square that defines the morphisms of the functor in *Cross-Domain-State-Preservation.Functor-Laws*.

record *gobj* =
$obj\text{-}states :: global\text{-}state\ set$
$obj\text{-}step\ \ :: (reg\text{-}action \times asset\text{-}id) \Rightarrow global\text{-}state \Rightarrow global\text{-}state\ option$

definition *F* :: $nat \Rightarrow gobj$ **where**
$F\ k = (\!|\ obj\text{-}states = deg\text{-}carrier\ k,\ obj\text{-}step = deg\text{-}step\ k\ |\!)$

definition *natural-transformation* ::
$gobj \Rightarrow gobj \Rightarrow (global\text{-}state \Rightarrow global\text{-}state) \Rightarrow bool$ **where**
$natural\text{-}transformation\ Fhi\ Flo\ \eta \longleftrightarrow$
$\quad (\forall\, gs.\ gs \in obj\text{-}states\ Fhi \longrightarrow \eta\ gs \in obj\text{-}states\ Flo)$
$\quad \wedge\ (\forall\, opn\ gs\ gs'.\ gs \in obj\text{-}states\ Fhi \longrightarrow obj\text{-}step\ Fhi\ opn\ gs = Some\ gs'$
$\quad\quad\quad \longrightarrow obj\text{-}step\ Flo\ opn\ (\eta\ gs) = Some\ (\eta\ gs'))$
$\quad \wedge\ (\forall\, opn\ gs.\ gs \in obj\text{-}states\ Fhi \longrightarrow obj\text{-}step\ Fhi\ opn\ gs = None$
$\quad\quad\quad \longrightarrow obj\text{-}step\ Flo\ opn\ (\eta\ gs) = None)$

Vertical composition of natural transformations is a natural transformation. This is the *global-state*-level analogue of *preservation-compose*: the naturality square of the lower transformation is stacked on that of the upper one, exactly as the second morphism's square is stacked on the first there. It is the structural fact that makes the whole degree ladder cohere.

lemma *nt-vertical-compose*:
  assumes *hi*: *natural-transformation A B* η
    and *lo*: *natural-transformation B C* ϑ
  shows *natural-transformation A C* $(\vartheta \circ \eta)$
proof −
  from *hi* **have**
      hi1: $\bigwedge gs.\ gs \in obj\text{-}states\ A \Longrightarrow \eta\ gs \in obj\text{-}states\ B$
    and *hi2*: $\bigwedge opn\ gs\ gs'.\ gs \in obj\text{-}states\ A \Longrightarrow obj\text{-}step\ A\ opn\ gs = Some\ gs'$
        $\Longrightarrow obj\text{-}step\ B\ opn\ (\eta\ gs) = Some\ (\eta\ gs')$
    and *hi3*: $\bigwedge opn\ gs.\ gs \in obj\text{-}states\ A \Longrightarrow obj\text{-}step\ A\ opn\ gs = None$
        $\Longrightarrow obj\text{-}step\ B\ opn\ (\eta\ gs) = None$
    unfolding *natural-transformation-def* **by** *blast+*
  from *lo* **have**
      lo1: $\bigwedge gs.\ gs \in obj\text{-}states\ B \Longrightarrow \vartheta\ gs \in obj\text{-}states\ C$
    and *lo2*: $\bigwedge opn\ gs\ gs'.\ gs \in obj\text{-}states\ B \Longrightarrow obj\text{-}step\ B\ opn\ gs = Some\ gs'$
        $\Longrightarrow obj\text{-}step\ C\ opn\ (\vartheta\ gs) = Some\ (\vartheta\ gs')$
    and *lo3*: $\bigwedge opn\ gs.\ gs \in obj\text{-}states\ B \Longrightarrow obj\text{-}step\ B\ opn\ gs = None$
        $\Longrightarrow obj\text{-}step\ C\ opn\ (\vartheta\ gs) = None$
    unfolding *natural-transformation-def* **by** *blast+*
  show *?thesis*
    unfolding *natural-transformation-def*
  proof (*intro conjI allI impI*)
    fix *gs* **assume** $gs \in obj\text{-}states\ A$
    then show $(\vartheta \circ \eta)\ gs \in obj\text{-}states\ C$
      using *hi1 lo1* **by** (*simp add*: *comp-def*)
  next
    fix *opn gs gs'*
    assume *a*: $gs \in obj\text{-}states\ A$ **and** *s*: $obj\text{-}step\ A\ opn\ gs = Some\ gs'$
    have $obj\text{-}step\ C\ opn\ (\vartheta\ (\eta\ gs)) = Some\ (\vartheta\ (\eta\ gs'))$
      using *lo2*[*OF hi1*[*OF a*] *hi2*[*OF a s*]] **.**
    then show $obj\text{-}step\ C\ opn\ ((\vartheta \circ \eta)\ gs) = Some\ ((\vartheta \circ \eta)\ gs')$
      by (*simp add*: *comp-def*)
  next
    fix *opn gs*
    assume *a*: $gs \in obj\text{-}states\ A$ **and** *s*: $obj\text{-}step\ A\ opn\ gs = None$
    have $obj\text{-}step\ C\ opn\ (\vartheta\ (\eta\ gs)) = None$
      using *lo3*[*OF hi1*[*OF a*] *hi3*[*OF a s*]] **.**
    then show $obj\text{-}step\ C\ opn\ ((\vartheta \circ \eta)\ gs) = None$
      by (*simp add*: *comp-def*)
  qed
qed

The key commutation: broadcasting at degree k after forgetting chain *Suc k* equals forgetting chain *Suc k* after broadcasting at degree *Suc k*. This is the heart of the naturality square: on chains $c \leq k$ both sides broadcast, on chain *Suc k* both sides clear, and on higher chains both sides leave *gs* alone.

```
lemma broadcast-forget-commute:
  broadcast-le k aid v (degree-forget (Suc k) gs)
   = degree-forget (Suc k) (broadcast-le (Suc k) aid v gs)
proof (rule global-state.equality, goal-cases chains locks more)
  case chains
  show ?case
  proof (intro ext)
    fix c aid'
    consider (le) c ≤ k | (top) c = Suc k | (gt) Suc k < c by linarith
    then show gs-chains (broadcast-le k aid v (degree-forget (Suc k) gs)) c aid'
              = gs-chains (degree-forget (Suc k) (broadcast-le (Suc k) aid v gs)) c
aid'
    proof cases
      case le
      then have c ≠ Suc k and c ≤ Suc k by linarith+
      with le show ?thesis
        by (simp add: broadcast-le-chains degree-forget-chains)
    next
      case top
      then have ¬ c ≤ k by linarith
      with top show ?thesis
        by (simp add: broadcast-le-chains degree-forget-chains)
    next
      case gt
      then have ¬ c ≤ k and ¬ c ≤ Suc k and c ≠ Suc k by linarith+
      then show ?thesis
        by (simp add: broadcast-le-chains degree-forget-chains)
    qed
  qed
next
  case locks then show ?case by simp
next
  case more then show ?case by simp
qed
```

The degree demotion *degree-forget* (*Suc k*) is a natural transformation *F* (*Suc k*) ⇒ *F k*: it maps degree-*Suc k* states to degree-*k* states, and its naturality square commutes (forget-after-step equals step-after-forget, in both the enabled and the disabled case). Because the hub chain *0* is never the dropped chain *Suc k*, demotion leaves the transition's enabling condition — the hub's regulatory state — untouched.

```
theorem degree-natural-transformation:
  natural-transformation (F (Suc k)) (F k) (degree-forget (Suc k))
  unfolding natural-transformation-def
proof (intro conjI allI impI)
  fix gs assume gsin: gs ∈ obj-states (F (Suc k))
  show degree-forget (Suc k) gs ∈ obj-states (F k)
```

```
    using gsin by (fastforce simp: F-def deg-carrier-def degree-forget-exists)
next
  fix opn gs gs'
  assume gs ∈ obj-states (F (Suc k))
     and step: obj-step (F (Suc k)) opn gs = Some gs'
  obtain act aid where opn: opn = (act, aid) by (cases opn)
  from step opn have deg-step (Suc k) (act, aid) gs = Some gs'
    by (simp add: F-def)
  then obtain s s' where
        hub: get-reg-state gs 0 aid = Some s
    and tr: reg-transition s act = Some s'
    and gs': gs' = broadcast-le (Suc k) aid s' gs
    by (auto simp: deg-step-def split: option.splits)
  have hub': get-reg-state (degree-forget (Suc k) gs) 0 aid = Some s
    using hub by (simp add: degree-forget-reg-eq)
  have obj-step (F k) opn (degree-forget (Suc k) gs)
        = Some (broadcast-le k aid s' (degree-forget (Suc k) gs))
    using hub' tr by (simp add: F-def deg-step-def opn)
  also have broadcast-le k aid s' (degree-forget (Suc k) gs)
            = degree-forget (Suc k) (broadcast-le (Suc k) aid s' gs)
    by (rule broadcast-forget-commute)
  also have ... = degree-forget (Suc k) gs' using gs' by simp
  finally show obj-step (F k) opn (degree-forget (Suc k) gs)
               = Some (degree-forget (Suc k) gs') .
next
  fix opn gs
  assume gs ∈ obj-states (F (Suc k))
     and step: obj-step (F (Suc k)) opn gs = None
  obtain act aid where opn: opn = (act, aid) by (cases opn)
  from step opn have none: deg-step (Suc k) (act, aid) gs = None
    by (simp add: F-def)
  have key: get-reg-state (degree-forget (Suc k) gs) 0 aid = get-reg-state gs 0 aid
    by (simp add: degree-forget-reg-eq)
  show obj-step (F k) opn (degree-forget (Suc k) gs) = None
  proof (cases get-reg-state gs 0 aid)
    case None
    then show ?thesis using key by (simp add: F-def deg-step-def opn)
  next
    case (Some s)
    with none have reg-transition s act = None
      by (simp add: deg-step-def split: option.splits)
    with key Some show ?thesis by (simp add: F-def deg-step-def opn)
  qed
qed
```

Decision B: natural transformations of degree functors compose. The two-step demotion $F\ (k+2) \Rightarrow F\ k$ is a natural transformation, obtained by vertically composing $F\ (k+2) \Rightarrow F\ (k+1)$ with $F\ (k+1) \Rightarrow F\ k$. This is what lifts the degree ladder from a point-to-point collection of forgetful maps to a structurally coherent tower: the whole chain $S_3 \Rightarrow S_2 \Rightarrow S_1 \Rightarrow S_0$ is natural at once. This is a layer that, to our knowledge, `ADS_Functor` does not develop: it closes functors under composition but does not build a hierarchy of functors with natural transformations between them.

theorem *nt-compose*:
  natural-transformation (*F* (*Suc* (*Suc k*))) (*F k*)
    (*degree-forget* (*Suc k*) ∘ *degree-forget* (*Suc* (*Suc k*)))
proof (*rule nt-vertical-compose*)
  show *natural-transformation* (*F* (*Suc* (*Suc k*))) (*F* (*Suc k*)) (*degree-forget* (*Suc* (*Suc k*)))
    using *degree-natural-transformation*[*of Suc k*] **.**
next
  show *natural-transformation* (*F* (*Suc k*)) (*F k*) (*degree-forget* (*Suc k*))
    using *degree-natural-transformation*[*of k*] **.**
qed

51 Functoriality of the Degree Transition Functor on Action Words

The natural-transformation results above operate on single regulatory operations. To make F a functor in the genuine sense — and the degree demotion a natural transformation on the whole one-object action category, not only on its generators — we must extend the transition step to action *words* and prove it preserves the category structure (identity word and word concatenation). The action category is the free monoid of regulatory action sequences; its non-degenerate content is supplied by the regulatory transition relation, which composes the sequence into a single composite state.

fun *reg-run* :: *reg-state* ⇒ *reg-action list* ⇒ *reg-state option* **where**
  reg-run s [] = *Some s*
| *reg-run s* (*a* # *w*) =
    (*case reg-transition s a of None* ⇒ *None* | *Some s′* ⇒ *reg-run s′ w*)

Regulatory composition: running a concatenated action word is running the first part and then, from the resulting state, the second. This is the monoid structure the degree functor must transport.

lemma *reg-run-append*:
  reg-run s (*v* @ *w*) = (*case reg-run s v of None* ⇒ *None* | *Some s′* ⇒ *reg-run s′ w*)
  by (*induction v arbitrary*: *s*) (*auto split*: *option.splits*)

fun *deg-run* ::
  nat ⇒ *asset-id* ⇒ *reg-action list* ⇒ *global-state* ⇒ *global-state option* **where**
  deg-run k aid [] *gs* = *Some gs*
| *deg-run k aid* (*a* # *w*) *gs* =
    (*case deg-step k* (*a*, *aid*) *gs of None* ⇒ *None* | *Some gs′* ⇒ *deg-run k aid w gs′*)

Broadcast overwrite: a second degree-k broadcast of the same asset obliterates the first. This is not a free-monoid triviality — it holds because the broadcast set is the chains that *already* hold the asset (which the first broadcast does not change) so the second writes over exactly the same chains. It is the structural reason a word of synchronizations collapses to a single one.

lemma *broadcast-le-overwrite*:
  broadcast-le k aid v2 (*broadcast-le k aid v1 gs*) = *broadcast-le k aid v2 gs*
proof (*rule global-state.equality*, *goal-cases chains locks more*)
  case *chains*
  show *?case*
  proof (*intro ext*)
    fix *c aid′*
    show *gs-chains* (*broadcast-le k aid v2* (*broadcast-le k aid v1 gs*)) *c aid′*
        = *gs-chains* (*broadcast-le k aid v2 gs*) *c aid′*
    proof (*cases c* ≤ *k*)
      case *True*
      show *?thesis*
        by (*cases aid′* = *aid*; *cases gs-chains gs c aid*)
          (*simp-all add*: *broadcast-le-chains True*)
    next
      case *False*
      then show *?thesis* **by** (*simp add*: *broadcast-le-chains*)
    qed
  qed
next
  case *locks* **then show** *?case* **by** (*simp add*: *broadcast-le-locks*)
next
  case *more* **then show** *?case* **by** *simp*
qed

Hub tracking: because the hub chain *0* is always within the coupling reach (*0* ≤ k), a broadcast that finds the asset at the hub leaves the hub carrying exactly the broadcast value, so the next step in a word reads the updated state.

lemma *broadcast-le-hub*:
  assumes *get-reg-state gs 0 aid* = *Some s0*
  shows *get-reg-state* (*broadcast-le k aid v gs*) *0 aid* = *Some v*
  using *assms broadcast-le-reg-in*[*of 0 k aid v gs*] **by** *simp*

The collapse, on an already-broadcast state. Running an action word at degree k from a state of the form *broadcast-le k aid v gs* reduces to the regulatory composite *reg-run v w* broadcast once. The induction is where the degree/broadcast structure does real work: each successive step reads the hub value left by the previous broadcast (*broadcast-le-hub*) and overwrites it (*broadcast-le-overwrite*).

lemma *deg-run-on-broadcast*:
  assumes *asset-exists gs 0 aid*

```
  shows deg-run k aid w (broadcast-le k aid v gs)
        = (case reg-run v w of None ⇒ None
           | Some sn ⇒ Some (broadcast-le k aid sn gs))
  using assms
proof (induction w arbitrary: v)
  case Nil
  show ?case by simp
next
  case (Cons a w)
  have hub: get-reg-state (broadcast-le k aid v gs) 0 aid = Some v
  proof −
    from Cons.prems obtain ast where get-asset-state gs 0 aid = Some ast
      by (auto simp: asset-exists-def)
    then have get-reg-state gs 0 aid = Some (as-reg-state ast)
      by (simp add: get-reg-state-def)
    then show ?thesis by (rule broadcast-le-hub)
  qed
  show ?case
  proof (cases reg-transition v a)
    case None
    then show ?thesis using hub by (simp add: deg-step-def)
  next
    case (Some v')
    have step: deg-step k (a, aid) (broadcast-le k aid v gs)
              = Some (broadcast-le k aid v' gs)
      using hub Some by (simp add: deg-step-def broadcast-le-overwrite)
    show ?thesis using Cons.IH[OF Cons.prems, of v'] Some by (simp add: step)
  qed
qed
```

The general collapse: a non-empty word of degree-k synchronizations on an asset held at the hub equals broadcasting the composite regulatory state once.

```
lemma deg-run-collapse:
  assumes get-reg-state gs 0 aid = Some s0
  shows deg-run k aid (a # w) gs
        = (case reg-run s0 (a # w) of None ⇒ None
           | Some sn ⇒ Some (broadcast-le k aid sn gs))
proof −
  have ex: asset-exists gs 0 aid
    using assms by (auto simp: asset-exists-def get-reg-state-def get-asset-state-def
                    split: option.splits)
  show ?thesis
  proof (cases reg-transition s0 a)
    case None
    then show ?thesis using assms by (simp add: deg-step-def)
  next
    case (Some s1)
    have deg-step k (a, aid) gs = Some (broadcast-le k aid s1 gs)
```

    using *assms Some* **by** (*simp add*: *deg-step-def*)
  then show *?thesis*
    using *deg-run-on-broadcast*[*OF ex*, *of k w s1*] *Some* **by** *simp*
 qed
qed

The degree-k step keeps a state inside the degree-k carrier: a broadcast rewrites regulatory values but never changes which chains hold the asset, so the support bound $c \leq k$ is preserved.

lemma *deg-step-carrier*:
 assumes $gs \in$ *deg-carrier k* **and** *deg-step k* (*a*, *aid*) *gs* = *Some gs′*
 shows *gs′* $\in$ *deg-carrier k*
proof −
 from *assms*(*2*) **obtain** *s s′* **where**
  get-reg-state gs 0 aid = *Some s* **and** *reg-transition s a* = *Some s′*
  and *gs′*: *gs′* = *broadcast-le k aid s′ gs*
  by (*auto simp*: *deg-step-def split*: *option.splits*)
 have $\bigwedge$*c aid″*. *asset-exists gs′ c aid″* = *asset-exists gs c aid″*
  unfolding *gs′* **by** (*simp add*: *broadcast-le-exists*)
 with *assms*(*1*) **show** *?thesis* **by** (*auto simp*: *deg-carrier-def*)
qed

Functor laws on action words. The identity word acts as the identity, and a concatenation of words is the Kleisli composition of the two segment maps: *deg-run* is a functor from the free monoid of regulatory action sequences into the Kleisli category of the option monad on *global-state*. These two laws are the categorical skeleton; the semantic content that the functor is non-degenerate — that its action on a word is the regulatory composite broadcast once, not an arbitrary fold — is *deg-run-collapse*.

lemma *deg-run-Nil*: *deg-run k aid* [] = *Some*
 by (*simp add*: *fun-eq-iff*)

lemma *deg-run-append*:
 deg-run k aid (*v* @ *w*) *gs* = *Option.bind* (*deg-run k aid v gs*) (*deg-run k aid w*)
 by (*induction v arbitrary*: *gs*) (*auto split*: *option.splits*)

Non-degeneracy witness. A genuine two-action regulatory word — *FREEZE* then *CONFISCATE* — composes to *CONFISCATED* through the intermediate *FROZEN*, and at the degree level its second synchronization overwrites the first: the word [*FREEZE*, *CONFISCATE*] broadcasts *CONFISCATED* while its prefix [*FREEZE*] broadcasts *FROZEN*. The two differ, so the composition is not idempotent padding; the functor transports real regulatory content.

lemma *deg-run-nondegenerate*:
 assumes *get-reg-state gs 0 0* = *Some ACTIVE*
 shows *deg-run k 0* [*FREEZE*, *CONFISCATE*] *gs* = *Some* (*broadcast-le k 0 CONFISCATED gs*)

and *deg-run k 0* [*FREEZE*] *gs* = *Some* (*broadcast-le k 0 FROZEN gs*)
and *broadcast-le k 0 CONFISCATED gs* ≠ *broadcast-le k 0 FROZEN gs*
∨ (∀ *c*≤*k*. ¬ *asset-exists gs c 0*)
proof −
have *f1*: *reg-transition ACTIVE FREEZE* = *Some FROZEN* **by** *simp*
have *f2*: *reg-transition FROZEN CONFISCATE* = *Some CONFISCATED* **by** *simp*
have *rc*: *reg-run ACTIVE* [*FREEZE*, *CONFISCATE*] = *Some CONFISCATED* **using** *f1 f2* **by** *simp*
have *rf*: *reg-run ACTIVE* [*FREEZE*] = *Some FROZEN* **using** *f1* **by** *simp*
show *deg-run k 0* [*FREEZE*, *CONFISCATE*] *gs* = *Some* (*broadcast-le k 0 CONFISCATED gs*)
using *deg-run-collapse*[*OF assms*, *of k FREEZE* [*CONFISCATE*]] *rc* **by** *simp*
show *deg-run k 0* [*FREEZE*] *gs* = *Some* (*broadcast-le k 0 FROZEN gs*)
using *deg-run-collapse*[*OF assms*, *of k FREEZE* []] *rf* **by** *simp*
show *broadcast-le k 0 CONFISCATED gs* ≠ *broadcast-le k 0 FROZEN gs*
∨ (∀ *c*≤*k*. ¬ *asset-exists gs c 0*)
proof (*cases* ∃ *c*≤*k*. *asset-exists gs c 0*)
case *True*
then obtain *c* **where** *ck*: *c* ≤ *k* **and** *ae*: *asset-exists gs c 0* **by** *blast*
from *ae* **obtain** *s* **where** *s*: *get-reg-state gs c 0* = *Some s*
by (*auto simp*: *asset-exists-def get-reg-state-def get-asset-state-def split*: *option.splits*)
have *c1*: *get-reg-state* (*broadcast-le k 0 CONFISCATED gs*) *c 0* = *Some CONFISCATED*
using *ck s* **by** (*simp add*: *broadcast-le-reg-in*)
have *c2*: *get-reg-state* (*broadcast-le k 0 FROZEN gs*) *c 0* = *Some FROZEN*
using *ck s* **by** (*simp add*: *broadcast-le-reg-in*)
have *broadcast-le k 0 CONFISCATED gs* ≠ *broadcast-le k 0 FROZEN gs*
proof
assume *broadcast-le k 0 CONFISCATED gs* = *broadcast-le k 0 FROZEN gs*
then have *get-reg-state* (*broadcast-le k 0 CONFISCATED gs*) *c 0*
= *get-reg-state* (*broadcast-le k 0 FROZEN gs*) *c 0* **by** *simp*
with *c1 c2* **show** *False* **by** *simp*
qed
then show *?thesis* **by** *blast*
next
case *False*
then show *?thesis* **by** *blast*
qed
qed

The single-step naturality square of *degree-natural-transformation*, re-exposed at the level of *deg-step* for use inside the word induction below.

lemma *deg-step-forget-some*:
assumes *gs* ∈ *deg-carrier* (*Suc k*) **and** *deg-step* (*Suc k*) *opn gs* = *Some gs′*
shows *deg-step k opn* (*degree-forget* (*Suc k*) *gs*) = *Some* (*degree-forget* (*Suc k*) *gs′*)
proof −

obtain *a b* **where** *opn*: *opn* = (*a*, *b*) **by** (*cases opn*)
show *?thesis*
using *degree-natural-transformation*[*of k*] *assms*
unfolding *opn natural-transformation-def F-def* **by** *auto*
qed

lemma *deg-step-forget-none*:
assumes *gs* ∈ *deg-carrier* (*Suc k*) **and** *deg-step* (*Suc k*) *opn gs* = *None*
shows *deg-step k opn* (*degree-forget* (*Suc k*) *gs*) = *None*
proof −
obtain *a b* **where** *opn*: *opn* = (*a*, *b*) **by** (*cases opn*)
show *?thesis*
using *degree-natural-transformation*[*of k*] *assms*
unfolding *opn natural-transformation-def F-def* **by** *auto*
qed

Naturality on the whole action category. The single-step square above lifts, by induction on the action word, to the statement that degree demotion commutes with running an *entire* word: forgetting the top chain and then running at degree *k* equals running at degree *Suc k* and then forgetting. Together with the functor laws *deg-run-Nil* and *deg-run-append*, this promotes *degree-natural-transformation* from a square on generators to a genuine natural transformation between the degree functors *F* (*Suc k*) and *F k* on the free action category — the functor-tower structure the entry claims.

theorem *degree-forget-natural-run*:
assumes *gs* ∈ *deg-carrier* (*Suc k*)
shows *deg-run k aid w* (*degree-forget* (*Suc k*) *gs*)
= *map-option* (*degree-forget* (*Suc k*)) (*deg-run* (*Suc k*) *aid w gs*)
using *assms*
proof (*induction w arbitrary*: *gs*)
case *Nil*
show *?case* **by** *simp*
next
case (*Cons a w*)
show *?case*
proof (*cases deg-step* (*Suc k*) (*a*, *aid*) *gs*)
case *None*
then have *deg-step k* (*a*, *aid*) (*degree-forget* (*Suc k*) *gs*) = *None*
using *deg-step-forget-none*[*OF Cons.prems*] **by** *simp*
then show *?thesis* **using** *None* **by** *simp*
next
case (*Some gs′*)
have *deg-step k* (*a*, *aid*) (*degree-forget* (*Suc k*) *gs*) = *Some* (*degree-forget* (*Suc k*) *gs′*)
using *deg-step-forget-some*[*OF Cons.prems Some*] **.**
moreover have *gs′* ∈ *deg-carrier* (*Suc k*)
using *deg-step-carrier*[*OF Cons.prems Some*] **.**
ultimately show *?thesis* **using** *Cons.IH Some* **by** *simp*
qed

qed

52 Degree Classes and Hierarchy Monotonicity

Each asset carries a required coupling strength, its *degree*, ranging over the four classes $S_0..S_3$. An asset belongs to the degree-k class when its required strength is at least k; since a requirement of at least k' entails a requirement of at least any $k \le k'$, the classes are nested (*reduction-containment*). A system, in turn, has a capability degree; it is the comparison of the asset's required degree with the system's capability degree that decides over- and under-provisioning.

The assignment of degrees to assets is operational data, not structure: the whole degree-class layer below is therefore parametric in an arbitrary assignment *asset-degree* :: *asset-id* ⇒ *nat*, fixed in an anonymous context. Every theorem of the layer holds for every assignment; a concrete example assignment witnesses non-vacuity at the end of the section.

Processing an asset at system degree d reconciles it across the chains the system couples (those with index $\le d$): it broadcasts the hub chain's regulatory value over that reach. Reconciling to the hub value is the state-level action of a degree-d synchronization. The processing machinery is independent of the degree assignment, so it lives outside the parametric context.

definition *process-at-degree* :: *nat* ⇒ *global-state* ⇒ *asset-id* ⇒ *global-state* **where**
  process-at-degree d gs aid =
    (*case get-reg-state gs 0 aid of*
      None ⇒ *gs*
    | *Some v* ⇒ *broadcast-le d aid v gs*)

On a consistent state every chain holding the asset already agrees with the hub, so broadcasting the hub value over any reach leaves the state consistent.

lemma *broadcast-le-consistent*:
  assumes *cons*: *consistent-state gs* **and** *hub*: *get-reg-state gs 0 aid* = *Some v*
  shows *consistent-state* (*broadcast-le d aid v gs*)
  unfolding *consistent-state-def*
proof (*intro allI impI*)
  fix *c1 c2 aid″ s1 s2*
  assume *g1*: *get-reg-state* (*broadcast-le d aid v gs*) *c1 aid″* = *Some s1*
    and *g2*: *get-reg-state* (*broadcast-le d aid v gs*) *c2 aid″* = *Some s2*
  have *val*: ⋀*c s. get-reg-state* (*broadcast-le d aid v gs*) *c aid″* = *Some s* ⟹ *aid″* = *aid* ⟹ *s* = *v*
  proof −
    fix *c s*
    assume *gc*: *get-reg-state* (*broadcast-le d aid v gs*) *c aid″* = *Some s* **and** *eqaid*: *aid″* = *aid*

```
    show s = v
    proof (cases c ≤ d)
      case True
      with gc eqaid have (case get-reg-state gs c aid of None ⇒ None | Some - ⇒
Some v) = Some s
        by (simp add: broadcast-le-reg-in)
      then show s = v by (auto split: option.splits)
    next
      case False
      with gc eqaid have get-reg-state gs c aid = Some s by (simp add: broad-
cast-le-reg-out)
      with cons hub show s = v unfolding consistent-state-def by blast
    qed
  qed
  show s1 = s2
  proof (cases aid'' = aid)
    case True
    from val[OF g1 True] val[OF g2 True] show ?thesis by simp
  next
    case False
    then have get-reg-state gs c1 aid'' = Some s1 and get-reg-state gs c2 aid'' =
Some s2
      using g1 g2 by (simp-all add: broadcast-le-reg-other-asset)
    then show ?thesis using cons unfolding consistent-state-def by blast
  qed
qed
```

Validity preservation under processing holds with no reference to degrees at all: on a consistent state the broadcast value agrees with every chain it overwrites, and the broadcast touches no lock. Isolating this as its own lemma records that the degree hypothesis of the over-provisioning headline below is not load-bearing for validity — the regime-sensitive content, where over- and under-provisioning genuinely diverge, is the pair *over-provisioning-guarantees* / *no-downward-safety*.

```
lemma processing-preserves-validity:
  assumes valid-state gs
  shows valid-state (process-at-degree d gs aid)
proof (cases get-reg-state gs 0 aid)
  case None
  then show ?thesis using assms by (simp add: process-at-degree-def)
next
  case (Some v)
  from assms have c: consistent-state gs and nl: no-locked-without-reason gs
    by (simp-all add: valid-state-def)
  have consistent-state (broadcast-le d aid v gs)
    using broadcast-le-consistent[OF c Some] .
  moreover have no-locked-without-reason (broadcast-le d aid v gs)
    using nl by (simp add: no-locked-without-reason-def is-locked-def)
  ultimately show ?thesis using Some by (simp add: process-at-degree-def valid-state-def)
```

qed

The timestamp order `lamport_hb` underlying the causal boundary is likewise independent of the degree assignment; it is a plain strict order on integer timestamps, not a distributed causal order.

definition *lamport-hb* :: *timestamp* ⇒ *timestamp* ⇒ *bool* **where**
lamport-hb t1 t2 ⟷ *t1* < *t2*

The degree-class layer, parametric in the degree assignment. Everything proved inside this context holds for an *arbitrary* assignment *asset-degree*; on export each constant and theorem generalises over the assignment.

context
fixes *asset-degree* :: *asset-id* ⇒ *nat*
begin

definition *degree-capable* :: *nat* ⇒ *asset-id* ⇒ *bool* **where**
degree-capable k aid ⟷ *k* ≤ *asset-degree aid*

Reduction containment: the degree classes form a descending chain, so membership in a higher class entails membership in every lower one. This is the partial-order closure of the hierarchy.

theorem *reduction-containment*:
k ≤ *k′* ⟹ *degree-capable k′ aid* ⟶ *degree-capable k aid*
unfolding *degree-capable-def* **by** *simp*

Validity preservation in the over-provisioning regime (auxiliary). A system preserves global validity when it processes an asset. This is a *degree-free* fact: its proof is exactly *processing-preserves-validity*, and it carries no provisioning hypothesis (an earlier redundant one was removed). It is therefore recorded as an auxiliary lemma aliasing that fact, *not* the headline monotonicity result. The genuinely degree-sensitive, one-directional monotonicity is carried by *over-provisioning-guarantees* together with *no-downward-safety* below, where the provisioning hypothesis is load-bearing.

lemma *hierarchy-monotonicity*:
assumes *valid-state gs*
shows *valid-state* (*process-at-degree system-degree gs aid*)
using *assms* **by** (*rule processing-preserves-validity*)

The coupling guarantee a degree-d system actually delivers: after processing, all of the asset's *required* chains — those within its degree class, index ≤ *asset-degree aid* — that hold the asset agree on its regulatory state.

definition *guarantees-preservation* :: *nat* ⇒ *global-state* ⇒ *asset-id* ⇒ *bool* **where**
guarantees-preservation d gs aid ⟷
(∀ *c1 c2*. *c1* ≤ *asset-degree aid* ⟶ *c2* ≤ *asset-degree aid*
⟶ *get-reg-state* (*process-at-degree d gs aid*) *c1 aid* ≠ *None*
⟶ *get-reg-state* (*process-at-degree d gs aid*) *c2 aid* ≠ *None*
⟶ *get-reg-state* (*process-at-degree d gs aid*) *c1 aid*

```
          = get-reg-state (process-at-degree d gs aid) c2 aid)
```

Under over-provisioning the guarantee is met: every required chain lies within the system's coupling reach ($\leq$ *asset-degree aid* $\leq d$), so all are driven to the single hub value.

```
theorem over-provisioning-guarantees:
  assumes deg: asset-degree aid ≤ d and val: valid-state gs
  shows guarantees-preservation d gs aid
proof (cases get-reg-state gs 0 aid)
  case None
  then have proc: process-at-degree d gs aid = gs by (simp add: process-at-degree-def)
  show ?thesis
    unfolding guarantees-preservation-def proc
  proof (intro allI impI)
    fix c1 c2
    assume c1 ≤ asset-degree aid and c2 ≤ asset-degree aid
       and get-reg-state gs c1 aid ≠ None and get-reg-state gs c2 aid ≠ None
    then obtain s1 s2 where
      s1: get-reg-state gs c1 aid = Some s1 and s2: get-reg-state gs c2 aid = Some
s2
      by auto
    have s1 = s2 using val s1 s2 unfolding valid-state-def consistent-state-def
by blast
    with s1 s2 show get-reg-state gs c1 aid = get-reg-state gs c2 aid by simp
  qed
next
  case (Some v)
  then have proc: process-at-degree d gs aid = broadcast-le d aid v gs
    by (simp add: process-at-degree-def)
  show ?thesis
    unfolding guarantees-preservation-def
  proof (intro allI impI)
    fix c1 c2
    assume c1d: c1 ≤ asset-degree aid and c2d: c2 ≤ asset-degree aid
       and d1: get-reg-state (process-at-degree d gs aid) c1 aid ≠ None
       and d2: get-reg-state (process-at-degree d gs aid) c2 aid ≠ None
    from c1d deg have l1: c1 ≤ d by simp
    from c2d deg have l2: c2 ≤ d by simp
    have get-reg-state (process-at-degree d gs aid) c1 aid = Some v
      using l1 d1 unfolding proc by (auto simp: broadcast-le-reg-in split: op-
tion.splits)
    moreover have get-reg-state (process-at-degree d gs aid) c2 aid = Some v
      using l2 d2 unfolding proc by (auto simp: broadcast-le-reg-in split: op-
tion.splits)
    ultimately show get-reg-state (process-at-degree d gs aid) c1 aid
              = get-reg-state (process-at-degree d gs aid) c2 aid by simp
  qed
qed
```

No downward safety. Under-provisioning carries no preservation guarantee: if the asset's required degree exceeds the system's capability, then some state defeats every preservation claim. The witness places the hub at *ACTIVE* and a required chain just beyond the system's reach (index *Suc system-degree*) at *FROZEN*; processing reconciles only the chains within reach, leaving the out-of-reach required chain in disagreement. This is the formal counterpart of over-provisioning safety: the monotonicity is genuinely one-directional.

```
theorem no-downward-safety:
  assumes asset-degree aid > system-degree
  shows ¬ (∀ gs. guarantees-preservation system-degree gs aid)
proof −
  define ast0 where ast0 = (| as-asset-id = aid, as-reg-state = ACTIVE |)
  define ast1 where ast1 = (| as-asset-id = aid, as-reg-state = FROZEN |)
  define gs0 where gs0 =
    (| gs-chains = (λc. if c = 0 then (λa. if a = aid then Some ast0 else None)
                        else if c = Suc system-degree then (λa. if a = aid then Some
ast1 else None)
                        else (λa. None)),
       gs-locks = (λa. False) |)
  have c0: get-reg-state gs0 0 aid = Some ACTIVE
    by (simp add: gs0-def get-reg-state-def get-asset-state-def ast0-def)
  have c1: get-reg-state gs0 (Suc system-degree) aid = Some FROZEN
    by (simp add: gs0-def get-reg-state-def get-asset-state-def ast1-def)
  have proc: process-at-degree system-degree gs0 aid = broadcast-le system-degree
aid ACTIVE gs0
    using c0 by (simp add: process-at-degree-def)
  have p0: get-reg-state (process-at-degree system-degree gs0 aid) 0 aid = Some
ACTIVE
    using proc c0 by (simp add: broadcast-le-reg-in)
  have p1: get-reg-state (process-at-degree system-degree gs0 aid) (Suc system-degree)
aid = Some FROZEN
    using proc c1 by (simp add: broadcast-le-reg-out)
  have le0: (0::nat) ≤ asset-degree aid by simp
  have le1: Suc system-degree ≤ asset-degree aid using assms by simp
  have ¬ guarantees-preservation system-degree gs0 aid
  proof
    assume guarantees-preservation system-degree gs0 aid
    then have get-reg-state (process-at-degree system-degree gs0 aid) 0 aid
              = get-reg-state (process-at-degree system-degree gs0 aid) (Suc sys-
tem-degree) aid
      unfolding guarantees-preservation-def using le0 le1 p0 p1 by blast
    with p0 p1 show False by simp
  qed
  then show ?thesis by blast
qed
```

The causal-consistency boundary. The classes S_1 and S_2 are separated by whether the asset requires *causal* consistency, in the sense of Lamport's happened-before order: assets at degree $\geq$ *2* do, lower ones do not. We model happened-before by the strict timestamp order and show the boundary is well-defined on (*asset*, *system*) pairs: the requirement is a single-valued function of the asset's degree, it is met whenever the system over-provisions the asset, and the underlying happened-before relation is a strict partial order (irreflexive and asymmetric; in fact linear, as timestamps are linearly ordered), so the causal boundary it induces is itself well-formed.

The degree index corresponds to the product's synchronization-degree hierarchy: $F\ k$ realises sync degree S_k for $k \leq 3$ (S0 static, S1 unidirectional observation, S2 bidirectional coupling, S3 atomic state binding), with the causal boundary at $k = 2$, the S1/S2 transition below.

definition *requires-causal* :: *asset-id* ⇒ *bool* **where**
requires-causal aid ⟷ *2* ≤ *asset-degree aid*

definition *causal-consistent-at* :: *asset-id* ⇒ *nat* ⇒ *bool* **where**
causal-consistent-at aid d ⟷ (*requires-causal aid* ⟶ *2* ≤ *d*)

theorem *boundary-well-defined*:
(*causal-consistent-at aid d* ⟷ (*2* ≤ *asset-degree aid* ⟶ *2* ≤ *d*))
∧ (*asset-degree aid* ≤ *d* ⟶ *causal-consistent-at aid d*)
∧ (∀ *t1 t2*. *lamport-hb t1 t2* ⟶ ¬ *lamport-hb t2 t1*)
∧ (∀ *t*. ¬ *lamport-hb t t*)
by (*auto simp*: *causal-consistent-at-def requires-causal-def lamport-hb-def*)

end

Non-vacuity of the parametric layer: a concrete example assignment, cycling the four degree classes over asset identifiers. The over-provisioning guarantee instantiates to it directly.

definition *example-degree* :: *asset-id* ⇒ *nat* **where**
example-degree aid = *aid mod 4*

corollary *example-degree-over-provisioning*:
example-degree aid ≤ *d* ⟹ *valid-state gs*
⟹ *guarantees-preservation example-degree d gs aid*
by (*rule over-provisioning-guarantees*)

Static promotion safety. A promotion re-assigns an asset's required degree; statically — between synchronizations — the change is just a different assignment parameter, so as long as the promoted degree stays within the system's capability, the over-provisioning guarantee transfers verbatim to the new assignment. In-flight promotion (a re-assignment crossing a live synchronization) is out of scope for this entry; subsequent work on retry-queue semantics covers it.

corollary *static-promotion-safety*:
assumes *asset-degree′ aid* ≤ *system-degree* **and** *valid-state gs*

shows *guarantees-preservation asset-degree′ system-degree gs aid*
using *assms* **by** (*rule over-provisioning-guarantees*)

end

theory *External-Instance*
imports *Proof-Automation*
begin

53 The TCP Endpoint State Machine (RFC 793 Subset)

A single-endpoint view of the TCP connection lifecycle, restricted to six states and five packet events. The transitions follow RFC 793: a passive open receives a *SYN* in *listen*; an active open in *SYN-sent* is completed by the peer's *SYN+ACK*; the passive side completes the handshake on the final *ACK*; a *FIN* begins the teardown, and the acknowledgement of the *FIN* closes the connection (the two *FIN−WAIT* stages of RFC 793 are folded into one, a standard simplification of the one-endpoint view). A *RST* aborts any connection in flight.

Two modelling notes. First, *CLOSED* is terminal: a re-connection is a new connection identity (a new tracker entry), not a transition of the old one. Second, a *RST* in *listen* is ignored by RFC 793; the partial transition function returns *None* there, as for every other state/event pair the subset does not admit.

datatype *tcp-state* =
LISTEN | *SYN-SENT* | *SYN-RCVD* | *ESTABLISHED* | *FIN-WAIT* | *CLOSED*

datatype *tcp-event* = *EvSyn* | *EvSynAck* | *EvAck* | *EvFin* | *EvRst*

fun *tcp-transition* :: *tcp-state* ⇒ *tcp-event* ⇒ *tcp-state option* **where**
tcp-transition LISTEN EvSyn = Some SYN-RCVD
| *tcp-transition SYN-SENT EvSynAck = Some ESTABLISHED*
| *tcp-transition SYN-RCVD EvAck = Some ESTABLISHED*
| *tcp-transition ESTABLISHED EvFin = Some FIN-WAIT*
| *tcp-transition FIN-WAIT EvAck = Some CLOSED*
| *tcp-transition SYN-SENT EvRst = Some CLOSED*
| *tcp-transition SYN-RCVD EvRst = Some CLOSED*
| *tcp-transition ESTABLISHED EvRst = Some CLOSED*
| *tcp-transition FIN-WAIT EvRst = Some CLOSED*
| *tcp-transition - - = None*

definition *tcp-states* :: *tcp-state set* **where**
tcp-states = {*LISTEN*, *SYN-SENT*, *SYN-RCVD*, *ESTABLISHED*, *FIN-WAIT*, *CLOSED*}

definition *tcp-events* :: *tcp-event set* **where**
tcp-events = {*EvSyn*, *EvSynAck*, *EvAck*, *EvFin*, *EvRst*}

definition *tcp-terminal* :: *tcp-state set* **where**
tcp-terminal = {*CLOSED*}

lemma *tcp-states-UNIV*: *tcp-states* = *UNIV*
unfolding *tcp-states-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *finite-tcp-states*: *finite tcp-states*
unfolding *tcp-states-def* **by** *simp*

lemma *finite-tcp-state-UNIV*: *finite* (*UNIV* :: *tcp-state set*)
using *finite-tcp-states* **by** (*simp add*: *tcp-states-UNIV*)

lemma *closed-terminal*: *tcp-transition CLOSED e* = *None*
by (*cases e*) *simp-all*

54 The Connection-Tracker Representation

The tracker entry is a structured record: a phase tag together with the peer-endpoint metadata the tracker keeps for a connection in flight. The well-formedness invariant ties the auxiliary field to the tag — the peer is recorded exactly while a connection attempt or connection exists — mirroring the layer-crossing pattern of the regulatory development. The tracker's state space is the image of the representation map, and its transition function is the lift of the endpoint transition through that map, guarded by the image domain.

datatype *ct-phase* =
CtNew | *CtSynSent* | *CtSynRecv* | *CtEstablished* | *CtFinWait* | *CtClosed*

record *ct-entry* =
ct-state :: *ct-phase*
ct-peer :: *nat option*

definition *default-peer* :: *nat* **where**
default-peer = *0*

fun *tcp-to-ct* :: *tcp-state* ⇒ *ct-entry* **where**
tcp-to-ct LISTEN = (| *ct-state* = *CtNew*, *ct-peer* = *None* |)
| *tcp-to-ct SYN-SENT* = (| *ct-state* = *CtSynSent*, *ct-peer* = *Some default-peer* |)
| *tcp-to-ct SYN-RCVD* = (| *ct-state* = *CtSynRecv*, *ct-peer* = *Some default-peer* |)
| *tcp-to-ct ESTABLISHED* = (| *ct-state* = *CtEstablished*, *ct-peer* = *Some default-peer* |)

| *tcp-to-ct FIN-WAIT* = (| *ct-state* = *CtFinWait*, *ct-peer* = *Some default-peer* |)
| *tcp-to-ct CLOSED* = (| *ct-state* = *CtClosed*, *ct-peer* = *None* |)

fun *ct-to-tcp* :: *ct-entry* ⇒ *tcp-state* **where**
ct-to-tcp p = (*case ct-state p of*
CtNew ⇒ *LISTEN*
| *CtSynSent* ⇒ *SYN-SENT*
| *CtSynRecv* ⇒ *SYN-RCVD*
| *CtEstablished* ⇒ *ESTABLISHED*
| *CtFinWait* ⇒ *FIN-WAIT*
| *CtClosed* ⇒ *CLOSED*)

definition *valid-ct-entry* :: *ct-entry* ⇒ *bool* **where**
valid-ct-entry p ⟷ (*ct-state p* ∈ {*CtNew*, *CtClosed*}) = (*ct-peer p* = *None*)

lemma *tcp-to-ct-valid*: *valid-ct-entry* (*tcp-to-ct s*)
by (*cases s*) (*auto simp*: *valid-ct-entry-def*)

lemma *ct-to-tcp-to-ct-id*: *ct-to-tcp* (*tcp-to-ct s*) = *s*
by (*cases s*) *simp-all*

definition *ct-states* :: *ct-entry set* **where**
ct-states = *range tcp-to-ct*

definition *ct-terminal* :: *ct-entry set* **where**
ct-terminal = {*tcp-to-ct CLOSED*}

55 The Tracked Event Subset and the Tracker Transition

The tracker observes the wire-visible handshake and teardown packets and does not track *RST*: an aborted connection is expired by the tracker's timeout machinery rather than by following the reset packet, the standard conntrack behaviour. The source action set of the preservation morphism is therefore the strict subset of tracked events — the action-scoping pattern — and the tracker has its own event alphabet, mapped one-to-one from that subset.

datatype *ct-event* = *CT-SYN* | *CT-SYNACK* | *CT-ACK* | *CT-FIN*

definition *tracked-events* :: *tcp-event set* **where**
tracked-events = {*EvSyn*, *EvSynAck*, *EvAck*, *EvFin*}

definition *ct-events* :: *ct-event set* **where**
ct-events = {*CT-SYN*, *CT-SYNACK*, *CT-ACK*, *CT-FIN*}

fun *tcp-to-ct-event* :: *tcp-event* ⇒ *ct-event* **where**

 *tcp-to-ct-event EvSyn* = *CT-SYN*
| *tcp-to-ct-event EvSynAck* = *CT-SYNACK*
| *tcp-to-ct-event EvAck* = *CT-ACK*
| *tcp-to-ct-event EvFin* = *CT-FIN*
| *tcp-to-ct-event EvRst* = *CT-SYN*

— The last clause is unused by the morphism, since $actions_s$ is the tracked subset, which excludes *EvRst*; it only makes the function total, exactly as in the escalation action map.

fun *ct-event-to-tcp* :: *ct-event* ⇒ *tcp-event* **where**
ct-event-to-tcp CT-SYN = *EvSyn*
| *ct-event-to-tcp CT-SYNACK* = *EvSynAck*
| *ct-event-to-tcp CT-ACK* = *EvAck*
| *ct-event-to-tcp CT-FIN* = *EvFin*

definition *ct-transition* :: *ct-entry* ⇒ *ct-event* ⇒ *ct-entry option* **where**
ct-transition p e =
(*if p* ∈ *ct-states*
then map-option tcp-to-ct (*tcp-transition* (*ct-to-tcp p*) (*ct-event-to-tcp e*))
else None)

lemma *ct-event-roundtrip*:
e ∈ *tracked-events* ⟹ *ct-event-to-tcp* (*tcp-to-ct-event e*) = *e*
by (*auto simp*: *tracked-events-def*)

56 Discharging the Instance with the Generic Methods

The helper lemmas mirror, point for point, those of the regulatory development: finiteness in the simplifier's normal form, terminal absorption, closure, domain, the naturality of the representation map on the tracked subset, and the terminal/well-definedness facts of the map. They are declared into the discharge collections, after which the machine and preservation instances close by one method invocation each.

lemma *ct-states-finite*: *finite ct-states*
by (*auto simp*: *ct-states-def finite-tcp-state-UNIV intro*: *finite-imageI*)

lemma *ct-terminal-subset*: *ct-terminal* ⊆ *ct-states*
unfolding *ct-terminal-def ct-states-def* **by** *blast*

lemma *ct-closed-absorbing*:
⟦ *p* ∈ *ct-terminal*; *e* ∈ *ct-events* ⟧ ⟹ *ct-transition p e* = *None*
by (*auto simp*: *ct-terminal-def ct-transition-def ct-to-tcp-to-ct-id closed-terminal*)

lemma *ct-transition-closed*:
⟦ *p* ∈ *ct-states*; *e* ∈ *ct-events*; *ct-transition p e* = *Some p'* ⟧ ⟹ *p'* ∈ *ct-states*
unfolding *ct-transition-def ct-states-def*

by (*auto split*: *if-splits option.splits*)

lemma *ct-transition-outside-states*:
$p \notin$ *ct-states* $\Longrightarrow$ *ct-transition p e* = *None*
by (*simp add*: *ct-transition-def*)

lemma *tcp-to-ct-in-states*: *tcp-to-ct s* $\in$ *ct-states*
by (*simp add*: *ct-states-def*)

lemma *tcp-to-ct-terminal*:
$s \in$ *tcp-terminal* $\Longrightarrow$ *tcp-to-ct s* $\in$ *ct-terminal*
by (*auto simp*: *tcp-terminal-def ct-terminal-def*)

lemma *tcp-to-ct-naturality-some*:
$[\![$ $s \in$ *tcp-states*; $e \in$ *tracked-events*; *tcp-transition s e* = *Some s'* $]\!]$
$\Longrightarrow$ *ct-transition* (*tcp-to-ct s*) (*tcp-to-ct-event e*) = *Some* (*tcp-to-ct s'*)
by (*simp add*: *ct-transition-def tcp-to-ct-in-states ct-to-tcp-to-ct-id*
ct-event-roundtrip
del: *ct-to-tcp.simps tcp-to-ct.simps*)

lemma *tcp-to-ct-naturality-none*:
$[\![$ $s \in$ *tcp-states*; $e \in$ *tracked-events*; *tcp-transition s e* = *None* $]\!]$
$\Longrightarrow$ *ct-transition* (*tcp-to-ct s*) (*tcp-to-ct-event e*) = *None*
by (*simp add*: *ct-transition-def tcp-to-ct-in-states ct-to-tcp-to-ct-id*
ct-event-roundtrip
del: *ct-to-tcp.simps tcp-to-ct.simps*)

lemmas [*discharge-simps*] =
tcp-states-UNIV finite-tcp-state-UNIV
tcp-events-def tracked-events-def ct-events-def
tcp-terminal-def closed-terminal
ct-states-finite ct-terminal-subset
ct-closed-absorbing ct-transition-outside-states
tcp-to-ct-naturality-some tcp-to-ct-naturality-none
tcp-to-ct-terminal tcp-to-ct-in-states

lemmas [*discharge-intros*] =
ct-transition-closed

lemmas [*discharge-dels*] =
tcp-to-ct.simps ct-to-tcp.simps

The two state machines and the preservation morphism, each discharged by the corresponding generic method. No obligation is weakened: the statements are the full locale predicates.

theorem *tcp-state-machine*:
state-machine tcp-states tcp-events tcp-transition tcp-terminal
by *discharge-state-machine*

theorem *conntrack-state-machine*:
  state-machine ct-states ct-events ct-transition ct-terminal
  by *discharge-state-machine*

theorem *tcp-conntrack-preservation*:
  state-preservation tcp-states tracked-events tcp-transition tcp-terminal
    ct-states ct-events ct-transition ct-terminal
    tcp-to-ct tcp-to-ct-event
  by *discharge-preservation*

Registered form of the morphism, giving access to the locale's sequential payload: every tracked packet sequence accepted by the endpoint is mirrored, packet for packet, by the tracker, ending in the corresponding tracker entry.

interpretation *tcp-ct*:
  state-preservation tcp-states tracked-events tcp-transition tcp-terminal
    ct-states ct-events ct-transition ct-terminal
    tcp-to-ct tcp-to-ct-event
  by (*rule tcp-conntrack-preservation*)

corollary *tracked-sequence-mirrored*:
  assumes $s \in$ *tcp-states*
    and $\forall\, e \in$ *set es*. $e \in$ *tracked-events*
    and *tcp-ct.source.apply-actions s es* = *Some* s'
  shows *tcp-ct.target.apply-actions* (*tcp-to-ct s*) (*map tcp-to-ct-event es*)
      = *Some* (*tcp-to-ct* s')
  using *assms* **by** (*rule tcp-ct.sequential-preservation*)

Non-vacuity witnesses: the three-way handshake is accepted by the endpoint machine, and its tracked image steps the tracker entry from the fresh entry to the established entry.

lemma *three-way-handshake-endpoint*:
  tcp-transition LISTEN EvSyn = *Some SYN-RCVD*
  tcp-transition SYN-RCVD EvAck = *Some ESTABLISHED*
  by *simp-all*

lemma *three-way-handshake-tracked*:
  ct-transition (*tcp-to-ct LISTEN*) *CT-SYN* = *Some* (*tcp-to-ct SYN-RCVD*)
  ct-transition (*tcp-to-ct SYN-RCVD*) *CT-ACK* = *Some* (*tcp-to-ct ESTABLISHED*)
  by (*simp-all add*: *ct-transition-def tcp-to-ct-in-states ct-to-tcp-to-ct-id*
    del: *ct-to-tcp.simps tcp-to-ct.simps*)

end

theory *Canton-Bridge*
  imports
    Functor-Laws
    ADS-Functor.ADS-Construction
    ADS-Functor.Canton-Transaction-Tree

begin

57 Auxiliary list predicate for synchronization sequences

A blinding path: each element of the list is a blinding of its successor. As blindings narrow what is revealed, the last element is the most-revealed view of the sequence and every earlier element is a partial view of it.

definition *blinding-path* :: $('d \Rightarrow 'd \Rightarrow bool) \Rightarrow 'd\ list \Rightarrow bool$ **where**
blinding-path bl xs $\longleftrightarrow (\forall\, i.\ Suc\ i < length\ xs \longrightarrow bl\ (xs\ !\ i)\ (xs\ !\ Suc\ i))$

58 Composite functor laws: cross-domain preservation after the ADS Merkle functor

Inside *authenticated-state* the ADS layer supplies a thin category: objects are authenticated values, and the unique morphism $a \to b$ is the blinding relation *bo a b*, a preorder (reflexive and transitive by the Merkle interface). The cross-domain layer supplies another thin category: objects are global states and the morphism $s \to s'$ is *state-refines* $s\ s'$, also a preorder. The extraction map *extract-map* is the composite functor from the first category to the second: it sends a value to the state it extracts to, and a blinding morphism to a refinement morphism. The next four results are the category axioms of this composite functor together with the associativity of the lifted merge (join) algebra.

context *authenticated-state*
begin

Identity is preserved: the identity (blinding) morphism on *a*, which exists by reflexivity of *bo*, is sent to the identity (refinement) morphism on the extracted state.

lemma *cdsp-ads-id*:
assumes *extract-map a = Some sa*
shows *bo a a* $\wedge$ *valid-state sa* $\wedge$ *state-refines sa sa*
using *mk.reflp extract-preserves-validity*[*OF assms*] *state-refines-refl*
by (*simp add*: *reflp-def*)

The functor's action on a single morphism: a blinding $a \to b$ is sent to a refinement *extract a* $\to$ *extract b*, with both extracted states valid. This is *blinded-view-preserves-validity* with the validity hypothesis on *b* discharged internally from *extract-preserves-validity*, so the morphism map is total on the blinding preorder.

lemma *cdsp-ads-morphism*:
assumes *bo*: *bo a b* **and** *eb*: *extract-map b = Some sb*

shows ∃ *sa. extract-map a = Some sa* ∧ *valid-state sa* ∧ *valid-state sb* ∧ *state-refines sa sb*
proof –
  have *hab*: *h a = h b* **using** *bo-hash-eq*[*OF bo*] **.**
  obtain *sa* **where** *ea*: *extract-map a = Some sa* **and** *sr*: *state-refines sa sb*
    using *extract-under-blinding*[*OF hab bo eb*] **by** *blast*
  have *valid-state sa* **using** *extract-preserves-validity*[*OF ea*] **.**
  moreover have *valid-state sb* **using** *extract-preserves-validity*[*OF eb*] **.**
  ultimately show *?thesis* **using** *ea sr* **by** *blast*
qed

Composition is preserved (closed). Two composable blinding morphisms $a \to b$ and $b \to c$ compose in the source category to $a \to c$ (transitivity of *bo*), and the functor sends the composite to the composite of the images *extract a* → *extract b* → *extract c* (transitivity of *state-refines*). This is the cross-domain, authenticated analogue of *preservation-compose*.

theorem *cdsp-ads-compose*:
  assumes *ab*: *bo a b* **and** *bc*: *bo b c* **and** *ec*: *extract-map c = Some sc*
  shows *bo a c*
      ∧ (∃ *sa sb. extract-map a = Some sa* ∧ *extract-map b = Some sb*
          ∧ *valid-state sa* ∧ *valid-state sb* ∧ *valid-state sc*
          ∧ *state-refines sa sb* ∧ *state-refines sb sc* ∧ *state-refines sa sc*)
proof –
  have *boac*: *bo a c* **using** *mk.transp ab bc* **by** (*metis transpD*)
  obtain *sb* **where** *eb*: *extract-map b = Some sb*
    and *srbc*: *state-refines sb sc* **and** *vsb*: *valid-state sb* **and** *vsc*: *valid-state sc*
    using *cdsp-ads-morphism*[*OF bc ec*] **by** *blast*
  obtain *sa* **where** *ea*: *extract-map a = Some sa*
    and *srab*: *state-refines sa sb* **and** *vsa*: *valid-state sa*
    using *cdsp-ads-morphism*[*OF ab eb*] **by** *blast*
  have *srac*: *state-refines sa sc* **using** *state-refines-trans*[*OF srab srbc*] **.**
  show *?thesis* **using** *boac ea eb vsa vsb vsc srab srbc srac* **by** *blast*
qed

Associativity. Three composable blinding morphisms yield the same composite $a \to d$ regardless of the bracketing; in this thin category the composite morphism is the relation *bo a d* (and its image *state-refines sa sd*), which is bracketing-independent because the relations are propositional. The substantive associativity of the lifted merge appears in the merge-associativity result below.

theorem *cdsp-ads-assoc*:
  assumes *ab*: *bo a b* **and** *bc*: *bo b c* **and** *cd*: *bo c d*
    and *ed*: *extract-map d = Some sd*
  shows *bo a d* ∧ (∃ *sa. extract-map a = Some sa* ∧ *valid-state sa* ∧ *state-refines sa sd*)
proof –
  have *bo b d* **using** *mk.transp bc cd* **by** (*metis transpD*)
  hence *boad*: *bo a d* **using** *mk.transp ab* **by** (*metis transpD*)

obtain *sa* **where** *extract-map a = Some sa valid-state sa state-refines sa sd*
using *cdsp-ads-morphism*[*OF boad ed*] **by** *blast*
thus *?thesis* **using** *boad* **by** *blast*
qed

The functor's action on the monoidal (merge) product: a merge of two views is sent to a join of the two extracted states, with the joined state valid and refining each input. This lifts *extract-respects-merging* to a theorem carrying validity, and is exactly *authenticated-preservation-soundness* read as the product map of the composite functor.

lemma *cdsp-ads-merge*:
assumes *mab*: *m a b = Some ab*
and *ea*: *extract-map a = Some sa* **and** *eb*: *extract-map b = Some sb*
shows $\exists$ *sab. extract-map ab = Some sab* $\wedge$ *valid-state sab*
$\wedge$ *state-refines sa sab* $\wedge$ *state-refines sb sab*
using *authenticated-preservation-soundness*[*OF mab ea*
extract-preserves-validity[*OF ea*] *eb extract-preserves-validity*[*OF eb*]]
by *blast*

Associativity of the lifted merge/join algebra. The ADS merge is associative (Merkle-interface axiom *mk.assoc*), so the three-way combination of authenticated views is independent of the bracketing: both $(a \cdot b) \cdot c$ and $a \cdot (b \cdot c)$ land on the same merged value, hence (the extraction being a function) on the same joined state. This is the substantive "associativity preserved" of the composite functor.

theorem *cdsp-ads-merge-assoc*:
assumes *mab*: *m a b = Some ab* **and** *mabc*: *m ab c = Some abc*
shows $\exists$ *bc. m b c = Some bc* $\wedge$ *m a bc = Some abc*
proof −
have $m\ b\ c \gg\!= m\ a = m\ a\ b \gg\!= m\ c$ **by** (*rule mk.assoc*[*symmetric*])
also have $m\ a\ b \gg\!= m\ c = m\ c\ ab$ **using** *mab* **by** *simp*
also have *m c ab = m ab c* **by** (*rule mk.commute*)
also have ... = *Some abc* **using** *mabc* **by** *simp*
finally have $m\ b\ c \gg\!= m\ a = Some\ abc$ **.**
thus *?thesis* **by** (*cases m b c*) *auto*
qed

end

59 Authenticity preserved along whole synchronization sequences

Four authenticity properties are shown to hold not across a single blinding or merge step but along the *whole* synchronization sequence:

1. **validity** — every view in the sequence extracts to a valid state (theorem *sequence-authenticity-preservation*);

2. **need-to-know (refinement)** — every view is a partial view refining the most-revealed endpoint (theorem *sequence-authenticity-preservation*);

3. **hash soundness** — the whole sequence commits to one authenticating root, in both the blinding direction (theorem *blinding-path-hash-soundness*) and the merge direction (theorem *merge-seq-hash*);

4. **inclusion integrity** — theorem *sequence-inclusion-integrity*.

Scope qualifier (read carefully). The inclusion result is *state-level*: it is about *revealed holdings* — a holding exhibited by any view in the sequence is genuinely present in the endpoint with the same regulatory state, and no view fabricates a chain the endpoint does not authenticate. This is *not* a claim about the *concrete Merkle inclusion path* (the witness list authenticating a leaf against a root hash); that construction lives in *ADS-Functor.Inclusion-Proof-Construction* and belongs to the concrete recursive-tree layer, not to this abstract layer.

context *authenticated-state*
begin

59.1 Need-to-know blinding along a sequence

Along a blinding path, an earlier (more blinded) element blinds any later element: the blinding relation is monotone along the path by transitivity.

```
lemma blinding-path-mono:
  assumes path: blinding-path bo xs and ij: i ≤ j and j: j < length xs
  shows bo (xs ! i) (xs ! j)
proof −
  from ij obtain d where jd: j = i + d using le-Suc-ex by blast
  from j jd show ?thesis unfolding jd
  proof (induction d arbitrary: j)
    case 0
    show ?case using mk.reflp by (simp add: reflp-def)
  next
    case (Suc d)
    have lt: i + d < length xs using Suc.prems by simp
    have ih: bo (xs ! i) (xs ! (i + d)) using Suc.IH lt by simp
    have step: bo (xs ! (i + d)) (xs ! Suc (i + d))
      using path Suc.prems unfolding blinding-path-def by simp
    show ?case using mk.transp ih step by (metis transpD add-Suc-right)
  qed
qed
```

Sequence-level need-to-know preservation. Given a synchronization sequence presented as a blinding path whose most-revealed endpoint (the last element) extracts to a valid cross-domain state, *every* intermediate view extracts to a valid state that refines the endpoint. Authenticity (validity together with the refinement guarantee of a partial view) is preserved along the whole sequence, not merely across one blinding step. This generalises *blinded-view-preserves-validity*.

theorem *sequence-authenticity-preservation*:
  assumes *path*: *blinding-path bo xs* **and** *ne*: $xs \neq []$
    and *elast*: *extract-map* (*last xs*) = *Some s-last*
    and *i*: $i <$ *length xs*
  shows $\exists\, s_i$. *extract-map* (*xs* ! *i*) = *Some* s_i $\wedge$ *valid-state* s_i $\wedge$ *state-refines* s_i *s-last*
proof −
  have *le*: $i \leq$ *length xs* − *1* **using** *i* **by** *simp*
  have *lt*: *length xs* − *1* < *length xs* **using** *ne* **by** (*cases xs*) *auto*
  have *bo* (*xs* ! *i*) (*xs* ! (*length xs* − *1*))
    using *blinding-path-mono*[*OF path le lt*] **.**
  also have *xs* ! (*length xs* − *1*) = *last xs* **using** *ne* **by** (*simp add*: *last-conv-nth*)
  finally have *bo* (*xs* ! *i*) (*last xs*) **.**
  from *cdsp-ads-morphism*[*OF this elast*] **show** *?thesis* **by** *blast*
qed

Hash soundness along the sequence. Every view in a blinding path commits to the same root hash as the most-revealed endpoint; a blinded view cannot present a different root. This is the sequence-level form of the ADS guarantee *bo-hash-eq*: along the whole synchronization sequence there is a single authenticating root, so any inclusion claim made anywhere along the sequence is checked against the same commitment.

theorem *blinding-path-hash-soundness*:
  assumes *path*: *blinding-path bo xs* **and** *ne*: $xs \neq []$ **and** *i*: $i <$ *length xs*
  shows *h* (*xs* ! *i*) = *h* (*last xs*)
proof −
  have *le*: $i \leq$ *length xs* − *1* **using** *i* **by** *simp*
  have *lt*: *length xs* − *1* < *length xs* **using** *ne* **by** (*cases xs*) *auto*
  have *bo* (*xs* ! *i*) (*xs* ! (*length xs* − *1*)) **using** *blinding-path-mono*[*OF path le lt*] **.**
  moreover have *xs* ! (*length xs* − *1*) = *last xs* **using** *ne* **by** (*simp add*: *last-conv-nth*)
  ultimately show *?thesis* **using** *bo-hash-eq* **by** *simp*
qed

Inclusion-proof integrity along the sequence. Every holding revealed by any view in the sequence is genuinely included in the most-revealed endpoint with the same regulatory state, and the revealing chain is connected in the endpoint. No view along the sequence can therefore exhibit a holding that the endpoint does not authenticate. Together with *blinding-path-hash-soundness* (one root for the whole sequence) this is the state-level reading of inclusion-proof integrity preserved across the synchronization sequence.

At this abstract layer inclusion reduces to the refinement guarantee (*sequence-authenticity-preservation*) read through *state-refines-def*: a partial view's revealed leaves are exactly the leaves it shares with the endpoint. The concrete Merkle inclusion-path machinery lives one level down in *ADS-Functor.Inclusion-Proof-Construction*.

theorem *sequence-inclusion-integrity*:
  assumes *path*: *blinding-path bo xs* **and** *ne*: $xs \neq []$
    and *elast*: *extract-map* (*last xs*) = *Some s-last*
    and *i*: $i <$ *length xs* **and** *ei*: *extract-map* (*xs* ! *i*) = *Some* s_i
    and *rev*: *get-reg-state* s_i *c aid* = *Some r*
  shows *get-reg-state s-last c aid* = *Some r* ∧ *c* ∈ *connected-chains s-last aid*
proof −
  obtain s' **where** es': *extract-map* (*xs* ! *i*) = *Some* s' **and** *sr*: *state-refines* s' *s-last*
    using *sequence-authenticity-preservation*[*OF path ne elast i*] **by** *blast*
  from es' *ei* **have** *eq*: $s' = s_i$ **by** *simp*
  have *conn*: *c* ∈ *connected-chains* s_i *aid*
    using *rev* **unfolding** *connected-chains-def asset-exists-def*
      get-reg-state-def get-asset-state-def **by** (*auto split*: *option.splits*)
  have *c* ∈ *connected-chains s-last aid*
      ∧ *get-reg-state* s_i *c aid* = *get-reg-state s-last c aid*
    using *sr eq conn* **unfolding** *state-refines-def* **by** *metis*
  thus *?thesis* **using** *rev* **by** *simp*
qed

The same guarantee phrased over the reflexive-transitive closure: any view reachable from *b* by any number of blinding steps extracts to a valid state refining *b*. (The closure collapses because *bo* is already a preorder, which is precisely why arbitrary-length sequences add no new obligation.)

corollary *reachable-view-authenticity*:
  assumes *reach*: bo^{**} *a b* **and** *eb*: *extract-map b* = *Some sb*
  shows ∃ *sa*. *extract-map a* = *Some sa* ∧ *valid-state sa* ∧ *state-refines sa sb*
proof −
  from *reach* **have** *bo a b*
  proof (*induction rule*: *rtranclp-induct*)
    case *base*
    show *?case* **using** *mk.reflp* **by** (*simp add*: *reflp-def*)
  next
    case (*step y z*)
    show *?case* **using** *mk.transp step.IH step.hyps*(*2*) **by** (*metis transpD*)
  qed
  from *cdsp-ads-morphism*[*OF this eb*] **show** *?thesis* **by** *blast*
qed

59.2 Re-revealing merge along a sequence

The n-ary right fold of the (locale) merge along a non-empty sequence of authenticated views. It is defined with the locale merge *m* fixed, so its induction principle keeps *m* in place (a theory-level fold parameterised by the merge would generalise it away under induction).

fun *merge-seq* :: *'d list* ⇒ *'d option* **where**
  merge-seq [] = *None*
| *merge-seq* [*x*] = *Some x*

| *merge-seq* $(x \# y \# ys) =$
  $(case\ merge\text{-}seq\ (y \# ys)\ of\ None \Rightarrow None \mid Some\ z \Rightarrow m\ x\ z)$

A single merge preserves the common hash, since each input blinds to the merge and blinding respects hashes.

lemma *merge-preserves-hash*:
  assumes $m\ a\ b = Some\ ab$
  shows $h\ ab = h\ a$
proof −
  have *bo a ab* **using** *mk.join assms* **by** *blast*
  thus *?thesis* **using** *bo-hash-eq* **by** *simp*
qed

Sequence-level hash preservation for the merge fold. Combining a whole hash-compatible sequence of partial views yields a value with that same common hash: re-revealing along the sequence never changes the authenticating root. This is the merge-direction companion of *blinding-path-hash-soundness* (which covers the blinding direction), so along either form of synchronization sequence the root hash is preserved.

theorem *merge-seq-hash*:
  assumes $merge\text{-}seq\ xs = Some\ r$ **and** $xs \neq []$
    and $\forall x \in set\ xs.\ h\ x = h\ (hd\ xs)$
  shows $h\ r = h\ (hd\ xs)$
  using *assms*
proof (*induction xs rule*: *merge-seq.induct*)
  case (*3 x y ys*)
  obtain *z* **where** *z*: $merge\text{-}seq\ (y \# ys) = Some\ z$ **and** *mxz*: $m\ x\ z = Some\ r$
    using *3.prems*(*1*) **by** (*auto split*: *option.splits*)
  have $h\ r = h\ x$ **using** *merge-preserves-hash*[*OF mxz*] **.**
  thus *?case* **by** *simp*
qed *simp-all*

Sequence-level merging soundness. A whole synchronization sequence of partial authenticated views over the same committed object (all sharing a hash) folds into a single combined view that extracts to a valid cross-domain state refining every contributor. This is the n-ary generalisation of *authenticated-preservation-soundness*: re-revealing along the entire sequence preserves validity and reconstructs a consistent joint view.

theorem *sequence-merge-soundness*:
  $[\![\ xs \neq [];\ \forall x \in set\ xs.\ \exists s.\ extract\text{-}map\ x = Some\ s;$
    $\forall x \in set\ xs.\ h\ x = h\ (hd\ xs)\ ]\!]$
  $\Longrightarrow \exists ab\ s.\ merge\text{-}seq\ xs = Some\ ab \wedge extract\text{-}map\ ab = Some\ s \wedge valid\text{-}state\ s$
    $\wedge\ (\forall x \in set\ xs.\ \exists sx.\ extract\text{-}map\ x = Some\ sx \wedge state\text{-}refines\ sx\ s)$
proof (*induction xs rule*: *merge-seq.induct*)
  case *1*
  then show *?case* **by** *simp*
next
  case (*2 x*)

obtain *sx* **where** *ex*: *extract-map* x = *Some sx* **using** *2.prems* **by** *auto*
have *valid-state sx* **using** *extract-preserves-validity*[*OF ex*] **.**
thus *?case* **using** *ex state-refines-refl* **by** *auto*
next
case (*3 x y ys*)
have *ne-rest*: $(y \# ys) \neq []$ **by** *simp*
have *hx*: $\bigwedge w.\ w \in set\ (y \# ys) \Longrightarrow h\ w = h\ x$
using *3.prems*(*3*) **by** *auto*
have *hrest*: $\forall w \in set\ (y \# ys).\ h\ w = h\ (hd\ (y \# ys))$ **using** *hx* **by** *auto*
have *erest*: $\forall w \in set\ (y \# ys).\ \exists s.$ *extract-map* w = *Some* s
using *3.prems*(*2*) **by** *auto*
obtain *z sz* **where** *mz*: *merge-seq* $(y \# ys)$ = *Some* z
and *ez*: *extract-map* z = *Some sz* **and** *vz*: *valid-state sz*
and *refz*: $\forall w \in set\ (y \# ys).\ \exists sw.$ *extract-map* w = *Some sw* $\wedge$ *state-refines sw sz*
using *3.IH*[*OF ne-rest erest hrest*] **by** *blast*
obtain *sx* **where** *ex*: *extract-map* x = *Some sx* **using** *3.prems*(*2*) **by** *auto*
have *hz*: $h\ z = h\ x$
proof −
have $h\ z = h\ (hd\ (y \# ys))$ **using** *merge-seq-hash*[*OF mz ne-rest hrest*] **.**
thus *?thesis* **using** *hx* **by** *simp*
qed
have $\exists ab.\ m\ x\ z$ = *Some ab*
using *hz mk.merge-respects-hashes*[*of x z*] **by** *simp*
then obtain *xz* **where** *mxz*: $m\ x\ z$ = *Some xz* **by** *blast*
have *mseq*: *merge-seq* $(x \# y \# ys)$ = *Some xz* **using** *mz mxz* **by** *simp*
obtain *sxz* **where** *exz*: *extract-map xz* = *Some sxz* **and** *vxz*: *valid-state sxz*
and *rx*: *state-refines sx sxz* **and** *rz*: *state-refines sz sxz*
using *authenticated-preservation-soundness*[*OF mxz ex extract-preserves-validity*[*OF ex*]
ez extract-preserves-validity[*OF ez*]] **by** *blast*
have $\forall w \in set\ (x \# y \# ys).\ \exists sw.$ *extract-map* w = *Some sw* $\wedge$ *state-refines sw sxz*
proof
fix w **assume** $w \in set\ (x \# y \# ys)$
then consider $w = x \mid w \in set\ (y \# ys)$ **by** *auto*
then show $\exists sw.$ *extract-map* w = *Some sw* $\wedge$ *state-refines sw sxz*
proof *cases*
case *1*
then show *?thesis* **using** *ex rx* **by** *blast*
next
case *2*
then obtain *sw* **where** *extract-map* w = *Some sw* **and** *state-refines sw sz*
using *refz* **by** *blast*
then show *?thesis* **using** *state-refines-trans*[*OF - rz*] **by** *blast*
qed
qed
then show *?case* **using** *mseq exz vxz* **by** *blast*
qed

end

60 Instantiation on the ADS blindable-position functor

We instantiate *authenticated-state* on a concrete ADS construction: the blindable-position functor of *ADS-Functor.ADS-Construction* applied to the Oraclizer Merkle interface *merkle-interface-auth*. The carrier is (*reg-state* × *chain-id set*, *reg-state*) $blindable_m$: an authenticated datum is either *Unblinded* (r, P) (the fully revealed consensus state r with revealed-chain set P) or *Blinded* x (a hash-only commitment that reveals nothing). The blindable hash, blinding order and merge are inherited from the ADS construction; the Merkle-interface obligation is discharged by *merkle-blindable* from *merkle-interface-auth*, so no new interface proof is needed.

The empty extracted view does not depend on the (unreachable) consensus label, so all hash-only commitments extract to the same state.

lemma *auth-state-empty-eq*: *auth-state* r {} = *auth-state* r' {}
  by (*simp add*: *auth-state-def*)

Extraction: a revealed datum yields its cross-domain state; a hash-only commitment yields the empty (no-chains) view, which is valid and refines every state. The commitment must still extract to a state – a blinding of a value must itself be extractable – so it maps to the empty view rather than to *None*.

fun *bl-extract* :: (*reg-state* × *chain-id set*, *reg-state*) $blindable_m$ ⇒ *global-state option* **where**
  bl-extract (*Unblinded a*) = *Some* (*auth-state* (*fst a*) (*snd a*))
| *bl-extract* (*Blinded* -)   = *Some* (*auth-state ACTIVE* {})

interpretation *oss-blindable*:
  authenticated-state
    hash-blindable auth-hash
    blinding-of-blindable auth-hash auth-bo
    merge-blindable auth-hash auth-merge
    bl-extract
proof (*rule authenticated-state.intro*)
  show *merkle-interface* (*hash-blindable auth-hash*)
      (*blinding-of-blindable auth-hash auth-bo*) (*merge-blindable auth-hash auth-merge*)
    by (*rule merkle-blindable*[*OF merkle-interface-auth*])
next
  show *authenticated-state-axioms* (*hash-blindable auth-hash*)
      (*blinding-of-blindable auth-hash auth-bo*) (*merge-blindable auth-hash auth-merge*)
bl-extract
  proof

— extract respects merging
fix *a b ab sa sb*
assume *Hab*: *hash-blindable auth-hash a* = *hash-blindable auth-hash b*
  and *Mab*: *merge-blindable auth-hash auth-merge a b* = *Some ab*
  and *ea*: *bl-extract a* = *Some sa* **and** *eb*: *bl-extract b* = *Some sb*
show ∃ *sab*. *bl-extract ab* = *Some sab* ∧ *state-join sa sb sab*
proof (*cases a*)
  case (*Unblinded p*) **note** *A* = *this*
  show *?thesis*
  proof (*cases b*)
    case (*Unblinded q*) **note** *B* = *this*
    have *fp*: *fst p* = *fst q* **using** *Hab* **by** (*simp add*: *A B auth-hash-def*)
    have *ab-eq*: *ab* = *Unblinded* (*fst p*, *snd p* ∪ *snd q*)
      using *Mab fp* **by** (*simp add*: *A B auth-merge-def*)
    have *sa-eq*: *sa* = *auth-state* (*fst p*) (*snd p*) **using** *ea* **by** (*simp add*: *A*)
    have *sb-eq*: *sb* = *auth-state* (*fst p*) (*snd q*) **using** *eb fp* **by** (*simp add*: *B*)
    have *bl-extract ab* = *Some* (*auth-state* (*fst p*) (*snd p* ∪ *snd q*)) **by** (*simp add*: *ab-eq*)
    moreover have *state-join sa sb* (*auth-state* (*fst p*) (*snd p* ∪ *snd q*))
      unfolding *sa-eq sb-eq* **by** (*rule state-join-auth*)
    ultimately show *?thesis* **by** *blast*
  next
    case (*Blinded y*) **note** *B* = *this*
    have *ab-eq*: *ab* = *Unblinded p* **using** *Mab Hab* **by** (*simp add*: *A B auth-hash-def*)
    have *sa-eq*: *sa* = *auth-state* (*fst p*) (*snd p*) **using** *ea* **by** (*simp add*: *A*)
    have *sb-eq*: *sb* = *auth-state ACTIVE* {} **using** *eb* **by** (*simp add*: *B*)
    have *bl-extract ab* = *Some* (*auth-state* (*fst p*) (*snd p*)) **by** (*simp add*: *ab-eq*)
    moreover have *state-join sa sb* (*auth-state* (*fst p*) (*snd p*))
      unfolding *sa-eq sb-eq*
      using *state-join-auth*[*of fst p snd p* {}] *auth-state-empty-eq*[*of ACTIVE fst p*]
      by *simp*
    ultimately show *?thesis* **by** *blast*
  qed
next
  case (*Blinded x*) **note** *A* = *this*
  show *?thesis*
  proof (*cases b*)
    case (*Unblinded q*) **note** *B* = *this*
    have *ab-eq*: *ab* = *Unblinded q* **using** *Mab Hab* **by** (*simp add*: *A B auth-hash-def*)
    have *sa-eq*: *sa* = *auth-state ACTIVE* {} **using** *ea* **by** (*simp add*: *A*)
    have *sb-eq*: *sb* = *auth-state* (*fst q*) (*snd q*) **using** *eb* **by** (*simp add*: *B*)
    have *bl-extract ab* = *Some* (*auth-state* (*fst q*) (*snd q*)) **by** (*simp add*: *ab-eq*)
    moreover have *state-join sa sb* (*auth-state* (*fst q*) (*snd q*))
      unfolding *sa-eq sb-eq*
      using *state-join-auth*[*of fst q* {} *snd q*] *auth-state-empty-eq*[*of ACTIVE fst q*]

```
        by simp
      ultimately show ?thesis by blast
    next
      case (Blinded y) note B = this
      have ab-eq: ab = Blinded y using Mab Hab by (simp add: A B)
      have sa-eq: sa = auth-state ACTIVE {} using ea by (simp add: A)
      have sb-eq: sb = auth-state ACTIVE {} using eb by (simp add: B)
      have bl-extract ab = Some (auth-state ACTIVE {}) by (simp add: ab-eq)
      moreover have state-join sa sb (auth-state ACTIVE {})
        unfolding sa-eq sb-eq using state-join-auth[of ACTIVE {} {}] by simp
      ultimately show ?thesis by blast
    qed
  qed
next
  — extract under blinding
  fix a b sb
  assume Hab: hash-blindable auth-hash a = hash-blindable auth-hash b
    and BOab: blinding-of-blindable auth-hash auth-bo a b
    and eb: bl-extract b = Some sb
  show ∃ sa. bl-extract a = Some sa ∧ state-refines sa sb
  proof (cases b)
    case (Unblinded q) note B = this
    show ?thesis
    proof (cases a)
      case (Unblinded p) note A = this
      have bo: auth-bo p q using BOab by (simp add: A B)
      have rfst: fst p = fst q and sub: snd p ⊆ snd q
        using bo by (simp-all add: auth-bo-def)
     have sr: state-refines (auth-state (fst p) (snd p)) (auth-state (fst p) (snd q))
        using sub by (rule state-refines-auth)
      have bl-extract a = Some (auth-state (fst p) (snd p)) by (simp add: A)
      moreover have state-refines (auth-state (fst p) (snd p)) sb
        using sr rfst eb by (simp add: B)
      ultimately show ?thesis by blast
    next
      case (Blinded x) note A = this
      have sr: state-refines (auth-state (fst q) {}) (auth-state (fst q) (snd q))
        using empty-subsetI by (rule state-refines-auth)
      have bl-extract a = Some (auth-state (fst q) {})
        by (simp add: A auth-state-empty-eq[of ACTIVE fst q])
      moreover have state-refines (auth-state (fst q) {}) sb
        using sr eb by (simp add: B)
      ultimately show ?thesis by blast
    qed
  next
    case (Blinded y) note B = this
    — Only a blinded value can blind to a blinded value, and then they are equal.
    have a = Blinded y using BOab by (cases a) (simp-all add: B)
    then have bl-extract a = Some (auth-state ACTIVE {}) by simp
```

moreover have *sb* = *auth-state ACTIVE* {} **using** *eb* **by** (*simp add*: *B*)
ultimately show *?thesis* **using** *state-refines-refl* **by** *auto*
qed
next
— extract preserves validity
fix *a s*
assume *bl-extract a* = *Some s*
then show *valid-state s*
by (*cases a*) (*auto simp*: *auth-state-valid*)
qed
qed

Non-degeneracy of the blindable instance: revealed two-chain views merge to their union, and a hash-only commitment merges back to the revealed view.

lemma *oss-blindable-nontrivial*:
merge-blindable auth-hash auth-merge (*Unblinded* (*ACTIVE*, {*0*})) (*Unblinded* (*ACTIVE*, {*Suc 0*}))
= *Some* (*Unblinded* (*ACTIVE*, {*0*, *Suc 0*}))
bl-extract (*Unblinded* (*ACTIVE*, {*0*, *Suc 0*})) = *Some* (*auth-state ACTIVE* {*0*, *Suc 0*})
merge-blindable auth-hash auth-merge (*Blinded* (*Content ACTIVE*)) (*Unblinded* (*ACTIVE*, {*0*}))
= *Some* (*Unblinded* (*ACTIVE*, {*0*}))
by (*simp-all add*: *auth-merge-def auth-hash-def insert-commute*)

Regression witness exhibiting a non-vacuous instance for the sequence-level results above. The generalised predicate *blinding-path* is satisfied by a genuinely increasing need-to-know sequence at this instance: a hash-only commitment, then a one-chain revealed view, then a two-chain revealed view, each blinding its successor. The most-revealed endpoint extracts to a non-degenerate two-chain state. This pins *authenticated-state.sequence-authenticity-preservation*, *authenticated-state.blinding-path-hash-soundness* and *authenticated-state.sequence-inclusion-integrity* to a non-vacuous instance.

lemma *blinding-path-witness*:
blinding-path (*blinding-of-blindable auth-hash auth-bo*)
[*Blinded* (*Content ACTIVE*),
Unblinded (*ACTIVE*, {*0*::*nat*}),
Unblinded (*ACTIVE*, {*0*::*nat*, *Suc 0*})]
unfolding *blinding-path-def*
by (*auto simp*: *nth-Cons′ auth-hash-def auth-bo-def*)

lemma *blinding-path-witness-endpoint*:
bl-extract (*Unblinded* (*ACTIVE*, {*0*::*nat*, *Suc 0*})) = *Some* (*auth-state ACTIVE* {*0*, *Suc 0*})
connected-chains (*auth-state ACTIVE* {*0*::*nat*, *Suc 0*}) (*0*::*nat*) = {*0*, *Suc 0*}
by (*simp-all add*: *auth-state-connected*)

61 Instantiation on the top-level Canton transaction commitment

Theory *ADS-Functor.Canton-Transaction-Tree* formalises the production Canton transaction tree as an authenticated data structure, but over abstract content: *view-data*, *view-metadata*, *common-metadata* and *participant-metadata* are introduced by **typedecl** and carry no regulatory meaning, so a cross-domain state cannot be read out of $transaction_m$ without an inadmissible axiom relating those opaque types to the regulatory model. We therefore model the same public structure with concrete content. A Canton transaction is $Transaction_m$ applied to one top-level blindable position over a payload of metadata and a view list; we mirror exactly this one-constructor-over-a-blindable shape, taking the payload to be the cross-domain regulatory datum — the consensus regulatory state (in the role of the common metadata) together with the set of chains whose views attest it (the per-participant view list, abstracted to its attesting-chain set). Hash, blinding and merge are the ADS blindable-position operations on the payload: precisely the top-level authenticated operation Canton performs, since $Transaction_m$ wraps a single $blindable_m$. The Merkle interface and the three coherence obligations are transported from the blindable instance *oss-blindable* through the constructor bijection, so no new axiom and no residual proof obligation arise. The one open item is model fidelity: whether this payload faithfully abstracts Canton's metadata-and-view-list datatype. That is a sanity check of the model against the Canton specification, not a proof gap.

datatype *reg-transaction* =
  RegTransaction (*the-reg-tx*: (*reg-state* × *chain-id set*, *reg-state*) $blindable_m$)

definition *rtx-hash* :: *reg-transaction* ⇒ *reg-state* $blindable_h$ **where**
  rtx-hash t = *hash-blindable auth-hash* (*the-reg-tx t*)

definition *rtx-bo* :: *reg-transaction* ⇒ *reg-transaction* ⇒ *bool* **where**
  rtx-bo s t = *blinding-of-blindable auth-hash auth-bo* (*the-reg-tx s*) (*the-reg-tx t*)

definition *rtx-merge* :: *reg-transaction* ⇒ *reg-transaction* ⇒ *reg-transaction option* **where**
  rtx-merge s t =
    map-option RegTransaction
      (*merge-blindable auth-hash auth-merge* (*the-reg-tx s*) (*the-reg-tx t*))

definition *rtx-extract* :: *reg-transaction* ⇒ *global-state option* **where**
  rtx-extract t = *bl-extract* (*the-reg-tx t*)

The single constructor is a bijection, so the Merkle interface of the payload blindable transports to the transaction wrapper.

lemma *merkle-reg-transaction*: *merkle-interface rtx-hash rtx-bo rtx-merge*

proof –
  have *bind-simp*: *rtx-merge a b* $\ggg$= *rtx-merge c*
      = *map-option RegTransaction*
        (*merge-blindable auth-hash auth-merge* (*the-reg-tx a*) (*the-reg-tx b*)
          $\ggg$= *merge-blindable auth-hash auth-merge* (*the-reg-tx c*)) **for** *a b c*
    by (*cases merge-blindable auth-hash auth-merge* (*the-reg-tx a*) (*the-reg-tx b*))
      (*simp-all add*: *rtx-merge-def*)
  have *mrh*: (*rtx-hash a* = *rtx-hash b*) = ($\exists$ *ab*. *rtx-merge a b* = *Some ab*) **for** *a b*
    by (*cases merge-blindable auth-hash auth-merge* (*the-reg-tx a*) (*the-reg-tx b*))
      (*auto simp*: *rtx-hash-def rtx-merge-def oss-blindable.mk.merge-respects-hashes*)
  have *midem*: *rtx-merge a a* = *Some a* **for** *a*
    by (*simp add*: *rtx-merge-def oss-blindable.mk.idem*)
  have *mcomm*: *rtx-merge a b* = *rtx-merge b a* **for** *a b*
    by (*simp add*: *rtx-merge-def oss-blindable.mk.commute*)
  have *massoc*: *rtx-merge a b* $\ggg$= *rtx-merge c* = *rtx-merge b c* $\ggg$= *rtx-merge a* **for** *a b c*
    using *bind-simp*[*of a b c*] *bind-simp*[*of b c a*]
      oss-blindable.mk.assoc[*of the-reg-tx a the-reg-tx b the-reg-tx c*] **by** *simp*
  have *mbod*: *rtx-bo a b* = (*rtx-merge a b* = *Some b*) **for** *a b*
    by (*cases merge-blindable auth-hash auth-merge* (*the-reg-tx a*) (*the-reg-tx b*))
      (*auto simp*: *rtx-bo-def rtx-merge-def oss-blindable.mk.bo-def reg-transaction.expand*)
  show *?thesis*
    by (*rule merkle-interface.intro*[*OF mrh midem mcomm massoc mbod*])
qed

Extraction reads the revealed cross-domain datum out of the transaction's top-level commitment; the three coherence obligations transport from the blindable instance *oss-blindable*.

interpretation *canton-authenticated*:
  authenticated-state rtx-hash rtx-bo rtx-merge rtx-extract
proof (*rule authenticated-state.intro*)
  show *merkle-interface rtx-hash rtx-bo rtx-merge* **by** (*rule merkle-reg-transaction*)
next
  show *authenticated-state-axioms rtx-hash rtx-bo rtx-merge rtx-extract*
  proof
    — extract respects merging, transported through the constructor
    fix *a b ab sa sb*
    assume *H*: *rtx-hash a* = *rtx-hash b* **and** *M*: *rtx-merge a b* = *Some ab*
      and *ea*: *rtx-extract a* = *Some sa* **and** *eb*: *rtx-extract b* = *Some sb*
     have *h'*: *hash-blindable auth-hash* (*the-reg-tx a*) = *hash-blindable auth-hash* (*the-reg-tx b*)
      using *H* **by** (*simp add*: *rtx-hash-def*)
    have *m'*: *merge-blindable auth-hash auth-merge* (*the-reg-tx a*) (*the-reg-tx b*)
          = *Some* (*the-reg-tx ab*)
      using *M* **by** (*auto simp*: *rtx-merge-def split*: *option.splits*)
   have *ea'*: *bl-extract* (*the-reg-tx a*) = *Some sa* **using** *ea* **by** (*simp add*: *rtx-extract-def*)
   have *eb'*: *bl-extract* (*the-reg-tx b*) = *Some sb* **using** *eb* **by** (*simp add*: *rtx-extract-def*)
    obtain *sab* **where** *bl-extract* (*the-reg-tx ab*) = *Some sab state-join sa sb sab*
      using *oss-blindable.extract-respects-merging*[*OF h' m' ea' eb'*] **by** *blast*

then show $\exists$ *sab. rtx-extract ab* = *Some sab* $\wedge$ *state-join sa sb sab*
by (*auto simp*: *rtx-extract-def*)
next
— extract under blinding, transported through the constructor
fix *a b sb*
assume *H*: *rtx-hash a* = *rtx-hash b* **and** *BO*: *rtx-bo a b*
and *eb*: *rtx-extract b* = *Some sb*
have *h′*: *hash-blindable auth-hash* (*the-reg-tx a*) = *hash-blindable auth-hash* (*the-reg-tx b*)
using *H* **by** (*simp add*: *rtx-hash-def*)
have *bo′*: *blinding-of-blindable auth-hash auth-bo* (*the-reg-tx a*) (*the-reg-tx b*)
using *BO* **by** (*simp add*: *rtx-bo-def*)
have *eb′*: *bl-extract* (*the-reg-tx b*) = *Some sb* **using** *eb* **by** (*simp add*: *rtx-extract-def*)
obtain *sa* **where** *bl-extract* (*the-reg-tx a*) = *Some sa state-refines sa sb*
using *oss-blindable.extract-under-blinding*[*OF h′ bo′ eb′*] **by** *blast*
then show $\exists$ *sa. rtx-extract a* = *Some sa* $\wedge$ *state-refines sa sb*
by (*auto simp*: *rtx-extract-def*)
next
— extract preserves validity
fix *a s*
assume *rtx-extract a* = *Some s*
then have *bl-extract* (*the-reg-tx a*) = *Some s* **by** (*simp add*: *rtx-extract-def*)
then show *valid-state s* **using** *oss-blindable.extract-preserves-validity* **by** *blast*
qed
qed

Non-degeneracy: a revealed two-chain transaction joins with a hash-only commitment to the same chains, re-revealing the cross-domain view.

lemma *canton-authenticated-nontrivial*:
rtx-merge (*RegTransaction* (*Unblinded* (*ACTIVE*, {*0*})))
(*RegTransaction* (*Unblinded* (*ACTIVE*, {*Suc 0*})))
= *Some* (*RegTransaction* (*Unblinded* (*ACTIVE*, {*0*, *Suc 0*})))
rtx-extract (*RegTransaction* (*Unblinded* (*ACTIVE*, {*0*, *Suc 0*})))
= *Some* (*auth-state ACTIVE* {*0*, *Suc 0*})
rtx-merge (*RegTransaction* (*Blinded* (*Content ACTIVE*)))
(*RegTransaction* (*Unblinded* (*ACTIVE*, {*0*})))
= *Some* (*RegTransaction* (*Unblinded* (*ACTIVE*, {*0*})))
by (*simp-all add*: *rtx-merge-def rtx-extract-def auth-merge-def auth-hash-def insert-commute*)

62 Recursive instantiation on the Canton transaction tree

The coarse instance *canton-authenticated* above models a Canton transaction as one top-level blindable over an abstract payload. Here we keep the public *recursive* shape: a view is the rose tree $\mathit{rose\text{-}tree}_m$ of *ADS-Functor.ADS-Construction* — the very datatype the public Canton formalisation uses to build view_m ($\mathit{view}_m \cong \mathit{view\text{-}content}\ \mathit{rose\text{-}tree}_m$, with *merge-view* = *merge-tree* ...). We instantiate that content-agnostic machine with concrete regulatory content: each node carries the *chain-id* it attests, and the children are its subviews. No new datatype, no new Merkle proof: the interface is inherited from *merkle-tree*.

type-synonym *reg-view* = (*chain-id*, *chain-id*) $\textit{rose-tree}_m$

abbreviation *rv-hash* :: (*reg-view*, *chain-id* $\textit{rose-tree}_h$) *hash* **where**
rv-hash ≡ *hash-tree* (*hash-discrete* :: *chain-id* ⇒ *chain-id*)
abbreviation *rv-bo* :: *reg-view blinding-of* **where**
rv-bo ≡ *blinding-of-tree hash-discrete* (*blinding-of-discrete* :: *chain-id blinding-of*)
abbreviation *rv-merge* :: *reg-view merge* **where**
rv-merge ≡ *merge-tree hash-discrete* (*merge-discrete* :: *chain-id merge*)

lemma *merkle-reg-view*: *merkle-interface rv-hash rv-bo rv-merge*
by (*rule merkle-tree*[*OF merkle-discrete*])

Extraction collects the attested chains from every revealed node, recursing into subviews; a blinded node contributes nothing. By the Merkle property a subview is reachable only through its (revealed) parent. Lemma *reg-chains-blinding* below makes this monotone: blinding a node can only shrink the attested-chain set.

fun *reg-chains* :: *reg-view* ⇒ *chain-id set* **where**
reg-chains ($\textit{Tree}_m$ (*Unblinded* (*c*, *kids*))) = *insert c* ($\bigcup$*x*∈*set kids*. *reg-chains x*)
| *reg-chains* ($\textit{Tree}_m$ (*Blinded* -)) = {}

62.1 Extraction commutes with blinding and merge

A pointwise containment along a list lifts to the union of the collected chains.

lemma *UN-reg-chains-mono*:
assumes *list-all2* (λ*x y*. *reg-chains x* ⊆ *reg-chains y*) *xs ys*
shows ($\bigcup$*x*∈*set xs*. *reg-chains x*) ⊆ ($\bigcup$*y*∈*set ys*. *reg-chains y*)
using *assms* **by** (*induction xs ys rule*: *list-all2-induct*) *auto*

Positional characterisation of the ADS list merge (used for the children lists of a rose-tree node). The length alignment is *derived*, not assumed: the lemmas *merge-list-NC* / *merge-list-CN* below reject lists of different length, exactly because the list hash (*map*) commits to the length.

context begin
interpretation *list-R1* **.**

lemma *merge-list-Cons*:
merge-list m (*x* # *xs*) (*y* # *ys*) =
(*case m x y of None* ⇒ *None* | *Some z* ⇒ *map-option* ((#) *z*) (*merge-list m xs ys*))
by (*simp add*: *merge-list-def merge-F-def merge-R1.simps*
list-to-list-R1.simps list-R1-to-list-simps merge-sum.simps merge-prod.simps
merge-discrete-def option.map-comp o-def split: *option.splits*)

lemma *merge-list-NN* [*simp*]: *merge-list m* [] [] = *Some* []
  by (*simp add*: *merge-list-def merge-F-def merge-R1.simps*
      list-to-list-R1.simps list-R1-to-list-simps merge-sum.simps merge-discrete-def)

lemma *merge-list-CN* [*simp*]: *merge-list m* (*x* # *xs*) [] = *None*
  by (*simp add*: *merge-list-def merge-F-def merge-R1.simps list-to-list-R1.simps merge-sum.simps*)

lemma *merge-list-NC* [*simp*]: *merge-list m* [] (*y* # *ys*) = *None*
  by (*simp add*: *merge-list-def merge-F-def merge-R1.simps list-to-list-R1.simps merge-sum.simps*)

end

The collected chains of a list-merge are the union of the contributors' chains, given that the elementwise merge has that property.

lemma *reg-chains-merge-list*:
  assumes *merge-list m k1 k2* = *Some k*
    and $\bigwedge$*s t u*. *s* ∈ *set k1* ⟹ *m s t* = *Some u* ⟹ *reg-chains u* = *reg-chains s* ∪ *reg-chains t*
  shows ($\bigcup$*s*∈*set k*. *reg-chains s*) = ($\bigcup$*s*∈*set k1*. *reg-chains s*) ∪ ($\bigcup$*t*∈*set k2*. *reg-chains t*)
  using *assms*
proof (*induction k1 arbitrary*: *k2 k*)
  case *Nil*
  then show *?case* **by** (*cases k2*) *auto*
next
  case (*Cons x xs*)
  show *?case*
  proof (*cases k2*)
    case *Nil* **then show** *?thesis* **using** *Cons.prems*(*1*) **by** *simp*
  next
    case (*Cons y ys*)
    from *Cons.prems*(*1*) **obtain** *z k′* **where** *z*: *m x y* = *Some z*
      and *k′*: *merge-list m xs ys* = *Some k′* **and** *keq*: *k* = *z* # *k′*
      by (*auto simp*: *Cons merge-list-Cons split*: *option.splits*)
    have *hz*: *reg-chains z* = *reg-chains x* ∪ *reg-chains y*
      using *Cons.prems*(*2*)[*of x y z*] *z* **by** *simp*
    have *ih*: ($\bigcup$*s*∈*set k′*. *reg-chains s*) = ($\bigcup$*s*∈*set xs*. *reg-chains s*) ∪ ($\bigcup$*t*∈*set ys*. *reg-chains t*)
      using *k′* **by** (*intro Cons.IH*) (*use Cons.prems*(*2*) **in** *auto*)
    show *?thesis* **using** *keq hz ih* **by** (*simp add*: *Cons*) *blast*
  qed
qed

Blinding shrinks the revealed chains, recursively through subviews: a blinded view's chains are contained in those of the view it blinds.

lemma *reg-chains-blinding*:

rv-bo a b $\Longrightarrow$ *reg-chains a* $\subseteq$ *reg-chains b*
proof (*induction a arbitrary*: *b rule*: *rose-tree*$_m$*.induct*)
case (*Tree*$_m$ *x b*)
obtain *t2* **where** *bt2*: *b* = *Tree*$_m$ *t2* **by** (*cases b*)
note *rt* = *Tree*$_m$*.prems*[*unfolded bt2 blinding-of-tree-simps*]
show *?case*
proof (*cases x*)
case (*Blinded h*) **then show** *?thesis* **by** *simp*
next
case (*Unblinded p*)
obtain *c kids1* **where** *p*: *p* = (*c*, *kids1*) **by** (*cases p*)
from *rt* **obtain** *kids2* **where** *t2eq*: *t2* = *Unblinded* (*c*, *kids2*) **and**
la: *list-all2 rv-bo kids1 kids2*
by (*auto simp*: *Unblinded p rel-prod-inject*)
have *la2*: *list-all2* (λ*s t. reg-chains s* $\subseteq$ *reg-chains t*) *kids1 kids2*
proof (*rule list-all2-all-nthI*)
show *length kids1* = *length kids2* **using** *la list-all2-lengthD* **by** *blast*
fix *n* **assume** *n*: *n* < *length kids1*
have *m1*: (*c*, *kids1*) $\in$ *set1-blindable*$_m$ *x* **by** (*simp add*: *Unblinded p*)
have *sm*: *kids1* $\in$ *snds* (*c*, *kids1*) **by** (*simp add*: *prod-set-simps*)
have *memn*: *kids1* ! *n* $\in$ *set kids1* **using** *n* **by** *simp*
have *rvn*: *rv-bo* (*kids1* ! *n*) (*kids2* ! *n*) **using** *la n list-all2-nthD* **by** *blast*
show *reg-chains* (*kids1* ! *n*) $\subseteq$ *reg-chains* (*kids2* ! *n*)
using *Tree*$_m$*.IH*[*OF m1 sm memn rvn*] **.**
qed
from *UN-reg-chains-mono*[*OF la2*] **show** *?thesis*
by (*auto simp*: *Unblinded p bt2 t2eq*)
qed
qed

Merging re-reveals exactly the union of the two contributors' chains, recursively through subviews.

lemma *reg-chains-merge*:
rv-merge a b = *Some ab* $\Longrightarrow$ *reg-chains ab* = *reg-chains a* $\cup$ *reg-chains b*
proof (*induction a arbitrary*: *b ab rule*: *rose-tree*$_m$*.induct*)
case (*Tree*$_m$ *x b ab*)
obtain *y* **where** *bY*: *b* = *Tree*$_m$ *y* **by** (*cases b*)
obtain *z* **where** *abz*: *ab* = *Tree*$_m$ *z* **by** (*cases ab*)
from *Tree*$_m$*.prems*[*unfolded bY abz merge-tree.simps*]
have *mz*: *merge-rt-F*$_m$ *hash-discrete merge-discrete* (*hash-tree hash-discrete*) *rv-merge x y* = *Some z*
by (*auto split*: *option.splits*)
note *mb* = *mz*[*unfolded merge-rt-F*$_m$*-def*]
show *?case*
proof (*cases x*)
case (*Blinded hx*)
from *mb* **have** *z* = *y* $\vee$ ($\exists$ *q. y* = *Unblinded q* $\wedge$ *z* = *Unblinded q*)
by (*cases y*) (*auto simp*: *Blinded split*: *if-splits*)
then show *?thesis* **using** *bY abz Blinded* **by** *auto*

next
case (*Unblinded px*)
obtain *c1 k1* **where** *px*: $px = (c1, k1)$ **by** (*cases px*)
show *?thesis*
proof (*cases y*)
case (*Blinded hy*)
from *mb* **have** $z =$ *Unblinded* $(c1, k1)$
by (*auto simp*: *Unblinded px Blinded split*: *if-splits*)
then show *?thesis* **using** *bY abz Unblinded px Blinded* **by** *simp*
next
case (*Unblinded py*)
obtain *c2 k2* **where** *py*: $py = (c2, k2)$ **by** (*cases py*)
from *mb* **obtain** *k* **where** *c12*: $c1 = c2$ **and** *kk*: *merge-list rv-merge k1 k2* = *Some k*
and *zk*: $z =$ *Unblinded* $(c1, k)$
by (*auto simp*: ‹$x =$ *Unblinded px*› *px Unblinded py merge-discrete-def*
split: *option.splits if-splits*)
have *IHc*: $\bigwedge s\ t\ u.\ s \in set\ k1 \Longrightarrow rv\text{-}merge\ s\ t = Some\ u$
$\Longrightarrow reg\text{-}chains\ u = reg\text{-}chains\ s \cup reg\text{-}chains\ t$
proof −
fix *s t u* **assume** *sset*: $s \in set\ k1$ **and** *st*: $rv\text{-}merge\ s\ t = Some\ u$
have *m1*: $(c1, k1) \in set1\text{-}blindable_m\ x$ **by** (*simp add*: ‹$x =$ *Unblinded px*› *px*)
have *sm*: $k1 \in snds\ (c1, k1)$ **by** (*simp add*: *prod-set-simps*)
show $reg\text{-}chains\ u = reg\text{-}chains\ s \cup reg\text{-}chains\ t$
using $Tree_m.IH[OF\ m1\ sm\ sset\ st]$ **.**
qed
have $(\bigcup s{\in}set\ k.\ reg\text{-}chains\ s) = (\bigcup s{\in}set\ k1.\ reg\text{-}chains\ s) \cup (\bigcup t{\in}set\ k2.\ reg\text{-}chains\ t)$
using *reg-chains-merge-list*[*OF kk IHc*] **.**
then show *?thesis*
using *bY abz zk c12* **by** (*simp add*: ‹$x =$ *Unblinded px*› *px Unblinded py*)
qed
qed
qed

Regression witnesses for the recursive structure. First, deep-nested chains are not dropped: extraction genuinely recurses into subviews. Second, list-merge length alignment is enforced (mismatched lengths are rejected), so it is derived from the hash rather than assumed. Third, a blinded node contributes nothing. The single-consensus invariant (load-bearing, requiring the extraction map) is exhibited with the interpretation below.

lemma *reg-chains-deep-nesting*:
$reg\text{-}chains\ (Tree_m\ (Unblinded\ (0{::}chain\text{-}id, [Tree_m\ (Unblinded\ (Suc\ 0, []))]))) = \{0, Suc\ 0\}$
by *simp*

lemma *reg-chains-blinded-contributes-nothing*:
$reg\text{-}chains\ (Tree_m\ (Blinded\ h)) = \{\}$

by *simp*

lemma *merge-list-length-aligned*:
merge-list rv-merge $[t]$ $[]$ $=$ *None*
merge-list rv-merge $[]$ $[t]$ $=$ *None*
by *simp-all*

62.2 The recursive transaction-tree instance

A Canton transaction wraps its consensus metadata and its view list in one top-level blindable ($\mathit{transaction}_m = \mathit{Transaction}_m\ (\mathit{blindable}_m\ \ldots)$). We mirror that exactly: the carrier is a top-level blindable over a payload *reg-state* $\times$ *reg-view list* — the consensus regulatory state together with the list of recursive view trees. Hash, blinding and merge come from the public product/list/blindable building blocks, so the Merkle interface is inherited (lemma *merkle-reg-tx* below).

Honesty (model-fidelity boundary, read carefully). The recursive view structure is now *faithful*: subviews nest as in Canton, and subview-level blinding is alive (the witnesses below). Two points remain modelling choices, *not* a 1:1 match with Canton's datatype. First, leaf content is concrete *chain-id*/*reg-state*, whereas Canton's leaves are the opaque *view-data*/*view-metadata*: this is *recursive-faithful, content-abstracted.* Second, the consensus *reg-state* is modelled as a *bare, non-independently-blindable* field of the payload, while Canton's $\mathit{common\text{-}metadata}_m$ is itself an independently blindable position. This is what structurally forces the single-consensus invariant (a view is reachable only by revealing the whole transaction, hence its consensus), which in turn gives validity with no assumption. The price is a scope limit: the case "a view is revealed while the consensus is blinded" is *out of this model's scope.* Lifting it would require an independently-blindable consensus and a fresh consistency argument; that is recorded as future work and as the residual fidelity check for the Canton authors, not closed here.

type-synonym *reg-transaction-tree* $=$
$(\mathit{reg\text{-}state} \times \mathit{reg\text{-}view\ list},\ \mathit{reg\text{-}state} \times \mathit{chain\text{-}id\ rose\text{-}tree}_h\ \mathit{list})\ \mathit{blindable}_m$

abbreviation *rtt-chash* :: $(\mathit{reg\text{-}state} \times \mathit{reg\text{-}view\ list},\ \mathit{reg\text{-}state} \times \mathit{chain\text{-}id\ rose\text{-}tree}_h\ \mathit{list})\ \mathit{hash}$
where *rtt-chash* $\equiv$ *hash-prod hash-discrete* (*hash-list rv-hash*)
abbreviation *rtt-cbo* :: (*reg-state* $\times$ *reg-view list*) *blinding-of*
where *rtt-cbo* $\equiv$ *blinding-of-prod blinding-of-discrete* (*blinding-of-list rv-bo*)
abbreviation *rtt-cmerge* :: (*reg-state* $\times$ *reg-view list*) *merge*
where *rtt-cmerge* $\equiv$ *merge-prod merge-discrete* (*merge-list rv-merge*)
abbreviation *rtt-hash* :: $(\mathit{reg\text{-}transaction\text{-}tree},\ (\mathit{reg\text{-}state} \times \mathit{chain\text{-}id\ rose\text{-}tree}_h\ \mathit{list})\ \mathit{blindable}_h)\ \mathit{hash}$
where *rtt-hash* $\equiv$ *hash-blindable rtt-chash*
abbreviation *rtt-bo* :: *reg-transaction-tree blinding-of*
where *rtt-bo* $\equiv$ *blinding-of-blindable rtt-chash rtt-cbo*

abbreviation *rtt-merge* :: *reg-transaction-tree merge*
  where *rtt-merge* ≡ *merge-blindable rtt-chash rtt-cmerge*

lemma *merkle-reg-tx-content*: *merkle-interface rtt-chash rtt-cbo rtt-cmerge*
  by (*rule merkle-product*[*OF merkle-discrete merkle-list*[*OF merkle-reg-view*]])

lemma *merkle-reg-tx*: *merkle-interface rtt-hash rtt-bo rtt-merge*
  by (*rule merkle-blindable*[*OF merkle-reg-tx-content*])

The two list-level corollaries of the recursive lemmas above, lifted over the view list of a transaction.

lemma *reg-chains-views-blinding*:
  assumes *list-all2 rv-bo va vb*
  shows $(\bigcup v{\in}set\ va.\ reg\text{-}chains\ v) \subseteq (\bigcup w{\in}set\ vb.\ reg\text{-}chains\ w)$
proof (*rule UN-reg-chains-mono*)
  show *list-all2* $(\lambda x\ y.\ reg\text{-}chains\ x \subseteq reg\text{-}chains\ y)$ *va vb*
    using *assms* **by** (*auto elim*!: *list-all2-mono reg-chains-blinding*)
qed

lemma *reg-chains-views-merge*:
  assumes *merge-list rv-merge va vb* = *Some v*
  shows $(\bigcup x{\in}set\ v.\ reg\text{-}chains\ x) = (\bigcup x{\in}set\ va.\ reg\text{-}chains\ x) \cup (\bigcup y{\in}set\ vb.\ reg\text{-}chains\ y)$
proof (*rule reg-chains-merge-list*[*OF assms*])
  fix *s t u* **assume** *rv-merge s t* = *Some u*
  then show *reg-chains u* = *reg-chains s* ∪ *reg-chains t* **by** (*rule reg-chains-merge*)
qed

Extraction: a revealed transaction yields the cross-domain state in which every attested chain (collected recursively from the revealed views) holds the asset in the consensus state; a fully blinded transaction yields the empty view.

fun *rtt-extract* :: *reg-transaction-tree* ⇒ *global-state option* **where**
  rtt-extract (*Unblinded* (*r*, *tvs*)) = *Some* (*auth-state r* $(\bigcup v{\in}set\ tvs.\ reg\text{-}chains\ v)$)
| *rtt-extract* (*Blinded* -) = *Some* (*auth-state ACTIVE* {})

interpretation *reg-tx-authenticated*:
  authenticated-state rtt-hash rtt-bo rtt-merge rtt-extract
proof (*rule authenticated-state.intro*)
  show *merkle-interface rtt-hash rtt-bo rtt-merge* **by** (*rule merkle-reg-tx*)
next
  note *mlist* = *merkle-list*[*OF merkle-reg-view*]
  show *authenticated-state-axioms rtt-hash rtt-bo rtt-merge rtt-extract*
  proof
    — extract respects merging
    fix *a b ab sa sb*
    assume *Hab*: *rtt-hash a* = *rtt-hash b* **and** *Mab*: *rtt-merge a b* = *Some ab*
      and *ea*: *rtt-extract a* = *Some sa* **and** *eb*: *rtt-extract b* = *Some sb*
    show ∃ *sab*. *rtt-extract ab* = *Some sab* ∧ *state-join sa sb sab*

proof (*cases a*)
  case (*Unblinded pa*) **note** $A = this$
  obtain *ra va* **where** *pa*: $pa = (ra, va)$ **by** (*cases pa*)
  show *?thesis*
  proof (*cases b*)
    case (*Unblinded pb*) **note** $B = this$
    obtain *rb vb* **where** *pb*: $pb = (rb, vb)$ **by** (*cases pb*)
    have *hh*: *ra* = *rb* ∧ *map rv-hash va* = *map rv-hash vb*
      using *Hab* **by** (*simp add*: *A B pa pb*)
    then have *req*: *ra* = *rb* **and** *meq*: *hash-list rv-hash va* = *hash-list rv-hash vb* **by** *simp-all*
    from *meq* **obtain** *v* **where** *mlv*: *merge-list rv-merge va vb* = *Some v*
      using *merkle-interface.merge-respects-hashes*[*OF mlist*] **by** *blast*
    have *ab-eq*: *ab* = *Unblinded* (*ra*, *v*)
      using *Mab req mlv* **by** (*simp add*: *A B pa pb merge-discrete-def*)
    have *sa-eq*: *sa* = *auth-state ra* (⋃*x*∈*set va*. *reg-chains x*) **using** *ea* **by** (*simp add*: *A pa*)
    have *sb-eq*: *sb* = *auth-state ra* (⋃*y*∈*set vb*. *reg-chains y*) **using** *eb req* **by** (*simp add*: *B pb*)
    have *uv*: (⋃*x*∈*set v*. *reg-chains x*)
        = (⋃*x*∈*set va*. *reg-chains x*) ∪ (⋃*y*∈*set vb*. *reg-chains y*)
      using *reg-chains-views-merge*[*OF mlv*] **.**
    have *rtt-extract ab* = *Some* (*auth-state ra* ((⋃*x*∈*set va*. *reg-chains x*)
                      ∪ (⋃*y*∈*set vb*. *reg-chains y*)))
      by (*simp add*: *ab-eq uv*)
    moreover have *state-join sa sb* (*auth-state ra* ((⋃*x*∈*set va*. *reg-chains x*)
                      ∪ (⋃*y*∈*set vb*. *reg-chains y*)))
      unfolding *sa-eq sb-eq* **by** (*rule state-join-auth*)
    ultimately show *?thesis* **by** *blast*
  next
    case (*Blinded hb*) **note** $B = this$
    have *ab-eq*: *ab* = *Unblinded* (*ra*, *va*) **using** *Mab Hab* **by** (*simp add*: *A B pa*)
    have *sa-eq*: *sa* = *auth-state ra* (⋃*x*∈*set va*. *reg-chains x*) **using** *ea* **by** (*simp add*: *A pa*)
    have *sb-eq*: *sb* = *auth-state ACTIVE* {} **using** *eb* **by** (*simp add*: *B*)
    have *rtt-extract ab* = *Some* (*auth-state ra* (⋃*x*∈*set va*. *reg-chains x*))
      by (*simp add*: *ab-eq*)
    moreover have *state-join sa sb* (*auth-state ra* (⋃*x*∈*set va*. *reg-chains x*))
      unfolding *sa-eq sb-eq*
      using *state-join-auth*[*of ra* ⋃*x*∈*set va*. *reg-chains x* {}]
          auth-state-empty-eq[*of ACTIVE ra*] **by** *simp*
    ultimately show *?thesis* **by** *blast*
  qed
next
  case (*Blinded ha*) **note** $A = this$
  show *?thesis*
  proof (*cases b*)
    case (*Unblinded pb*) **note** $B = this$
    obtain *rb vb* **where** *pb*: $pb = (rb, vb)$ **by** (*cases pb*)

have *ab-eq*: *ab* = *Unblinded* (*rb*, *vb*) **using** *Mab Hab* **by** (*simp add*: *A B pb*)
have *sa-eq*: *sa* = *auth-state ACTIVE* {} **using** *ea* **by** (*simp add*: *A*)
have *sb-eq*: *sb* = *auth-state rb* ($\bigcup$*y*∈*set vb. reg-chains y*) **using** *eb* **by** (*simp add*: *B pb*)
have *rtt-extract ab* = *Some* (*auth-state rb* ($\bigcup$*y*∈*set vb. reg-chains y*))
by (*simp add*: *ab-eq*)
moreover have *state-join sa sb* (*auth-state rb* ($\bigcup$*y*∈*set vb. reg-chains y*))
unfolding *sa-eq sb-eq*
using *state-join-auth*[*of rb* {} $\bigcup$*y*∈*set vb. reg-chains y*]
auth-state-empty-eq[*of ACTIVE rb*] **by** *simp*
ultimately show *?thesis* **by** *blast*
next
case (*Blinded hb*) **note** *B* = *this*
have *ab-eq*: *ab* = *Blinded hb* **using** *Mab Hab* **by** (*simp add*: *A B*)
have *sa-eq*: *sa* = *auth-state ACTIVE* {} **using** *ea* **by** (*simp add*: *A*)
have *sb-eq*: *sb* = *auth-state ACTIVE* {} **using** *eb* **by** (*simp add*: *B*)
have *rtt-extract ab* = *Some* (*auth-state ACTIVE* {}) **by** (*simp add*: *ab-eq*)
moreover have *state-join sa sb* (*auth-state ACTIVE* {})
unfolding *sa-eq sb-eq* **using** *state-join-auth*[*of ACTIVE* {} {}] **by** *simp*
ultimately show *?thesis* **by** *blast*
qed
qed
next
— extract under blinding
fix *a b sb*
assume *Hab*: *rtt-hash a* = *rtt-hash b* **and** *BOab*: *rtt-bo a b*
and *eb*: *rtt-extract b* = *Some sb*
show ∃ *sa. rtt-extract a* = *Some sa* ∧ *state-refines sa sb*
proof (*cases b*)
case (*Unblinded pb*) **note** *B* = *this*
obtain *rb vb* **where** *pb*: *pb* = (*rb*, *vb*) **by** (*cases pb*)
show *?thesis*
proof (*cases a*)
case (*Unblinded pa*) **note** *A* = *this*
obtain *ra va* **where** *pa*: *pa* = (*ra*, *va*) **by** (*cases pa*)
have *bo*: *rtt-cbo* (*ra*, *va*) (*rb*, *vb*) **using** *BOab* **by** (*simp add*: *A B pa pb*)
then have *req*: *ra* = *rb* **and** *la*: *list-all2 rv-bo va vb*
by (*simp-all add*: *rel-prod-inject*)
have *sr*: ($\bigcup$*x*∈*set va. reg-chains x*) ⊆ ($\bigcup$*y*∈*set vb. reg-chains y*)
using *reg-chains-views-blinding*[*OF la*] **.**
have *srf*: *state-refines* (*auth-state ra* ($\bigcup$*x*∈*set va. reg-chains x*))
(*auth-state ra* ($\bigcup$*y*∈*set vb. reg-chains y*))
using *sr* **by** (*rule state-refines-auth*)
have *rtt-extract a* = *Some* (*auth-state ra* ($\bigcup$*x*∈*set va. reg-chains x*))
by (*simp add*: *A pa*)
moreover have *state-refines* (*auth-state ra* ($\bigcup$*x*∈*set va. reg-chains x*)) *sb*
using *srf req eb* **by** (*simp add*: *B pb*)
ultimately show *?thesis* **by** *blast*
next

case (*Blinded ha*) **note** *A* = *this*
have *sr*: *state-refines* (*auth-state rb* {}) (*auth-state rb* ($\bigcup$ *y*∈*set vb*. *reg-chains y*))
using *empty-subsetI* **by** (*rule state-refines-auth*)
have *rtt-extract a* = *Some* (*auth-state rb* {})
by (*simp add*: *A auth-state-empty-eq*[*of ACTIVE rb*])
moreover have *state-refines* (*auth-state rb* {}) *sb*
using *sr eb* **by** (*simp add*: *B pb*)
ultimately show *?thesis* **by** *blast*
qed
next
case (*Blinded hb*) **note** *B* = *this*
have *a* = *Blinded hb* **using** *BOab* **by** (*cases a*) (*simp-all add*: *B*)
then have *rtt-extract a* = *Some* (*auth-state ACTIVE* {}) **by** *simp*
moreover have *sb* = *auth-state ACTIVE* {} **using** *eb* **by** (*simp add*: *B*)
ultimately show *?thesis* **using** *state-refines-refl* **by** *auto*
qed
next
— extract preserves validity: single consensus per transaction, hence consistent
fix *a s*
assume *rtt-extract a* = *Some s*
then show *valid-state s*
by (*cases a*) (*auto simp*: *auth-state-valid*)
qed
qed

Multi-level (subview) non-degeneracy witness. A transaction with one view that has two subviews (chains *1* and *2* nested under chain *0*). Revealing everything attests all three chains; blinding *only the chain-2 subview* — a subview-level, not top-level, blinding — drops exactly chain *2*, leaving chains *0* and *1* attested. Selective disclosure is alive below the root.

definition *demo-subview1* :: *reg-view* **where**
demo-subview1 = $Tree_m$ (*Unblinded* (*Suc 0*, []))
definition *demo-subview2* :: *reg-view* **where**
demo-subview2 = $Tree_m$ (*Unblinded* (*Suc* (*Suc 0*), []))
definition *demo-view* :: *reg-view* **where**
demo-view = $Tree_m$ (*Unblinded* (*0*, [*demo-subview1*, *demo-subview2*]))
definition *demo-view-sv2blinded* :: *reg-view* **where**
demo-view-sv2blinded = $Tree_m$ (*Unblinded* (*0*, [*demo-subview1*, $Tree_m$ (*Blinded* (*Garbage 0*))]))

lemma *demo-subview-disclosure*:
reg-chains demo-view = {*0*, *Suc 0*, *Suc* (*Suc 0*)}
reg-chains demo-view-sv2blinded = {*0*, *Suc 0*}
by (*auto simp*: *demo-view-def demo-view-sv2blinded-def demo-subview1-def demo-subview2-def*)

lemma *demo-multilevel-extract*:
rtt-extract (*Unblinded* (*ACTIVE*, [*demo-view*]))
= *Some* (*auth-state ACTIVE* {*0*, *Suc 0*, *Suc* (*Suc 0*)})

rtt-extract (*Unblinded* (*ACTIVE*, [*demo-view-sv2blinded*]))
  = *Some* (*auth-state ACTIVE* {*0*, *Suc 0*})
by (*simp-all add*: *demo-subview-disclosure*)

The single-consensus invariant is load-bearing. If a model allowed a chain to be attested in a regulatory state different from another chain's (which a per-node *reg-state* would permit), validity would collapse: the state below pins chain *0* to *ACTIVE* and chain *Suc 0* to *FROZEN* on the same asset, and is *not* valid. The recursive extraction *rtt-extract* never produces such a state, because every attested chain shares the *one* consensus *reg-state* of its transaction — exactly the structural reason validity holds with no assumption.

definition *rogue-inconsistent-state* :: *global-state* **where**
  rogue-inconsistent-state =
   (| *gs-chains* = (λ*c a*.
      if a = *0* ∧ *c* = *0*
       then Some (| *as-asset-id* = *0*, *as-reg-state* = *ACTIVE* |)
      else if a = *0* ∧ *c* = *Suc 0*
       then Some (| *as-asset-id* = *0*, *as-reg-state* = *FROZEN* |)
      else None),
    gs-locks = (λ-. *False*) |)

lemma *single-consensus-load-bearing*:
  ¬ *valid-state rogue-inconsistent-state*
  by (*auto simp*: *rogue-inconsistent-state-def valid-state-def consistent-state-def*
        get-reg-state-def get-asset-state-def)

Non-degeneracy of the recursive transaction instance: a single-chain transaction extracts to a genuine one-chain cross-domain state, and a transaction with a non-empty view list genuinely merges at the transaction-tree level (the *extract-respects-merging* path is exercised directly, not only through *reg-chains-merge*).

lemma *reg-tx-authenticated-nontrivial*:
  rtt-extract (*Unblinded* (*ACTIVE*, [$Tree_m$ (*Unblinded* (*0*, []))]))
   = *Some* (*auth-state ACTIVE* {*0*})
  rtt-merge (*Unblinded* (*ACTIVE*, [$Tree_m$ (*Unblinded* (*0*, []))]))
       (*Unblinded* (*ACTIVE*, [$Tree_m$ (*Unblinded* (*0*, []))]))
   = *Some* (*Unblinded* (*ACTIVE*, [$Tree_m$ (*Unblinded* (*0*, []))]))
 by (*simp-all add*: *merge-discrete-def merge-list-Cons merge-tree.simps merge-rt-*F_m*-def*)

63 Path-level Merkle inclusion for the recursive view tree

The sequence-level result *authenticated-state.sequence-inclusion-integrity* is stated at the level of revealed holdings. Here we sharpen it to the concrete Merkle *inclusion path*. The recursive view *reg-view* is exactly an instance of the generic rose-tree inclusion-proof machinery of *ADS-Functor.Inclusion-Proof-Construction*: we take the source-content embedding to be the identity on *chain-id* (*rv-embed*) and the content hash to be *id*. An inclusion proof is a zipper in which the target subview is revealed while every node on the path to the root and all off-path siblings are blinded (*blind-path*). Because the discrete content already satisfies the blinding-order locale (*blinding-of-on-discrete*), the two soundness results transfer with no new assumption. This is the same construction the public Canton formalisation applies to its own view tree; it operates on the view rose tree, so the consensus scope limit of the preceding section is untouched.

abbreviation *rv-embed* :: *chain-id* ⇒ *chain-id* **where**
  rv-embed ≡ *id*

abbreviation *zippers-reg-view* :: *chain-id zipper* ⇒ (*chain-id*, *chain-id*) $zipper_m$ *list* **where**
  zippers-reg-view ≡ *zippers-rose-tree rv-embed hash-discrete*

abbreviation *blind-reg-path* :: *chain-id path* ⇒ (*chain-id*, *chain-id*) $path_m$ **where**
  blind-reg-path ≡ *blind-path rv-embed hash-discrete*

abbreviation *embed-reg-path* :: *chain-id path* ⇒ (*chain-id*, *chain-id*) $path_m$ **where**
  embed-reg-path ≡ *embed-path rv-embed*

Inclusion soundness: every enumerated inclusion proof commits to the same authenticating root hash as the fully revealed view tree, so no inclusion proof can fabricate a tree the root does not authenticate.

theorem *reg-view-inclusion-same-hash*:
  assumes $z \in$ *set* (*zippers-reg-view* (*p*, *t*))
  shows *rv-hash* ($tree\text{-}of\text{-}zipper_m$ *z*)
    = *rv-hash* ($tree\text{-}of\text{-}zipper_m$ (*embed-reg-path p*, *embed-source-tree rv-embed t*))
  using *assms* **by** (*rule zippers-rose-tree-same-hash′*)

Each inclusion proof is a blinding of the canonical inclusion-path tree, which carries the same root hash as the fully revealed tree: a partial, need-to-know view that refines the whole. The single content premise is discharged from the discrete building block, so no assumption is added.

theorem *reg-view-inclusion-blinding-of*:
  assumes $z \in$ *set* (*zippers-reg-view* (*p*, *t*))
  shows *rv-bo* ($tree\text{-}of\text{-}zipper_m$ *z*)
      ($tree\text{-}of\text{-}zipper_m$ (*blind-reg-path p*, *embed-source-tree rv-embed t*))
  by (*rule zippers-rose-tree-blinding-of*[*OF blinding-of-on-discrete assms*])

The attested chains carried by an inclusion proof are contained in those of the fully revealed tree: a path-level reading of need-to-know, from *reg-chains-blinding*.

corollary *reg-view-inclusion-chains-sound*:
  assumes $z \in$ *set* (*zippers-reg-view* (*p*, *t*))
  shows *reg-chains* ($tree\text{-}of\text{-}zipper_m$ *z*)

$\subseteq$ *reg-chains* (*tree-of-zipper*$_m$ (*blind-reg-path p*, *embed-source-tree rv-embed t*))
using *reg-chains-blinding*[*OF reg-view-inclusion-blinding-of*[*OF assms*]] **.**

Non-degeneracy. Over a genuine two-level view tree the canonical inclusion endpoint reveals all three attested chains, and a selective inclusion proof in the enumeration exposes its target subview (chains *0* and *Suc 0*) while the unrelated sibling (chain *Suc* (*Suc 0*)) stays blinded. Selective disclosure is alive at the path level.

definition *demo-src* :: *chain-id rose-tree* **where**
demo-src = *Tree* (*0*, [*Tree* (*Suc 0*, []), *Tree* (*Suc* (*Suc 0*), [])])

lemma *reg-view-inclusion-nonvacuous-endpoint*:
reg-chains (*tree-of-zipper*$_m$ (*embed-reg-path* [], *embed-source-tree rv-embed demo-src*))
= {*0*, *Suc 0*, *Suc* (*Suc 0*)}
by (*auto simp*: *demo-src-def embed-path-def*)

lemma *reg-view-inclusion-nonvacuous-selective*:
$\exists$ *z* $\in$ *set* (*zippers-reg-view* ([], *demo-src*)). *reg-chains* (*tree-of-zipper*$_m$ *z*) = {*0*, *Suc 0*}
by (*force simp*: *demo-src-def zippers-rose-tree.simps zipper-children.simps blind-path-def embed-path-def*)

Load-bearing regression witness. A forged zipper that fabricates a different chain is neither in the legitimate enumeration nor matched by the same-hash guarantee, so the membership premise of *reg-view-inclusion-same-hash* is load-bearing: dropping it makes the theorem false.

lemma *reg-view-inclusion-forgery-excluded*:
([], *embed-source-tree rv-embed* (*Tree* (*Suc 0*, [])))
$\notin$ *set* (*zippers-reg-view* ([], *Tree* (*0*::*chain-id*, [])))
$\wedge$ *rv-hash* (*tree-of-zipper*$_m$ ([], *embed-source-tree rv-embed* (*Tree* (*Suc 0*, []))))
$\neq$ *rv-hash* (*tree-of-zipper*$_m$ (*embed-reg-path* [], *embed-source-tree rv-embed* (*Tree* (*0*::*chain-id*, []))))
by (*simp add*: *zippers-rose-tree.simps zipper-children.simps embed-path-def hash-rt-F*$_m$*-alt-def*)

end

64 Acknowledgments

The regulatory state model (five states, seven actions, and the transition exclusions) is motivated by the Regulatory Compliance Protocol (RCP), a framework developed by the Oraclizer research team that systematically classifies requirements from 15 global financial regulatory authorities for tokenized capital markets.